



Assessment Cover Sheet

Assessment Title	Thesis Document		
Assessment Type	Uncontrolled	Individual	Not must-pass
Due Date	17 th December 2024	Course Code	IT7099
Course Title	IT Project		
Internal Moderator's Name	Husain Alyusuf		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in red below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is 8 weeks
3. This assessment carries 30 marks assessing CILO 1, CILO 2 and CILO 4.
4. The materials allowed for use in this assessment are Thesis (Design + Technical) document.
5. The **use of generative AI tools is strictly prohibited**.
6. References consulted (if any) must be properly acknowledged and cited.
7. The assessment has a total of XXX pages.

Learner ID	202101314	Date Submitted	19/05/2025
Learner Name	Mohamed Yanis Moussai		
Programme Code			
Programme Title	Bachelor of Information and Communications Technology		
Lecturer's Name	Cyril Anthony		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature	Yanis
-------------------	-------

Do not write beyond this line. For assessor use only.

Assessor's Name			
Marking Date		Maks Obtained	

Comments:

THESIS DOCUMENT ASSESSMENT DESCRIPTION

Bella Pizza – A Handcrafted Full-Stack E-Commerce Website

A Thesis Submitted in Fulfillment of the Requirements of the
Degree of

Bachelor of Science

In Information & Communication Technology

At

Bahrain Polytechnic

July 2025

Title



Copyright

© 2025

Mohamed Yanis Moussai

All rights reserved

© 2025 || Bella Pizza || 202101314 || Mohamed Yanis Moussai

Declaration

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name: Mohamed Yanis Moussai Date:18/4/2025

Approval Signatures

APPROVED FOR THE ICT PROGRAMME

_____ Thesis Supervisor

Date:

_____ Technical Writing Tutor

Date:

Abstract

The increasing reliance on third-party delivery platforms has presented significant challenges for small food businesses in Bahrain, including high commission fees, limited data ownership, and lack of customization. This project presents *Bella Pizza*, a cloud-based e-commerce web application designed to streamline online ordering and business operations for a local pizzeria. The system enables customers to browse menu items by category, manage a shopping cart, and complete secure transactions through Amazon Payment Services. Administrators can manage menu items, orders, categories, and customer accounts via a centralized dashboard. Developed entirely by a single individual using the LAMP stack, this project demonstrates that a scalable, professional-grade restaurant management system can be achieved without the need for a large development team. The solution promotes digital independence for small businesses by reducing operational costs and enhancing direct customer engagement.

Keywords: E-commerce, Restaurant Management System, Full-Stack Development, Online Food Ordering, LAMP Stack, Amazon Payment Services, Web Application

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my project supervisor for their continuous support, guidance, and constructive feedback throughout the development of this thesis. Their expertise was instrumental in shaping the direction and execution of the project.

I would also like to extend my appreciation to the ICT faculty at Bahrain Polytechnic for equipping me with the theoretical and technical knowledge that laid the foundation for this work. The modules in project management, technical writing, and system design were particularly influential in preparing me to handle the challenges of a full-scale implementation.

A special thanks goes to my peers and colleagues for their encouragement and for being a source of motivation during critical phases of the project. Their input and shared experiences have enriched the overall outcome.

Lastly, I am deeply grateful to my family for their unwavering support, patience, and understanding. Their encouragement has been essential to my academic journey and personal growth.

Table of Contents

Copyright	4
Declaration.....	5
Approval Signatures	6
Abstract	7
Acknowledgements.....	8
List of Figures	14
List of Tables.....	26
List of Abbreviations	26
Introduction	27
I. Project Rationale	27
II. Project Objectives	27
III. Prior Work.....	27
IV. Hypothesis	28
V. Proposed Solution.....	28
VI. Description of the Report.....	28
Background	30
I. Related Theory.....	30
II. Project Technology	30
III. Related Work	32
Mama Joz	33
McDonald's	35
Methodology – Solution Design	36
System Architecture Diagram	36
Deployment Diagram	38
.....	39
Use Case Diagram	40
RDB Structure	41
Wireframe	42
Sequence Diagrams	43
Add/edit/delete menu items.....	43
Ordering system.....	45

Methodology – Implementation.....	46
Development Environment Setup	46
Development Tools and IDE	48
Database Access and Management.....	48
Port Forwarding for Public Access	50
System Integration.....	51
Backend and Frontend Integration.....	51
Folder and Component Structure	52
AJAX-Based Data Interactions	54
Coding Tools and Libraries Used	55
Visual Studio Code.....	55
External Libraries and APIs.....	56
Security Setup.....	58
Session-Based Access Control	58
CSRF Protection.....	58
Prepared Statements with PDO	59
HTTPS Configuration	59
Secure Login and Session Handling.....	60
Input Validation and Error Handling	60
Real-time network and device monitoring	60
Device Configuration and System Setup.....	62
Container Architecture and Service Integration	62
Port Exposure and SSL Configuration.....	64
Volume and Data Persistence	65
Network Access and Port Forwarding.....	65
Service Management Tools	66
Description by Components.....	67
Login and Registration Functionality	67
View Menu Functionality.....	74
Add to Cart Functionality	76
Payment Functionality with APS Redirection	79
Email Functionality	83

Customer View Orders	85
Customer Manage Profile	87
Manage Menu Functionality	90
Implementation Overview.....	90
Order Management Functionality	97
Employee Management Functionality	100
CSV Export Functionality	104
Logs Functionality	106
Search and Filter Functionality.....	111
Security Implementation	113
Testing	117
Test Plan	117
Objectives.....	117
Scope	117
Roles and Responsibilities	118
Tools and Environment.....	118
Types of Testing Conducted.....	118
Test Participants	118
Functionality Test Cases and Results	119
Registration and Authentication	119
Creating an Order and Payment.....	119
Admin and Management Features	120
Performance and Security.....	121
Acceptance Testing Process and Results.....	122
Usability Testing results and statistics	123
Final Review from Participants	124
Conclusion	124
Discussions, LESPI, Conclusion.....	125
System Functionality	125
Summary of Achieved Objectives	126
Project Issues (with Backup Plan/Proposed Solutions/Actions)	128
Backup Plan	130

Backing up data	130
Security and Access Control	131
Disaster Recovery Plan	132
Version Control	132
Future Work (Upgrades – Modifications)	133
Synopsis of My Experience	134
Bahraini Perspectives	135
Legal, Ethical, Social, and Professional Issues	136
Legal Issues	136
Ethical Issues	136
Professional Issues	137
Conclusion	138
Appendices	139
Appendix I: System and User Manuals.....	139
Appendix II: Detailed Design	166
User Interface Design	166
Appendix III: Detailed Implementation	177
I. Setting up the Environment	177
II. Creating Containers and Virtual Hosts	178
Dockerfile.....	178
Docker-Compose.yml File.....	181
ssl config file.....	185
Images	188
III. Creating the Database	189
IV. Developing the Backend Logic.....	194
IV. A. CSRF Page.....	194
IV. B. Session Secure Configuration	197
IV. C. Home Page.....	199
IV. D. Login Page	203
IV. E. Admin and Employee Dashboard Page.....	215
IV. F. Edit Menu Page	221
IV. G. Add Menu Item Page	229

IV. H. Edit Menu Item Page	237
IV. I. Manage Orders Page.....	247
IV. J. Manage Employees Page	256
IV. L. Edit Employee Page	271
IV. M. Soft-Delete Employee	278
IV. N. Business Analytics Page	281
IV. O. System Logs	291
IV. P. Real-time Delivery tracking System	298
IV. Q. Customer Dashboard.....	309
IV. R. View Cart Page.....	315
IV. S. Payment Gateway	323
IV. T. Response Page	329
IV. U. Order Placement Page.....	334
IV. V. My orders Page	338
IV. W. Customer Edit Profile.....	343
References	346

List of Figures

Figure 1 Mama Joz UI	33
Figure 2 Mama Joz checkout	34
Figure 3 Mcdonald's UI	35
Figure 4 System Architecture diagram.....	37
Figure 5 Deployment Diagram	39
Figure 6 Use-Case Diagram.....	40
Figure 7 Relational Database	41
Figure 8 add/edit/delete menu items Sequence Diagram.....	43
Figure 9 Ordering System Sequence Diagram	45
Figure 10 Docker Compose configuration for the Bella Pizza development environment.	47
Figure 11 VS Code setup with Bella Pizza source code and Docker integration.	48
Figure 12 WSL2 terminal running Docker Compose for Bella Pizza containers.....	49
Figure 13 Bella Pizza web interface accessed via Docker on localhost.	49
Figure 14 Public access to Bella Pizza website via external IP address.	50
Figure 15 Router port forwarding rules to allow public access to the Bella Pizza system.	50
Figure 16 Frontend structure of the user dashboard integrating PHP-generated menu data.	51
Figure 17 Organized folder structure of the Bella Pizza system showing modular components.	53
Figure 18 AJAX request from frontend JavaScript to a backend PHP script for live updates.....	54
Figure 19 Development of Bella Pizza using Visual Studio Code and PHP-specific extensions.....	55
Figure 20 Example of AJAX integration in the frontend JavaScript code.	55
Figure 21 Drag-and-drop menu sorting feature using SortableJS library.	56
Figure 22 Custom CSRF token implementation to secure form submissions.....	57
Figure 23 Role-based session validation enforcing secure page access.....	58
Figure 24 CSRF protection implemented using hidden token fields and server-side validation.	58
Figure 25 Secure database interaction using PDO to prevent SQL injection.	59
Figure 26 HTTPS used to ensure encrypted traffic between client and server.....	59
Figure 27 Wazuh Interface.....	61
Figure 28 Docker Compose configuration managing PHP, MySQL, and phpMyAdmin on an internal network.	63
Figure 29 Apache configuration file enabling local HTTPS support for secure testing. ..	64
Figure 30 External port forwarding enabled for public access to the Bella Pizza system.	65

Figure 31 Login Logic	68
Figure 32 registration logic	69
Figure 33 Google login logic	70
Figure 34 google login handler	71
Figure 35 Login Form View	73
Figure 37 Query database for menu items snippet	74
Figure 36 Menu Structure code	74
Figure 38 Menu View	75
Figure 39 add to cart logic	76
Figure 40cart retrieval logic on view cart page	76
Figure 41 Display each item in cart view logic	77
Figure 42 successful addition to cart	77
Figure 43 cart view	78
Figure 44 Proceed to payment button that sets the total order value to a session variable and redirects user to the payment gateway page	79
Figure 45 API request params configuration and posting data to APS.....	80
Figure 46 APS redirect for payment	81
Figure 47 successful payment message	82
Figure 48 Filled form for payment	82
Figure 49 Email functionality using PHPMailer for customer order confirmations.	83
Figure 50 Received Confirmation Email	84
Figure 51 get customer's orders	85
Figure 52 My orders view.....	86
Figure 53 Profile UI	87
Figure 54 Role based action buttons	88
Figure 55 Employee tailored view	89
Figure 56Admin Dashboard.....	89
Figure 57 edit menu logic	90
Figure 58Add menu item logic	91
Figure 59 Toggle item availability logic	91
Figure 60 add category logic.....	92
Figure 61edit category logic	92
Figure 62 delete category logic	92
Figure 63 Edit menu items view	93
Figure 64 Add menu item form	94
Figure 65 edit menu form	95
Figure 66 Category management view	96
Figure 67 Role based search logic	97
Figure 69 Status Update buttons logic	98
Figure 68 Assign Driver Logic.....	98
Figure 70 Assign drivers page	99

Figure 71 Orders View.....	99
Figure 72 Add employee Form	101
Figure 73 Employees management view	101
Figure 74 Edit Employee form.....	102
Figure 75 delete employee logic	103
Figure 76 Export Employees CSV logic	104
Figure 77Analytics page download CSV logic	104
Figure 78 Employees CSV file	105
Figure 79 Analytics report CSV file	105
Figure 80 Log function logic	106
Figure 81 calling log funtions.....	107
Figure 82 log page included in files	107
Figure 83 Logs view	107
Figure 84 Create new chart logic for items sold.....	109
Figure 85 Data summarization to turn raw data into meaningful summaries	109
Figure 86 Analytics View	110
Figure 87 Search Logic in employees table	111
Figure 88 Filtering Logic in analytics page.....	112
Figure 89 filter view in analytics page	112
Figure 90 search view in edit menu page	112
Figure 91 csrf validation.....	113
Figure 92 Prepared statements used.....	113
Figure 93verifying users through session variables.....	114
Figure 94 input validation for registration	114
Figure 95 Input sanitization	115
Figure 96 Wazuh interface.....	116
Figure 97 Ease of Navigation, Visual Appeal, and Satisfaction, Graph.....	123
Figure 98 Overall Experience Graph.....	123
Figure 99 Backup successfully created	130
Figure 100 export database script.....	130
Figure 101 script to create zip file of project directory	131
Figure 102 turn windows features on or off	139
Figure 103 check Windows Subsystem for Linux.....	139
Figure 104 Ubuntu in the microsoft store	140
Figure 105 Ubuntu running after setup on windows 11	140
Figure 106 bella pizza directory	141
Figure 107 bella-pizza-docker folder	141
Figure 108 bella-pizza-docker directory	141
Figure 109 Copied content into directory	142
Figure 110 Downloading docker desktop.....	142
Figure 111 WSL integration	143

Figure 112 New Containers Created	144
Figure 113 Router Interface	145
Figure 114 new port map configuration	146
Figure 115Creating project at supabase	148
Figure 116 Sign in providers	149
Figure 117 enable google sign in	150
Figure 118 google cloud Page	150
Figure 119 moving to Oauth page	151
Figure 120create OAuth client	151
Figure 121Configuring OAuth Client.....	152
Figure 122Getting Client ID and Client Secret.....	153
Figure 123Insert client ID and secret here	154
Figure 125 Index page	155
Figure 126 Registering A new User	155
Figure 127 Log in with new account	156
Figure 128 Adding items to cart	156
Figure 129 Cart view	157
Figure 130 APS view	157
Figure 131 server response	158
Figure 132 order placement upon successful payment response.....	158
Figure 133 Received Confirmation Email.....	158
Figure 134my orders button	159
Figure 135order details view	159
Figure 136 my orders view.....	159
Figure 137 Admin dashboard view	160
Figure 138 Menu Management View	161
Figure 139 Manage Orders view	162
Figure 140 Employee CSV file.....	163
Figure 141 Employee List	163
Figure 142 Analytics View	164
Figure 143 Colour palette	167
Figure 144 Example Nav bar	168
Figure 145 Homepage Structure	169
Figure 146Login page Structure	170
Figure 147 User dashboard structure	171
Figure 148 Admin/Employee dashboard structure	172
Figure 149 Edit Menu structure.....	173
Figure 150 Manage orders structure	174
Figure 151 Analytics Page structure	176
Figure 152 Docker Desktop and Ubuntu Setup on Windows 11	177
Figure 153 Base image using PHP 8.2 with Apache	178

Figure 154 Enable Apache modules for URL rewriting and SSL	178
Figure 155 Install MySQL-related PHP extensions	178
Figure 156 Prepare SSL directory in Apache.....	179
Figure 157 Copy SSL certificate and private key to the container	179
Figure 158 Activate the Apache SSL virtual host.....	179
Figure 159 Dockerfile full screenshot.....	180
Figure 160 Docker Compose file version	181
Figure 161 PHP-Apache container setup for Bella Pizza.....	181
Figure 162 MySQL 8 container for data persistence.....	182
Figure 163 GUI for MySQL database administration	183
Figure 164 Persistent volume and network for container communication	183
Figure 165 docker compose yml full screenshot	184
Figure 166 Define SSL VirtualHost for HTTPS	185
Figure 167 Define server identity and document root	185
Figure 168 Enable SSL and bind self-signed certificate.....	185
Figure 169 Enforce strong encryption for secure connections.....	186
Figure 170 Grant access and allow .htaccess overrides.....	186
Figure 171 Define SSL-specific log file locations	186
Figure 172 default-ssl full screenshot	187
Figure 173 Multi-container created.....	188
Figure 174 Website working through https.....	188
Figure 175 Stores menu categories.....	189
Figure 176 Stores customer registration and contact data	189
Figure 177 Manages delivery logistics	190
Figure 178 Employee user management	190
Figure 179 Centralized activity logging	191
Figure 180 Defines all menu items.....	191
Figure 181 Stores order transactions	192
Figure 182 Line items per order	192
Figure 183 Records payment transactions	192
Figure 184 Enforcing foreign key relationships	193
Figure 185 If the session does not yet have a CSRF token, generate a cryptographically secure 32-byte random string (hex-encoded) and store it in \$_SESSION['csrf_token'].194	
Figure 186 This function checks that the incoming POST request includes a csrf_token field matching the session's token, using hash_equals() to prevent timing attacks. If validation fails, it returns HTTP 400 and stops execution.	195
Figure 187 Csrf token returned 200 "ok"	196
Figure 188 Success Csrf token message	196
Figure 189 Here we specify five critical flags for the session cookie—tying its lifetime to the browser session, scoping it site-wide, enforcing HTTPS-only transport, blocking JavaScript access, and using SameSite=Lax to reduce CSRF risk.....	197

Figure 190 Initialize or resume the session, causing PHP to emit the cookie with the previously defined parameters.	197
Figure 191 Session cookie attributes in Chrome DevTools confirming Secure, HttpOnly, and SameSite=Lax settings	198
Figure 192 Import session cookie configuration and CSRF helper before any output.	199
Figure 193 Process logout requests only after verifying the CSRF token.	200
Figure 194 Code snippet from index.php that logs unique visitors to the visitors table by IP and user agent.	200
Figure 195 “Login” or a CSRF-protected “Logout” button based on authentication state, plus a persistent “View Menu” link.	201
Figure 196 Bella Pizza welcome page showing the “Login” controls and “View Menu” button on desktop.	202
Figure 197 user logged in cookies	202
Figure 198 Import centralized session cookie settings and CSRF helpers at the very start.	203
Figure 199 Initialize PHP session, connect to MySQL via MySQLi, and include the logging helper.....	204
Figure 200 Prevent logged-in visitors from re-seeing the login/register page by redirecting them to their dashboard.	204
Figure 201 Both login and registration forms include the session’s csrf_token in a hidden field to be validated on submit.	205
Figure 202 Prevent logged-in visitors from re-seeing the login/register page by redirecting them to their dashboard.	206
Figure 203 Validate inputs, hash password, insert new customer, and log each step.	207
Figure 204 google login redirection	208
Figure 205 Google OAuth Callback Handler	209
Figure 206 Google Login Session Processor	210
Figure 207 Login and Registration Form.....	212
Figure 208 Successful Registration Message	213
Figure 209 Invalid Login	213
Figure 210 Invalid Input message in registration.....	214
Figure 211 google login user created.....	214
Figure 212 Securely start the session, enforce authentication, and handle logout with CSRF validation.	215
Figure 213 Dashboard header displaying the user’s role and a CSRF-protected logout button.	216
Figure 214 Section showing the logged-in username for confirmation and personalization.	216
Figure 215 Conditional rendering of dashboard actions based on the user’s role.....	217
Figure 216 Footer displaying the current year dynamically.	217
Figure 217 Dashboard Admin View	219

Figure 218 Dashboard Delivery Driver View	220
Figure 219 Secures the session, loads CSRF helpers, connects to the database, and enforces role-based access control.	222
Figure 220 Processes availability checkbox toggles via POST, updates the database, and logs the action.	222
Figure 221 Handles AJAX POSTs for drag-and-drop reordering of menu items and categories, persisting the new display order.	223
Figure 222 Loads all categories and, for each, queries its menu items in the configured display order.	223
Figure 223 Renders the live-search input and collapsible <details> blocks grouping each category's items.....	224
Figure 224 Displays each menu item row with ID, name, price, an auto-submitting availability checkbox form, and an Edit link.	224
Figure 225 jQuery and SortableJS power live search filtering, availability toggles, and drag-and-drop reordering with AJAX updates.	225
Figure 226 Real-time client-side filtering of menu items using the search bar.....	227
Figure 227 The complete menu management interface with collapsible categories and action controls.	227
Figure 228 Drag-and-drop in use—reordering categories (or items) dynamically with SortableJS.	228
Figure 229 Toggled Menu Item to Unavailable	228
Figure 230 Securely start session, set up CSRF, logging, connect to the database, enforce role-based access, and load categories.....	229
Figure 231 On POST, validate CSRF, sanitize inputs, handle secure image upload, insert the new item via a prepared statement, and log the action.	230
Figure 232 The form markup includes semantic labels, a CSRF hidden input, a category dropdown, and a drag-and-drop area with live preview.....	231
Figure 233 JavaScript to power the custom file-drop zone and render a live image preview.....	232
Figure 234 Initial Add Menu Item screen showing all input fields and the drag-and-drop area.	234
Figure 235 File drop area highlighting when a file is dragged over it.	235
Figure 236 Confirmation message shown once the menu item is added successfully.	236
Figure 237 Live preview of the uploaded menu item image.	236
Figure 238 Secure session setup, CSRF helper, DB connection, and access control..	237
Figure 239 Fetch the menu item to edit and list of categories.....	238
Figure 240 Validate CSRF, process form inputs, handle file upload, update the database, and log the change.	239
Figure 241 Persistent navigation bar and feedback banner.....	240
Figure 242 Inputs for editing text, numeric, selection, and checkbox values.....	240

Figure 243 Vanilla JS handlers for drag-and-drop uploads and live preview of the selected image.	241
Figure 244 “Edit Menu Item” form showing prefilled values and the current item image.	243
Figure 245 Highlighted drop zone when a file is dragged over, ready for upload.....	244
Figure 246 Instant image preview after selecting or dropping a file in the edit form. ...	245
Figure 247 Successful menu edit message.....	246
Figure 248 Secure session, CSRF support, logging helper, PDO initialization, and access control.	247
Figure 249 Listen for POSTs with order_id and new_status, update the order’s status, log the change, and return a simple success response.....	248
Figure 250 Retrieve orders (filtered by driver for delivery role), then split them into today’s and past orders based on the date.	249
Figure 251 Encapsulate action-button HTML and collapsible table rendering logic....	250
Figure 252 Handle AJAX POST for status updates and AJAX GET for section refresh. ..	251
Figure 253 Navigation bar, search input, and container for dynamically loaded orders.	251
Figure 254 jQuery-driven handlers for live search, delegated AJAX form submits, full-section reloads, and timed polling.....	252
Figure 255 Manage Orders interface with collapsible sections for today’s and past orders.	254
Figure 256 Real-time order filtering by customer name or ID.	254
Figure 257 Inline status update—from “Pending” to “Preparing”—handled via AJAX without a full page reload.....	255
Figure 258 Secure session start, CSRF inclusion, and access control before any data is shown	256
Figure 259 Query all employees in descending ID order and collect them into the \$employees array.	257
Figure 260 A sticky top navbar provides navigation back to the main dashboard or a logout link, and the .container wraps the page content.	257
Figure 261 Top-of-page controls include an “Add Employee” link, an “Export CSV” trigger, a live search input, and a summary count.	258
Figure 262 A table lists each employee’s details with a colored role badge and inline “Update”/“Delete” controls.	259
Figure 263 jQuery-powered live search, AJAX soft-delete with CSRF, and client-side CSV generation.	260
Figure 264 The full Manage Employees interface, with role badges, action buttons, and search controls.	262
Figure 266 Real-time filtering of employee records by name or ID, with the total count reflecting only visible rows.	262
Figure 265 Downloading the visible employee list as a CSV file	262

Figure 267 Employee CSV file in Excel.....	262
Figure 268 Confirmation dialog to prevent accidental deletion.	263
Figure 269 Loading session hardening, CSRF utilities, logging helper, and establishing DB connection, then enforcing role-based access.	264
Figure 270 Validating CSRF, sanitizing input, checking duplicates, hashing passwords, inserting new employee, and logging outcomes.....	265
Figure 271 An endpoint returning “taken” or “available” for live validation of usernames or emails.	266
Figure 272 The employee form with hidden CSRF token, fields for username/email, and a small script for live availability checks.	267
Figure 273 Add Employee Form	269
Figure 274 Success message in adding employee.....	270
Figure 275 Load security modules, establish DB connection, and restrict access to admin/support users.	271
Figure 276 AJAX endpoint returning “taken” or “available” for username/email, excluding the current employee record.	272
Figure 277 Retrieve the employee’s existing data into an associative array for form prefill.	272
Figure 278 Validate CSRF token, bind form values, execute the update, log the change, and reload the record.	273
Figure 279 The update form pre-populated with current values, including hidden CSRF token.....	274
Figure 280 Vanilla JS listens on username/email inputs, calls the ?check= endpoint with the current id, and updates status displays.	275
Figure 281 Edit Employee form pre-filled	276
Figure 282 Employee Updated successfully message.....	277
Figure 283 Soft-delete handler that marks an employee’s is_deleted flag via AJAX, enforcing role-based access and returning appropriate HTTP status codes.	278
Figure 284 Employee 12 and 13 Set is_deleted as 1	280
Figure 285 Employee 12 and 13 Do not show up on UI.....	280
Figure 286 Start session, enforce that only admins may access analytics.....	281
Figure 287 Connect to the database, determine the filter range, and compute total orders and revenue.	282
Figure 288 Retrieve breakdowns by status, top-selling items, and daily revenue for charts and tables.	283
Figure 289 Code block that calculates total logins, new customers, and returning customers using date-based filtering.....	284
Figure 290 Date filters, cards for summary metrics, tables for breakdowns, and canvases for charts.	285
Figure 291 JavaScript to render bar and line charts with Chart.js, toggle auto-refresh, and generate a CSV report client-side.	287

Figure 292 Analytics Table View.....	289
Figure 293 Analytics Graphs.....	289
Figure 294 analytics report in excel.....	290
Figure 295 Downloading Analytics Report	290
Figure 296 Start a secure session, connect to MySQL, and require admin access.	291
Figure 297 Read and sanitize filter inputs, then assemble a dynamic WHERE clause plus corresponding parameter and type arrays for prepared statements.	292
Figure 298 First execute a COUNT(*) with bound parameters to get the total number of matching logs, then prepare and execute the paginated SELECT using the same filters plus LIMIT/OFFSET.	293
Figure 299 Render the filter form with preserved values, display the paginated log entries in a responsive table, and generate pagination links based on the total count.	294
Figure 300 System Logs View	296
Figure 301 Search Funtion in System Logs.....	296
Figure 302 Filetring messages by Level "Warning"	297
Figure 303 Checks if a customer is logged in before allowing access to the tracking page.....	299
Figure 304 Retrieves and sanitizes order_id and driver_id from the URL.	299
Figure 305 Verifies that the logged-in customer owns the order and that the driver matches the assignment.	299
Figure 306 Renders the map container and navigation elements for the user interface.	300
Figure 307 Loads the Leaflet.js library from a CDN to enable interactive map rendering.	300
Figure 308 Initializes the Leaflet map with OpenStreetMap tiles and a placeholder for the driver marker.	300
Figure 309 Periodically fetches the driver's coordinates from the server and updates the map marker.	301
Figure 310 Establishes a PDO connection to the RestaurantDB database.....	301
Figure 311 Queries the driver_location table to get the driver's latest latitude and longitude.	301
Figure 312 Sends the driver's location to the frontend as a JSON object or an empty array if not found.	301
Figure 313 Ensures only logged-in delivery drivers can access the location sharing page	302
Figure 314 Displays instructions and status messages to the driver.....	302
Figure 315 Sends the driver's current latitude and longitude to the server using a POST request.	303
Figure 316 Uses the browser's geolocation API to capture and continuously track the driver's position.	304

Figure 317 Confirms the user is a logged-in delivery driver before accepting location data.	305
Figure 318 Extracts latitude and longitude from the POST request's JSON body.	305
Figure 319 Inserts or updates the driver's current location using their session-linked ID.	305
Figure 320 Sends a JSON response confirming the location update was successful. .	306
Figure 321 "out for delivery" order showing "track delivery" button.	307
Figure 322 Map UI showing very accurate location of driver	307
Figure 323 Successfully sent GPS location	308
Figure 324 Share location page prompting drive to accept sharing their location	308
Figure 325 Protect the page behind login and fetch the current customer's username	309
Figure 326 Snippet from dashboard_user.php that logs a login and checks whether the customer is new or returning.	310
Figure 327 Securely update the session-based shopping cart when the user submits a form	311
Figure 328 Build two arrays: one of categories, the other grouping available menu items by category.	311
Figure 329 Render a sticky header with branding, quick links, and a logout form.	312
Figure 330 Display a thematic banner and greet the customer by name, showing any success messages.	312
Figure 331 Loop over each category to render horizontally scrollable item cards with add-to-cart forms.	313
Figure 332 Customer Dashboard View	314
Figure 333 Successful add to cart message.	314
Figure 334 Ensure only logged-in users with items in their cart can proceed, then establish a PDO connection.	316
Figure 335 Determines if the user's profile contains valid phone and address information before proceeding to payment.	317
Figure 336 Handle quantity updates or clear the cart entirely.	318
Figure 337 Retrieve details for each menu item in the cart.	318
Figure 338 Display an editable cart table, update/clear buttons, and a "Proceed to Payment" form.	319
Figure 339 Prevents users with incomplete profiles from accessing the payment gateway.	319
Figure 340 Offer navigation back to the menu and handle an empty cart.	320
Figure 341 Empty Cart View	321
Figure 342 Clearing cart Confirmation Message.	321
Figure 343 The Cart page lets users adjust quantities, view subtotals, and proceed to payment.	321
Figure 344 Warning message displayed if user profile is not complete	322

Figure 345 Ensure only authenticated users with items in their cart can proceed, then define the APS credentials.	323
Figure 346 Build the parameter array, sort it, then create the APS signature.	324
Figure 347 Render a hidden form with all APS parameters plus signature, then auto-redirect.	325
Figure 348 Starting Cart with Items	326
Figure 349 Redirected to payment gateway where user needs to input payment details	326
Figure 350 Payment Form Filled.....	327
Figure 351 Successful Payment Message from APS	328
Figure 352 Failed Payment Message from APS	328
Figure 353 Restore the specific session from APS and load core session/CSRF configuration	329
Figure 354 Compute and compare the APS signature, then flag success if the status code is 14.....	330
Figure 355 On validated success, fire off the order confirmation email.	330
Figure 356 Five-second delayed redirect to finalize the order, with a no-JS fallback. ..	331
Figure 357 Display success, failure, or tampering messages in a contrasting dark theme.	332
Figure 358 Failed Payment Message	333
Figure 359 Successful Payment Message	333
Figure 360 Ensure only a proper POST from our payment flow with a valid session and non-empty cart.	334
Figure 361 Establish a PDO connection to the RestaurantDB with exception mode enabled.....	335
Figure 362 Fetch current prices in bulk, compute per-item subtotals, and accumulate the order total.	335
Figure 363 Display the order number and total paid in a clean, centered card.	336
Figure 364 User-facing confirmation screen showing successful order placement and total charged.	337
Figure 365 Verified new order and associated items in the database after POSTing to place_order.php.....	337
Figure 366 Secure session start, CSRF include, and fetch the logged-in customer's username.....	338
Figure 367 Load the customer's orders and apply the optional search filter.	339
Figure 368 Search input followed by a responsive table of orders or an empty-state message.....	340
Figure 369 My orders Page	342
Figure 370 Search function used	342
Figure 371 Ensures that only logged-in users can access and edit their profile.	343

Figure 372 Determines the user table based on login method and retrieves profile data.	344
Figure 373 Validates submitted form data for correctness and formatting.	344
Figure 374 Ensures that the updated username is unique across both user tables. ...	344
Figure 375 Saves the new user data to the database.	345
Figure 376 Screenshot of the Bella Pizza profile update form with name, phone, and address fields.	345

List of Tables

Table 1 Project Tasks	28
Table 2 Project Technologies Used.....	30
Table 3 Project Design Technologies used	32
Table 4 Registration and authentication	119
Table 5 Creating an order and payment.....	119
Table 6 Admin and Management Features	120
Table 7 Performance and Security table	121
Table 8 Participant test process and result Table	122
Table 9 Participant Survey	123
Table 10 Achieved Objectives.....	127
Table 11 Project issues and proposed solutions	128

List of Abbreviations

Abbreviation	Definition
APS	Amazon Payment System
LAMP	Linux, Apache, MySQL, PHP
RDB	Relational Database
DB	Database
CSV	Comma-Separated Values file
UML	Unified Modeling Language
NAV	Navigation
UI	User Interface

Introduction

I. Project Rationale

Small and medium-sized food businesses in Bahrain often rely on third-party delivery platforms to reach customers online. While convenient, this dependency introduces significant drawbacks, including high service fees, limited access to customer data, and constrained customization of business processes. Moreover, these businesses are vulnerable to platform outages—when third-party services experience downtime, the restaurants suffer direct financial losses due to missed or failed orders. For local pizzerias like Bella Pizza, the need for a reliable and independent solution is clear. The rise of cloud-based web technologies provides a cost-effective and scalable opportunity for such businesses to develop their own online ordering systems and take full control of their digital operations. This project addresses these challenges by delivering a complete web-based restaurant management and ordering system that eliminates reliance on external intermediaries.

II. Project Objectives

The primary objective is to build a full-stack web application that enables customers to browse menu items, place orders, and make secure payments online. Simultaneously, it offers an administrative interface for managing orders, menu items, and customer data. The system is built using the LAMP stack and aims to demonstrate how a single developer can produce a high-quality, scalable product suitable for commercial use. The key goals are as follows:

- Deliver a user-friendly web interface for ordering food online.
- Implement real-time order and cart functionality.
- Enable secure credit card payment via Amazon Payment Services.
- Provide an admin panel for managing menu, orders, and customer accounts.
- Ensure responsive design and smooth user experience.

III. Prior Work

Existing platforms like Talabat and Hungerstation offer similar functionality but at a financial and operational cost to small businesses. These platforms also limit customization and prevent full data ownership. Several open-source food ordering systems exist, but most require significant technical adaptation or lack modern UI standards. This project aims to bridge that gap by providing a customizable, standalone solution built from scratch.

IV. Hypothesis

A single-developer solution can successfully deliver a complete restaurant ordering and management system that is scalable, secure, and user-friendly, thereby eliminating the need for third-party delivery services and reducing operational overhead for small food businesses.

V. Proposed Solution

The project delivers a cloud-hosted web application tailored for Bella Pizza. Customers can register, browse menu items grouped by category, add items to a cart, and complete orders using a secure payment gateway. The backend allows administrators to manage content through a modern dashboard. The solution is designed to be scalable, responsive, and easy to maintain.

VI. Description of the Report

This thesis is structured to present the complete life cycle of the system development process. It begins with the background theory and technologies used, followed by the system design and implementation. Later sections cover testing and evaluation, discussion of ethical and social impacts, and future development opportunities. All technical components are documented with diagrams, test cases, and appendices to provide a comprehensive view of the solution.

Table 1 Project Tasks

Task ID	Functionality	User Role	Description
F1	User Registration and Login	Customer	Allows new users to register and existing users to log in securely
F2	Menu Browsing by Category	Customer	Displays menu items grouped by categories using collapsible sections
F3	Add to Cart with Quantity Selection	Customer	Enables customers to select quantity and add items to a session-based cart
F4	View and Manage Cart	Customer	Displays cart items with options to update quantity or remove them

F5	Checkout and Online Payment	Customer	Facilitates order placement and payment via Amazon Payment Services
F6	View Order History	Customer	Allows users to see their previous orders and statuses
F7	Admin Login	Admin	Secure login for administrative users
F8	Dashboard Overview	Admin	Displays summary of orders, system activity, and quick navigation links
F9	Manage Menu Items	Admin	Add, edit, or delete menu items including images and prices
F10	Manage Categories	Admin	Add, edit, or delete menu categories
F11	Manage Orders	Admin	View, filter, and update the status of orders (e.g., Pending, Completed)
F12	Manage Customers	Admin	View and update customer details
F13	Responsive UI with Dark Mode	All Users	Ensures accessible interface across devices, with dark/light theme toggle
F14	Secure Session Management	All Users	Maintains user login and cart across sessions securely
F15	System Feedback (e.g., success messages, validation warnings)	All Users	Provides real-time alerts to improve user experience

Background

I. Related Theory

Online food ordering systems have become an essential part of modern food service businesses, especially in urban regions where convenience, speed, and digital access drive consumer behavior. These systems typically allow customers to browse a digital menu, customize orders, and place transactions online, streamlining the entire ordering process for both the customer and the business.

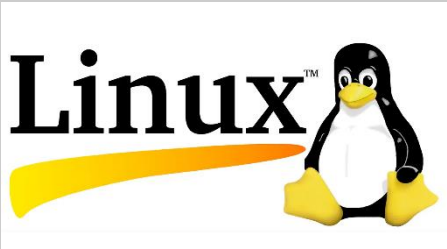
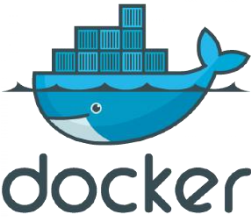
In the context of Bahrain, many small to medium-sized restaurants depend heavily on third-party delivery platforms. While these platforms offer visibility, they introduce disadvantages such as service fees, limited control over customer interaction, and dependency risks during outages. A self-hosted online ordering system allows businesses to maintain full control over their operations and branding.

From a system architecture perspective, such applications follow a client-server model, where the front end presents user interfaces and the backend processes logic, data storage, and interactions with external services such as payment gateways. Technologies like session management, relational databases, and HTTP protocols form the core of these systems.

II. Project Technology

The Bella Pizza project utilizes a stack of widely adopted web technologies, collectively referred to as the LAMP stack (Linux, Apache, MySQL, PHP), along with other modern tools to deliver a scalable and responsive web application:

Table 2 Project Technologies Used

Technology	Category	Purpose
Linux 	Operating System	Provides a stable and secure environment for deploying web servers and databases, enabling full control over configurations and server resources.
Docker 	Container Manager	Packages the application and its dependencies into isolated containers, ensuring consistent deployment across different systems and simplifying environment setup.

PHP 	Backend Scripting	Handles server-side logic, form processing, and database interaction.
MySQL 	Database Management	Stores structured data for users, orders, menu items, payments, etc.
HTML5 / CSS3 / JavaScript 	Frontend Development	Builds the user interface, layout, responsiveness, and interactive features.
XAMPP 	Development Environment	Local server stack for running Apache, PHP, and MySQL during development.
Amazon Payment Services 	Payment Gateway	Enables secure credit card payments for customer checkout.
Gmail SMTP (via PHPMailer) 	Email Service Integration	Sends confirmation and notification emails to users upon registration and order completion using a configured SMTP server.
Apache Web Server 	Web Server	Hosts PHP scripts and serves dynamic content to users.

Json 	Data Format / Interchange Format	JSON is used to exchange structured data between the frontend and backend, such as driver location updates and login information.
Supabase 	Authentication Platform / Backend as a Service	Supabase is used to handle Google OAuth login, allowing users to sign in securely without creating a password manually.

Table 3 Project Design Technologies used

Tool/Technology	Category	Purpose
Draw.io	Diagramming Tool	Used to create system architecture diagrams, ERDs, and flowcharts.

These technologies were chosen for their reliability, availability of documentation, compatibility with local hosting environments, and ease of deployment. They also allow a single developer to implement, test, and maintain the entire application effectively.

III. Related Work

Existing third-party food delivery platforms such as Talabat, Jahez, and Hungerstation, as well as individual restaurant websites, offer end-to-end ordering experiences. However, restaurants—especially local ones that build their own websites—often face significant limitations. These websites frequently lack critical features such as secure payment gateway integration, real-time order tracking, or even a decent interface. On the other hand, third-party platforms lock businesses into commission-based pricing models, limit branding flexibility, and retain ownership of customer data. Several open-source alternatives exist, but they typically require extensive customization and are not localized for the Bahraini market. Some small businesses still rely on social media or messaging apps to take orders, which can be error-prone and inefficient during peak hours. This project builds upon lessons learned from these systems, offering a fully localized, secure, and standalone solution that gives restaurants full control while ensuring convenience for the customer.

Mama Joz

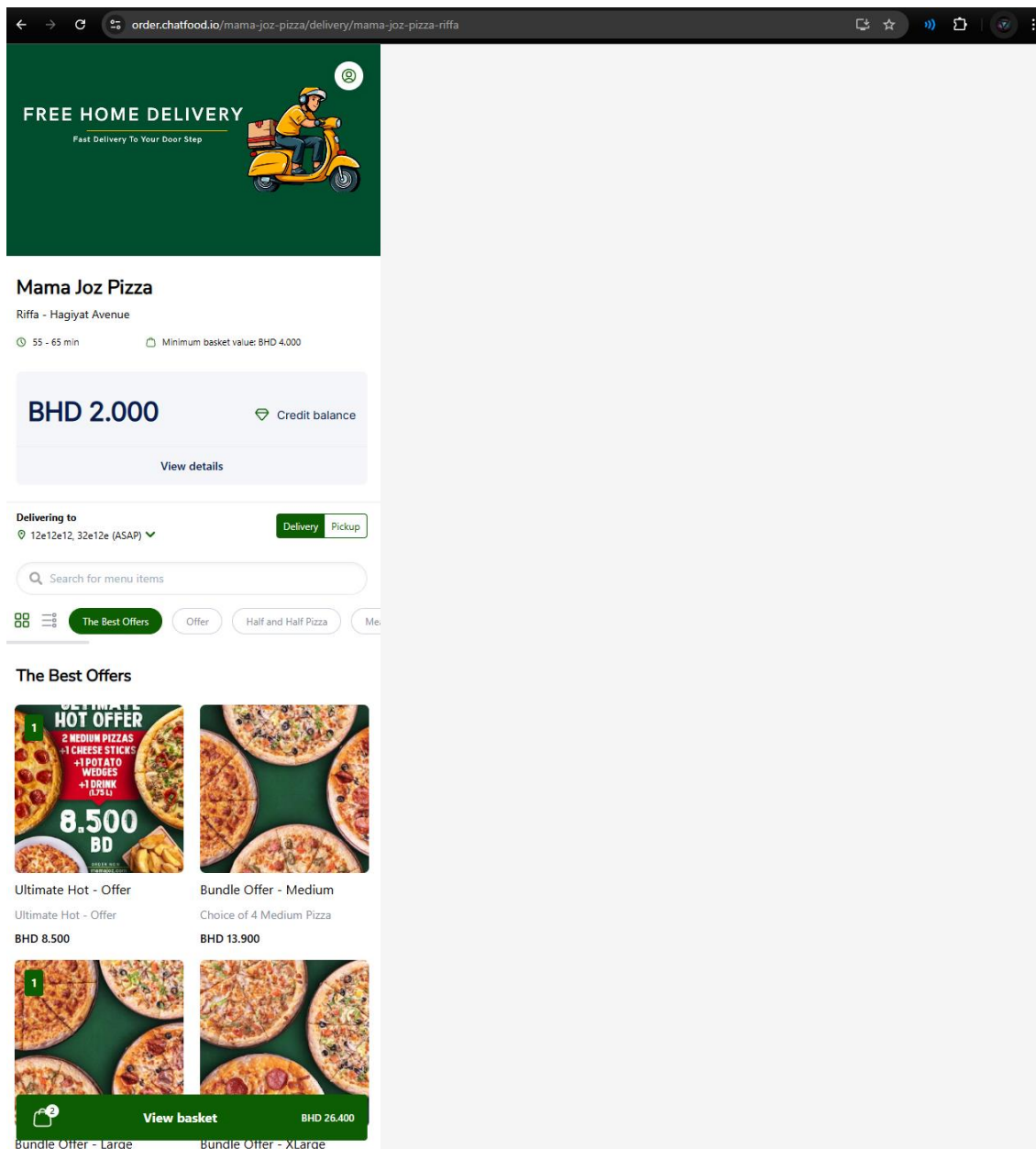


Figure 1 Mama Joz UI

While researching local restaurant ordering systems, I attempted to place an order through Mama Joz's official website. However, instead of offering a built-in ordering feature, the site redirected me to a third-party service, order.chatfood, to complete the transaction. The experience was disappointing—the user interface appeared to be designed exclusively for mobile devices. On desktop, the ordering panel was crammed awkwardly into the left side of the screen, occupying less than 30% of the available space. The remaining 70% of the screen was completely empty, giving the page an unpolished, broken appearance. This example highlights a common issue with restaurants relying on third-party or poorly integrated ordering platforms, where the user experience is severely compromised on devices outside the target design.

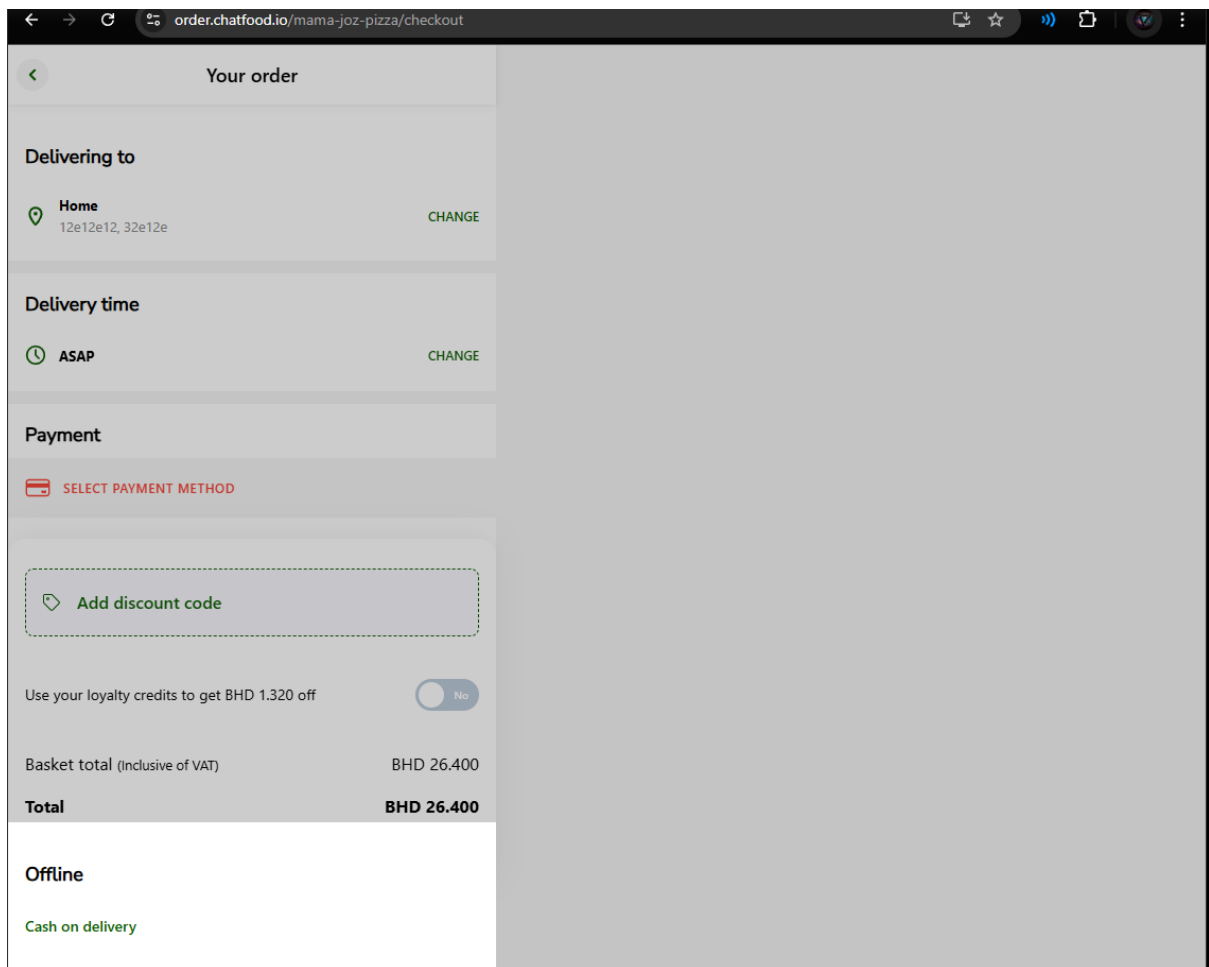


Figure 2 Mama Joz checkout

Upon reaching the checkout page, the limitations became even more apparent — the only available payment method was **cash on delivery**. There was no option to pay using credit or debit cards, which significantly reduces convenience for customers who prefer secure, cashless transactions. This lack of integrated online payment functionality not only hinders the user experience but also reflects a broader issue faced by many local restaurants: the absence of essential features like payment gateways in their digital ordering systems.

McDonald's

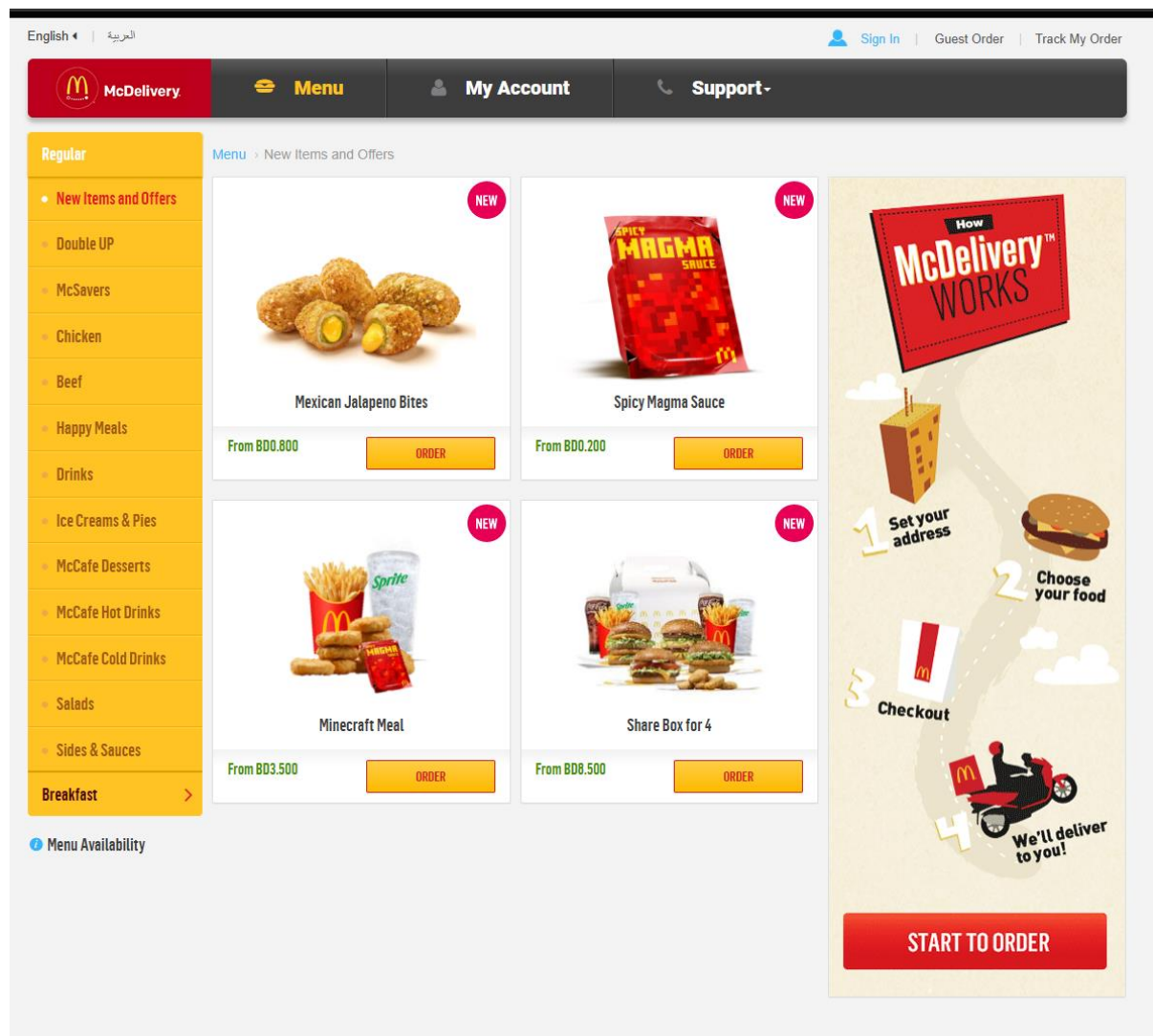


Figure 3 Mcdonald's UI

Even large franchises like McDonald's in Bahrain reveal gaps in their digital experience. While their ordering UI is at least responsive and adapts to different screen sizes, the overall design feels outdated and overly technical. The interface lacks the modern, user-friendly aesthetic expected from a global multi-billion dollar brand. This contrast highlights a wider trend where even well-funded platforms may neglect intuitive design or fail to prioritize user experience, further emphasizing the need for local solutions like Bella Pizza to strike the right balance between functionality, responsiveness, and visual appeal.

Methodology – Solution Design

System Architecture Diagram

A system architecture diagram provides a high-level representation of how the major components of a system are structured and how they interact with each other. For the Bella Pizza web application, this diagram is essential to understand the flow of data, the separation of roles, and the logical placement of each core service. It helps developers, administrators, and stakeholders quickly grasp how the application is organized, where each functionality resides, and how different users access the system.

Including such a diagram is important not only for documentation purposes but also for guiding deployment, troubleshooting, and scaling decisions. It outlines which services are accessible to the public and which are restricted for internal use, highlighting key security and design considerations.

The Bella Pizza system consists of three main components: the Web Server, the Database Server, and phpMyAdmin.

- The Web Server, hosted on a public network, handles all user-facing and administrative interfaces. Customers access the system through a browser to browse the menu, place orders, and complete payments. Employees such as administrators and support staff log in through the same interface but are granted additional privileges, such as managing orders, employees, and menu items. The webserver will be
- The Database Server, hosted on a private network, securely stores all data including user accounts, orders, menu information, and logs. It is not directly accessible to external users. Instead, the web server communicates with the database to perform all read and write operations.
- phpMyAdmin is a web-based tool used for managing the database through a graphical interface. It is reserved for trusted users such as system administrators and is not exposed to public users. phpMyAdmin connects directly to the database server to support tasks such as database inspection, backups, and manual updates when necessary. It will also be hosted on a private network.

The Bella Pizza system also integrates with various external and internal APIs to enhance functionality and enable real-time interactions. These include the Supabase API for handling secure Google OAuth authentication, the HTML5 Geolocation API for capturing live GPS coordinates from delivery drivers, and the Leaflet/OpenStreetMap API for rendering dynamic delivery maps on the customer side. Additionally, custom-built internal JSON APIs handle secure data exchange between the frontend and backend, such as cart updates, driver location tracking, and session management.

These APIs ensure smooth integration between components, improve system responsiveness, and maintain a modular architecture that can scale or adapt to new features over time.

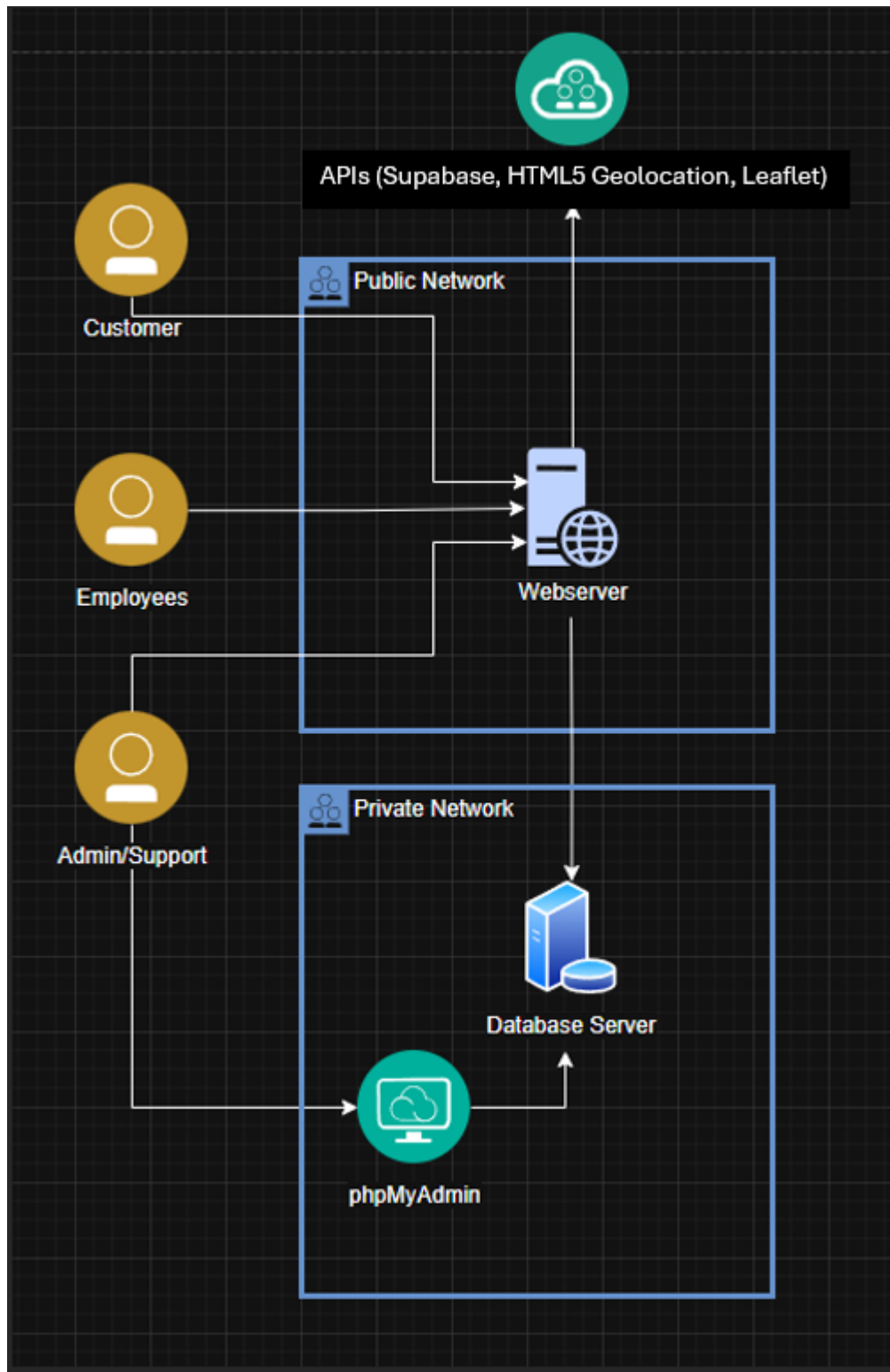


Figure 4 System Architecture diagram

Deployment Diagram

The deployment diagram illustrates the physical and virtual infrastructure used to run the Bella Pizza system. It highlights how the application components—such as the web server, database, and supporting services—are deployed within a containerized environment using Docker Desktop and WSL2 on a Windows host machine.

The Bella Pizza system is deployed using Docker Desktop on Windows 11, with WSL2 (Windows Subsystem for Linux) as the backend. Development and command-line operations are performed inside a WSL2 Ubuntu environment, which is integrated with Docker Desktop. The Docker Engine, running in a separate WSL2 distribution (docker-desktop), manages containers and associated resources.

The diagram shows key components such as:

- Ubuntu (WSL2) – where source code lives and Docker CLI commands are issued.
- Docker-desktop (WSL2) – where
- Docker Engine – the core daemon responsible for managing containers, volumes, and networks.
- Containers – including the Apache/PHP web server, MySQL database, and phpMyAdmin interface.
- Volumes – used for persistent storage of database data.
- Web Browser – used by admins and users to interact with the system through exposed ports (80, 443, or 8080).

Each element is represented with directional connections showing how data flows between the system components, including database queries, web requests, and Docker orchestration.

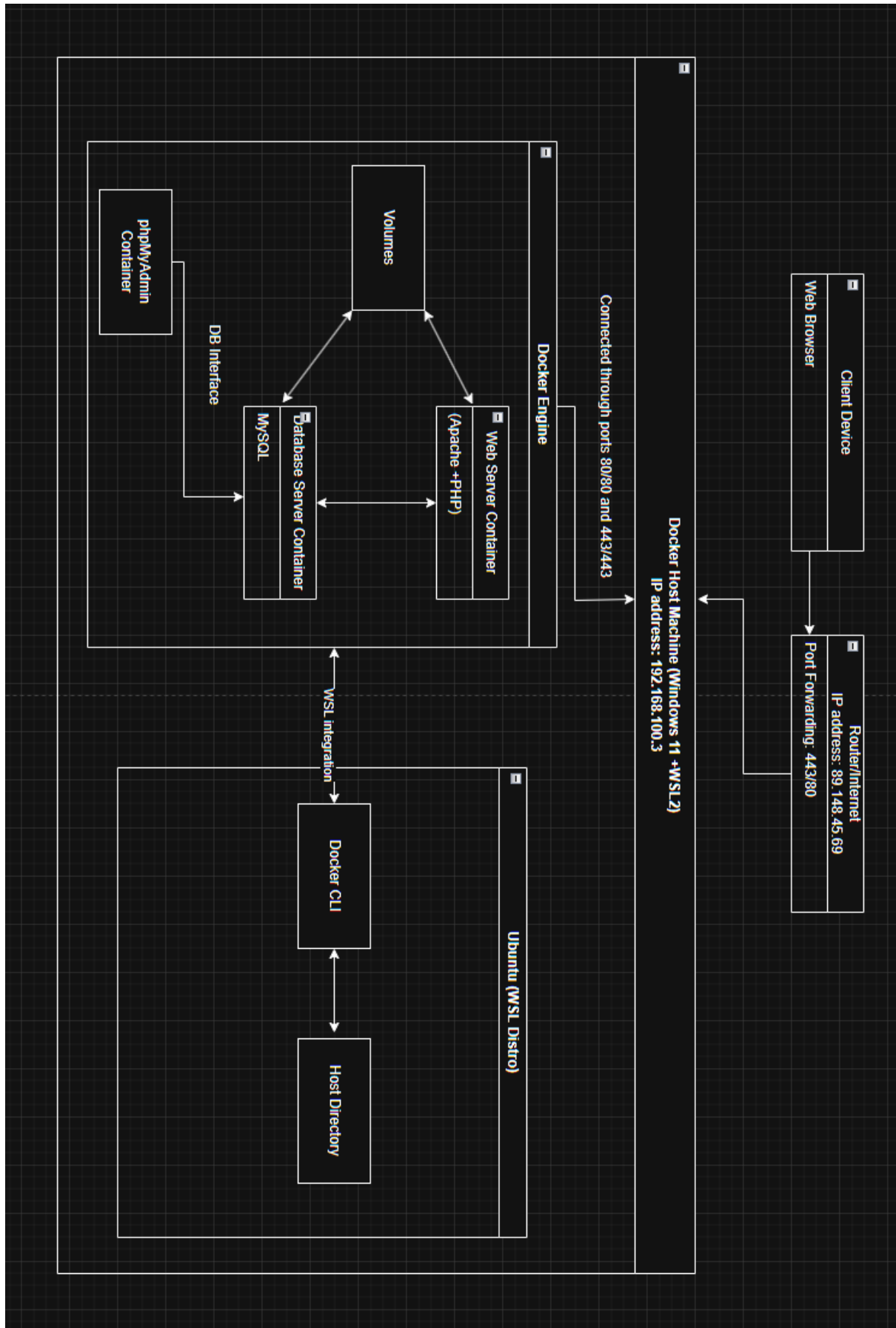


Figure 5 Deployment Diagram

Use Case Diagram

According to Rouse (2020), a use case diagram is a visual representation that describes the interactions between users (actors) and the system to achieve specific goals. It captures the functional requirements by illustrating what the system should do from the user's perspective. In the context of the Bella Pizza project, the use case diagram highlights two primary actors: customers and administrators. Customers interact with features such as account registration, menu browsing, cart management, and secure order placement. Administrators, on the other hand, access functionalities related to menu item management, category control, order processing, and user administration. This diagram was instrumental in identifying system boundaries, defining feature scope, and guiding the implementation of functional components.

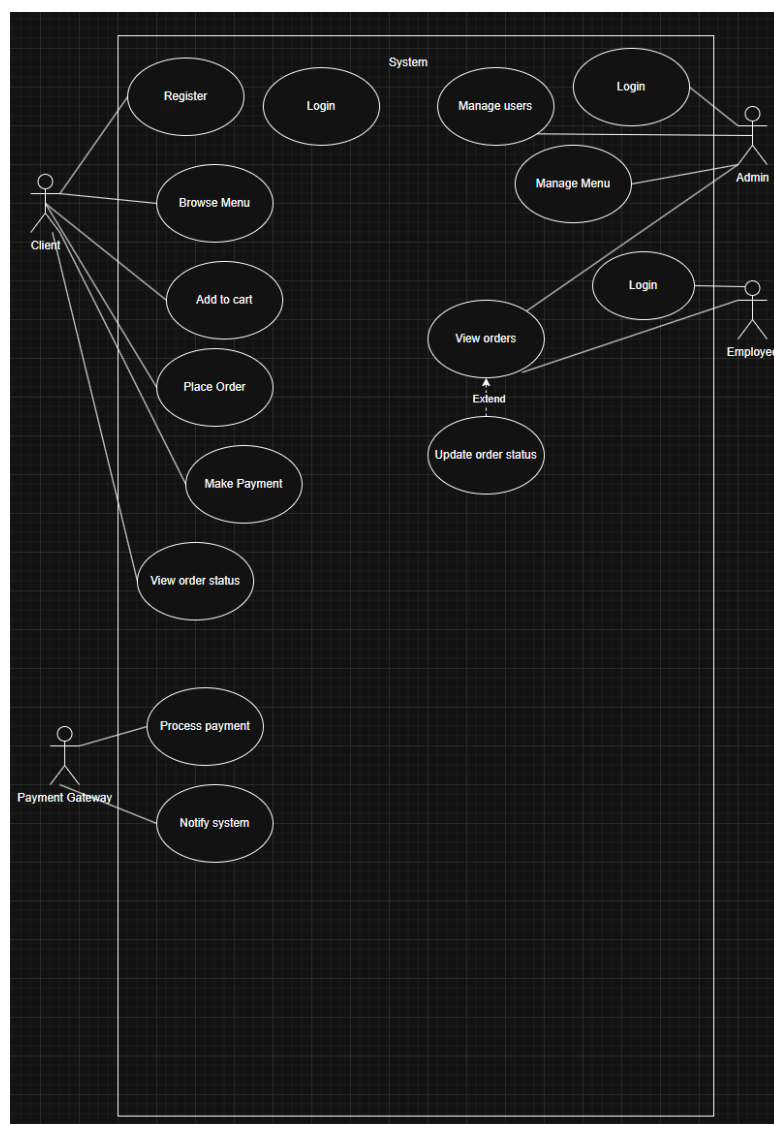


Figure 6 Use-Case Diagram

RDB Structure

A relational database (RDB) is a structured collection of data organized into tables, where relationships between data are established using primary and foreign keys. According to Codd's relational model (1970), data is represented in rows and columns, allowing for efficient querying, normalization, and data integrity. This model forms the foundation of modern database systems and is particularly suitable for transactional applications such as restaurant management systems.

In the Bella Pizza project, a relational database was designed using MySQL to support core functionalities such as user management, menu categorization, order tracking, payment processing, and delivery assignments. Each entity, including customers, orders, menu items, and employees, is represented by a dedicated table. Relationships between these entities—such as a customer placing multiple orders or an order containing multiple menu items—are maintained using foreign key constraints. This schema ensures referential integrity, reduces redundancy, and supports complex queries required for both administrative and customer-facing features.

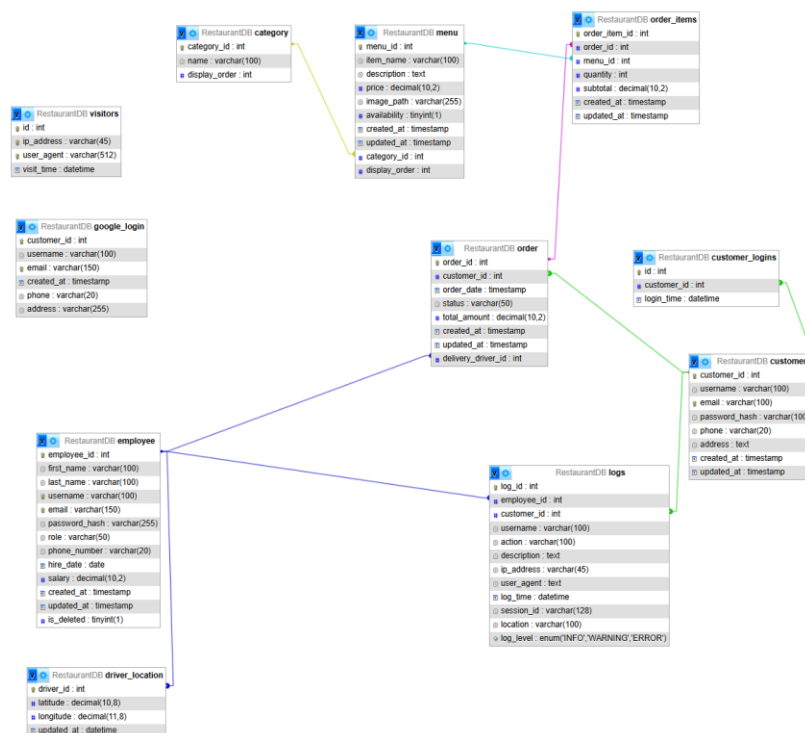


Figure 7 Relational Database

Wireframe

A wireframe is a simplified visual guide that represents the skeletal framework of a web application. It focuses on layout, navigation, and core functionality without including final design elements such as colours or images. Wireframes are widely used in early stages of system development to communicate interface structure and user flow before actual coding begins. As described by Garrett, wireframes help align design goals with user expectations by laying out interface components and their interactions in a logical and accessible format (Garrett).

For the Bella Pizza project, the wireframe was designed using Draw.io, a visual diagramming tool. It illustrates the key functionalities of the system, including user navigation through menu categories, shopping cart interaction, and order placement on the customer side. On the admin side, the wireframe maps out sections for managing orders, menu items, categories, and customer data. The design aimed to provide an intuitive, user-friendly interface while ensuring that both customer and administrative workflows are logically structured and easy to navigate.

The wireframe served as a reference point throughout the frontend development phase, ensuring consistency between the intended design and implemented features.

[Click here for the Wireframe](#)

(Download the file or open in draw.io)

Sequence Diagrams

Add/edit/delete menu items

A sequence diagram is a type of Unified Modelling Language (UML) diagram that models the flow of messages or interactions between different objects or actors in a system over time. It helps visualize how processes operate in a defined sequence and is particularly useful in identifying the order of operations in transactional systems such as web-based ordering platforms. According to Fowler, sequence diagrams clarify system behaviour by outlining which components are responsible for specific actions and how data flows between them.

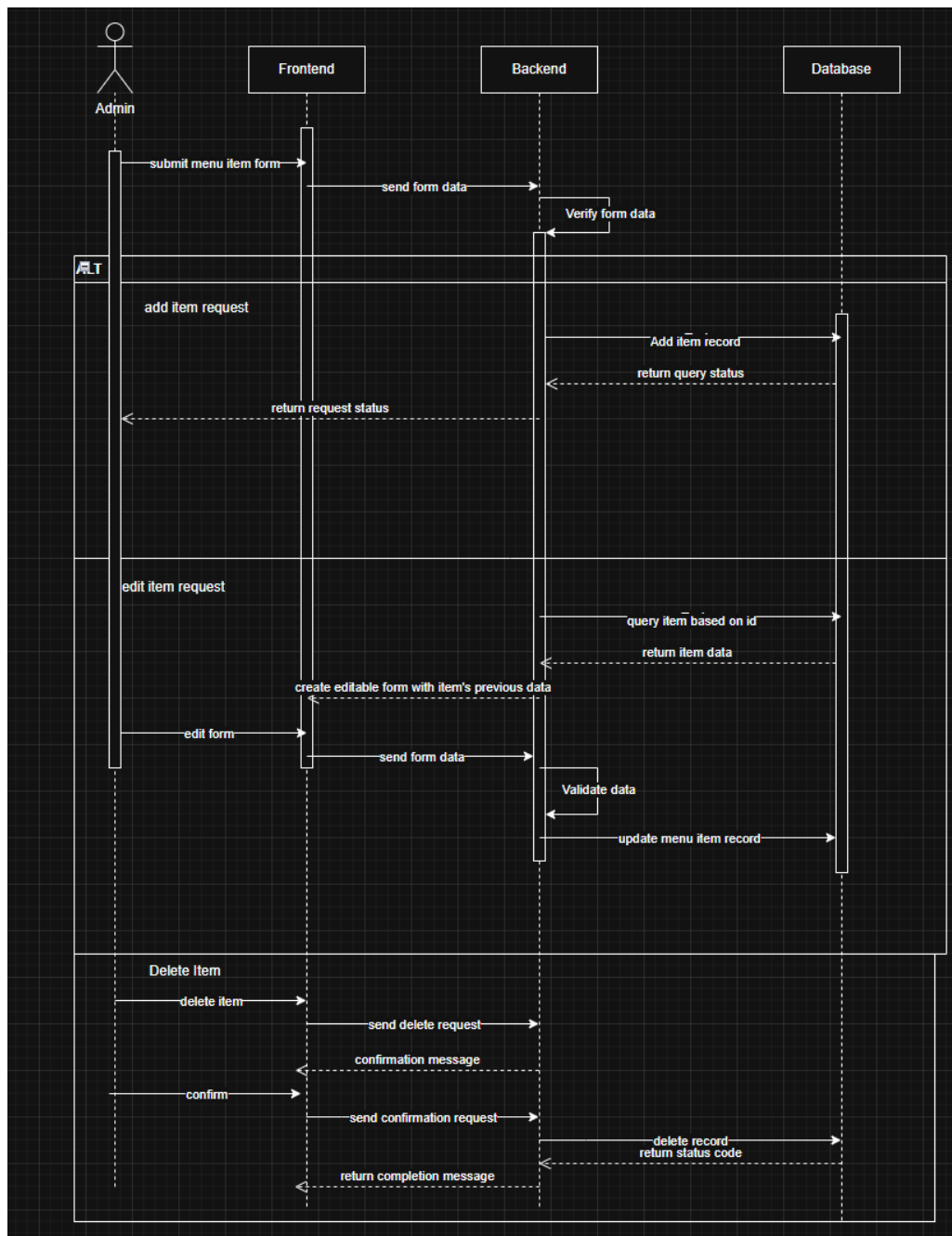


Figure 8 add/edit/delete menu items Sequence Diagram

The diagram illustrates a sequence diagram that outlines the process of an administrator adding, editing, or deleting menu items. The sequence begins with the administrator interacting with the frontend interface, where they can fill out a form to add or edit an item or click a delete button to remove an existing item. This action triggers a request to the backend, which validates the input and determines the appropriate operation. For add and edit actions, the backend verifies the form data for completeness and correctness. If valid, it sends a corresponding query to the database—either to insert a new record or update an existing one. In the case of deletion, the backend processes the request and issues a delete command directly to the database. After the operation is completed, the backend responds to the frontend with a success or error message, which is then displayed to the administrator, completing the sequence.

This interaction pattern—frontend request, backend validation and processing, database operation, and frontend feedback—is consistent across other administrative tasks in the system such as managing employees, categories, or orders. Each of these follows the same structure, ensuring consistency in behavior, code reuse, and predictable user interaction throughout the application.

Ordering system

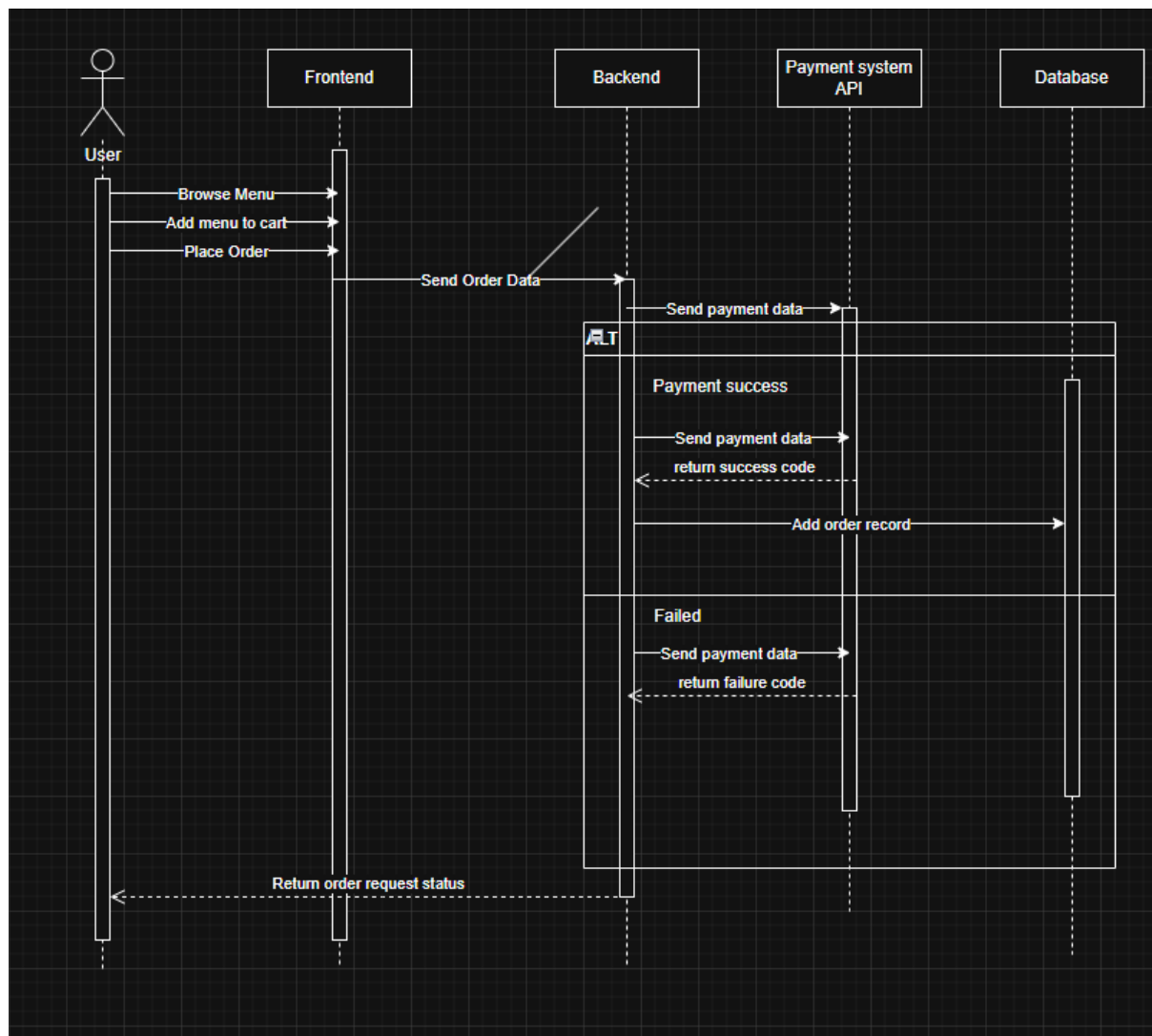


Figure 9 Ordering System Sequence Diagram

The diagram presents a sequence diagram illustrating the steps a user follows to place an order. The process begins at the frontend, where the user selects menu items and adds them to the cart. When the user proceeds to checkout, the frontend sends the order details to the backend for validation. Once validated, the backend redirects the user to the Payment System API's hosted payment page, along with the necessary payment data. The user enters their payment information directly on the API's page, and upon completion, the payment system redirects the user back to the website along with a response code. The backend receives this response, verifies its status, and if the payment is successful, stores the order information in the database. Finally, the user is presented with a confirmation message indicating whether the order was successfully placed.

Methodology – Implementation

Development Environment Setup

The Bella Pizza system was developed using a local containerized environment to ensure modularity, reproducibility, and ease of deployment. The core development environment consisted of Windows 11, enhanced with Windows Subsystem for Linux 2 (WSL2) and Docker Desktop, enabling seamless interaction between Linux-based services and Windows-based development tools.

To emulate a production-grade LAMP stack (Linux, Apache, MySQL, PHP), Docker Compose was used to orchestrate multiple containers. These included:

- A web container based on an Apache and PHP image to serve the PHP application files
- A MySQL container to manage the backend relational database
- phpMyAdmin, attached as an auxiliary service for easier inspection and manipulation of database tables during development

All services were defined in a `docker-compose.yml` file, with custom port mappings (80:80 or 8080:80, 443:443) and persistent volumes to retain MySQL data across sessions. Self-signed SSL certificates were also generated and mounted to the Apache container, enabling the use of HTTPS for secure local testing.

```

🔥 docker-compose.yml > ...
1  version: '3.8'
2
3  services:
4    > Run Service
5    web:
6      build:
7        context: .
8        dockerfile: Dockerfile
9      container_name: bella_web
10     ports:
11       - "8080:80"
12       - "443:443"
13     volumes:
14       - ./certs/selfsigned.crt:/etc/apache2/ssl/selfsigned.crt:ro
15       - ./certs/selfsigned.key:/etc/apache2/ssl/selfsigned.key:ro
16       - ./default-ssl.conf:/etc/apache2/sites-available/default-ssl.conf:ro
17       - ./app:/var/www/html
18     depends_on:
19       - db
20     restart: always
21     networks:
22       - bella_net
23     environment:
24       MYSQL_HOST: db
25       MYSQL_USER: root
26       MYSQL_PASSWORD: rootpass
27       MYSQL_DATABASE: RestaurantDB
28
29    > Run Service
30    db:
31      image: mysql:8.0
32      container_name: bella_db
33      restart: always
34      environment:
35        MYSQL_ROOT_PASSWORD: rootpass
36        MYSQL_DATABASE: restaurantdb
37        MYSQL_USER: user
38        MYSQL_PASSWORD: userpass
39      volumes:
40        - dbdata:/var/lib/mysql
41
42    > Run Service
43    phpmyadmin:
44      image: phpmyadmin/phpmyadmin
45      container_name: bella_phpmyadmin
46      restart: always
47      ports:
48        - "8081:80"
49      environment:
50        PMA_HOST: db
51        MYSQL_ROOT_PASSWORD: rootpass
52      depends_on:
53        - db
54      networks:
55        - bella_net
56
57    volumes:
58      dbdata:
59
60    networks:
61      bella_net:

```

Figure 10 Docker Compose configuration for the Bella Pizza development environment.

Development Tools and IDE

For code development, Visual Studio Code (VSCode) was used, configured with extensions such as Docker and WSL. The project source code was maintained within the /app directory, which was volume-mounted to the web container's document root (/var/www/html). This allowed for live reloads and immediate feedback during coding.

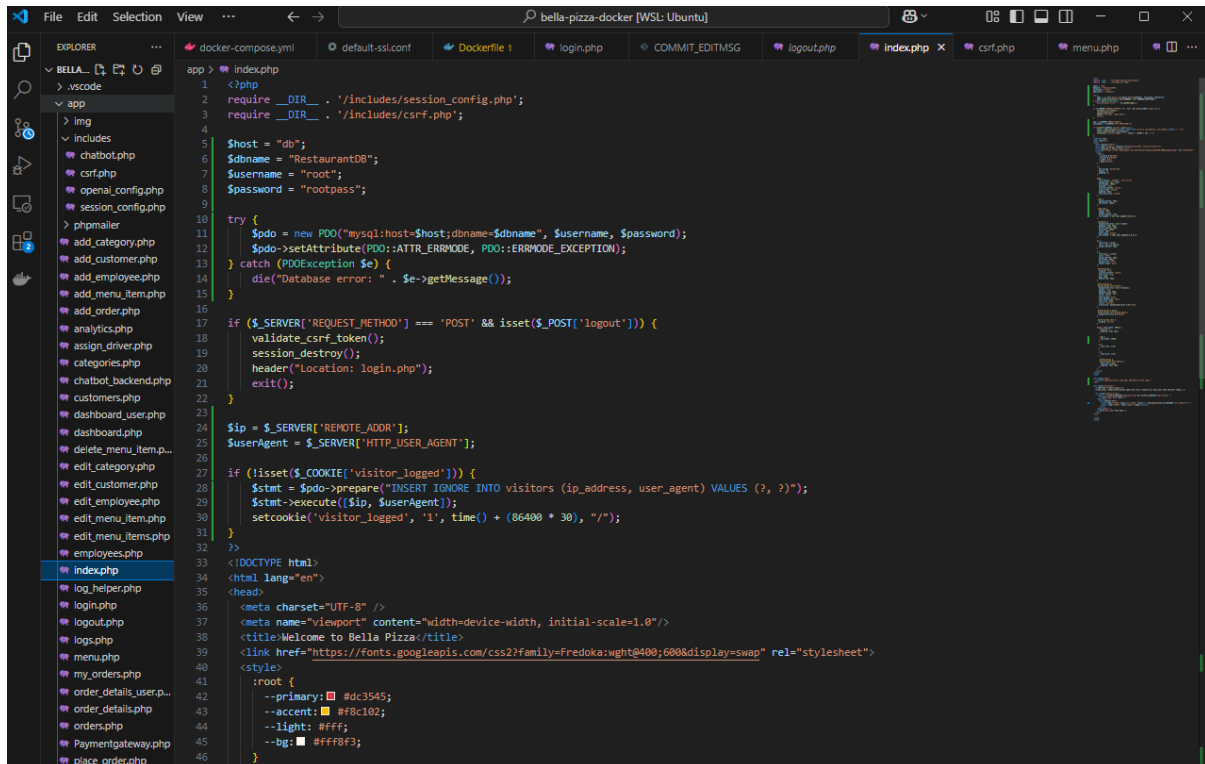


Figure 11 VS Code setup with Bella Pizza source code and Docker integration.

WSL2 provided a native Linux shell where Docker services could be managed directly through commands like `docker-compose up` and `docker ps`. All project files were accessible both from the Linux filesystem and through Windows Explorer by referencing the WSL share path (`\\wsl$`), allowing flexible file operations.

Database Access and Management

The MySQL database was accessed via both backend PHP scripts and phpMyAdmin at port 8081. Docker volumes ensured persistence across restarts.

```
bellapizza@Yanis: ~/bella-piz. x + v
bellapizza@Yanis:~/bella-pizza-docker$ docker compose up
WARN[0000] /home/bellapizza/bella-pizza-docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
  ✓ Container bella_db          Running      0.0s
  ✓ Container bella_web         Recreated 1.9s
  ✓ Container bella_phpmyadmin  Running   0.0s
Attaching to bella_db, bella_phpmyadmin, bella_web
bella_web | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.1
8.0.4. Set the 'ServerName' directive globally to suppress this message
bella_web | AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.1
8.0.4. Set the 'ServerName' directive globally to suppress this message
bella_web | [Fri May 09 09:44:12.952459 2025] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.62 (Debian)
PHP/8.2.28 OpenSSL/3.0.15 configured -- resuming normal operations
bella_web | [Fri May 09 09:44:12.952485 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'apache2 -D FOR
EGROUND'
```

Figure 12 WSL2 terminal running Docker Compose for Bella Pizza containers.

Browser-based testing was conducted using Google Chrome, and developer tools were employed to inspect element rendering, console logs, and AJAX responses. Network tab debugging was especially useful during the integration of features like real-time order polling and dynamic menu updates.

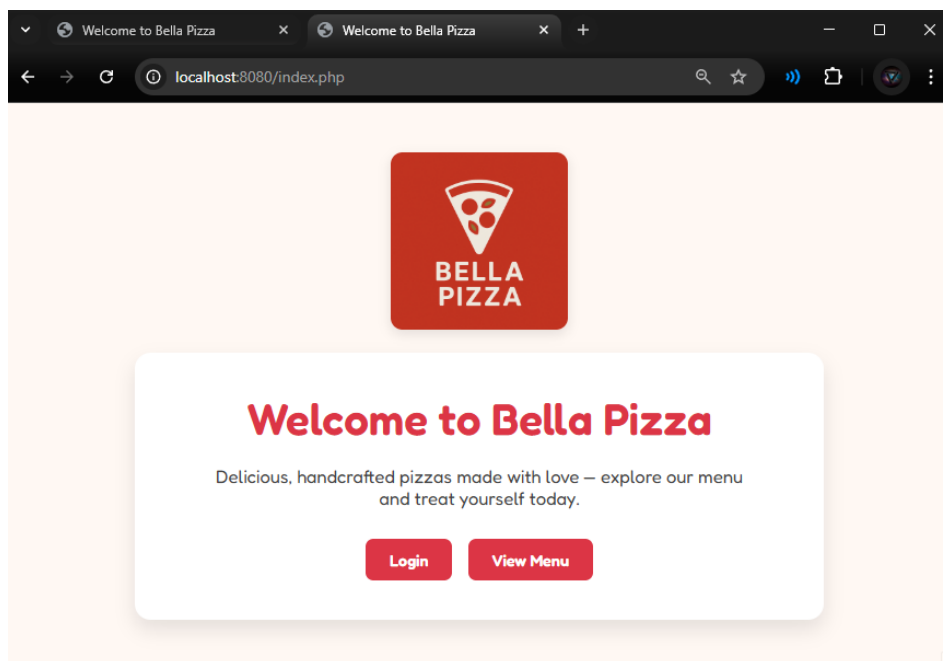


Figure 13 Bella Pizza web interface accessed via Docker on localhost.

This integrated development environment provided a robust foundation for building, testing, and refining the Bella Pizza system across all components of the stack.

Port Forwarding for Public Access

To make the Bella Pizza system publicly accessible for demonstration purposes, port forwarding was configured on the local router. Incoming traffic on port 80 or 8080 and 443 was forwarded to the internal IP address of the host machine running Docker. This allowed users to access the website from outside the local network using the host's public IP address, with appropriate firewall exceptions set for secure remote access.

Type:	☒ User-defined ☐ Application		
Application:	Select...		
Enable Port Mapping:	☒		
Mapping Name:	SSH		
WAN Name:	1_INTERNET_R_VII		
Internal Host:	192.168.100.3	*	Yanis
External Source IP Address:			
Protocol:	TCP	Internal port number:	2000 - 2000
External port number:	2000 - 2000	External source port number:	
Delete			

Type:	☒ User-defined ☐ Application		
Application:	Select...		
Enable Port Mapping:	☒		
Mapping Name:	bella pizza https		
WAN Name:	1_INTERNET_R_VII		
Internal Host:	192.168.100.3	*	Yanis
External Source IP Address:			
Protocol:	TCP	Internal port number:	443 - 443
External port number:	443 - 443	External source port number:	
Delete			

Figure 15 Router port forwarding rules to allow public access to the Bella Pizza system.

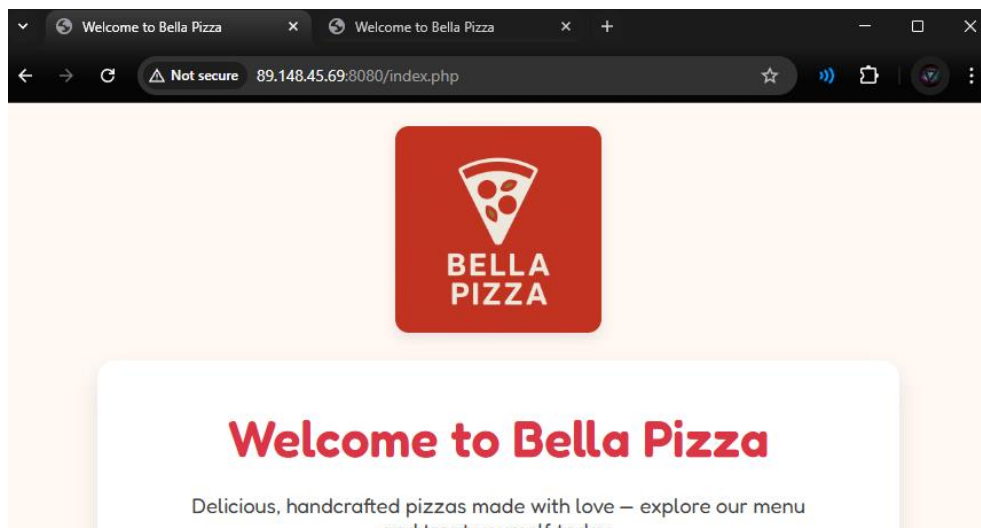


Figure 14 Public access to Bella Pizza website via external IP address.

System Integration

Backend and Frontend Integration

The Bella Pizza system follows a modular architecture where the frontend (HTML/CSS/JavaScript) is tightly integrated with the backend logic (PHP) and a MySQL database, ensuring dynamic content generation and full interactivity. This integration allowed for smooth data flow between the user interface and the underlying system logic, supporting features such as user registration, real-time order handling, cart functionality, and admin management.

The frontend consists of structured HTML templates, styled using custom CSS (with red and yellow branding colors) and enhanced with JavaScript for interactivity. Features such as collapsible category sections, availability toggles, drag-and-drop sorting, and AJAX-based search filters were all powered by frontend JavaScript, which communicated with PHP scripts using XMLHttpRequest or fetch().

The backend logic was implemented in PHP and served as the bridge between the frontend and the database. Server-side scripts handled tasks such as processing form submissions, managing user sessions, fetching menu data, logging activities, and updating order statuses. Each PHP page was modularized with reusable components to reduce redundancy and enhance maintainability.

```
94 <div class="container">
95 <h2>Edit Menu Items</h2>
96 <div class="search-bar">
97 <input type="text" id="searchInput" placeholder="Search menu items...">
98 </div>
99
100 <div class="category-group" id="categoryGroup">
101 <?php foreach ($categories as $cat): ?>
102 <div>
103 <summary><span class="drag-handle">≡</span> <?= htmlspecialchars($cat['name']) ?></summary>
104 <table class="menu-table" data-category-id="<?= $cat['category_id'] ?>">
105 <thead>
106 <tr><th>ID</th><th>Image</th><th>Name</th><th>Description</th><th>Price</th><th>Available</th>
107 <th>Actions</th></tr>
108 </thead>
109 <tbody class="sortable">
110 <?php foreach ($menu_by_category[$cat['category_id']] as $item): ?>
111 <tr data-id="<?= $item['menu_id'] ?>">
112 <td><?= $item['menu_id'] ?></td>
113 <td>
114 <?php if (empty($item['image_path'])): ?>
115 
117 <?php else: ?>
118 <span style="color:#aaa;">No image</span>
119 <?php endif; ?>
120 </td>
121 <td class="searchable"><?= htmlspecialchars($item['item_name']) ?></td>
122 <td><?= htmlspecialchars($item['description']) ?></td>
123 <td><?= number_format($item['price'], 2) ?> BD</td>
124 <td>
125 <form method="POST" style="display:inline;">
126 <input type="hidden" name="csrf_token" value="<?= $csrf_token ?>">
127 <input type="hidden" name="toggle_id" value="<?= $item['menu_id'] ?>">
128 <input type="hidden" name="available" value="<?= $item['availability'] ? 0 : 1 ?>">
129 <input type="checkbox" onchange="this.form.submit()" <?= $item['availability'] ? 'checked' : '' ?>
130 </form>
131 </td>
132 <td class="actions">
133 <a href="edit_menu_item.php?id=<?= $item['menu_id'] ?>">Edit</a>
134 </td>
135 </tr>
136 </tbody>
137 </table>
138 </div>
139 </div>
140 </div>
```

Figure 16 Frontend structure of the user dashboard integrating PHP-generated menu data.

All dynamic data was stored and retrieved from a MySQL 8.0 database, accessed via PDO (PHP Data Objects) for secure, parameterized queries. The database schema consisted of tables such as user, menu, category, order, order_items, employee, and log, each serving a specific purpose in the system. Foreign key constraints and indexing ensured data integrity and efficient lookup operations.

Folder and Component Structure

The project maintained a clean folder structure, with key directories such as:

- app/ - contains all the directories and files, holds the Dockerfile, docker compose, ssl config, and ssl certifications.
- app/includes/ - Contains reusable PHP files for session configuration, CSRF protection, sessionconfig
- app/img/ -Stores all menu item images and visual assets
- app/Root directory - Contains individual functional pages like dashboard_user.php, orders.php, add_employee.php, edit_menu_items.php, etc.
- /backups – contains all database and project backups
- /scripts – contains scripts to be run such as backup scripts
- /certs – contains Https certifications


```
bellapizza@Yanis: ~/bella-piz  X  +  v
bellapizza@Yanis:~/bella-pizza-docker$ tree -L 2
.
├── Dockerfile
├── README.md
├── app
│   ├── Paymentgateway.php
│   ├── add_category.php
│   ├── add_customer.php
│   ├── add_employee.php
│   ├── add_menu_item.php
│   ├── add_order.php
│   ├── analytics.php
│   ├── assign_driver.php
│   ├── categories.php
│   ├── chatbot_backend.php
│   ├── customers.php
│   ├── dashboard.php
│   ├── dashboard_user.php
│   ├── delete_menu_item.php
│   ├── driver_share_location.php
│   ├── driver_update_location.php
│   ├── edit_category.php
│   ├── edit_customer.php
│   ├── edit_employee.php
│   ├── edit_menu_item.php
│   ├── edit_menu_items.php
│   ├── employees.php
│   ├── get_driver_location.php
│   ├── logs
│   ├── includes
│   ├── index.php
│   ├── log_helper.php
│   ├── login.php
│   ├── login_handler.php
│   ├── logout.php
│   ├── logs.php
│   ├── menu.php
│   ├── my_orders.php
│   ├── order_details.php
│   ├── order_details_user.php
│   ├── orders.php
│   ├── phpmailer
│   ├── place_order.php
│   ├── profile.php
│   ├── response.php
│   ├── send_email.php
│   ├── soft_delete_employee.php
│   ├── supabase_callback.php
│   ├── track_delivery.php
│   └── view_cart.php
├── backups
│   ├── RestaurantDB_20250511_192101.sql
│   └── project_backup_20250509_234542.zip
├── certs
│   ├── selfsigned.crt
│   └── selfsigned.key
├── default-ssl.conf
├── docker-compose.yml
├── scripts
│   ├── backup_project.sh
│   └── export_db.sh
└── 8 directories, 52 files
bellapizza@Yanis:~/bella-pizza-docker$ |
```

Figure 17 Organized folder structure of the Bella Pizza system showing modular components.

AJAX-Based Data Interactions

AJAX requests enabled asynchronous interactions without reloading pages. For example, toggling item availability or live polling for orders was achieved using JavaScript that called backend PHP scripts, which then updated the database and responded with real-time data. Similarly, `edit_menu_items.php` uses asynchronous requests to toggle menu item availability and perform live search filtering.

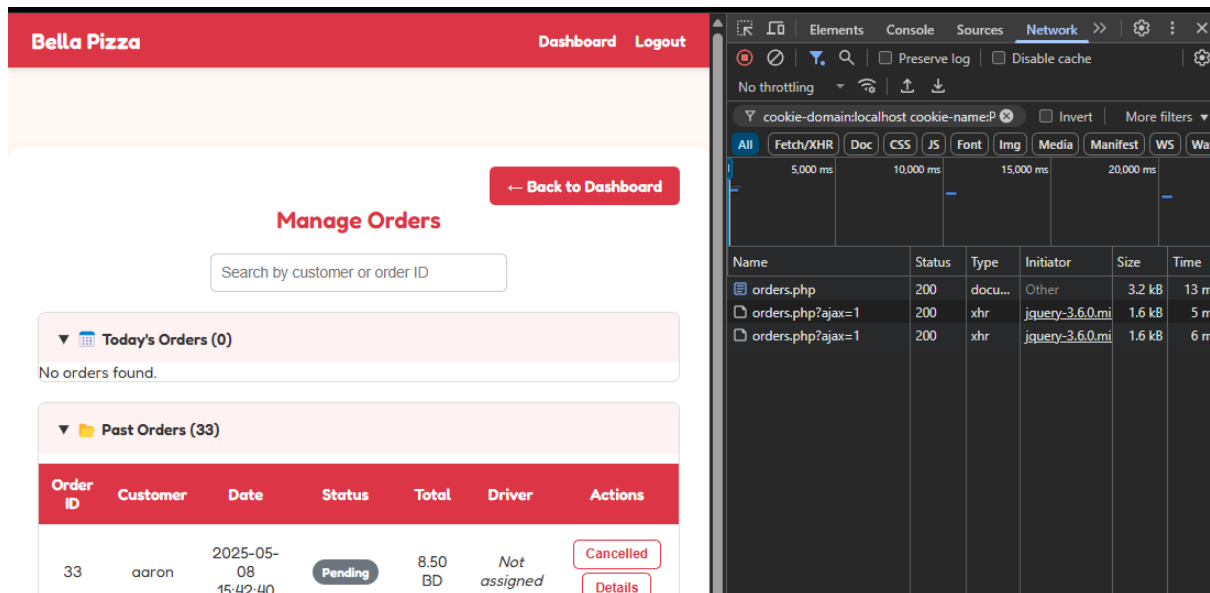


Figure 18 AJAX request from frontend JavaScript to a backend PHP script for live updates.

By integrating all components in a structured and layered approach, the system ensured maintainability, scalability, and a responsive user experience for both customers and administrators.

Coding Tools and Libraries Used

To efficiently develop, debug, and manage the Bella Pizza system, a combination of programming tools, software libraries, and browser utilities were used throughout the implementation phase. These tools enabled modular development, seamless integration, and consistent code quality across both frontend and backend components.

Visual Studio Code

The primary development environment was **Visual Studio Code (VS Code)**, chosen for its lightweight design, strong PHP support, and extensibility. Key extensions included:

- **Docker** – for managing containers directly from the editor
- **WSL** - to connect to the Linux Subsystem

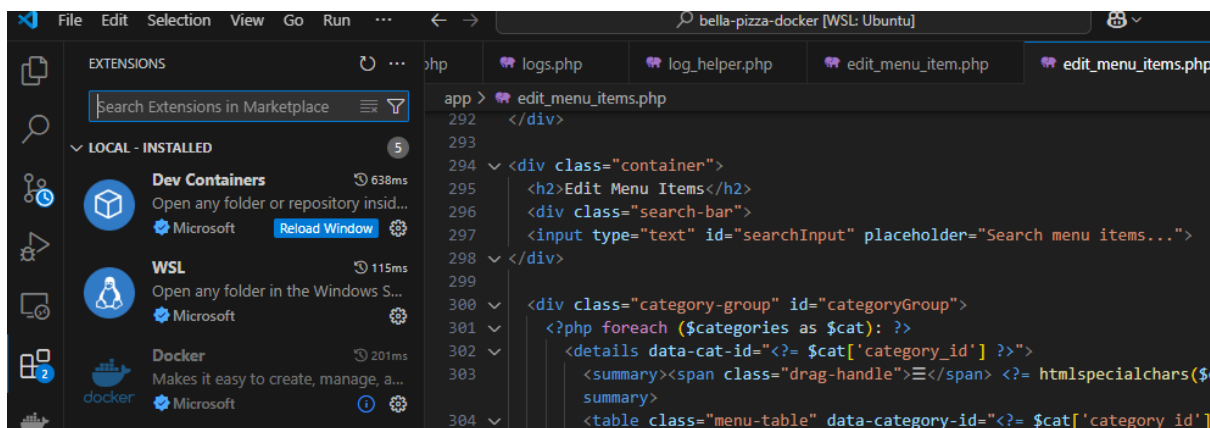


Figure 19 Development of Bella Pizza using Visual Studio Code and PHP-specific extensions.

On the backend, PHP 8 was used for server-side scripting, leveraging PDO (PHP Data Objects) to securely interact with the MySQL 8.0 database. PDO allowed the use of prepared statements, which helped prevent SQL injection attacks and maintained efficient data handling throughout the application.

For data storage and querying, MySQL was selected due to its wide support, performance, and compatibility with Docker containers. Tables were created using foreign keys to ensure relational integrity and were accessed through secure SQL queries executed via PHP scripts.

The frontend layer was primarily built using standard HTML5, CSS3, and JavaScript, with custom UI components styled in alignment with Bella Pizza's branding. Additionally, asynchronous features such as real-time updates, cart interaction, and availability toggles were built using AJAX via JavaScript `fetch()` and `XMLHttpRequest` APIs.

```
onEnd: function () {
  const catOrder = Array.from(document.querySelectorAll('#categoryGroup details')).map(d =>
    d.dataset.catId);
  $.post('/' + category_order: catOrder, csrf_token: '<?= $csrf_token ?>');
```

Figure 20 Example of AJAX integration in the frontend JavaScript code.

External Libraries and APIs

The project used several third-party libraries to add advanced functionality without bloating the system:

- PHPMailer – A robust library for sending emails, used for system notifications
- SortableJS – A lightweight JavaScript library that enabled drag-and-drop sorting for menu items and categories
- Google Fonts – Used for consistent and visually appealing typography
- Font Awesome – For scalable vector icons across the UI

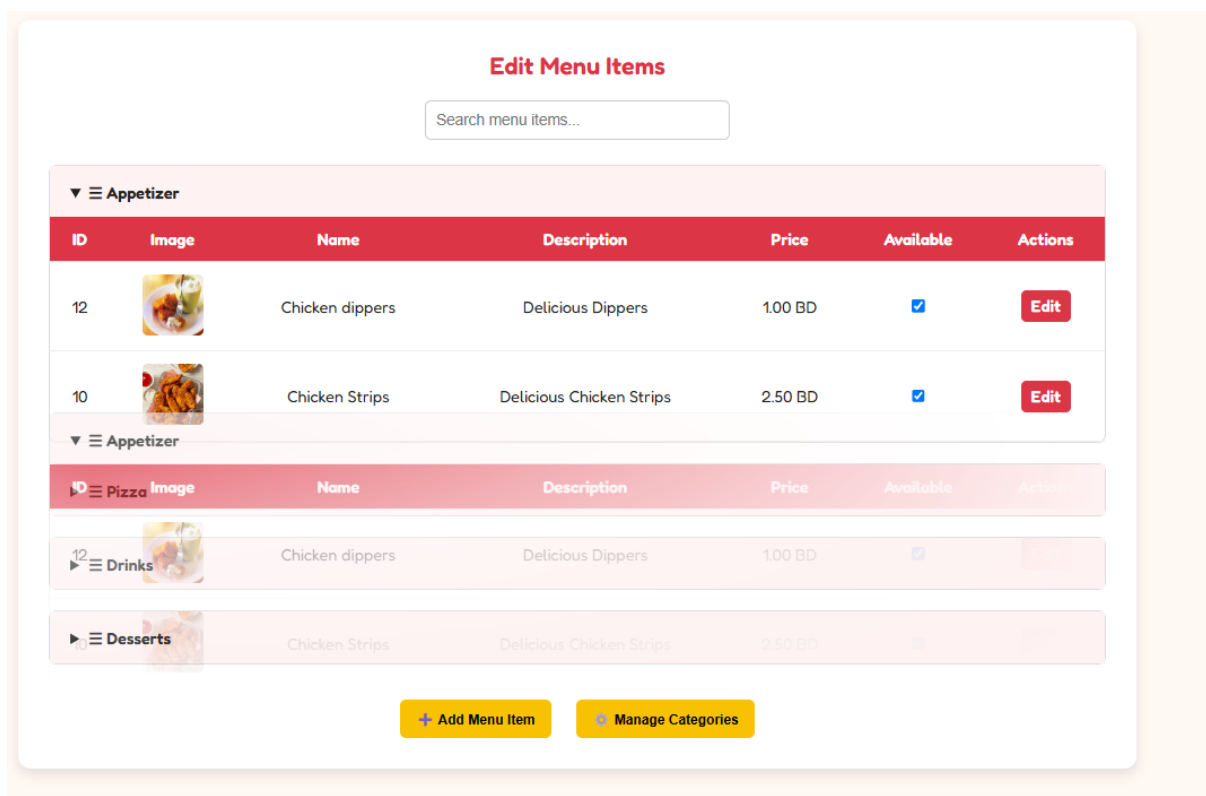


Figure 21 Drag-and-drop menu sorting feature using SortableJS library.

Additionally, tools like phpMyAdmin were used during development for easier database inspection, testing queries, and performing manual data entries. Debugging and performance testing were done using Chrome DevTools, which helped track AJAX activity, analyze network performance, and optimize frontend rendering.

All third-party dependencies used were either open-source or custom-embedded to ensure no external reliance on APIs or proprietary systems. Security libraries such as custom CSRF token validation were developed internally and included via includes/csrf.php.

```
> includes > 🐞 csrf.php
<?php

if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

function validate_csrf_token() {
    if (
        empty($_POST['csrf_token']) ||
        !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])
    ) {
        http_response_code(400);
        exit("Invalid CSRF token");
    }
}
```

Figure 22 Custom CSRF token implementation to secure form submissions.

This toolset and library selection helped ensure a maintainable, secure, and scalable solution while supporting rapid iteration throughout development.

Security Setup

Session-Based Access Control

Bella Pizza uses session variables to manage user authentication and restrict access to authorized roles only. Upon login, users are assigned a session containing their unique ID and role (e.g., admin, support, delivery, or customer). Each page that involves sensitive data or system-level access checks this session before allowing further operations.

Why it's important:

This ensures that users cannot access restricted pages by manually typing URLs, and prevents unauthorized actions such as viewing or modifying employee data or orders.

```
if (!isset($_SESSION["employee_id"])) {  
    header("Location: login.php");  
    exit();  
}
```

Figure 23 Role-based session validation enforcing secure page access.

CSRF Protection

To prevent Cross-Site Request Forgery (CSRF) attacks, all forms in the system include a hidden token generated in includes/csrf.php. This token is stored in the session and verified on every form submission. Any request lacking a valid token is rejected, thereby blocking forged submissions.

Why it's important:

CSRF tokens stop malicious sites from tricking authenticated users into performing unwanted actions (e.g., deleting a menu item without consent).

```
<form method="POST" style="display:inline;">  
    <input type="hidden" name="csrf_token" value="<?= $csrf_token ?>">
```

Figure 24 CSRF protection implemented using hidden token fields and server-side validation.

Prepared Statements with PDO

All user input is processed through prepared statements using PDO, where SQL queries are constructed with placeholders and executed with bound values. This prevents attackers from injecting malicious SQL code through form inputs.

Why it's important:

SQL injection is one of the most critical vulnerabilities in web applications. Using prepared statements ensures user input cannot alter the structure of the SQL query.

```
function fetchOrders($pdo, $employee_id, $role) {  
    if ($role === 'delivery') {  
        $stmt = $pdo->prepare("SELECT o.*, c.username AS customer_name, CONCAT(e.first_name, '  
        ', e.last_name) AS driver_name FROM `order` o LEFT JOIN customer c ON o.customer_id = c.  
        customer_id LEFT JOIN employee e ON o.delivery_driver_id = e.employee_id WHERE o.  
        delivery_driver_id = :employee_id ORDER BY o.order_date DESC");  
        $stmt->execute([':employee_id' => $employee_id]);  
    }  
}
```

Figure 25 Secure database interaction using PDO to prevent SQL injection.

HTTPS Configuration

To protect data in transit, Bella Pizza enables HTTPS using self-signed SSL certificates in the local development environment. All sensitive communication, such as login credentials or order data, is encrypted between the client and server.

Why it's important:

Without HTTPS, login and form data could be intercepted by attackers on the same network, leading to stolen credentials or hijacked sessions.

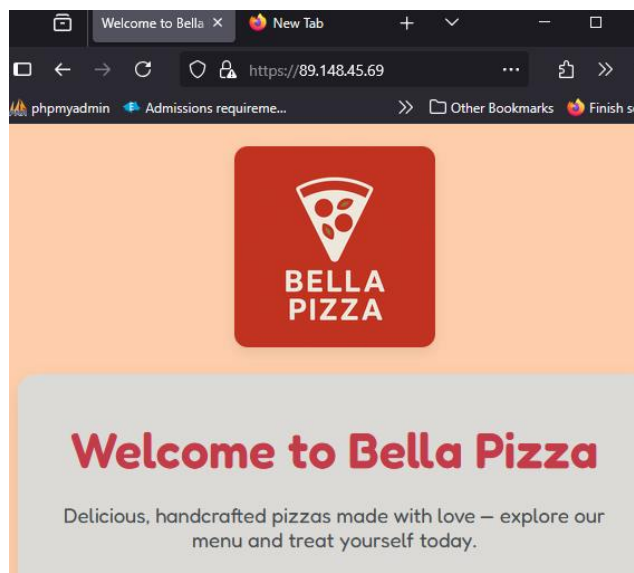


Figure 26 HTTPS used to ensure encrypted traffic between client and server.

Secure Login and Session Handling

Session IDs are generated securely upon login, and logout destroys the session immediately to prevent session hijacking. If password hashing (e.g., `password_hash()` and `password_verify()`) is used, it ensures passwords are never stored or transmitted in plain text.

Why it's important:

Proper session handling protects user identity and prevents attackers from impersonating a user if they steal a session token.

Input Validation and Error Handling

User input is validated on both the client side (via HTML5 attributes) and server side (via PHP logic) to ensure data integrity. Error messages are also controlled so sensitive details about the server or database are not revealed.

Why it's important:

Unvalidated input can break functionality or introduce vulnerabilities, while overly detailed error messages can aid attackers in probing the system.

These security measures, when combined, ensure the Bella Pizza system is protected against common web vulnerabilities and adheres to best practices in modern web development.

Real-time network and device monitoring

To ensure the Bella Pizza platform maintains a high level of security and real-time visibility over its infrastructure, Wazuh was integrated as a centralized security monitoring solution. Wazuh, an open-source Security Information and Event Management (SIEM) system, was deployed within the Docker-based architecture to monitor network activity, analyze logs, and detect potential intrusions across containers and host systems. It continuously collects and analyzes data from Apache access logs, PHP error logs, and operating system activity, enabling rapid identification of suspicious behavior such as unauthorized login attempts or unusual traffic patterns. In particular, Wazuh monitors incoming and outgoing network traffic, alerting administrators to anomalies such as port scans, excessive failed logins, or connections from blacklisted IP ranges. The system's active rules engine correlates events against threat intelligence feeds, offering both real-time alerts and historical analysis through its integrated dashboard. This level of visibility significantly enhances Bella Pizza's ability to detect and respond to potential threats, ensuring customer data confidentiality and system

integrity. Additionally, Wazuh's integration supports future scalability by allowing more agents to be added as new services or servers are introduced.

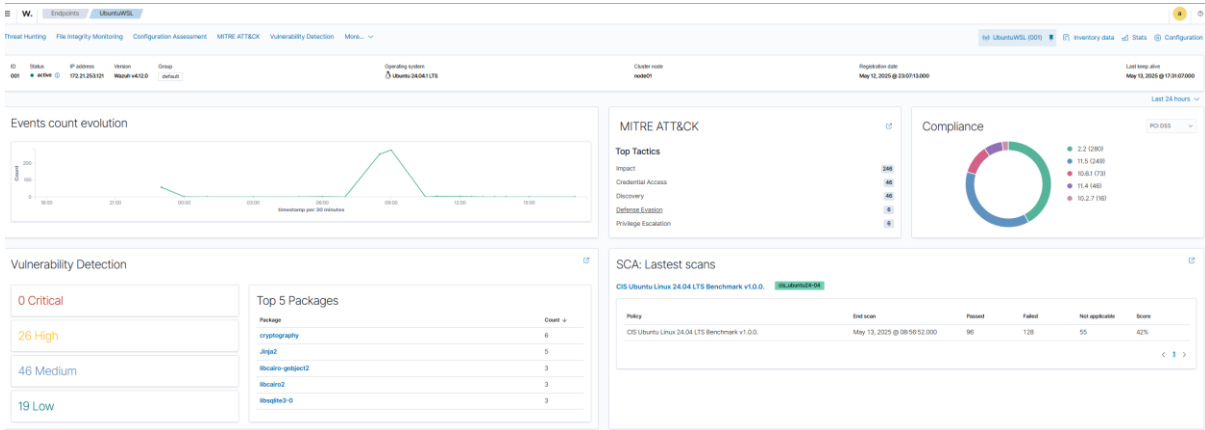


Figure 27 Wazuh Interface

Device Configuration and System Setup

This section focuses on how the Bella Pizza system was configured at the system level, including container networking, service orchestration, SSL setup, and port exposure. Unlike the development environment, this part details how the application's backend components and services were interconnected and accessed during deployment and testing.

Container Architecture and Service Integration

The system was deployed using Docker Compose, which defined and managed the three core services:

- Web service (web) – Runs Apache with PHP to serve the Bella Pizza application.
- Database service (db) – Hosts a MySQL 8.0 instance used to store all application data.
- Admin interface (phpmyadmin) – Web-based GUI for inspecting and managing the MySQL database.

These containers were interconnected using an internal Docker network called `bella_net`, allowing secure communication between services without exposing internal ports to the host.

```
docker-compose.yml X  default-ssl.conf  Dockerfile 1  login.php  COMMIT_E

docker-compose.yml > ...
1  version: '3.8'
2
3  > Run All Services
4  services:
5    > Run Service
6    web:
7      build:
8        context: .
9        dockerfile: Dockerfile
10     container_name: bella_web
11     ports:
12       - "8080:80"
13       - "443:443"
14     volumes:
15       - ./certs/selfsigned.crt:/etc/apache2/ssl/selfsigned.crt:ro
16       - ./certs/selfsigned.key:/etc/apache2/ssl/selfsigned.key:ro
17       - ./default-ssl.conf:/etc/apache2/sites-available/default-ssl.conf:ro
18       - ./app:/var/www/html
19     depends_on:
20       - db
21     restart: always
22     networks:
23       - bella_net
24     environment:
25       MYSQL_HOST: db
26       MYSQL_USER: root
27       MYSQL_PASSWORD: rootpass
28       MYSQL_DATABASE: RestaurantDB
29
30   > Run Service
31   db:
32     image: mysql:8.0
33     container_name: bella_db
34     restart: always
35     environment:
36       MYSQL_ROOT_PASSWORD: rootpass
37       MYSQL_DATABASE: restaurantdb
38       MYSQL_USER: user
39       MYSQL_PASSWORD: userpass
40     volumes:
41       - dbdata:/var/lib/mysql
42     networks:
43       - bella_net
44
45   > Run Service
46   phpmyadmin:
47     image: phpmyadmin/phpmyadmin
48     container_name: bella_phpmyadmin
49     restart: always
50     ports:
51       - "8081:80"
52     environment:
53       PMA_HOST: db
54       MYSQL_ROOT_PASSWORD: rootpass
55     depends_on:
56       - db
57     networks:
58       - bella_net
59
60 volumes:
61   dbdata:
62
63 networks:
64   bella_net:
```

Figure 28 Docker Compose configuration managing PHP, MySQL, and phpMyAdmin on an internal network.

Port Exposure and SSL Configuration

Port mappings were defined as follows:

- Port 8080 or 80 (host) → Port 80 (container) for HTTP
- Port 443 (host) → Port 443 (container) for HTTPS

Self-signed SSL certificates were created and stored in the certs/ folder. These were referenced in the default-ssl.conf Apache configuration file to enable HTTPS locally.

This ensured encrypted communication between the client browser and the server, even in a local or test environment.

```
default-ssl.conf
1  <IfModule mod_ssl.c>
2      <VirtualHost _default_:443>
3          ServerName localhost
4          DocumentRoot /var/www/html
5
6          # Enable SSL
7          SSLEngine on
8          SSLCertificateFile /etc/apache2/ssl/selfsigned.crt
9          SSLCertificateKeyFile /etc/apache2/ssl/selfsigned.key
10
11         # (Optional) Stronger security settings
12         SSLProtocol          all -SSLv3 -TLSv1 -TLSv1.1
13         SSLCipherSuite       HIGH:!aNULL:!MD5
14         SSLHonorCipherOrder  on
15
16         <Directory /var/www/html>
17             Options Indexes FollowSymLinks
18             AllowOverride All
19             Require all granted
20         </Directory>
21
22         # Log locations (optional)
23         ErrorLog ${APACHE_LOG_DIR}/error_ssl.log
24         CustomLog ${APACHE_LOG_DIR}/access_ssl.log combined
25     </VirtualHost>
26 </IfModule>
27
```

Figure 29 Apache configuration file enabling local HTTPS support for secure testing.

Volume and Data Persistence

Two persistent volumes were configured:

- One for the MySQL container (dbdata) to retain database records between restarts.
- One volume-mount of the app/ source code into the Apache web root (/var/www/html) for live development and file syncing.

This setup ensured no data loss and enabled seamless updates to the application logic during testing.

Network Access and Port Forwarding

To make the application available for demonstration outside the host machine, router-level port forwarding was configured:

- Inbound traffic on ports 8080 or 80 (HTTP) and 443 (HTTPS) was forwarded to the Docker host's internal IP.
- Firewall rules were added to permit external access on these ports.

This allowed testers and stakeholders to access the running Dockerized application remotely using a public IP.

Type:	☒ User-defined ☐ Application	
Application:	Select...	
Enable Port Mapping:	☒	
Mapping Name:	bella pizza https	
WAN Name:	1_INTERNET_R_VII	
Internal Host:	192.168.100.3	Yanis
External Source IP Address:		
Protocol:	TCP	Internal port number: 443 - 443
External port number:	443 - 443	External source port number: -
Delete		
Type:	☒ User-defined ☐ Application	
Application:	Select...	
Enable Port Mapping:	☒	
Mapping Name:	Bella Pizza	
WAN Name:	1_INTERNET_R_VII	
Internal Host:	192.168.100.3	Yanis
External Source IP Address:		
Protocol:	TCP	Internal port number: 80 - 80
External port number:	80 - 80	External source port number: -
Delete		

Figure 30 External port forwarding enabled for public access to the Bella Pizza system.

Service Management Tools

- Docker CLI was used to start, stop, and inspect containers.
- phpMyAdmin was accessed via `http://localhost:8081` for managing tables and testing queries.
- Chrome DevTools assisted in debugging real-time AJAX activity and verifying HTTPS headers.

This device and system-level configuration enabled a portable, secure, and scalable deployment environment for Bella Pizza, while also supporting external testing and validation without deploying to a full cloud server.

Security Monitoring and Log Analysis

To enhance the security posture of the Bella Pizza platform, Wazuh was deployed within the Docker environment as a centralized monitoring and intrusion detection solution:

- The Wazuh server container was configured to monitor Apache, PHP, and system logs from all other containers.
- Log forwarding was enabled from the web and database containers to Wazuh via mounted volumes and agent configuration.
- Real-time alerts were triggered for suspicious activities such as repeated failed logins, unexpected port scans, or modified configuration files.
- Wazuh dashboards were made accessible via HTTPS on a secured port for administrators to review logs and incident alerts.

This integration enabled continuous visibility into the system's behavior and improved incident response by identifying potential threats and misconfigurations across the application stack.

Description by Components

This section outlines the most important components of the Bella Pizza system. Each feature is explained in terms of functionality, integration, and implementation logic. Where relevant, key code segments and screenshots are included to illustrate behavior and design.

Login and Registration Functionality

Objective

To provide a secure login and registration interface for both employees and customers. It supports standard authentication using username and password, along with Google OAuth login for customers.

Implementation Overview

- **Session & CSRF Protection:** Includes session_config.php and csrf.php for secure session handling and CSRF token enforcement.
- **Redirect Logic:**
 - If an employee is already logged in, redirect to dashboard.php.
 - If a customer is already logged in, redirect to dashboard_user.php.
- **Login Handling:**
 - First checks the employee table for matching credentials using a prepared statement.
 - If not found, it checks the customer table.
 - On success, relevant user data is stored in \$_SESSION, and log_action() records the event.
- **Registration Handling:**
 - Registers new customers only. Validates inputs and inserts into the customer table with hashed password.
 - Logs successful and failed registration attempts.
- **Google Login (Supabase OAuth):**
 - Offers login with Google using Supabase's auth.signInWithOAuth API, redirecting to supabase_callback.php.

Login

```
if ($_SERVER["REQUEST_METHOD"] === 'POST') {
    validate_csrf_token();

    if (isset($_POST["login"])) {
        $username = trim($_POST["username"]);
        $password = $_POST["password"];

        $stmt = $conn->prepare("SELECT * FROM employee WHERE username = ?");
        $stmt->bind_param("s", $username);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows === 1) {
            $employee = $result->fetch_assoc();
            if (password_verify($password, $employee["password_hash"])) {
                session_regenerate_id(true);
                $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
                $_SESSION['employee_id'] = $employee["employee_id"];
                $_SESSION["username"] = $employee["username"];
                $_SESSION["email"] = $employee["email"];
                $_SESSION["phone"] = $employee["phone_number"];
                $_SESSION["role"] = $employee["role"];
                log_action("Login Success", "Employee logged in successfully", "INFO", $employee["employee_id"], null, $employee["username"]);
                header("Location: dashboard.php");
                exit();
            } else {
                log_action("Login Failed", "Invalid password for employee: $username", "WARNING", null, null, $username);
                $error = "Invalid password.";
            }
        } else {
            $stmt = $conn->prepare("SELECT * FROM customer WHERE username = ?");
            $stmt->bind_param("s", $username);
            $stmt->execute();
            $result = $stmt->get_result();

            if ($result->num_rows === 1) {
                $customer = $result->fetch_assoc();
                if (password_verify($password, $customer["password_hash"])) {
                    session_regenerate_id(true);
                    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
                    $_SESSION['user_id'] = $customer["customer_id"];
                    $_SESSION["username"] = $customer["username"];
                    $_SESSION["email"] = $customer["email"];
                    $_SESSION["phone"] = $customer["phone"];
                    log_action("Login Success", "Customer logged in successfully", "INFO", null, $customer["customer_id"], $customer["username"]);
                    $_SESSION['login_type'] = 'normal';
                    $_SESSION['user_id'] = $customer["customer_id"];

                    header("Location: dashboard_user.php");
                    exit();
                } else {
                    log_action("Login Failed", "Invalid password for customer: $username", "WARNING", null, null, $username);
                    $error = "Invalid password.";
                }
            } else {
                log_action("Login Failed", "Unknown username: $username", "WARNING", null, null, $username);
                $error = "User not found.";
            }
        }
    }
}
```

Figure 31 Login Logic

This block handles the login functionality for both employees and customers. When the login form is submitted, the system first validates the CSRF token to ensure the request is genuine. It then checks if the entered username exists in the employee table. If it finds a match, it verifies the password using `password_verify`. If the password is correct, the system regenerates the session ID for security, stores the employee's details in the session, logs the successful login using `log_action`, and

redirects the user to the employee dashboard. If the password is incorrect, it logs a warning and shows an error.

If the username is not found in the employee table, the system repeats the process for the customer table. If the customer login is successful, the session is updated with the customer's information, including a login_type of "normal", and the user is redirected to the customer dashboard. If login fails at any stage, it logs the failure and displays an appropriate error message. This dual-path login logic enables both employees and customers to use the same form securely.

Registration

```
if (isset($_POST["register"])) {
    $username = trim($_POST["reg_username"]);
    $password_raw = trim($_POST["reg_password"]);
    $email = trim($_POST["reg_email"]);
    $number = trim($_POST["reg_number"]);
    $address = trim($_POST["reg_address"]);

    if (empty($username) || strlen($username) < 3) {
        $error = "Username must be at least 3 characters long.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = "Invalid email address.";
    } elseif (empty($password_raw) || strlen($password_raw) < 6) {
        $error = "Password must be at least 6 characters long.";
    } elseif (!preg_match('/^[0-9]{8,15}$/', $number)) {
        $error = "Phone number must be between 8 to 15 digits.";
    } elseif (empty($address) || strlen($address) < 5) {
        $error = "Address must be at least 5 characters long.";
    } else {
        $password = password_hash($password_raw, PASSWORD_BCRYPT);
        $stmt = $conn->prepare("SELECT * FROM customer WHERE username = ? OR email = ?");
        $stmt->bind_param("ss", $username, $email);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows == 0) {
            $stmt = $conn->prepare("INSERT INTO customer (username, email, password_hash, phone, address) VALUES (?, ?, ?, ?, ?)");
            $stmt->bind_param("sssss", $username, $email, $password, $number, $address);
            if ($stmt->execute()) {
                log_action("Registration Success", "New customer registered", "INFO", null, $conn->insert_id, $username);
                $success = "Registration successful. You can now log in.";
            } else {
                log_action("Registration Error", "Failed to register: $username", "ERROR", null, null, $username);
                $error = "Registration failed. Please try again.";
            }
        } else {
            log_action("Registration Failed", "Duplicate username/email: $username", "WARNING", null, null, $username);
            $error = "Username or email already exists.";
        }
    }
}
```

Figure 32 registration logic

This block manages new customer registrations. When the registration form is submitted, the script first verifies the CSRF token. It then retrieves and sanitizes the input fields: username, password, email, phone number, and address. It performs strict validation on all fields—checking lengths, formats, and whether required inputs are present. If any validation fails, a specific error message is shown to guide the user.

If validation passes, the password is securely hashed using `password_hash`. The script then checks if the username or email already exists in the database to prevent duplicate accounts. If the user is unique, it inserts the new customer into the database using a prepared statement. Upon success, the event is logged, and a success message is shown. If the insertion fails or if the username/email already exists, an appropriate error is logged and displayed. This flow ensures that customer registrations are handled safely, cleanly, and consistently.

Google Login



```
<script type="module">
  import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';

  const supabase = createClient(
    'https://fvpkzssoudxcsejxvow.supabase.co',
    'redacted private key'
  );

  window.signInWithGoogle = async function () {
    const { error } = await supabase.auth.signInWithOAuth({
      provider: 'google',
      options: {
        redirectTo: 'https://89.148.45.69/supabase_callback.php'
      }
    });
    if (error) {
      alert("Login failed: " + error.message);
    }
  };
</script>
```

Figure 33 Google login logic

This JavaScript block enables users to log in using their Google accounts via Supabase's OAuth2 service. When the user clicks the "Sign in with Google" button on the login page, it triggers the `signInWithGoogle()` function. This function initializes the Supabase client using your project's URL and public API key. It then requests an OAuth login flow with Google as the provider.

If successful, the user is redirected to the specified `redirectTo` URL — in this case, `supabase_callback.php`, which you'll use to handle the returned session or user data from Supabase. If there's an error during login (e.g., popup blocked or user denied access), it will show an alert to notify the user.

This integration modernizes your authentication system by allowing users to skip manual registration or password-based login. It improves convenience and security, as authentication is delegated to Google. Supabase manages the OAuth handshake, and your server (in `supabase_callback.php`) can complete the login by extracting the user info and saving it in your system.

```

<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

$data = json_decode(file_get_contents("php://input"), true);

$email = $data['email'] ?? '';
$name = $data['user_metadata']['full_name'] ?? 'Google User';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) die("Connection failed");

// Check if email exists
$stmt = $conn->prepare("SELECT * FROM google_login WHERE email = ?");
$stmt->bind_param("s", $email);
$stmt->execute();
$result = $stmt->get_result();

if ($user = $result->fetch_assoc()) {
    $_SESSION['user_id'] = $user['customer_id'];
    $_SESSION['username'] = $user['username'];
    $_SESSION['email'] = $user['email'];
    $_SESSION['login_type'] = 'google';
} else {
    // Ensure unique username
    $original_name = $name;
    $check = $conn->prepare("SELECT 1 FROM google_login WHERE username = ?");
    $check->bind_param("s", $name);
    $check->execute();
    $check->store_result();

    $suffix = 1;
    while ($check->num_rows > 0) {
        $name = $original_name . rand(1000, 9999);
        $check->bind_param("s", $name);
        $check->execute();
        $check->store_result();
    }

    // Insert new user
    $stmt = $conn->prepare("INSERT INTO google_login (username, email) VALUES (?, ?)");
    $stmt->bind_param("ss", $name, $email);
    if ($stmt->execute()) {
        $_SESSION['user_id'] = $conn->insert_id;
        $_SESSION['username'] = $name;
        $_SESSION['email'] = $email;
        $_SESSION['login_type'] = 'google';
    } else {
        http_response_code(500);
        echo "Failed to insert user.";
        exit();
    }
}

http_response_code(200);

```

Figure 34 google login handler

This PHP script handles the backend processing for users who log in via Google using Supabase. When Supabase redirects the user to your site, the user's information (email and name) is sent in a POST request to this script from `supabase_callback.php`. The script first reads the JSON payload and extracts the user's email and name.

It then checks whether the email already exists in the `google_login` table. If the user already exists, it retrieves their information and initializes a new PHP session, storing key values like `user_id`, `username`, `email`, and setting the login type to `google`. If the email is not found, it proceeds to register the user by first checking if their full name is already taken as a username in the database. If it is, it appends a random number to ensure uniqueness.

Once a unique username is found, the script inserts the new user into the `google_login` table and then sets up the session using the new record's ID and the submitted data. The session now recognizes the user as logged in, and the response returns HTTP 200 to confirm success. If insertion fails, it returns an HTTP 500 error.

This script acts as the secure link between Supabase's Google authentication and your PHP session-based login system, enabling a seamless login experience using third-party credentials while maintaining internal user tracking.

Login

Username

Password

Login

 Sign in with Google

Register

Username

Email

Password

Phone Number

Address

Register

Figure 35 Login Form View

View Menu Functionality

Objective

Displays all available menu items to the user, organized by category in a collapsible, card-based layout. Each item includes an image, name, description, and price. Only items marked as “available” in the database are shown.

Implementation Overview

- Data is fetched using a LEFT JOIN between menu and category tables.
- Menu items are grouped by their category and displayed inside collapsible <details> elements for better organization.
- Each item includes the product image (from /img/), name, price, and an “Add to Cart” form.
- Only menu items with availability = 1 are shown.

```
$categories = $pdo->query("SELECT * FROM category ORDER BY display_order ASC, name ASC")->fetchAll(PDO::FETCH_ASSOC);
$category_items = [];
$itemStmt = $pdo->prepare("SELECT * FROM menu WHERE availability=1 AND category_id=? ORDER BY item_name");
foreach ($categories as $cat) {
    $itemStmt->execute([$cat['category_id']]);
    $category_items[$cat['category_id']] = $itemStmt->fetchAll(PDO::FETCH_ASSOC);
}
?>
```

Figure 37 Query database for menu items snippet

```
<?php foreach ($categories as $cat): ?>
    <section class="category">
        <h2><?= htmlspecialchars($cat['name']) ?></h2>
        <div class="cards">
            <?php foreach ($category_items[$cat['category_id']] as $item): ?>
                <div class="card">
                    
                    <div class="card-content">
                        <h3><?= htmlspecialchars($item['item_name']) ?></h3>
                        <p class="desc"><?= htmlspecialchars($item['description']) ?>: 'No description.' ?></p>
                        <div class="price"><?= number_format($item['price'],2) ?> BD</div>
                    </div>
                    <div class="card-actions">
                        <form method="POST">
                            <!-- CSRF token for add-to-cart -->
                            <input type="hidden" name="csrf_token"
                                value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>" />
                            <input type="hidden" name="menu_id" value="<?= $item['menu_id'] ?>" />
                            <input type="number" name="quantity" value="1" min="1" />
                            <button type="submit" name="add_to_cart">Add to Cart</button>
                        </form>
                    </div>
                </div>
            <?php endforeach; ?>
        </div>
    </section>
<?php endforeach; ?>
```

Figure 36 Menu Structure code

Delicious, Fast & Fresh!

Welcome, yannis!
Order your favorite meals below.

Appetizer



Chicken dippers

Delicious Dippers

1.00 BD

1

Add to Cart



Chicken Strips

Delicious Chicken Strips

2.50 BD

1

Add to Cart

Pizza



cheese pizza

Handcrafted Cheese Pizza For The Cheeselovers.

5.00 BD

1

Add to Cart



Meat Pizza

Meaty and Delicious Pizza

5.00 BD

1

Add to Cart



peperoni pizza

nice pizza with peperonni

3.00 BD

1

Add to Cart



Vegetarian Pizza

Delicious Vegeterian Pizza

4.00 BD

1

Add to Cart

Figure 38 Menu View

Add to Cart Functionality

Objective

Enables users to select specific quantities of menu items and add them to their shopping cart directly from the menu view. This action is performed without redirecting the user, using POST form submission and session storage.

Implementation Overview

- Each menu item displayed in the dashboard includes a quantity input and a form with a hidden menu_id.
- When submitted, the form sends the selected quantity and item ID to dashboard_user.php, which adds the item to the cart.
- The cart is stored in the \$_SESSION['cart'] array, keyed by menu_id, with the value representing quantity.
- If an item already exists in the cart, the quantity is incremented rather than overwritten.

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST["add_to_cart"])) {  
    validate_csrf_token();  
  
    $menu_id = (int)$_POST["menu_id"];  
    $quantity = max(1, (int)($_POST["quantity"] ?? 1));  
  
    if (!isset($_SESSION["cart"])) {  
        $_SESSION["cart"] = [];  
    }  
    $_SESSION["cart"][$menu_id] = ($_SESSION["cart"][$menu_id] ?? 0) + $quantity;  
    $message = "✅ Added to cart!";  
}
```

Figure 39 add to cart logic

```
if (!empty($cart)) {  
    $placeholders = implode(',', array_fill(0, count($cart), '?'));  
    $stmt = $pdo->prepare("SELECT * FROM menu WHERE menu_id IN ($placeholders)");  
    $stmt->execute(array_keys($cart));  
    $menu_items = $stmt->fetchAll(PDO::FETCH_ASSOC);  
}
```

Figure 40cart retrieval logic on view cart page


```

<form method="POST">
  <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
  <table>
    <tr>
      <th>Item</th>
      <th>Price</th>
      <th>Quantity</th>
      <th>Subtotal</th>
    </tr>
    <?php foreach ($menu_items as $item):
      $menu_id = $item['menu_id'];
      $qty = $cart[$menu_id];
      $subtotal = $qty * $item['price'];
      $total += $subtotal;
    ?>
    <tr>

```

Figure 41 Display each item in cart view logic

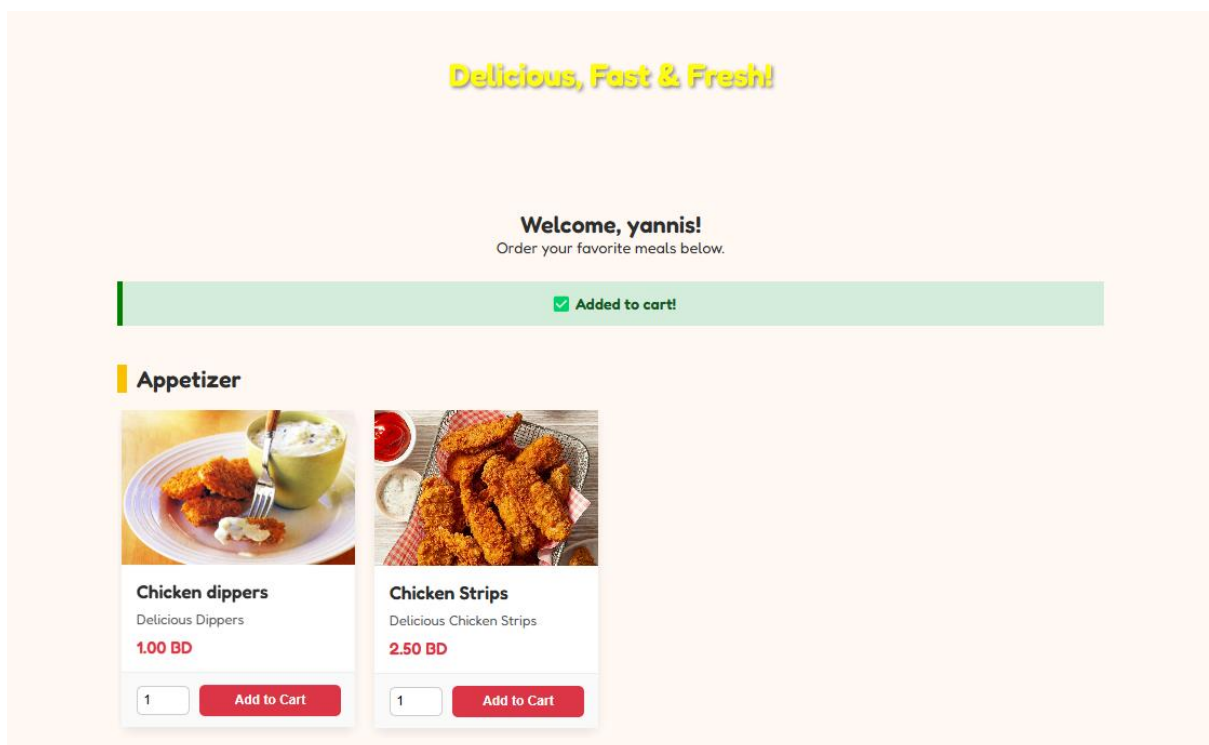


Figure 42 successful addition to cart

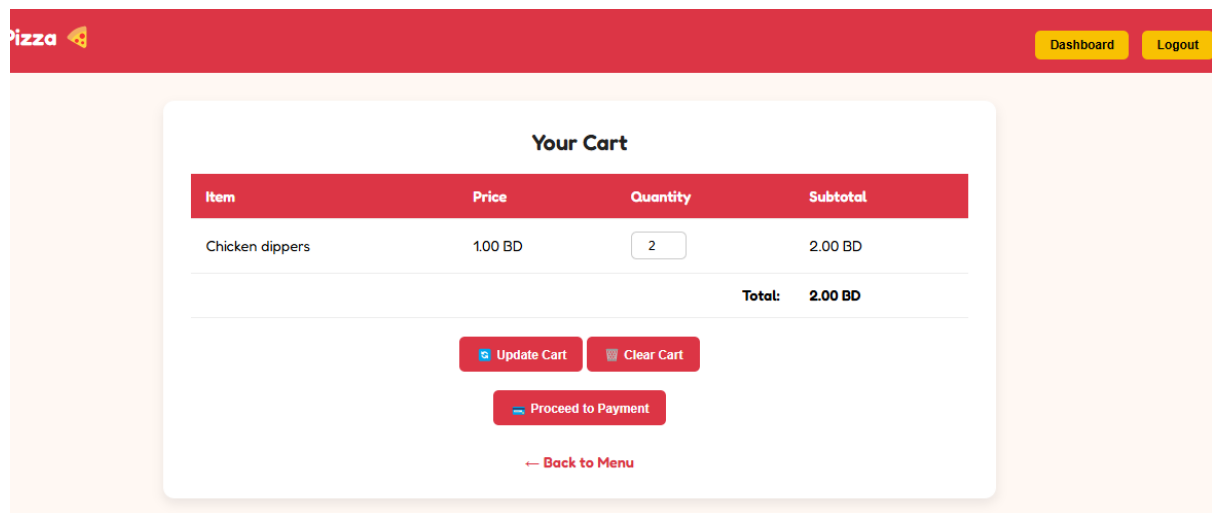


Figure 43 cart view

Payment Functionality with APS Redirection

Objective

Handles the final order placement after cart confirmation and redirects the user to a payment processing interface. Although not linked to a real payment gateway, this component simulates integration with Amazon Payment Services (APS) by preparing a redirect to a payment page or processor.

Implementation Overview

- The user's cart is validated and the total amount is calculated.
- Before inserting the order into the database, the system optionally redirects to a placeholder APS payment page (e.g., Paymentgateway.php) using `header("Location: ...")`.
- The Paymentgateway.php page could simulate interaction with a third-party processor or sandbox environment.
- Upon “completion,” a POST back or internal confirmation places the order and clears the cart.
- This prepares the system for integration with real payment APIs in the future.

```
<div class="actions inline-buttons" style="margin-top: 20px;">
  <form action="Paymentgateway.php" method="POST">
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
    <?php $_SESSION['cart_total_bhd'] = $total; ?>
    <button type="submit">➡ Proceed to Payment</button>
  </form>
```

Figure 44 Proceed to payment button that sets the total order value to a session variable and redirects user to the payment gateway page

To send a payment request to APS, the request_params associative array must be filled with the named keys and associated value, according to APS's documentation.

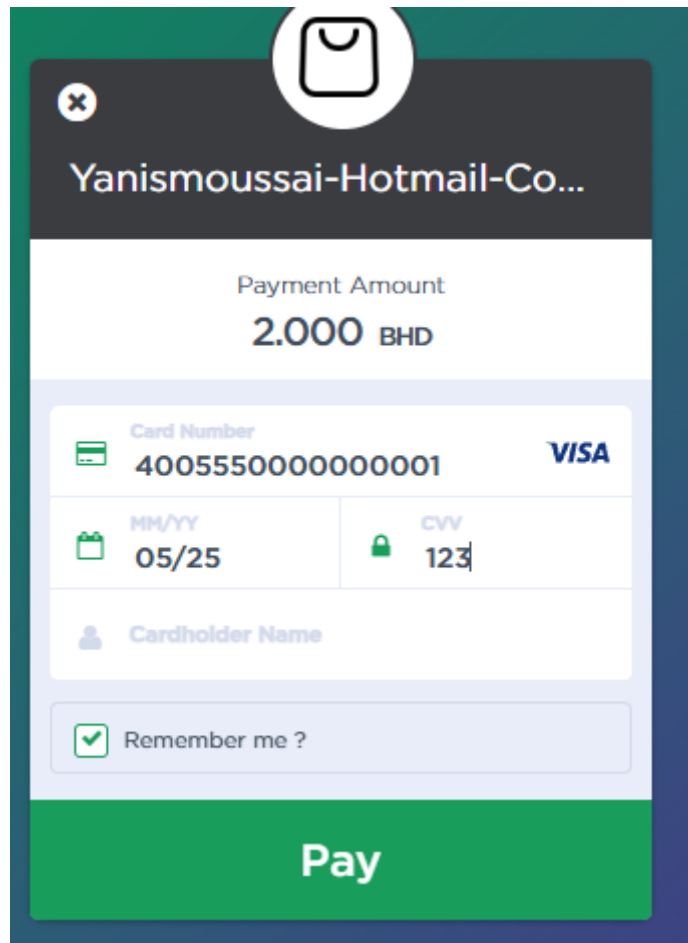
```
1  <?php
2  require_once 'includes/session_config.php';
3
4
5  if (!isset($_SESSION['user_id']) || empty($_SESSION['cart'])) {
6      header("Location: index.php");
7      exit;
8  }
9
10
11  $access_code = "IFafg4g3Zk0055igY1p1";
12  $merchant_identifier = "pCnTt3RP";
13  $sha_request_phrase = "12QW0MFnhxftV7Rev112nY!";
14  $currency = "BHD";
15  $language = "en";
16  $return_url = "http://localhost:8080/response.php?session_id=" . session_id();
17
18
19  $cart_total_bhd = $_SESSION['cart_total_bhd'];
20  $amount = $cart_total_bhd * 1000; // Convert to fils
21  $order_id = uniqid("BPZ_");
22
23
24  $request_params = [
25      'access_code'      => $access_code,
26      'merchant_identifier' => $merchant_identifier,
27      'merchant_reference' => $order_id,
28      'amount'           => $amount,
29      'currency'         => $currency,
30      'language'         => $language,
31      'command'          => 'PURCHASE',
32      'return_url'       => $return_url,
33      'customer_email'   => $_SESSION['email'] ?? 'guest@bellapizza.com',
34  ];
35
36
37  ksort($request_params);
38
39  $signature_string = $sha_request_phrase;
40  foreach ($request_params as $key => $value) {
41      $signature_string .= "$key=$value";
42  }
43  $signature_string .= $sha_request_phrase;
44
45  $signature = hash("sha256", $signature_string);
46
47
48  ?>
49  <!DOCTYPE html>
50  <html>
51  <head>
52      <meta http-equiv="refresh" content="0;url=https://sbcheckout.payfort.com/FortAPI/paymentPage" />
53  </head>
54  <body>
55      <form id="paymentForm" method="POST" action="https://sbcheckout.payfort.com/FortAPI/paymentPage">
56          <?php foreach ($request_params as $key => $value): ?>
57              <input type="hidden" name="<?= htmlspecialchars($key) ?>" value="<?= htmlspecialchars($value) ?>">
58          <?php endforeach; ?>
59          <input type="hidden" name="signature" value="<?= $signature ?>">
60      </form>
61
62      <script>
63          document.getElementById('paymentForm').submit();
64      </script>
65  </body>
66  </html>
67
68
```

Figure 45 API request params configuration and posting data to APS

The user is then redirected to APS's Payfort where he will be prompted to enter his credit card details

The screenshot shows a web browser window with the address bar displaying "sbcheckout.payfort.com/FortAPI/paymentPage?...". The page features a dark green and blue gradient background. A central white card displays the merchant name "Yanismoussai-Hotmail-Co..." and the payment amount "2.000 BHD". Below this, there is a form with fields for "Card Number", "MM/YY", "CVV", and "Cardholder Name". A "Remember me ?" checkbox is checked. A large green "Pay" button is at the bottom of the card. At the very bottom of the page, there are logos for "amazon payment services" and "pci DSS compliant".

Figure 46 APS redirect for payment



A mobile payment interface for 'Yanismoussai-Hotmail-Co...'. At the top, there's a close button (X) and a shopping bag icon. The payment amount is '2.000 BHD'. Below this, a card details form is shown with a Visa logo. The card number is '4005550000000001'. The expiration date is '05/25' and the CVV is '123'. The cardholder name field is empty. There is a 'Remember me ?' checkbox which is checked. At the bottom is a large green 'Pay' button.

Yanismoussai-Hotmail-Co...

Payment Amount
2.000 BHD

Card Number
4005550000000001 **VISA**

MM/YY
05/25

CVV
123

Cardholder Name

☒ Remember me ?

Pay

Figure 48 Filled form for payment

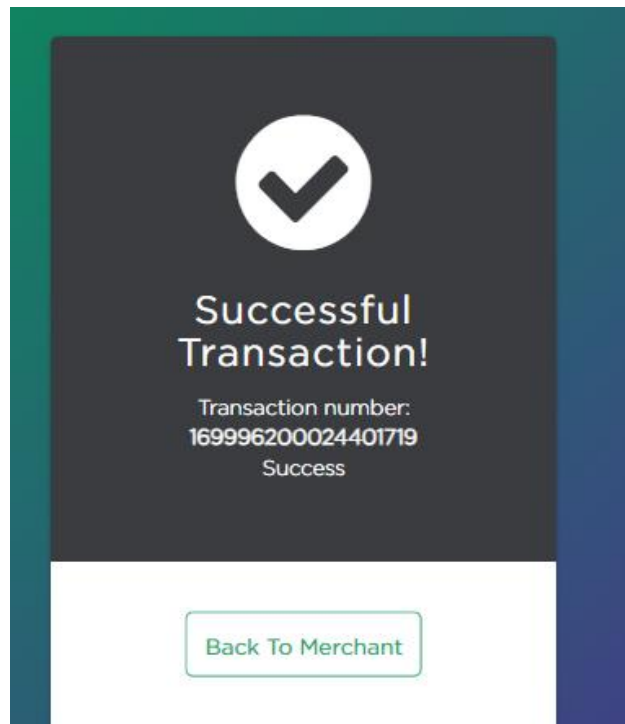


Figure 47 successful payment message

Email Functionality

Objective

Sends email notifications to customers upon specific actions, such as placing an order. The system uses the PHPMailer library to handle mail composition, formatting, and sending via SMTP or local mail servers.

Implementation Overview

- The PHPMailer library is included in the project under the app/phpmailer/ folder.
- An email is triggered after a successful action (e.g., order placement).
- The email contains details such as the order ID, total amount, and confirmation message.
- PHPMailer handles mail headers, SMTP settings, and character encoding.
- Language files within PHPMailer support internationalization (70+ languages included).

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

require 'phpmailer/src/Exception.php';
require 'phpmailer/src/PHPMailer.php';
require 'phpmailer/src/SMTP.php';

$mail = new PHPMailer(true);

try {
    $mail->isSMTP();
    $mail->Host = 'smtp.gmail.com';
    $mail->SMTPAuth = true;
    $mail->Username = 'noreplybellapizza@gmail.com';
    $mail->Password = 'ajcpxiyrrbkhqwl';
    $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
    $mail->Port = 587;

    $mail->setFrom('noreplybellapizza@gmail.com', 'Bella Pizza');
    $mail->addAddress($_SESSION['email']);

    $mail->isHTML(true);
    $mail->Subject = 'Your Bella Pizza Order Confirmation 🍕';

    $rawAmount = $_POST['amount'] ?? 0;
    $actualAmount = number_format($rawAmount / 1000, 3);

    $mail->Body = "
        <h2>Thank you for your order!</h2>
        <p>Your payment was successful. We're preparing your pizza now!</p>
        <p><strong>Order ID:</strong> " . htmlspecialchars($_POST['fort_id'] ?? 'N/A') . "<br>
        <strong>Amount:</strong> {$actualAmount} " . htmlspecialchars($_POST['currency'] ?? '') . "</p>
    ";

    $mail->AltBody = "Thank you for your order!\nOrder ID: " . ($_POST['fort_id'] ?? 'N/A') .
        "\nAmount: {$actualAmount} " . ($_POST['currency'] ?? '');

    $mail->send();
    echo 'Email has been sent ✅';
} catch (Exception $e) {
    echo "Email could not be sent. Mailer Error: {$mail->ErrorInfo}";
}
```

Figure 49 Email functionality using PHPMailer for customer order confirmations.

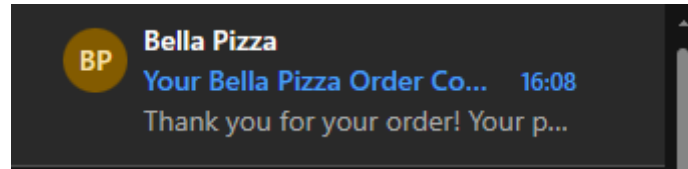


Figure 50 Received Confirmation Email

Customer View Orders

Objective

Allows logged-in customers to view all their previous orders, including order IDs, dates, total amounts, and statuses. Customers can use this page to track ongoing and past orders directly from their dashboard.

Implementation Overview

- When a customer logs in, their `user_id` is stored in the session.
- The `my_orders.php` page retrieves all orders associated with that customer using a prepared statement.
- The query joins the `order` and `order_items` tables to fetch relevant order details.
- Results are displayed in a structured table with the most recent orders listed first.
- Each order can be clicked to view more details .

```
$user_id = $_SESSION["user_id"];
$stmt = $pdo->prepare("SELECT username FROM customer WHERE customer_id = ?");
$stmt->execute([$user_id]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

$sql = "SELECT * FROM `order` WHERE customer_id = :customer_id ORDER BY order_date DESC";
$stmt = $pdo->prepare($sql);
$stmt->execute([':customer_id' => $user_id]);
$orders = $stmt->fetchAll(PDO::FETCH_ASSOC);

$search = $_GET['search'] ?? '';
```

Figure 51 get customer's orders

My Orders				
Search by ID, Status, or Date			Search	
Order ID	Date	Status	Total (BD)	Action
#34	09 May 2025, 13:08	Pending	2.00 BD	View Details
#32	05 May 2025, 14:54	Pending	1.00 BD	View Details
#31	01 May 2025, 16:04	Preparing	5.00 BD	View Details
#30	30 Apr 2025, 12:23	Cancelled	5.00 BD	View Details
#29	29 Apr 2025, 12:34	Completed	5.00 BD	View Details

Figure 52 My orders view

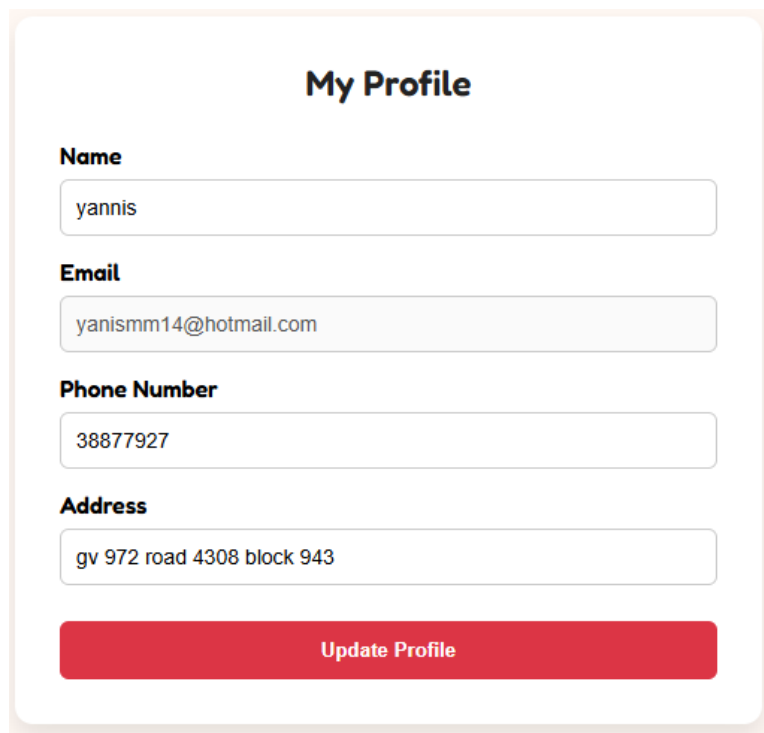
Customer Manage Profile

Objective

Allows logged-in customers to view and update their profile (username, phone, and address) securely from the dashboard.

Implementation Overview

- **Session Check:** Ensures only logged-in users can access the page. Redirects unauthenticated users to login.
- **Dual Login Support:** Detects if user logged in via Google or normal method to fetch data from the correct table.
- **Data Fetch:** Retrieves current profile info using a prepared SQL statement.
- **Form Handling:** On submission, validates input and checks for unique usernames across both login types.
- **Update Logic:** If valid, updates the user's profile in the database using a prepared statement.
- **Feedback:** Shows success or error messages based on outcome.



My Profile

Name
yannis

Email
yanismm14@hotmail.com

Phone Number
38877927

Address
gv 972 road 4308 block 943

Update Profile

Figure 53 Profile UI

Admin/Employee Dashboard

Objective

Provides a role-based dashboard interface for all employee types (admin, support, cashier, chef, delivery). This central panel acts as the employee landing page after login, giving access to tools and pages relevant to the employee's role.

Implementation Overview

- The page starts with a secure session check using `$_SESSION["employee_id"]`. Unauthorized users are redirected to the login page.
- The logged-in employee's username and role are displayed at the top of the dashboard for personalization.
- A CSRF-protected logout form allows employees to securely end their session.
- Available actions are dynamically generated using conditional logic based on the employee's role:
 - admin: Full access to all modules (menu, orders, employees, analytics, logs)
 - support: Limited access to menu and orders
 - cashier, chef, delivery: View orders only

```
<div class="welcome">
  <p>You are logged in as <strong><?= htmlspecialchars($_SESSION["username"]) ?></strong>.</p>
</div>

<div class="actions">
  <?php if ($role === 'admin'): ?>
    <a href="edit_menu_items.php" class="action-button">🍷 Manage Menu</a>
    <a href="orders.php" class="action-button">🍕 Manage Orders</a>
    <a href="employees.php" class="action-button">👤 Employees</a>
    <a href="analytics.php" class="action-button">📊 Business Analytics</a>
    <a href="logs.php" class="action-button">📋 View System Logs</a>
  <?php elseif ($role === 'support'): ?>
    <a href="edit_menu_items.php" class="action-button">🍷 Manage Menu</a>
    <a href="orders.php" class="action-button">🍕 Manage Orders</a>
  <?php elseif (in_array($role, ['cashier', 'chef', 'delivery'])): ?>
    <a href="orders.php" class="action-button">🍕 View Orders</a>
  <?php else: ?>
    <p style="text-align:center;">No actions available for your role.</p>
  <?php endif; ?>
</div>

<div class="footer">
```

Figure 54 Role based action buttons

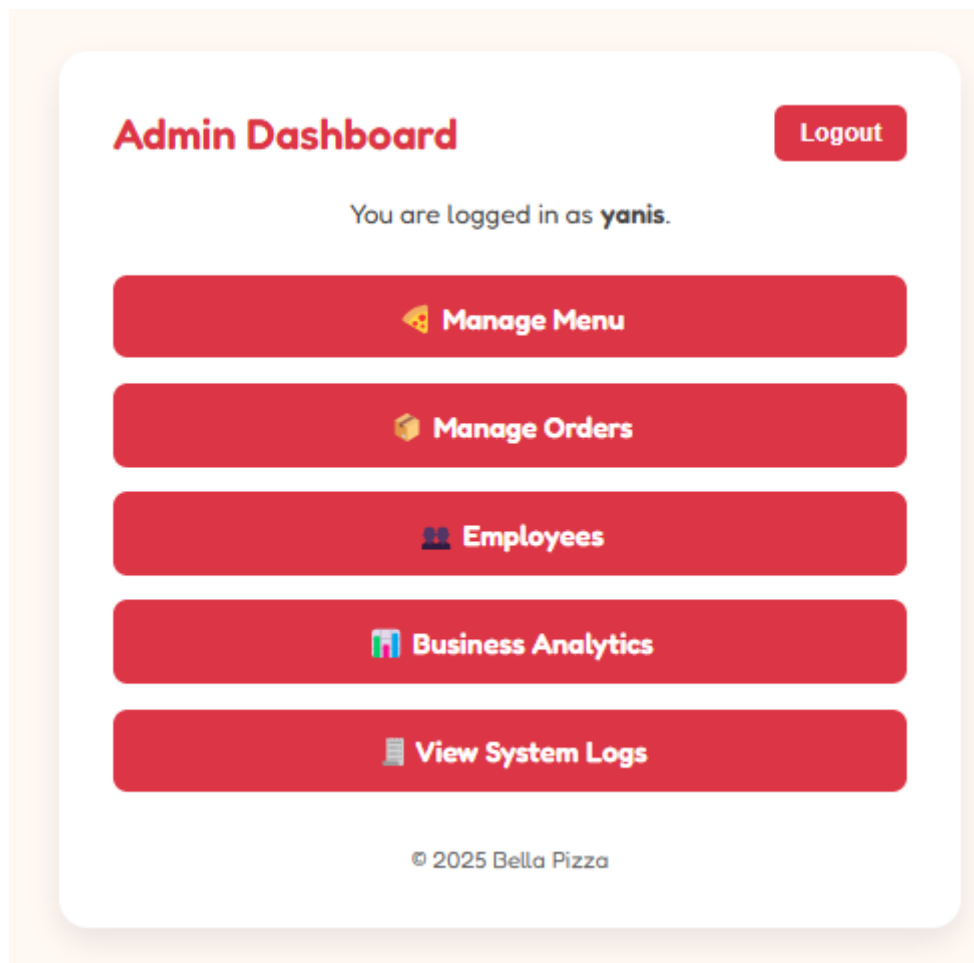


Figure 56Admin Dashboard

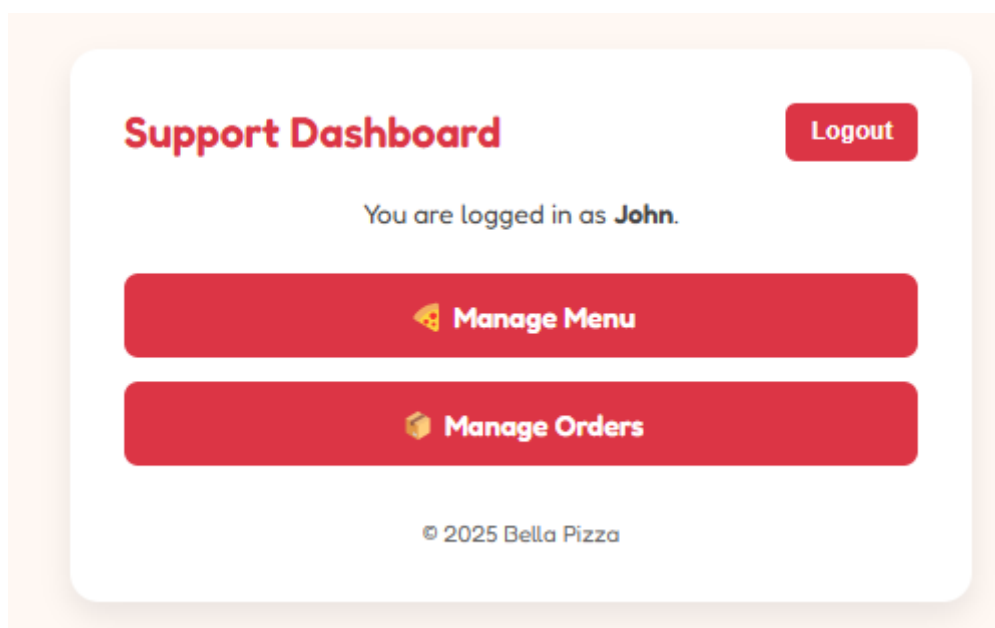


Figure 55 Employee tailored view

Manage Menu Functionality

Objective

Allows administrators and support staff to manage menu items and categories in a visual, interactive interface. This includes adding, editing, deleting, toggling availability, and sorting items by category using drag-and-drop.

Implementation Overview

- Menu items are retrieved from the database and grouped by their categories.
- Each category section is collapsible, and each item is displayed in a card layout with its image, name, price, availability status, and action buttons.
- Admins/support users can:
 - Add/Edit an item and category (redirect to edit_menu_item.php?id=)
 - Delete an item and category (AJAX or form)
 - Toggle availability via AJAX switch
 - Reorder items and categories via SortableJS drag-and-drop, which sends the new order to the backend using AJAX
- The UI is styled with card components and badge indicators to improve clarity and user interaction.

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {  
    validate_csrf_token();  
  
    $item_name = $_POST['item_name'];  
    $description = $_POST['description'];  
    $price = $_POST['price'];  
    $category_id = $_POST['category_id'];  
    $availability = isset($_POST['availability']) ? 1 : 0;  
    $image_path = $_POST['image_path'] ?? "";  
  
    if (isset($_FILES['image']) && $_FILES['image']['error'] === UPLOAD_ERR_OK) {  
        $target_dir = "img/";  
        $image_name = basename($_FILES["image"]["name"]);  
        $target_file = $target_dir . uniqid() . "." . $image_name;  
        $ext = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));  
        if (in_array($ext, ["jpg", "jpeg", "png", "gif"])) {  
            if (move_uploaded_file($_FILES["image"]["tmp_name"], $target_file)) {  
                $image_path = basename($target_file);  
            }  
        }  
    }  
  
    $stmt = $pdo->prepare("UPDATE menu SET item_name = :item_name, description = :description,  
price = :price, category_id = :category_id, availability = :availability, image_path =  
:image_path WHERE menu_id = :menu_id");  
    $stmt->execute([  
        ':item_name' => $item_name,  
        ':description' => $description,  
        ':price' => $price,  
        ':category_id' => $category_id,  
        ':availability' => $availability,  
        ':image_path' => $image_path,  
        ':menu_id' => $menu_id  
    ]);  
  
    log_action("Edit Menu Item", "Updated item '$item_name' (ID: $menu_id)", "INFO", $_SESSION  
["employee_id"], null, $_SESSION["username"]);  
  
    header("Location: edit_menu_item.php?id=$menu_id&success=1");  
    exit();  
}
```

Figure 57 edit menu logic

```

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    validate_csrf_token();

    $item_name = trim($_POST['item_name']);
    $description = trim($_POST['description']);
    $price = $_POST['price'];
    $category_id = $_POST['category_id'];
    $availability = isset($_POST['availability']) ? 1 : 0;

    $image_path = "";
    if (!empty($_FILES['image']['name']) && $_FILES['image']['error'] === UPLOAD_ERR_OK) {
        $target_dir = "img/";
        $file_name = uniqid() . "_" . basename($_FILES["image"]["name"]);
        $target_file = $target_dir . $file_name;
        $ext = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
        if (in_array($ext, ["jpg", "jpeg", "png", "gif"]) && move_uploaded_file($_FILES["image"]["tmp_name"], $target_file)) {
            $image_path = $file_name;
        }
    }

    $stmt = $pdo->prepare("
        INSERT INTO menu
        (item_name, description, price, category_id, availability, image_path)
        VALUES
        (:item_name, :description, :price, :category_id, :availability, :image_path)
    ");
    $stmt->execute([
        'item_name' => $item_name,
        'description' => $description,
        'price' => $price,
        'category_id' => $category_id,
        'availability' => $availability,
        'image_path' => $image_path
    ]);

    log_action(
        "Add Menu Item",
        "Menu item '$item_name' was added by employee {$$_SESSION['username']}",
        "INFO",
        $_SESSION["employee_id"],
        null,
        $_SESSION["username"]
    );

    $success = "Menu item added successfully!";
}

```

Figure 58 Add menu item logic

```

if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['toggle_id'])) {
    validate_csrf_token();
    $id = (int) $_POST['toggle_id'];
    $available = $_POST['available'] === '1' ? 1 : 0;
    $stmt = $pdo->prepare("UPDATE menu SET availability = ? WHERE menu_id = ?");
    $stmt->execute([$available, $id]);

    log_action("Item Availability Toggled", "Menu item ID $id marked as " . ($available ? "available" : "unavailable"), "INFO", $_SESSION['employee_id'], null, $_SESSION['username']);
    exit;
}

```

Figure 59 Toggle item availability logic

```

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST['name']);
    if ($name !== "") {
        $stmt = $pdo->prepare("INSERT INTO category (name) VALUES (?)");
        $stmt->execute([$name]);
        $message = "✅ Category added successfully!";
        log_action("Category Added", "New category '$name' was added.", "INFO", $_SESSION["employee_id"], null);
    } else {
        $message = "⚠ Please enter a category name.";
        log_action("Category Add Failed", "Attempted to add category with empty name.", "WARNING", $_SESSION["employee_id"], null);
    }
}
?>

```

Figure 60 add category logic

```

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST['name']);
    if (!empty($name)) {
        $update = $pdo->prepare("UPDATE category SET name = ? WHERE category_id = ?");
        $update->execute([$name, $category_id]);

        // Log the update
        log_action("Update Category", "Category ID $category_id renamed to '$name'", "INFO", $_SESSION["employee_id"], null, $_SESSION["username"]);

        $message = "Category updated!";
        header("Location: categories.php");
        exit;
    } else {
        $message = "Please enter a category name.";
    }
}
?>

```

Figure 61 edit category logic

```

if (isset($_GET['delete'])) {
    $stmt = $pdo->prepare("DELETE FROM category WHERE category_id = ?");
    $stmt->execute([$_GET['delete']]);
    $message = "Category deleted successfully!";
}

$categories = $pdo
    ->query("SELECT * FROM category ORDER BY category_id DESC")
    ->fetchAll(PDO::FETCH_ASSOC);
?>

```

Figure 62 delete category logic

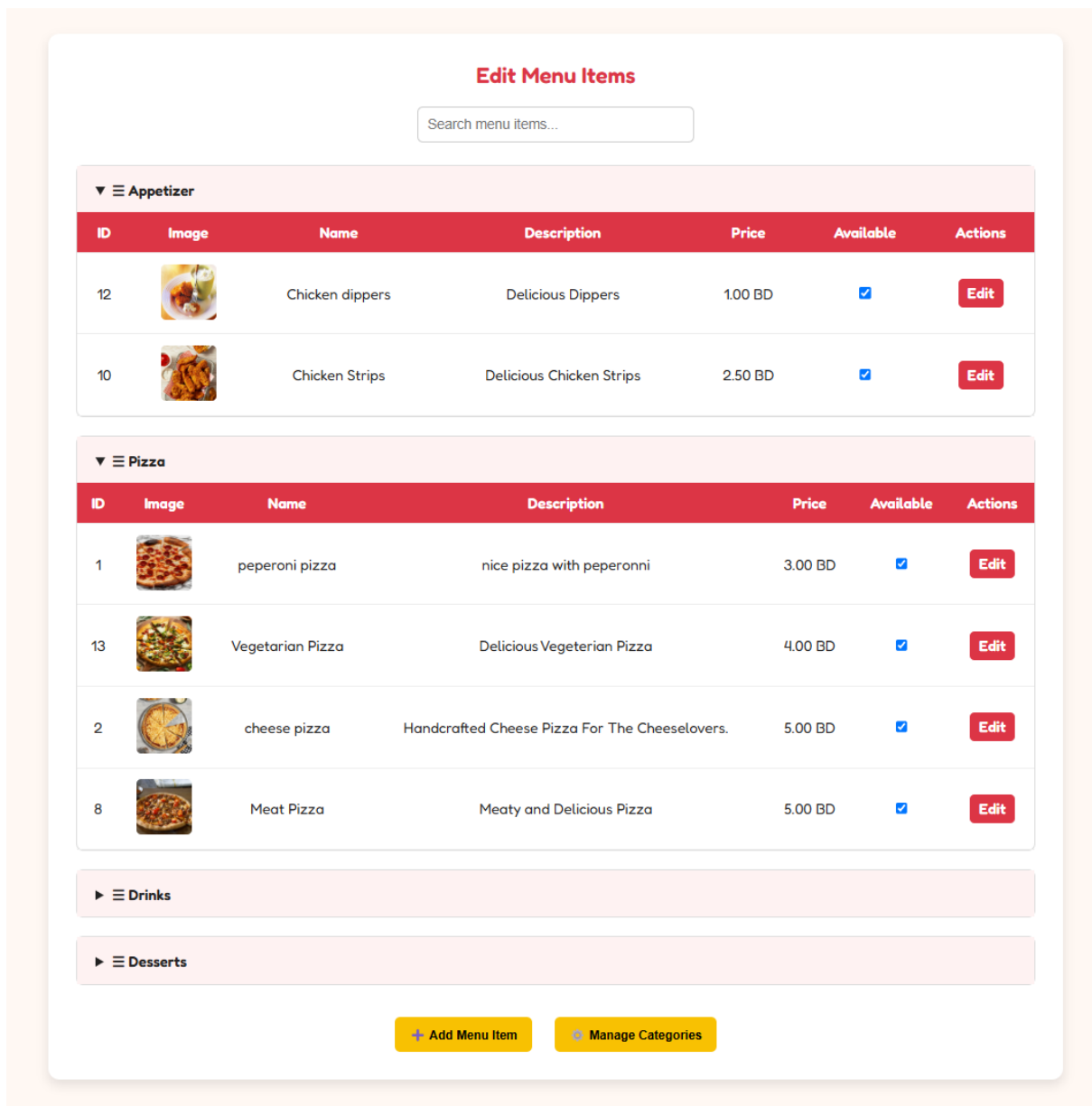


Figure 63 Edit menu items view

Add Menu Item

Item Name

Description

Price (BD)

Category

— Select Category —

Upload Image (optional)

Drag & Drop or Click to Browse

☒ Available

+ Add Item

← Back to Menu

Figure 64 Add menu item form

Edit Menu Item

Item Name:

Chicken dippers

Description:

Delicious Dippers

Price:

1.00

Category:

Appetizer

Upload New Image (optional):

Drag & Drop Image Here or Click to Browse



☒ Available

Update Item

← Back to Menu

Figure 65 edit menu form

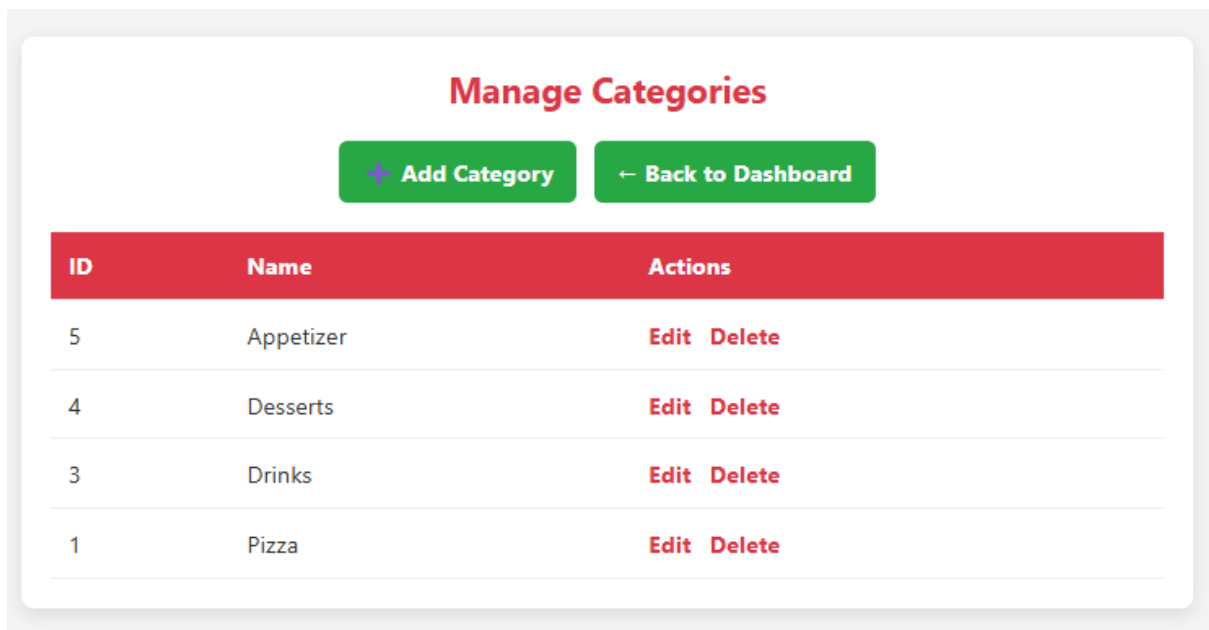


Figure 66 Category management view

Order Management Functionality

Objective

Enables employees to view and manage customer orders. Admin, support, cashier, chef, and delivery staff can access this interface to update order statuses, assign deliveries, and monitor real-time changes to orders throughout the day.

Implementation Overview

- The page is role-protected, accessible only to logged-in employees via `$_SESSION['employee_id']`.
- Orders are fetched from the database and grouped into today's orders and past orders.
- If the logged-in role is delivery, only orders assigned to that driver are shown.
- The interface includes action buttons to:
 - Mark orders as *Preparing*, *Out for Delivery*, or *Completed*
 - View order details such as item list and customer info
- Real-time polling with `setInterval()` and AJAX refreshes the order list every 10 seconds without reloading the page.
- All status updates are CSRF-protected POST requests, with logging included via `log_helper.php`.

```
function fetchOrders($pdo, $employee_id, $role) {
    if ($role === 'delivery') {
        $stmt = $pdo->prepare("SELECT o.*, c.username AS customer_name, CONCAT(e.first_name, ' ', e.
last_name) AS driver_name FROM `order` o LEFT JOIN customer c ON o.customer_id = c.customer_id LEFT
JOIN employee e ON o.delivery_driver_id = e.employee_id WHERE o.delivery_driver_id = :employee_id
ORDER BY o.order_date DESC");
        $stmt->execute([':employee_id' => $employee_id]);
    } else {
        $stmt = $pdo->query("SELECT o.*, c.username AS customer_name, CONCAT(e.first_name, ' ', e.last_name)
AS driver_name FROM `order` o LEFT JOIN customer c ON o.customer_id = c.customer_id LEFT JOIN
employee e ON o.delivery_driver_id = e.employee_id ORDER BY o.order_date DESC");
    }
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

Figure 67 Role based search logic

```

function statusBtn($id, $status, $class = '') {
    $token = $_SESSION['csrf_token'];
    return "<form method='POST' class='status-form'>
        <input type='hidden' name='csrf_token' value='$token'>
        <input type='hidden' name='order_id' value='$id'>
        <input type='hidden' name='new_status' value='$status'>
        <button class='$class' type='submit'>$status</button>
    </form>";
}

```

```

if (!in_array($o['status'], ['Completed', 'Cancelled'])) {
    if ($role === 'cashier') {
        if ($o['status'] === 'Pending') {
            echo statusBtn($o['order_id'], 'Preparing');
        } elseif ($o['status'] === 'Preparing') {
            echo statusBtn($o['order_id'], 'Out for Delivery');
            echo linkBtn('assign_driver.php', $o['order_id'], 'Assign Driver', 'btn-outline');
        }
    }
    if ($role === 'delivery' && $o['status'] === 'Out for Delivery') {
        echo statusBtn($o['order_id'], 'Completed');
    }
    echo statusBtn($o['order_id'], 'Cancelled', 'btn-outline');
}

echo linkBtn('order_details.php', $o['order_id'], 'Details', 'btn-outline');
echo "</td></tr>";
}

```

Figure 69 Status Update buttons logic

```

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['driver_id'])) {
    $driver_id = $_POST['driver_id'];
    $stmt = $pdo->prepare("UPDATE `order` SET delivery_driver_id = :driver_id WHERE order_id = :order_id");
    if ($stmt->execute([':driver_id' => $driver_id, ':order_id' => $order_id])) {
        $success = "Driver assigned successfully.";

        $driver = $pdo->prepare("SELECT first_name, last_name FROM employee WHERE employee_id = ?");
        $driver->execute([$driver_id]);
        $driver_data = $driver->fetch(PDO::FETCH_ASSOC);
        $driver_name = $driver_data ? $driver_data['first_name'] . ' ' . $driver_data['last_name'] : 'Unknown';

        log_action(
            "Assign Driver",
            "Assigned $driver_name to order #$order_id",
            "INFO",
            $employee_id,
            null,
            $username
        );
    } else {
        $error = "Failed to assign driver.";
    }
}

```

Figure 68 Assign Driver Logic

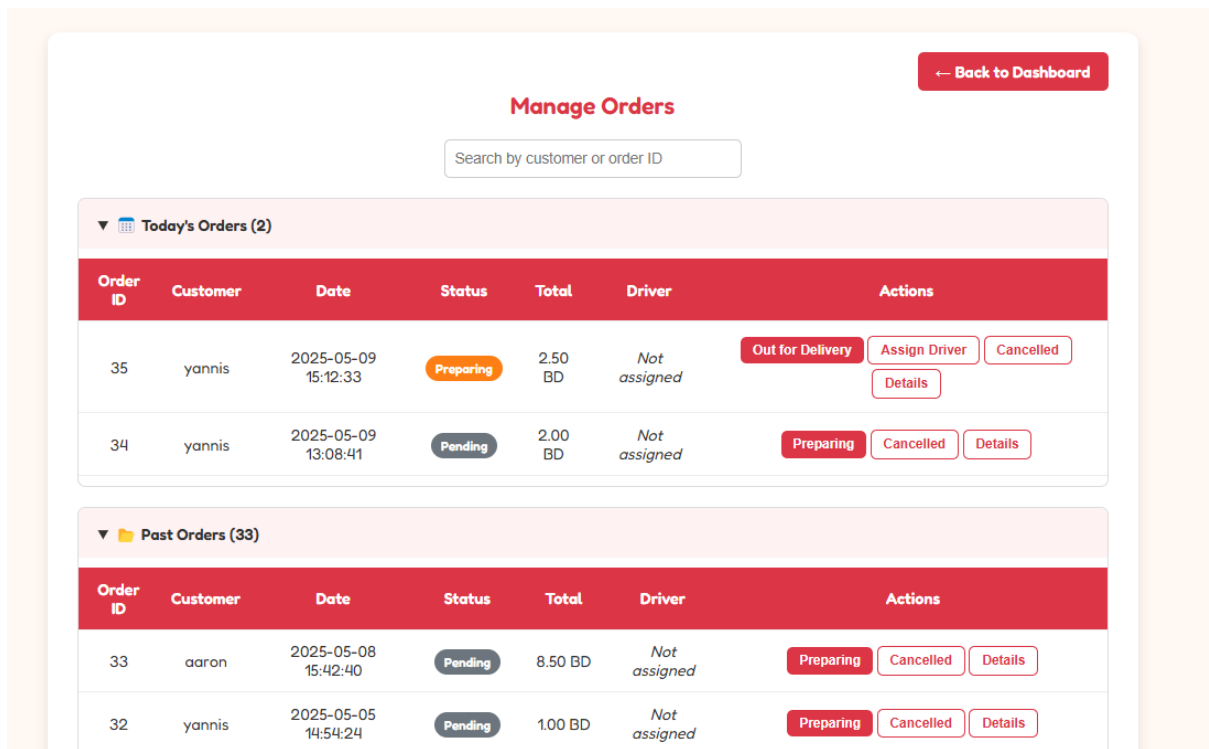


Figure 71 Orders View

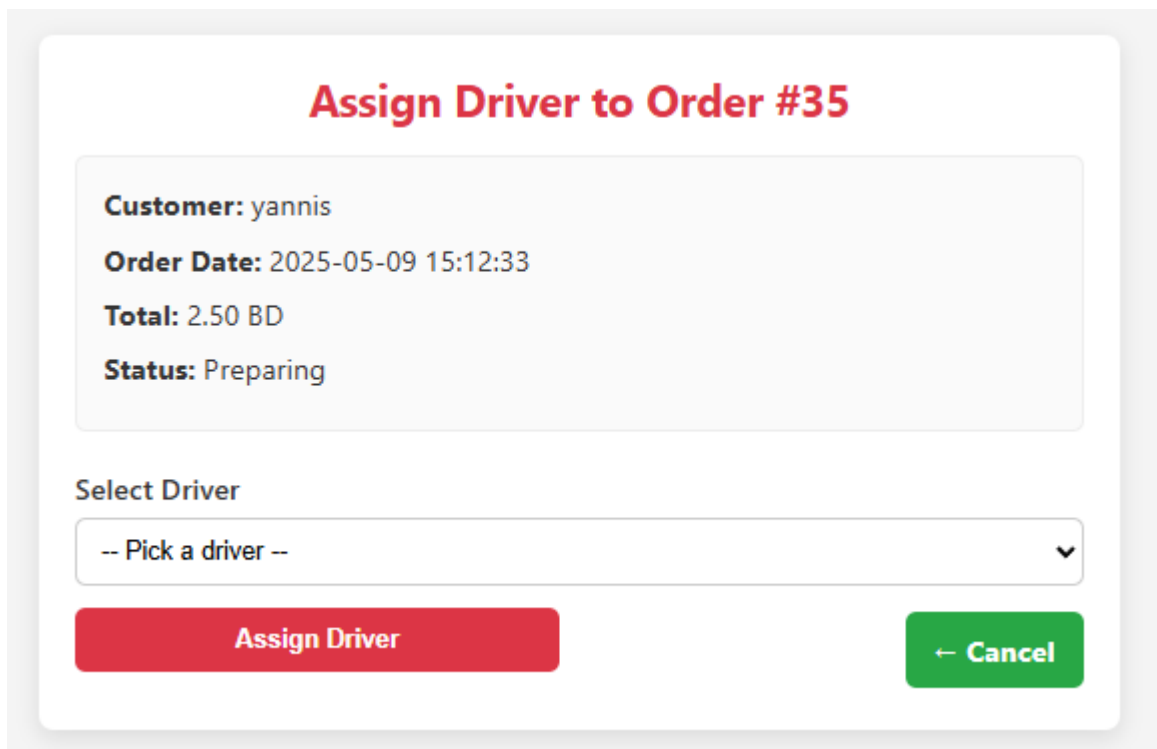


Figure 70 Assign drivers page

Employee Management Functionality

Objective

Allows administrators to view, add, edit, and delete employees. Each employee is assigned a specific role (admin, support, chef, cashier, delivery), which determines their access to different system features.

Implementation Overview

- Accessible only to users with the admin role (verified via `$_SESSION["role"] === 'admin'`).
- The main `employees.php` page displays a table of all employees with their:
 - ID
 - Name
 - Username
 - Email
 - Phone number
 - Role
 - Actions (Edit / Delete)
- Includes:
 - Live search to filter employees by name or username
 - Role badges styled with distinct colors (e.g., red for admin)
 - AJAX-based delete to remove an employee without reloading the page
 - CSV export button for generating staff reports
 - Pagination for easier browsing if many employees exist
 - Real-time count of total employees at the top of the page

+ Add Employee

Export CSV

Search employees...

Total Employees: 7

ID	Full Name	Username	Email	Phone	Role	Salary	Hire Date	Actions	
14	admin admin	admin	admin@hotmail.com	38877927	Admin	BD 1,000.00	2025-05-08	Update	Delete
11	Yanis Moussai	qwqwq	Drain@email.xom	38877927	Support	BD 12,222.00	2025-05-03	Update	Delete
5	jack jack	jack	jackemail@gmail.com	38877927	Delivery	BD 2,333.00	2025-04-16	Update	Delete
4	james james	james	james@gmail.com	38877927	Chef	BD 2,000.00	2025-04-16	Update	Delete
3	drake drake	drake	drake111@gmail.com	38877927	Cashier	BD 1,212.00	2025-04-16	Update	Delete
2	Yanis yanis	yanis	yanismm14@hotmail.com	38877927	Admin	BD 500.00	2025-04-16	Update	Delete
1	John John	John	John@gmail.com	12312312	Support	BD 500.00	2025-04-10	Update	Delete

Figure 73 Employees management view

Add New Employee

First Name

Last Name

Username

Email

Phone Number (optional)

Role

-- Select Role --

Salary (BHD)

Password

+ Add Employee

← Back to Employee List

Figure 72 Add employee Form

Edit Employee

First Name:

admin

Last Name:

admin

Username:

admin

Email:

admin@hotmail.com

Phone Number:

38877927

Role:

Admin



Salary:

1000.00

Update Employee

← Back to Employees

Figure 74 Edit Employee form

```

<?php
session_start();
✓ if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    http_response_code(403);
    exit("Unauthorized");
}

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
✓ if ($conn->connect_error) {
    http_response_code(500);
    exit("DB error");
}

✓ if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['id'])) {
    $id = intval($_POST['id']);
    $stmt = $conn->prepare("UPDATE employee SET is_deleted = 1 WHERE employee_id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    echo "OK";
    exit;
}

http_response_code(400);
echo "Bad Request";

```

Figure 75 delete employee logic

CSV Export Functionality

Objective

Enables administrators to export data (e.g., employee list, analytics, order reports) into a downloadable CSV file for offline viewing, reporting, or archival. This feature enhances business reporting and record-keeping.

Implementation Overview

- Export buttons are placed on pages such as employees.php and analytics.php.
- When clicked, the browser sends a GET or POST request to a dedicated export script (e.g., export_employees.php).
- The export script:
 - Validates the request
 - Fetches the data from the database using PDO
 - Sends appropriate HTTP headers to trigger a file download
 - Outputs CSV content line by line

```
$('#exportCsv').on('click', function () {
    let csv = "ID,Full Name,Username,Email,Phone,Role,Salary,Hire Date\n";
    $('#employeeTable tbody tr:visible').each(function () {
        const cols = $(this).find('td');
        csv += `${cols.eq(0).text()}`,`${cols.eq(1).text()}`,`${cols.eq(2).text()}`,`${cols.eq(3).text()}`,`${cols.eq(4).text()}`,`${cols.eq(5).text()}`,`${cols.eq(6).text()}`,`${cols.eq(7).text()}`"\n";
    });
    const blob = new Blob([csv], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'employees.csv';
    a.click();
    URL.revokeObjectURL(url);
});
```

Figure 76 Export Employees CSV logic

```
function downloadCSV() {
    const rows = <?= json_encode($csvRows) ?>;
    const csv = rows.map(r => r.join(",")).join("\n");

    const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
    const link = document.createElement('a');
    link.href = URL.createObjectURL(blob);
    link.download = `analytics_report_<?= $startDate ?>_to_<?= $endDate ?>.csv`;
    link.click();
}
```

Figure 77 Analytics page download CSV logic

A	B	C	D	E	F	G	H
ID	Full Name	Username	Email	Phone	Role	Salary	Hire Date
14	admin admin	admin	admin@hotmail.com	38877927	Admin	BD 1,000.00	08/05/2025
11	Yanis Moussai	qwqwq	Drain@email.xom	38877927	Support	BD 12,222.00	03/05/2025
5	jack jack	jack	jackemail@gmail.com	38877927	Delivery	BD 2,333.00	16/04/2025
4	james james	james	james@gmail.com	38877927	Chef	BD 2,000.00	16/04/2025
3	drake drake	drake	drake111@gmail.com	38877927	Cashier	BD 1,212.00	16/04/2025
2	Yanis yanis	yanis	yanismm14@hotmail.com	38877927	Admin	BD 500.00	16/04/2025
1	John John	John	John@gmail.com	12312312	Support	BD 500.00	10/04/2025

Figure 78 Employees CSV file

	A	B	C	D	E
1	Bella Pizza Analytics Report				
2	Date Range:	2025-05-01 to 2025-05-09			
3	Generated At:	09/05/2025 15:36			
4					
5	Total Orders	5			
6	Total Revenue	19.00 BD			
7	Total Visits				
8	Total Unique Visits				
9	Total Customer Logins	99			
10	New Customers	2			
11	Returning Customers	0			
12					
13	Status	Count	Total Amount		
14	Preparing	2	7.50 BD		
15	Pending	3	11.50 BD		
16					
17	Item Name	Total Sold	Total Amount		
18	Chicken dippers	4	4.00 BD		
19	cheese pizza	2	10.00 BD		
20	Chicken Strips	2	5.00 BD		
21					
22					

Figure 79 Analytics report CSV file

Logs Functionality

Objective

Captures and displays a record of system events such as logins, registrations, menu edits, order updates, and deletions. This log serves as an internal audit trail for administrators to monitor system activity, detect misuse, and maintain accountability.

Implementation Overview

- All critical actions are logged via a centralized helper function called `log_action()` located in `log_helper.php`.
- Each log entry records:
 - Action title (e.g., "Login Success", "Delete Employee")
 - Description
 - Severity level (INFO, WARNING, ERROR)
 - Actor (employee or customer ID + username)
 - IP address, User Agent, Session ID
 - Timestamp
- The `logs.php` page displays these logs in a searchable and filterable table.
- Admins can filter logs by role, action type, and keyword.
- Logs are stored in a dedicated logs table in the database and fetched using PDO.

```
$ip_address = $_SERVER['REMOTE_ADDR'] ?? 'unknown';
$user_agent = $_SERVER['HTTP_USER_AGENT'] ?? 'unknown';
$session_id = session_id() ?: 'no-session';
$location = $_SERVER['REQUEST_URI'] ?? 'unknown';
$log_time = date('Y-m-d H:i:s');

try {
    $stmt = $pdo->prepare("
        INSERT INTO logs
        (action, description, log_level, employee_id, customer_id, username, ip_address, user_agent,
        session_id, location, log_time)
        VALUES
        (:action, :description, :log_level, :employee_id, :customer_id, :username, :ip_address,
        :user_agent, :session_id, :location, :log_time)
    ");

    $stmt->execute([
        ':action' => $action,
        ':description' => $description,
        ':log_level' => strtoupper($level),
        ':employee_id' => $employee_id,
        ':customer_id' => $customer_id,
        ':username' => $username,
        ':ip_address' => $ip_address,
        ':user_agent' => $user_agent,
        ':session_id' => $session_id,
        ':location' => $location,
        ':log_time' => $log_time,
    ]);
} catch (PDOException $e) {
    error_log("Log insertion failed: " . $e->getMessage());
}
```

Figure 80 Log function logic

```
require_once "log_helper.php";
```

Figure 82 log page included in files

```
log_action("Login Success", "Customer logged in successfully", "INFO", null, $customer["customer_id"], $customer["username"]);
```

Figure 81 calling log funtions

System Logs

Search logs...

All Levels

dd/mm/yyyy

dd/mm/yyyy

Filter

ID	Level	Timestamp	Employee ID	Customer ID	Username	Action	Description	IP Address	User Agent	Session
70	INFO	2025-05-09 15:19:40	2	-	yanis	Login Success	Employee logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486
69	WARNING	2025-05-09 15:12:55	3	-	drake	Order Status Change	Order #35 marked as 'Preparing'	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486
68	INFO	2025-05-09 15:12:41	3	-	drake	Login Success	Employee logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486
67	INFO	2025-05-09 15:11:57	-	1	yannis	Login Success	Customer logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486
66	INFO	2025-05-09 14:53:00	2	-	yanis	Login Success	Employee logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486
65	INFO	2025-05-09 13:48:24	1	-	John	Login Success	Employee logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	7ede9250c144ac3486

Figure 83 Logs view

Analytics Functionality

Objective

Provides administrators with a visual, filterable dashboard that summarizes sales performance, customer behavior, and menu item statistics. The analytics page helps the business make data-driven decisions by offering both numeric summaries and interactive charts.

Implementation Overview

- Access Control: Only accessible to admin users through session role validation.
- Date Filtering: Admins can filter data by selecting a start and end date. This updates all stats, tables, and charts dynamically.
- Live Data: Includes auto-refresh functionality every 30 seconds to keep analytics up to date without manual reload.
- Customer Insights:
 - Total unique visits (based on new logins)
 - Total logins (customer sessions)
 - New vs. returning customers
- Sales Metrics:
 - Total number of orders
 - Total revenue (excluding cancelled orders)
 - Orders grouped by status (e.g., completed, pending)
- Menu Item Performance:
 - Items ranked by quantity sold
 - Total revenue per item
- Charts:
 - Bar chart of best-selling items
 - Line chart of daily revenue trends
- CSV Export: A full summary of analytics data is available for download in CSV format using a `downloadCSV()` JavaScript function.


```

$allItems = $pdo->prepare("
    SELECT m.item_name, SUM(oi.quantity) AS total_sold, SUM(oi.quantity * m.price) AS total_amount
    FROM order_items oi
    JOIN menu m ON oi.menu_id = m.menu_id
    JOIN `order` o ON o.order_id = oi.order_id
    $nonCancelledCondition
    GROUP BY oi.menu_id
    ORDER BY total_sold DESC
");
$allItems->execute($params);
$allItems = $allItems->fetchAll(PDO::FETCH_ASSOC);

$dailyRevenue = $pdo->prepare("
    SELECT DATE(order_date) AS day, SUM(total_amount) AS revenue
    FROM `order`
    $nonCancelledCondition
    GROUP BY day ORDER BY day
");
$dailyRevenue->execute($params);
$dailyDates = [];
$dailyValues = [];
foreach ($dailyRevenue as $row) {
    $dailyDates[] = $row['day'];
    $dailyValues[] = $row['revenue'];
}

```

Figure 85 Data summarization to turn raw data into meaningful summaries

```

new Chart(document.getElementById('itemsChart'), {
    type: 'bar',
    data: {
        labels: <?= json_encode($itemLabels) ?>,
        datasets: [{
            label: 'Items Sold',
            data: <?= json_encode($itemCounts) ?>,
            backgroundColor: '#dc3545'
        }]
    },
    options: { scales: { y: { beginAtZero: true } } }
});

```

Figure 84 Create new chart logic for items sold

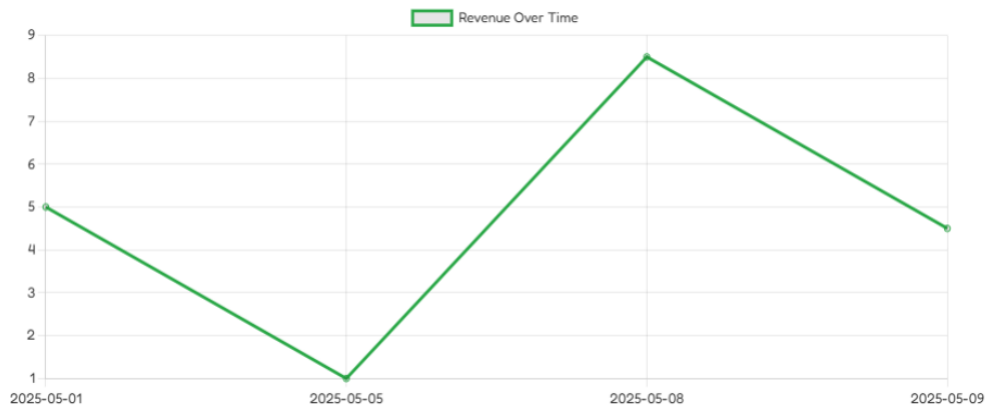
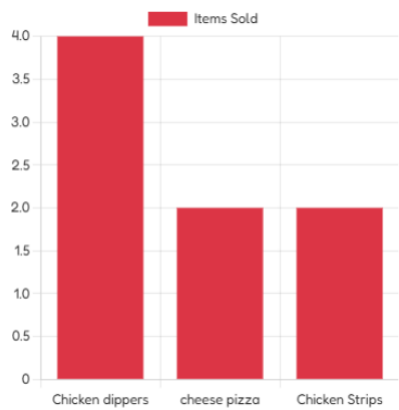
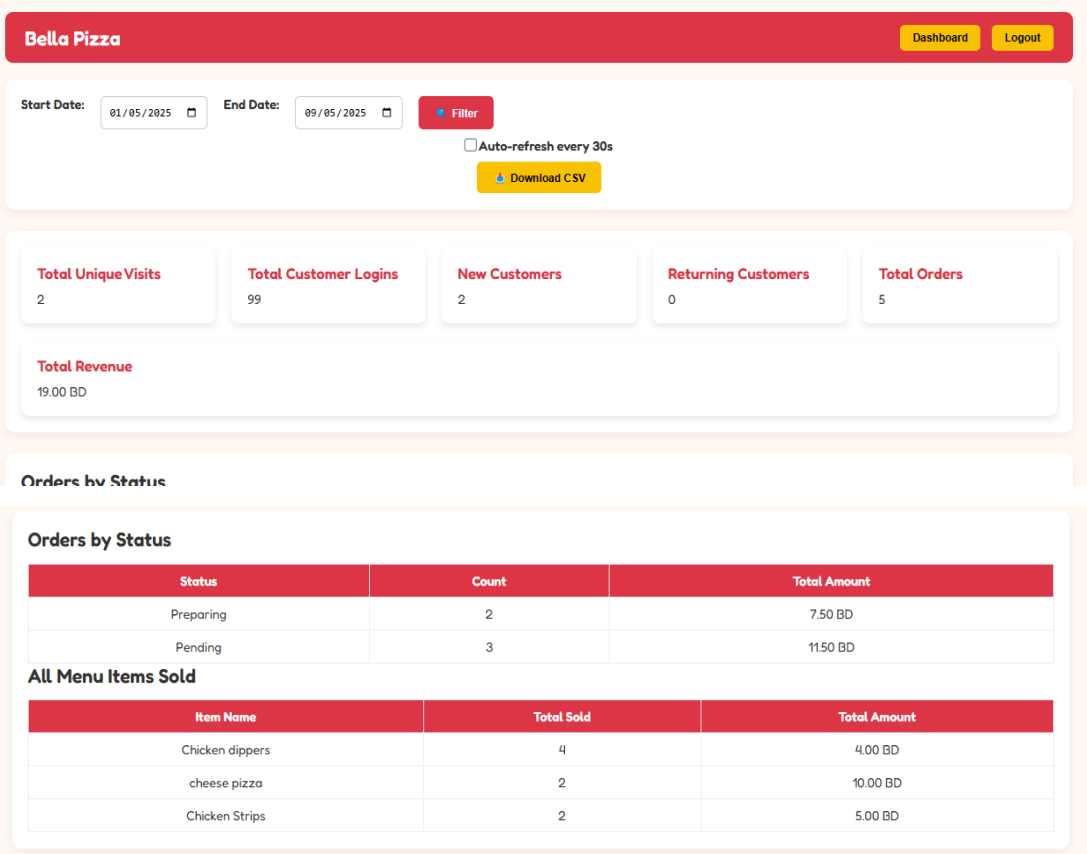


Figure 86 Analytics View

Search and Filter Functionality

Objective

Enables administrators to quickly locate and analyze specific data by applying keyword searches, role filters, severity levels, and date ranges. This improves usability and performance when dealing with large datasets.

Implementation Overview

- Search and filter inputs are displayed at the top of each data-driven page using HTML forms (<input>, <select>, <date>).
- When the form is submitted, the filter values are passed via GET parameters and used to construct a dynamic SQL query.
- Prepared statements are used to safely bind filter values (?) and prevent SQL injection.
- Results are paginated and reloaded based on current filters, ensuring a consistent browsing experience.
- Search and filters are available in:
 - logs.php: filter by log level, date range, and keyword
 - employees.php: live search by name or username
 - analytics.php: filter data by start and end date

```
$('#searchInput').on('input', function () {
  const val = $(this).val().toLowerCase();
  let count = 0;
  $('#employeeTable tbody tr').each(function () {
    const rowText = $(this).text().toLowerCase();
    const match = rowText.includes(val);
    $(this).toggle(match);
    if (match) count++;
  });
  $('#totalCount').text(count);
});
```

Figure 87 Search Logic in employees table

```
$startDate = $_GET['start'] ?? date('Y-m-01');
$endDate   = $_GET['end']   ?? date('Y-m-d');

$params = [':start' => $startDate, ':end' => $endDate];
$condition = "WHERE DATE(order_date) BETWEEN :start AND :end";
$nonCancelledCondition = $condition . " AND status != 'cancelled'";
```

Figure 88 Filtering Logic in analytics page

Start Date:

01/05/2025

End Date:

09/05/2025

Filter

Figure 89 filter view in analytics page

Edit Menu Items

strips

▼ ≡ Appetizer

ID	Image	Name	Description	Price	Available	Actions
10		Chicken Strips	Delicious Chicken Strips	2.50 BD	☒	Edit

Figure 90 search view in edit menu page

Security Implementation

Objective

Ensures that all user and employee interactions are protected from unauthorized access, tampering, and data leakage. The system applies multiple security layers including session validation, CSRF protection, prepared statements, and role-based access control.

Implementation Overview

The Bella Pizza system incorporates the following key security measures:

1. Session Management

- All sensitive pages begin by checking if the session is valid using `$_SESSION['employee_id']` or `$_SESSION['user_id']`.
- If the session is missing or the user role is invalid, the user is redirected to `login.php`.
- Session data includes user ID, role, email, and CSRF token.

2. CSRF Protection

- A CSRF token is generated when the session starts and stored in `$_SESSION['csrf_token']`.
- All sensitive forms include a hidden input with this token.
- Server-side scripts verify the token using `validate_csrf_token()` before processing any POST request.

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST["add_to_cart"])) {  
    validate_csrf_token();  
}
```

Figure 91 csrf validation

3. Prepared Statements

- All SQL queries that involve user input use `prepare()` and `bind_param()` or `execute([$param])` to prevent SQL injection.
- Example from login:

```
$itemStmt = $pdo->prepare("SELECT * FROM menu WHERE availability=1 AND category_id=? ORDER BY item_name");  
foreach ($categories as $cat) {  
    $itemStmt->execute([$cat['category_id']]);  
}
```

Figure 92 Prepared statements used

4. Role-Based Access Control (RBAC)

- Pages like dashboard.php, employees.php, and analytics.php verify the logged-in user's role:
- Each role (admin, support, cashier, delivery, chef) has limited visibility and access to different parts of the dashboard.

```
if (!isset($_SESSION["user_id"])) {  
    header("Location: login.php");  
    exit();  
}
```

Figure 93 verifying users through session variables

5. Soft Delete System

- Instead of permanently deleting records (e.g., employees), a soft delete flag (is_deleted = 1) is used.
- This ensures data integrity, allows recovery, and avoids cascading issues from accidental deletions.

6. Input Validation and Sanitization

- Registration and login inputs are validated for length, format, and pattern.
- Phone numbers, emails, and addresses are checked on both client and server sides.
- Passwords are hashed using password_hash() and verified using password_verify().

```
if (empty($username) || strlen($username) < 3) {  
    $error = "Username must be at least 3 characters long.";  
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    $error = "Invalid email address.";  
} elseif (empty($password_raw) || strlen($password_raw) < 6) {  
    $error = "Password must be at least 6 characters long.";  
} elseif (!preg_match('/^[0-9]{8,15}$/', $number)) {  
    $error = "Phone number must be between 8 to 15 digits.";  
} elseif (empty($address) || strlen($address) < 5) {  
    $error = "Address must be at least 5 characters long.";  
} else {  
    $password = password_hash($password_raw, PASSWORD_BCRYPT);  
}
```

Figure 94 input validation for registration

```

<td><?= $row['employee_id'] ?></td>
<td><?= htmlspecialchars($row['first_name'] . ' ' . $row['last_name']) ?></td>
<td><?= htmlspecialchars($row['username']) ?></td>
<td><?= htmlspecialchars($row['email']) ?></td>
<td><?= htmlspecialchars($row['phone_number']) ?></td>
<td><span class="badge <?= htmlspecialchars(strtolower($row['role'])) ?>"><?= htmlspecialchars(ucfirst($row['role'])) ?></span></td>

```

Figure 95 Input sanitization

7. Audit Logging

- All critical actions (login, logout, edit, delete, register) are recorded via `log_action()` into the logs table.
- Logs include: username, action type, severity level, timestamp, IP address, and user agent.

8. Real-time monitoring

- View Alerts: Access real-time alerts for suspicious or unauthorized activity across all monitored containers.
- Filter Events: Use filters by severity, source, or rule ID to investigate specific security incidents.
- Acknowledge Alerts: Mark alerts as reviewed to manage ongoing investigations or reduce noise.
- Create Custom Rules: Define new rules or modify existing ones to fine-tune detection based on Bella Pizza's infrastructure.
- Monitor Log Files: Inspect raw logs collected from Apache, PHP, Docker, and system files in a centralized view.
- Configure Agents: Add or update monitored agents (e.g., containers, WSL environments) to expand security coverage.
- Generate Reports: Export summaries of threats, agent status, and compliance data for audits or documentation.
- Visualize Trends: Use built-in dashboards to analyze security trends, login anomalies, and system changes over time.

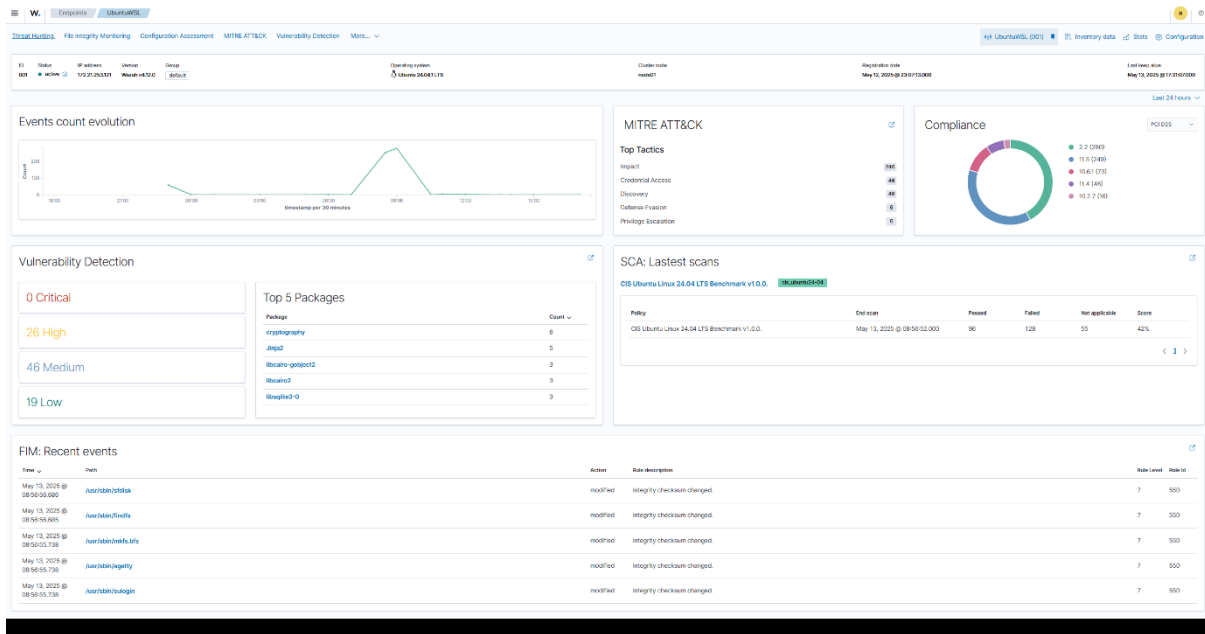


Figure 96 Wazuh interface

Testing

The testing phase was a critical step in validating the functional integrity, security, and overall usability of the Bella Pizza web-based ordering system. Various testing methodologies were applied to ensure the system performed as expected across different scenarios and usage patterns. This section outlines the testing strategies used, including functional testing, form validation, session and CSRF protection verification, and user feedback evaluations. Through systematic test cases, edge condition simulations, and security-oriented inspections, the system was thoroughly evaluated to identify and resolve potential issues prior to deployment. The aim was to confirm that the application meets its core requirements, functions reliably under typical and unexpected conditions, and ensures data security for both customers and staff users.

Test Plan

The Bella Pizza system underwent a structured testing plan to ensure that each feature fulfilled its intended purpose and met both functional and non-functional requirements. The primary goals of the testing plan were to verify the correctness of critical functionalities, validate user interactions, ensure data integrity, and confirm that access control and security protections were enforced effectively. The following summarizes the test strategy used:

Objectives

- Ensure that all core features (menu browsing, cart, checkout, employee/admin tools) work as expected.
- Verify that user roles (admin, support, cashier, delivery, customer) have correct access rights.
- Test that security features such as session validation and CSRF protection are functioning.
- Identify any UI/UX bugs or inconsistencies through user interaction testing.

Scope

- Functional features: customer ordering, menu editing, order status updates, employee and category management.
- Security aspects: session timeout, unauthorized access prevention, CSRF token enforcement.
- Usability testing: navigation clarity, form validation feedback, responsiveness on mobile.

Roles and Responsibilities

- Developer: Conducted unit and integration tests during development.
- Peer Reviewers: Performed exploratory testing and usability checks.
- End Users (sample testers): Provided feedback on the customer ordering experience.

Tools and Environment

- Browser-based manual testing (Chrome, Firefox, mobile view)
- VS Code terminal and Docker logs for backend behavior
- PHPMysqlAdmin for data verification
- Mailtrap for email testing (in sandbox mode)

Types of Testing Conducted

- Unit Testing: Performed during development for individual components such as menu item updates or login validation.
- Integration Testing: Tested the flow from cart update to payment to order placement.
- System Testing: Verified complete use cases like order processing and role-based dashboard access.
- Security Testing: Manual inspection of session handling, CSRF tokens, and route protection.
- User Acceptance Testing (UAT): Feedback gathered from testers simulating real customer usage.

Test Participants

Name	Age	Background
Abdulla Sooud	24	ICT student with experience in testing software. Focused in Database Management
Ahmad Faisal	22	ICT student working in web development. Mainly focuses in backend logic.
Yousif Abbas	22	Programming graduate focusing in cybersecurity and penetration testing.

Functionality Test Cases and Results

Registration and Authentication

Table 4 Registration and authentication

ID	Test Scenario	Expected Result	Actual Result	Pass/Fail
R1	User registers with valid details	Account is created and redirected to dashboard	Account successfully created	Pass
R2	User registers with existing email	Error message “Email already exists” is shown	Proper error message shown	Pass
R3	User registers with invalid email format	Validation prevents submission	Form blocks invalid email	Pass
R4	User logs in with correct credentials	User is logged in and redirected	Dashboard loads correctly	Pass
R5	User logs in with wrong password	Error message is displayed	Correct error displayed	Pass
R6	User attempts to access dashboard without login	Redirected to login page	Access denied and redirected	Pass

Creating an Order and Payment

Table 5 Creating an order and payment

ID	Test Scenario	Expected Result	Actual Result	Pass/Fail
O1	User adds item to cart	Cart updates with selected item	Cart shows correct quantity	Pass
O2	User updates quantity in cart	Cart total updates accordingly	Cart updates correctly	Pass
O3	User clears cart	Cart is emptied successfully	Cart shows as empty	Pass
O4	User proceeds to payment	Redirected to payment gateway	APS page loads correctly	Pass

O5	User completes payment successfully	Order is stored and confirmed	Order created and confirmation shown	Pass
O6	User cancels payment/ Payment failed	No order is saved and cart remains	No order ID generated	Pass

Admin and Management Features

Table 6 Admin and Management Features

ID	Test Scenario	Expected Result	Actual Result	Pass/Fail
A1	Admin logs in successfully	Access granted to admin dashboard	Dashboard loads	Pass
A2	Admin adds new employee	Employee appears in list	New employee record saved	Pass
A3	Admin edits employee	Updated values saved correctly	Info updated in database	Pass
A4	Admin deletes employee	Employee is soft deleted	Employee no longer shown	Pass
A5	Admin adds new menu item	Item appears in menu grid	Item saved and visible	Pass
A6	Admin edits menu item	Changes are reflected	Item updated correctly	Pass
A7	Admin deletes menu item	Menu item is soft deleted	Item removed from visible list	Pass
A8	Admin reorders menu	Menu is re-sorted visually	Drag-and-drop works correctly	Pass
A9	Admin exports employee CSV	CSV downloaded with full list	File exported correctly	Pass
A10	Admin views system logs	Logs displayed with filters	Logs accurate and searchable	Pass
A11	Admin uses search on menu	Matching items shown	Filter works correctly	Pass

A12	Admin searches in employee list	Filtered results displayed	Search returns matches	Pass
A13	Admin filters analytics by date	Revenue and order charts update	Charts refresh correctly	Pass
A14	Admin exports analytics CSV	File contains correct summaries	Exported successfully	Pass

Performance and Security

Table 7 Performance and Security table

ID	Test Scenario	Expected Result	Actual Result	Pass/Fail
S1	Attempt SQL Injection on login	Input is sanitized; query fails	Login blocked and input escaped	Pass
S2	Attempt SQL Injection on search bars	Special characters handled; no query error	No SQL execution; input ignored safely	Pass
S3	Try Cross-Site Scripting (XSS) via input	Script tags are sanitized and not executed	Page shows input as plain text	Pass
S4	Attempt CSRF on admin action (to delete employee)	CSRF token missing or invalid; action is blocked	Action denied without valid CSRF token	Pass
S5	Reuse expired session	User is redirected to login	Session expired and access denied	Pass
S6	Access admin page as user	Access is blocked	Redirected to login or access denied page	Pass

Acceptance Testing Process and Results

Acceptance testing ensures that the system meets the client's expectations and requirements before final delivery. It is conducted by representatives simulating real-world usage scenarios. For Bella Pizza, this phase involved evaluating the system's usability, feature completeness, and reliability under realistic conditions using predefined use cases derived from the functional requirements.

Table 8 Participant test process and result Table

Participant	Assigned Acceptance Testing Role	Result
Abdulla Sooud	Tested the complete customer journey, including user registration, cart updates, and completing an order using APS, and viewing orders.	Successfully completed all tasks. Ordering and payment worked as expected.
Ahmad Faisal	Evaluated all admin/employee features by logging in with various roles (admin, support, cashier) to manage menu, employees, system logs, and analytics.	All admin dashboard features worked correctly across different roles.
Yousif Abbas	Attempted security testing through CSRF, XSS, and SQL injection techniques to test application defences.	Application blocked all attacks; CSRF tokens and input sanitization proved effective.

Usability Testing results and statistics

General Feedback on use of system

Table 9 Participant Survey

Question	Abdulla Sooud	Ahmad Faisal	Yousif Abbas
Ease of Navigation (1–5)	5	5	5
Visual Appeal (1–5)	5	5	5
Task Completion Success (Yes/No)	Yes	Yes	Yes
Bugs Encountered (Yes/No)	No	No	No
Was the website slow or laggy? (Yes/No)	No	No	No
Overall Experience Rating (1–10)	10	9	8
Satisfaction with the Application (1–5)	5	4	4
Do you think the features you have tested are complete and deployment-ready? (Yes/No)	Yes	Yes	Yes

Graphs and Charts

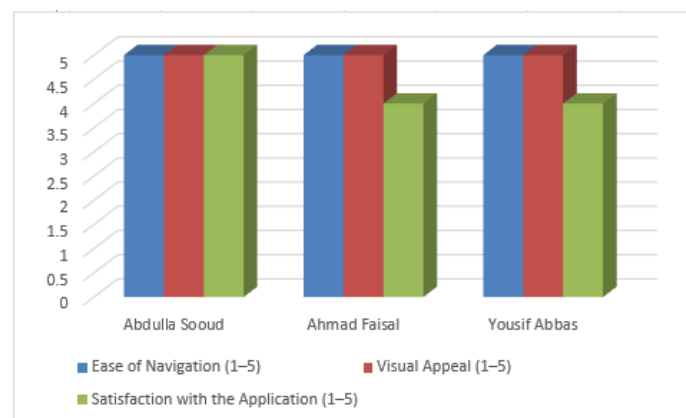


Figure 97 Ease of Navigation, Visual Appeal, and Satisfaction, Graph

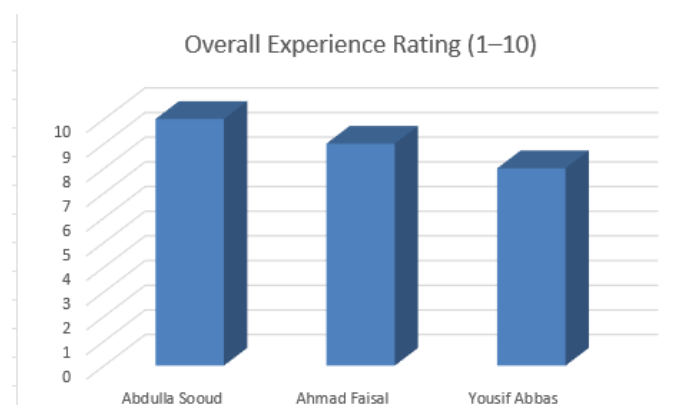


Figure 98 Overall Experience Graph

Final Review from Participants

Abdulla Sooud: “The Bella Pizza application offers a smooth and intuitive user experience. The interface is clean, professional, and easy to navigate, allowing customers to browse the menu, manage their cart, place orders, and keep track of orders with minimal effort. Features like clear visuals, responsive design, and organized categories make the process fast and user-friendly. Overall, the UI enhances usability and supports an efficient customer journey.”

Ahmad Faisal: “The admin and employee dashboard provides a complete overview of the business, making management tasks simple and efficient. The interface is well-organized, allowing staff to access employee records, menu items, orders, and categories with ease. Features like CSV exports, editable tables, and live search streamline operations. The business analytics and system logs offer valuable insights; technically through activity tracking and economically through revenue data, empowering better decision-making and operational control.”

Yousif Abbas: “The Bella Pizza web application incorporates multiple security measures to protect both users and system data. It uses CSRF tokens to prevent cross-site request forgery, input sanitization to guard against SQL injection, and output encoding to mitigate XSS (cross-site scripting) attacks. User sessions are securely managed, and only authorized roles can access sensitive features like employee management or system logs.”

Conclusion

Based on the feedback gathered through surveys and usability testing, all participants reported a fast, smooth and issue-free experience while interacting with the Bella Pizza web application. No bugs or malfunctions were encountered during their use of both the customer-facing and administrative interfaces. The participants praised the platform for its intuitive design, responsive layout, and comprehensive features. Furthermore, they expressed strong confidence in the system’s security measures, noting that the application feels well-protected and professionally developed. Overall, the results affirm that the application delivers a high-quality user experience while maintaining robust security standards.

Discussions, LESPI, Conclusion

System Functionality

The Bella Pizza web application successfully delivers all core functionalities as envisioned in the original system design. From a customer's perspective, the system allows users to browse categorized menu items, add items to a cart, adjust quantities, and complete their orders through a fully integrated payment gateway (Amazon Payment System - APS). The customer interface is clean, responsive, and easy to navigate, contributing to a smooth user journey.

On the administrative side, employees and managers can manage menu items, categories, employee records, customer orders, and delivery assignments. Key features like real-time order monitoring, drag-and-drop item reordering, availability toggles, and secure login access ensure both flexibility and control. System logs track every action, providing transparency and accountability. The analytics dashboard helps managers assess business performance using visual data like revenue trends, item sales, and filtered reporting.

Security features such as password hashing, CSRF protection, session hardening, and SQL injection/XSS prevention have been implemented and tested, further reinforcing the system's reliability. The completed platform operates as an end-to-end restaurant management solution tailored to both customer convenience and business efficiency.

Summary of Achieved Objectives

The Bella Pizza project set out to develop a complete web-based restaurant management system that serves both customers and business staff, with a strong focus on usability, performance, and security. The primary objectives included implementing:

- A customer-facing interface for browsing and ordering food online
- A dynamic cart and secure payment system
- An employee dashboard for managing menu items, orders, and users
- Role-based access control and system logging
- Visual business analytics and performance tracking
- Secure development practices to protect the system from common threats

All these objectives have been successfully met. Customers can create accounts, browse categorized menu items, and place orders using APS. The cart system supports quantity adjustment and clearing, and each transaction is securely handled with verification steps and order logging.

From the employee side, admins can add, update, and delete menu items, assign delivery drivers, and manage employee roles. Role-based access ensures that only authorized personnel can perform sensitive tasks. Logs are automatically generated for key actions, supporting accountability. The analytics section provides insight into revenue, order volume, and product performance, fulfilling the need for data-driven decision-making.

Table 10 Achieved Objectives

ID	Objective	Status	Description
1	Create a website for Bella Pizza where customers can place orders	Achieved	A complete website was built allowing users to browse the menu and order food online.
2	Develop a management system for admins and employees	Achieved	A dashboard was created for staff to manage orders, menu items, and other restaurant tasks.
3	Implement strong security features	Achieved	Security practices such as user authentication, session protection, and data validation were applied.
4	third-party applications	Achieved	The website allows Bella Pizza to handle orders directly without depending on external apps.
5	Build a modern, professional-looking website	Achieved	The website has a clean, user-friendly layout that reflects a professional restaurant brand.
6	Make the system easy to use for both customers and staff	Achieved	The design and structure were kept simple to make navigation and usage easy for all users.
7	Ensure the system supports key restaurant operations	Achieved	The system handles orders, deliveries, payments, and menu management to meet restaurant needs.
8	Provide tools for tracking business performance	Achieved	Analytics and reports are available to help managers understand sales and performance.
9	Test the system thoroughly to ensure it works as expected	Achieved	Multiple testing methods were applied to confirm the system functions correctly and reliably.
10	Significantly reduced order placement bottlenecking	Achieved	The restaurant's order placement system is not bottlenecked by lack of cashiers or phone numbers as the website is served simultaneously on multiple devices, it also manages orders simultaneously.

Project Issues (with Backup Plan/Proposed Solutions/Actions)

During the development of the Bella Pizza web application, several technical and design-related challenges arose, particularly in areas related to deployment, security, and user experience. These issues, while expected in any real-world IT project, provided valuable opportunities to improve the robustness and usability of the system. Each encountered issue was analysed, addressed with appropriate actions, and in some cases, led to permanent improvements in the codebase and project workflow. This section outlines the major problems faced, their underlying causes, and the solutions or backup strategies implemented to resolve them efficiently.

Table 11 Project issues and proposed solutions

Issue	Description	Proposed Solution	Justification / Benefits
Server Scalability	The current system runs on a single Dockerized Apache + MySQL stack, which may not handle high traffic or many concurrent users as the customer base grows.	Migrate backend services to a cloud platform like AWS, Azure, or GCP using services such as EC2 (for hosting), RDS (for database), and Elastic Load Balancer.	These platforms offer scalable infrastructure that automatically adjusts to traffic demand. This ensures minimal downtime, better response time, and eliminates the need to manually upgrade server resources.
Security Monitoring	While CSRF and XSS protection are implemented, there is no real-time threat monitoring or intrusion detection system.	Integrate security tools like ModSecurity, Fail2Ban, or use a Web Application Firewall (WAF) in the cloud.	These tools actively monitor and block suspicious activity, such as repeated login attempts or SQL injection patterns, reducing risk of attacks.
No Logging Centralization	Logs are written to local storage without centralized analysis. This can lead to delayed	Use ELK Stack (Elasticsearch, Logstash, Kibana) or Grafana with Loki for log centralization,	Centralized logs help developers and admins identify patterns, investigate bugs, and improve response time to

	responses during incidents.	search, and visualization.	incidents through better visibility.
Limited Payment Integration	Currently, the app only integrates with APS (Amazon Payment Services) in sandbox mode. No support for alternative or localized payment gateways.	Add support for multiple payment methods such as BenefitPay, PayPal.	More options increase conversion rate, improve user convenience, and reduce failed payments. Local methods like BenefitPay may be more trusted by Bahraini users.
Manual Restarts Process	The deployment still requires manual builds and restarts.	Set up CI/CD pipelines using GitHub Actions or GitLab CI to automate build, test, and deployment.	Saves developer time, reduces deployment errors, and ensures that new updates are delivered consistently and reliably.

Backup Plan

To ensure the continuity, reliability, and recoverability of the Bella Pizza system, a comprehensive backup plan has been designed. This strategy aims to protect essential data and configurations from accidental loss, corruption, or system failure. The backup plan focuses on three core elements of the system: the database, uploaded images, and SSL certificates; all of which are critical to system integrity and user trust.

Backing up data

- A) Database Backup Strategy: The MySQL database, which stores all customer accounts, menu items, order history, logs, and payment details, is backed up using Docker volume persistence. The volume dbdata is mounted to /var/lib/mysql, ensuring that database contents are retained even if the container is removed or restarted. The technician can manually run the script below to backup the database:

```
scripts > $ export_db.sh
1  #!/bin/bash
2
3  DATE=$(date +"%Y%m%d_%H%M%S")
4  FILENAME="RestaurantDB_${DATE}.sql"
5
6  docker exec bella_db mysqldump -u root -prootpass RestaurantDB > "$FILENAME"
7
8  if [ $? -eq 0 ]; then
9      echo "Backup successful: $FILENAME"
10 else
11     echo "Backup failed!"
12 fi
13
```

Figure 100 export database script

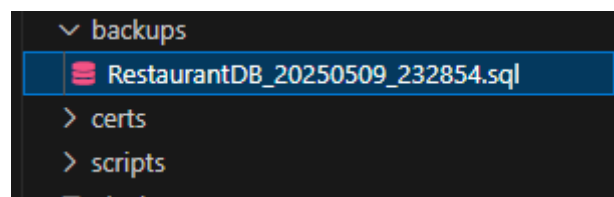


Figure 99 Backup successfully created

- B) File and Media Backup: the source code files and media images are stored in the Docker volume app/, mounted to the web server container at /var/www/html/. This ensures that data persists even during container rebuilds or system updates. To backup the data, the technician can use this script to create a zip file of the project directory:

```
scripts > $ backup_project.sh
1  #!/bin/bash
2
3  SOURCE_DIR="/home/bellapizza/bella-pizza-docker"
4  BACKUP_NAME="project_backup_$(date +%Y%m%d_%H%M%S).zip"
5  DEST_DIR="/home/bellapizza/bella-pizza-docker/backups/"
6
7
8  zip -r "$DEST_DIR/$BACKUP_NAME" "$SOURCE_DIR"
9
10
11 if [ $? -eq 0 ]; then
12 | echo "Backup created: $DEST_DIR/$BACKUP_NAME"
13 else
14 | echo "Backup failed!"
15 fi
16
```

Figure 101 script to create zip file of project directory

The best practice is to have as multiple backups stored in different areas.

Security and Access Control

Maintaining the server in a physically secure location is essential to protect the integrity, confidentiality, and availability of the Bella Pizza system. Unauthorized physical access to the server can result in data breaches, tampering, or even complete system compromise. By ensuring that only designated personnel; such as system administrators or trusted IT staff, are allowed to access the server environment, the organization significantly reduces the risk of accidental damage, theft, or malicious activity. Controlled physical access complements digital security measures by creating a layered defence strategy that safeguards critical infrastructure, sensitive customer information, and operational continuity.

Disaster Recovery Plan

In case of total system failure, recovery involves the following steps:

1. Re-deploy the Docker containers using docker-compose up.
2. Restore the database using the latest SQL dump.
3. Re-attach Docker volumes for images and SSL certs or restore them from backup.
4. Rebuild containers using the stored Dockerfile and configurations.

Version Control

Version control using GitHub plays a vital role in the development and maintenance of the Bella Pizza web application by enabling efficient code management, collaboration, and change tracking. GitHub serves as a remote repository that stores the project's source code, allowing the developer to commit, push, and track changes over time. This ensures that every modification is documented, reversible, and traceable, which is especially useful when debugging, testing new features, or rolling back to a previous state. Additionally, GitHub enhances collaboration by providing a centralized platform where updates can be reviewed, merged, and managed securely. By using GitHub, the Bella Pizza project benefits from improved development workflow, backup reliability, and transparency throughout its lifecycle.

Future Work (Upgrades – Modifications)

While the Bella Pizza system has successfully met its core objectives, there are several areas where future enhancements and refinements can be made to improve scalability, usability, security, and functionality. These improvements are suggested to accommodate future growth, enhance the user experience, and align with industry best practices.

1. Real-Time Order Notifications

Currently, order updates and status changes are refreshed on a timed polling basis. Future updates can introduce WebSocket-based real-time notifications for customers and employees.

- Benefit: Improves responsiveness and user experience by reducing waiting time and manual refreshes.

2. Mobile App Integration

Developing a native or hybrid mobile app for iOS and Android users would provide a more optimized and accessible platform for customers.

- Benefit: Expands reach and improves convenience, especially for customers who prefer ordering via mobile apps over web browsers.

3. Role-Based Access Control Panel (RBAC) Enhancements

While role-based access is implemented, it could be expanded with more granular permissions.

- Benefit: Allows more flexible delegation of responsibilities across different employee types, increasing control and security.

4. Enhanced Admin Analytics with AI Insights

The current analytics module shows essential statistics. Integrating machine learning can introduce sales forecasts, demand predictions, and customer behavior analysis.

- Benefit: Provides deeper business intelligence that helps with inventory, staffing, and marketing decisions.

5. Support for Multiple Branches

Future versions could support multi-branch management, allowing centralized control for Bella Pizza if it expands operations.

- Benefit: Scalability and centralization of operations and analytics across various physical locations.

6. Advanced Security Monitoring

In addition to existing CSRF/XSS protections, adding features like two-factor authentication (2FA), login attempt rate limiting, and behavioral anomaly detection would enhance security further.

- Benefit: Protects against evolving threats and increases user confidence in the platform.

8. Customer Support Chatbot

Integrating an AI-based chatbot or live support system for handling order queries or issues.

- Benefit: Enhances customer service and reduces response time for common questions.

9. Advanced Inventory Management

Adding backend features to automatically update ingredient stock levels based on orders and trigger restock alerts.

- Benefit: Helps in managing raw materials efficiently and reduces manual tracking.

Synopsis of My Experience

Working on the Bella Pizza project has been a transformative experience that allowed me to apply both my technical and project management skills in a real-world scenario. From designing the database schema to implementing complex features like secure login systems, role-based access control, and dynamic dashboards, I was able to gain hands-on experience in full-stack web development using PHP, MySQL, and Docker.

Throughout the project, I encountered several challenges — such as optimizing system performance, ensuring secure payment integration, and maintaining code modularity — but these hurdles significantly strengthened my problem-solving abilities. Debugging issues, handling SQL errors, and applying security measures like CSRF and XSS prevention gave me a deeper understanding of secure coding practices.

I also learned the importance of UI/UX in user retention and business impact. Designing separate interfaces for customers and employees, and receiving usability feedback, taught me how thoughtful interface design can improve usability and satisfaction.

Beyond technical skills, I developed better time management, testing discipline, and communication — especially when documenting features and addressing feedback. This project has equipped me with valuable experience that will be directly applicable in future academic and professional settings.

Bahraini Perspectives

The Bella Pizza project reflects the growing need in Bahrain for digital transformation in local businesses, particularly in the food and beverage sector. In a society where dining and food delivery are an essential part of daily life, this project aims to support and empower small to mid-sized restaurants by providing them with an affordable and customized digital ordering system.

Culturally, Bahrain has a strong preference for fast and convenient service. Many customers still rely on phone calls, Instagram pages or third-party delivery apps, which can be inefficient or costly. By developing a dedicated ordering website, Bella Pizza can reduce its dependence on external platforms, enhance direct customer engagement, and retain full control over its branding and service quality. This aligns well with the national push towards economic diversification and technological self-sufficiency under Bahrain's Vision 2030.

Moreover, the application respects local values by being simple, intuitive, and available in both English and (potentially in future updates) Arabic, which is important for accessibility among a diverse user base. The secure architecture and role-based dashboards reflect the increasing awareness of data privacy and cybersecurity within Bahraini institutions.

From an employment perspective, the admin and employee dashboards encourage transparency and responsibility, supporting better internal management — an area of growing interest for SMEs in Bahrain.

In summary, while the system introduces changes in how restaurant services are accessed and managed, it respects and uplifts Bahraini cultural values by empowering local businesses, preserving community ties through local branding, and promoting innovation in a culturally sensitive manner.

Legal, Ethical, Social, and Professional Issues

The Bella Pizza system, while innovative and practical, must navigate several important legal, ethical, social, and professional considerations to ensure it is safe, fair, and responsibly managed.

Legal Issues

The website handles sensitive customer data such as personal information, addresses, and payment-related identifiers. As such, it must comply with local and international data protection regulations, such as Bahrain's Personal Data Protection Law (PDPL). Failure to implement secure data storage, enforce user consent, or provide clear privacy policies could lead to legal consequences, including fines or bans.

Additionally, the integration with a third-party payment gateway (APS) places legal responsibility on the system to ensure encrypted, authenticated transactions. Any negligence in handling or validating payment data could lead to financial fraud and liability.

Ethical Issues

Ethically, the system must ensure that all users are treated fairly, regardless of their digital proficiency or background. For example, the UI should be accessible to users with disabilities (e.g., screen reader compatibility) and not designed in a way that manipulates customers (e.g., hidden charges or misleading discounts).

The use of admin logs raises ethical concerns as well. While it provides transparency, logging user behavior must be disclosed clearly and used only for legitimate system monitoring — not surveillance or employee micromanagement.

Ethical judgment is also required when handling employee data, including salary and role management, ensuring such details are protected from misuse or internal leaks.

Social Issues

Socially, the system contributes to the shift toward digital commerce in Bahrain. While it improves convenience and accessibility, it may reduce traditional social interactions between customers and restaurant staff. This change can affect the cultural fabric of service industries, potentially diminishing the sense of hospitality that is traditionally strong in Bahraini culture.

The system may also unintentionally exclude people who are less digitally literate, especially older customers or people with limited internet access, which may deepen the digital divide in the community.

Professional Issues

From a professional standpoint, maintaining system security, data integrity, and uptime reflects directly on the credibility of the developer. Poorly written code, lack of security updates, or ignoring vulnerabilities (e.g., XSS, CSRF, SQL injection) could harm not just the system but also the developer's reputation.

Furthermore, ICT professionals are expected to uphold principles like transparency, confidentiality, and reliability. Any form of plagiarism, undocumented third-party code usage, or intentional backdoors in the system would be serious violations of professional conduct.

Conclusion

The development of the Bella Pizza system has led to meaningful insights into building a secure, full-featured restaurant management and customer ordering platform.

Throughout the project, I gained practical experience in backend development, database design, modern UI/UX principles, and deploying applications securely using containerized infrastructure.

The project has highlighted the importance of user-centered design, especially in creating a seamless experience for both customers and staff. By integrating functionalities such as real-time menu management, automated order logging, secure payments, and analytics dashboards, the system demonstrates how digital solutions can effectively streamline restaurant operations and improve service quality.

This work also reinforces the significance of security in web applications, particularly for systems handling personal and financial data. Implementing protections such as CSRF tokens, hashed passwords, secure sessions, and HTTPS ensures both legal compliance and user trust.

However, this system also raises new questions for further exploration. For example, how can we scale this system to support high traffic in a production environment? How can we optimize real-time delivery tracking or integrate with mobile apps more effectively? These areas represent future development directions that could enhance the system's capabilities and commercial viability.

Practically, the Bella Pizza platform has immediate real-world applicability and can serve as a strong foundation for similar small-to-medium enterprise systems. Its modular structure, scalability, and clear UI make it adaptable for other industries as well.

In closing, this project reflects a complete development cycle — from planning and implementation to testing and deployment. It not only fulfilled its technical objectives but also offered deep educational value, reinforcing critical skills in system design, testing, user engagement, and secure development. It serves as a strong stepping stone for future innovations in digital service platforms.

Appendices

Appendix I: System and User Manuals

Setup Manual

To setup the environment for use please follow these instructions:

- 1) Open your search bar and type “features”, then click on “turn windows features on or off”

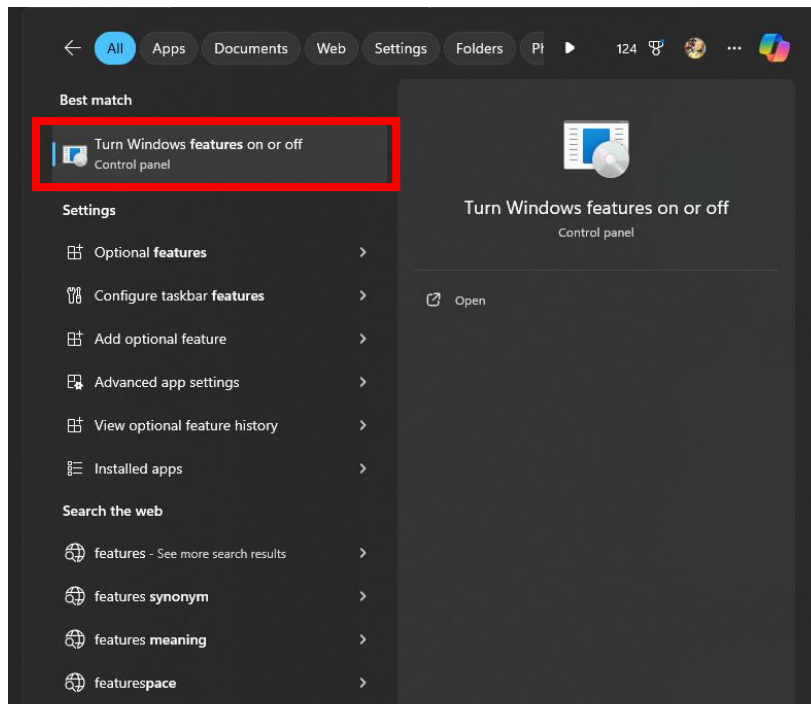


Figure 102 turn windows features on or off

- 2) Scroll down until you find “ Windows Subsystem for Linux, and make sure it is check marked

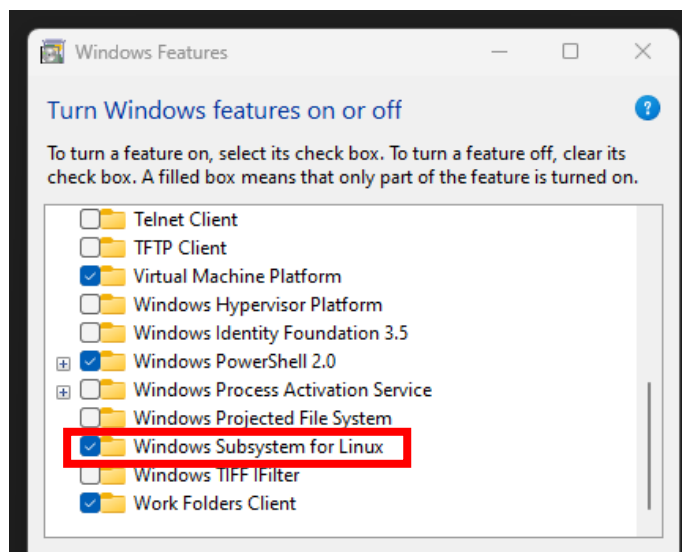


Figure 103 check Windows Subsystem for Linux

3) Install Ubuntu from the Microsoft store

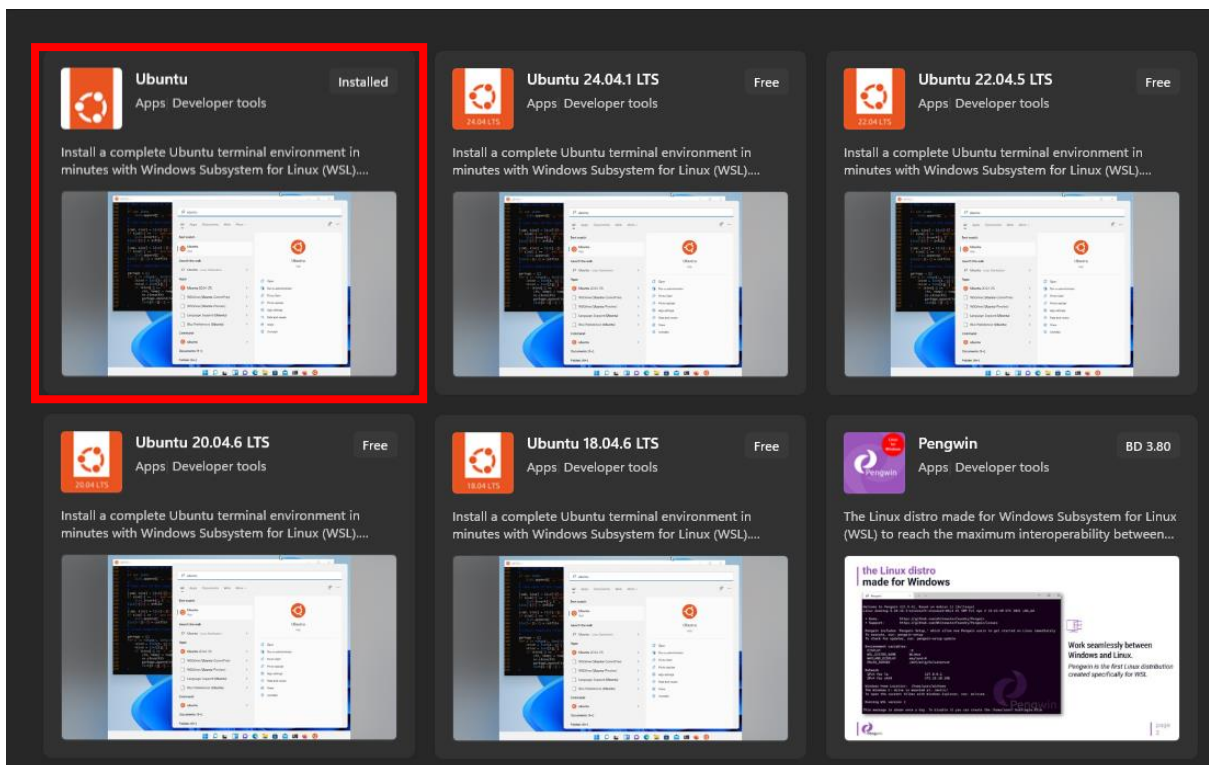


Figure 104 Ubuntu in the microsoft store

- 4) Once installed it should automatically popup, or else manually open the app. It will prompt you to create a username and then a password. Follow their instructions.

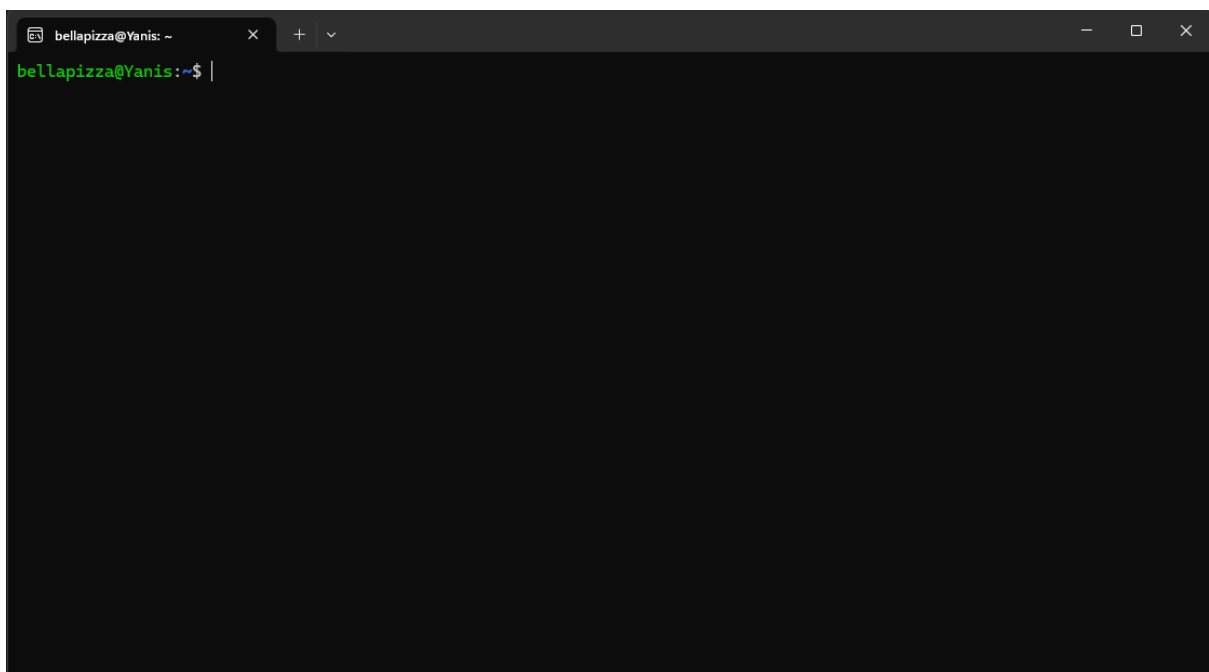
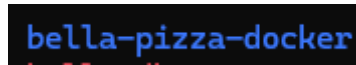


Figure 105 Ubuntu running after setup on windows 11

- 5) In the home directory create a directory called “bella-pizza-docker” using the command “mkdir bella-pizza-docker”, now when you type the command “ls”, you should see this:



```
bella-pizza-docker
```

Figure 106 bella pizza directory

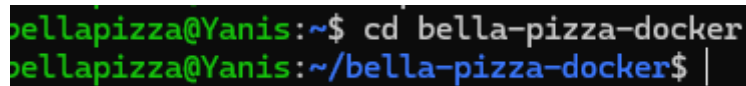
- 6) Go to this Link: https://drive.google.com/drive/folders/1EJ-T0Wvulg8kNOvEkkJtDB_oheYk_F20?usp=sharing

Download the “bella-pizza-docker” folder

Name	Owner	Last mo...	File size	
 bella-pizza-docker	 me	5:12 PM	—	

Figure 107 bella-pizza-docker folder

- 7) Now go back to ubuntu, type “cd bella-pizza-docker”, you should now be in the directory



```
bellapizza@Yanis:~$ cd bella-pizza-docker
bellapizza@Yanis:~/bella-pizza-docker$ |
```

Figure 108 bella-pizza-docker directory

- 8) Type “explorer.exe”. Windows should open a new explorer page of the directory. Copy all the contents from the downloaded folder (**not the folder itself**) into the folder. It should look like this:

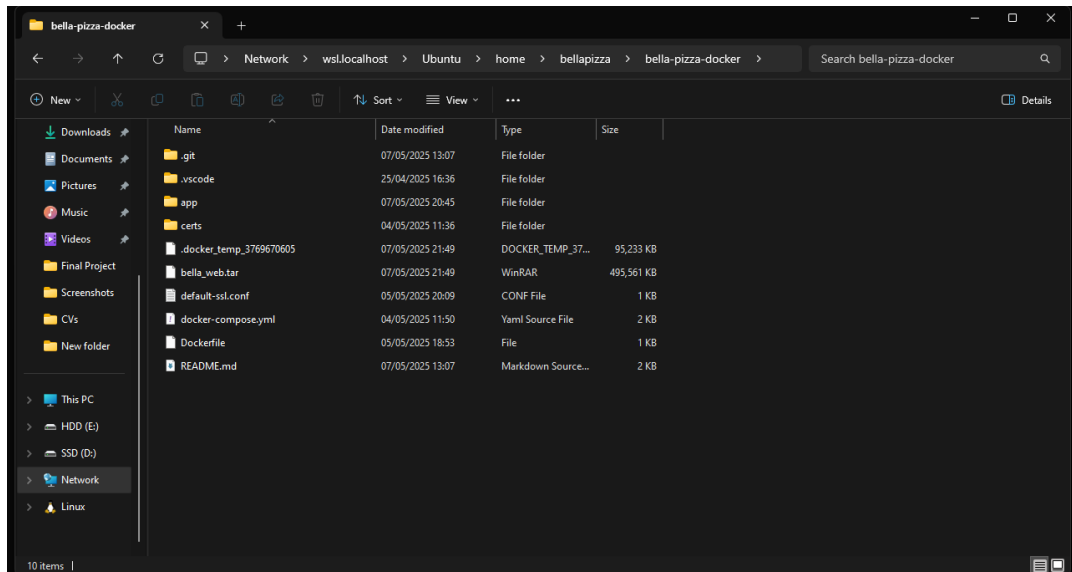


Figure 109 Copied content into directory

- 9) Now download Docker from here: <https://docs.docker.com/get-started/introduction/get-docker-desktop/>

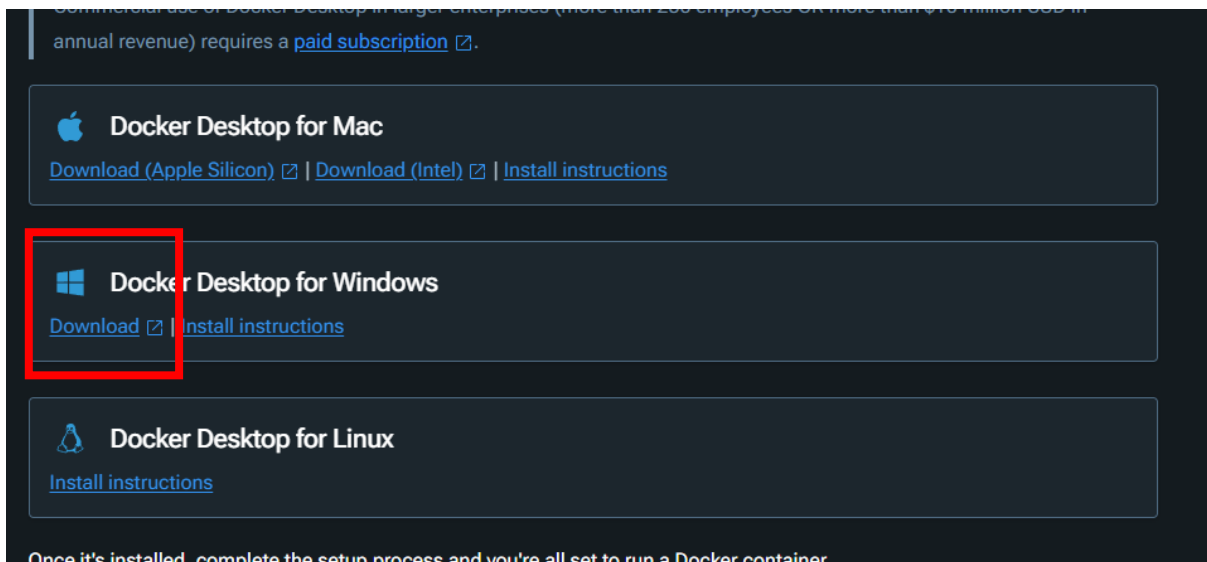


Figure 110 Downloading docker desktop

- 10) Follow the installation steps until you are finished, then click on the settings (the cog on the top right of the app, then resources tab, then the WSL subtab and make sure “enable integration with my default WSL distro is checked and that Ubuntu is enabled.

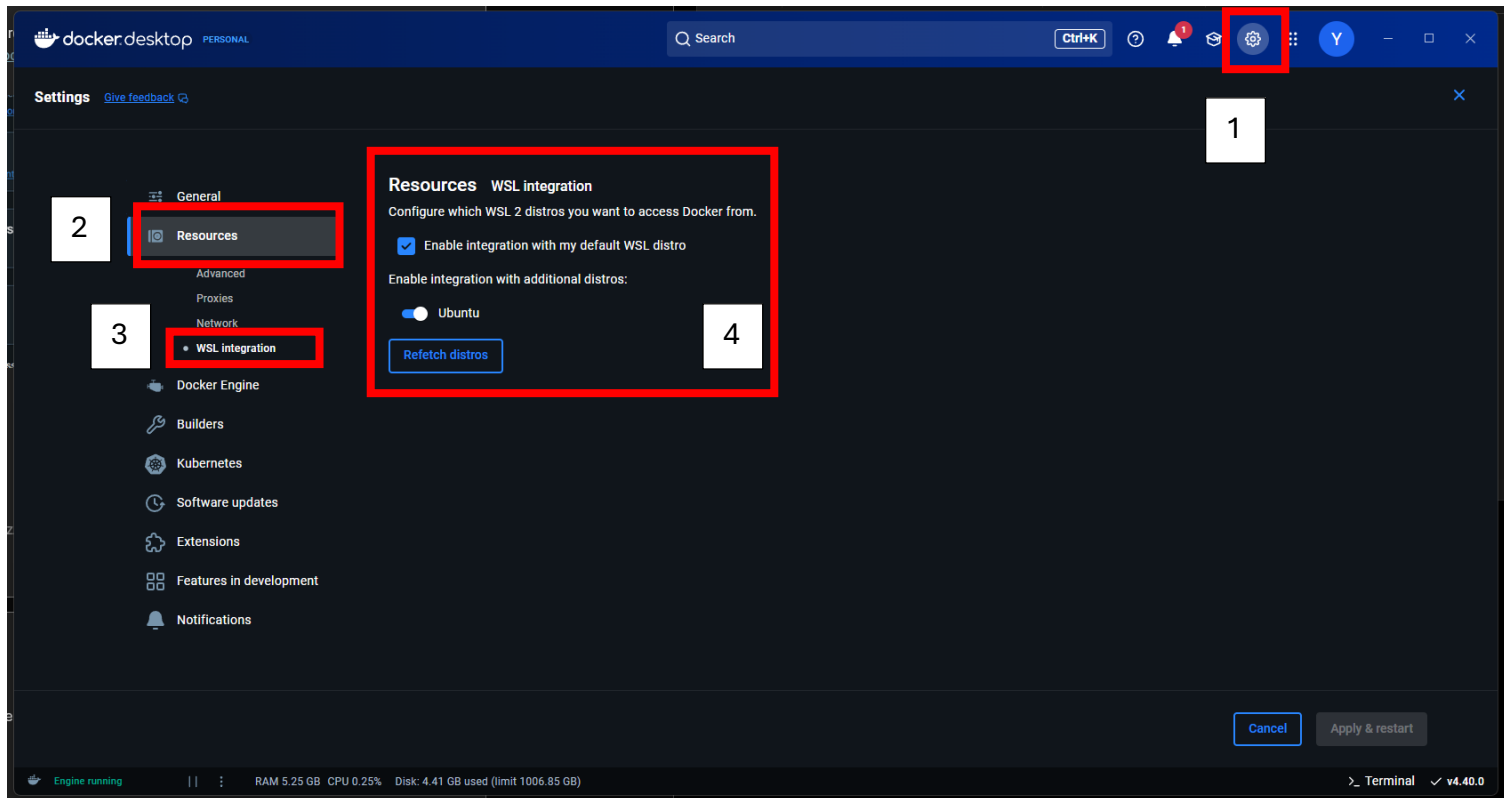


Figure 111 WSL integration

- 11) Press on apply and restart and wait for Docker to restart.

12) After restarting, go back to Ubuntu, and type the command “docker-compose up”. This process will take some time, as it is creating the containers. Once it is done go back to docker desktop, you should see these new containers:

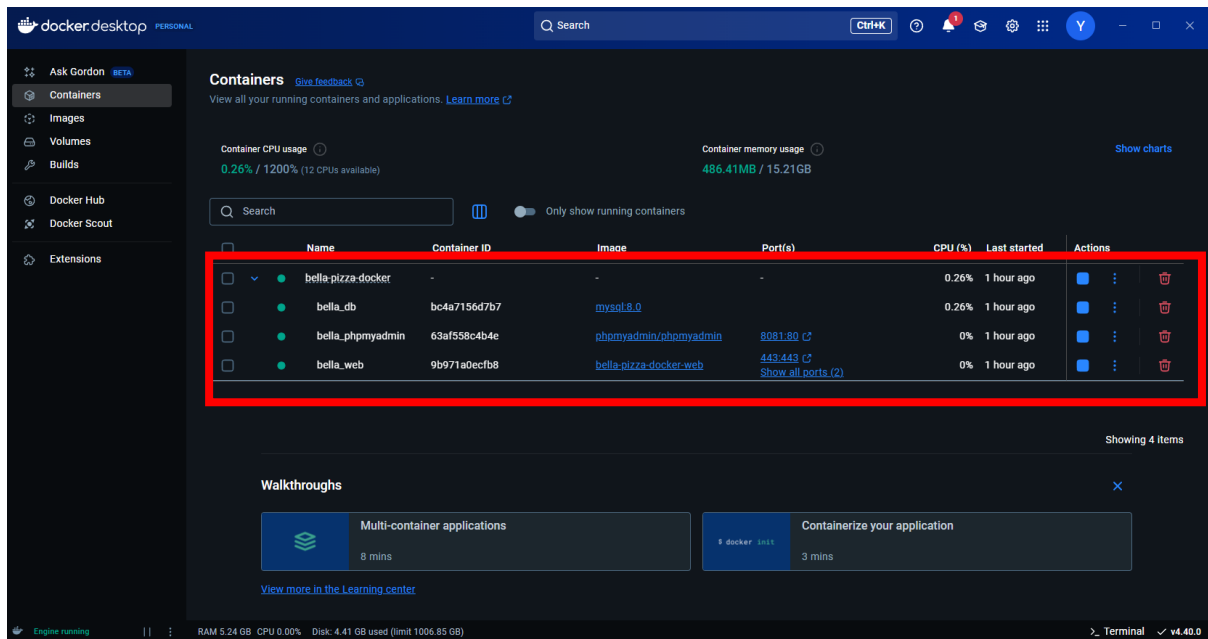


Figure 112 New Containers Created

13) you can now access the website on:

<http://localhost:80/>

or

<https://localhost/>

PHPmyAdmin (Database):

<http://localhost:8081/index.php?route=/>

(Username: root Password: rootpass)

- 14) To host the website on your pc, you need to port forward using a router and ethernet cable, in this case, we will be using Batelco's Huawei Router. First, connect the your pc to the router using the ethernet cable.
- 15) Now enter your router's interface using with this: <http://192.168.100.1/>



Figure 113 Router Interface

16) First open cmd and type “ipconfig” to find out your pc’s ip address:

```
Command Prompt
C:\Users\yanis>ipconfig

Windows IP Configuration

Ethernet adapter vEthernet (WSL (Hyper-V firewall)):

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::3215:5922:1366:f85a%18
    IPv4 Address. . . . . : 172.21.240.1
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : 2001:1a40:1424:5a00::1
    IPv6 Address. . . . . : 2001:1a40:1424:5a00:1395:222a:ed9a:1fb0
    Temporary IPv6 Address. . . . . : 2001:1a40:1424:5a00:14e1:f80e:6977:8d80
    Temporary IPv6 Address. . . . . : 2001:1a40:1424:5a00:2c4d:a33a:2cc7:ee09
    Temporary IPv6 Address. . . . . : 2001:1a40:1424:5a00:fc46:588c:da75:72c7
    Link-local IPv6 Address . . . . . : fe80::d7f4:7a50:a5d3:e540%9
    IPv4 Address. . . . . : 192.168.100.3
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::1%9
                                192.168.100.1

C:\Users\yanis>
```

17) Login, go to “Forward rules” tab, then the “port Mapping configuration” tab and click on “new”

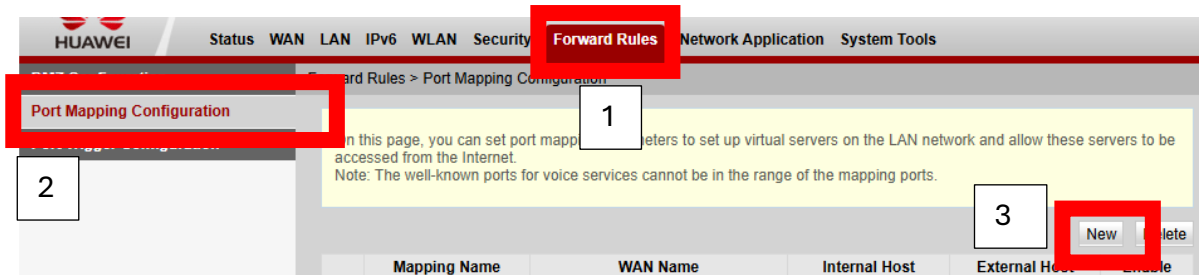


Figure 114 new port map configuration

- 18) Make sure enable port mapping is checked, and that the internal host's ip address matches and that external port and internal port number is set to 80. Then click apply.

Type:	☒ User-defined ☐ Application		
Application:	Select...		
Enable Port Mapping:	☒		
Mapping Name:	Bella Pizza		
WAN Name:	1_INTERNET_R_VI[		
Internal Host:	192.168.100.3	*	Yanis
External Source IP Address:			
Protocol:	TCP	Internal port number:	80 - 80 *
External port number:	80 - 80	External source port number:	
Delete			

- 19) Now go to your web browser and retrieve your public IP address from this link :
<https://whatismyipaddress.com/>

My IP Address is:

IPv6: ? [2001:1a40:1424:5a00:fc46:588c:da75:72c7](#)

IPv4: ? [89.148.45.69](#)

My IP Information:

ISP: Bahrain
Telcommunication Company
City: Madinat 'Isa
Region: Ash Shamaliyah
Country: Bahrain

Your location may be exposed!

[Show Complete IP Details](#)

Location not accurate?
[Update My IP Location](#)

Persian Gulf
الخليج العربي

Click for more details about
2001:1a40:1424:5a00:fc46:588c:da75:72c7

Bahrain
البحرين

Qatar
قطر

Al-Hofuf
الهوفوف

© OpenStreetMap

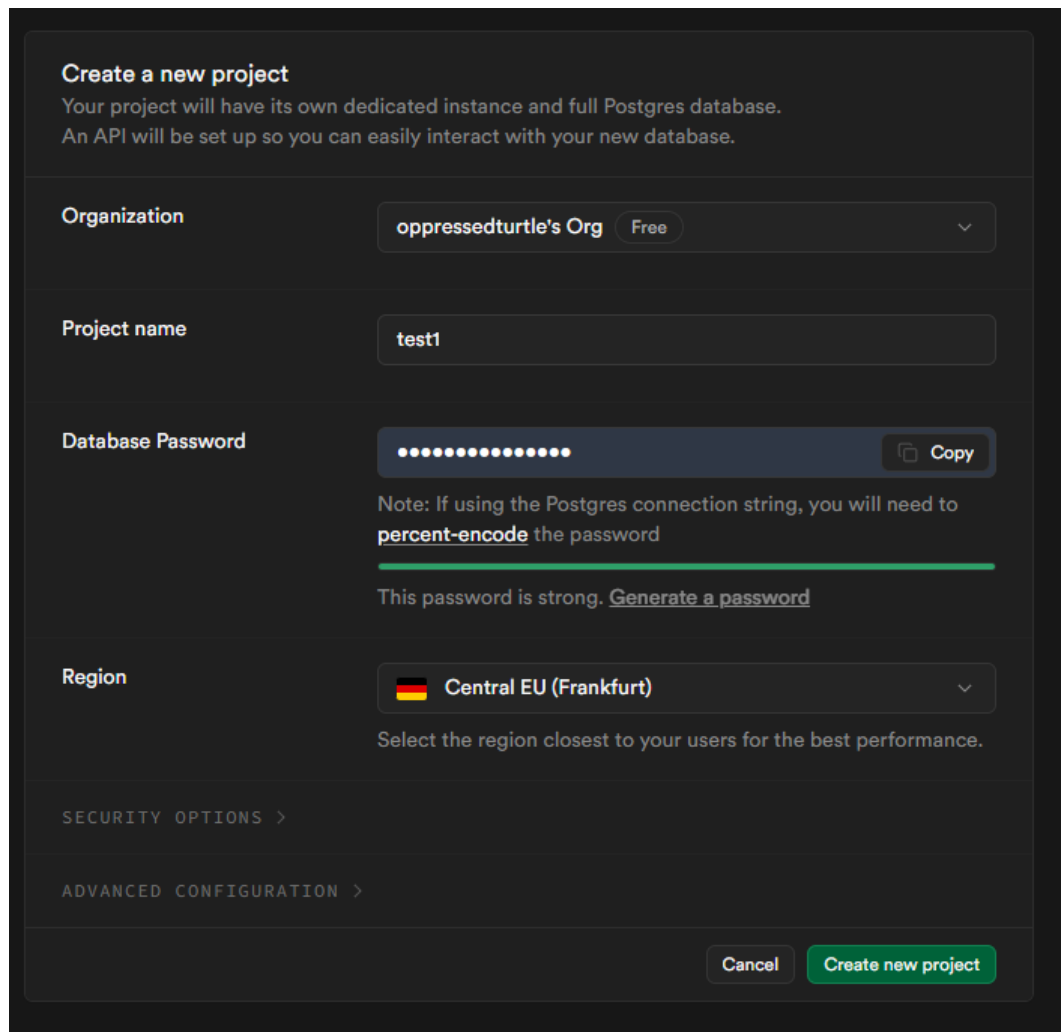
Bella Pizza is now public and can be accessed through

[https://\[Enter-Your-Public-IP\]](https://[Enter-Your-Public-IP])

or

[http://\[Enter-Your-Public-IP\]:8080/](http://[Enter-Your-Public-IP]:8080/)

20) To configure google authentication create and account at Supabase.com and create a project.



The image shows the 'Create a new project' form on the Supabase website. The form is dark-themed and contains the following fields and options:

- Create a new project**: Header text with a sub-note: 'Your project will have its own dedicated instance and full Postgres database. An API will be set up so you can easily interact with your new database.'
- Organization**: A dropdown menu showing 'oppressedturtle's Org' with a 'Free' tag and a downward arrow.
- Project name**: A text input field containing 'test1'.
- Database Password**: A password input field with masked characters (dots). To the right is a 'Copy' button with a clipboard icon.
- Note**: A text note stating: 'Note: If using the Postgres connection string, you will need to **percent-encode** the password'.
- Password Strength**: A green progress bar and text indicating 'This password is strong.' with a link to 'Generate a password'.
- Region**: A dropdown menu showing 'Central EU (Frankfurt)' with a German flag icon and a downward arrow.
- Region Note**: Text below the region dropdown: 'Select the region closest to your users for the best performance.'
- SECURITY OPTIONS >**: A link to expand security options.
- ADVANCED CONFIGURATION >**: A link to expand advanced configuration.
- Buttons**: At the bottom right, there are two buttons: 'Cancel' and 'Create new project' (highlighted in green).

Figure 115 Creating project at supabase

21) Go to the authentication Tab and then Sign in/ Providers Tab

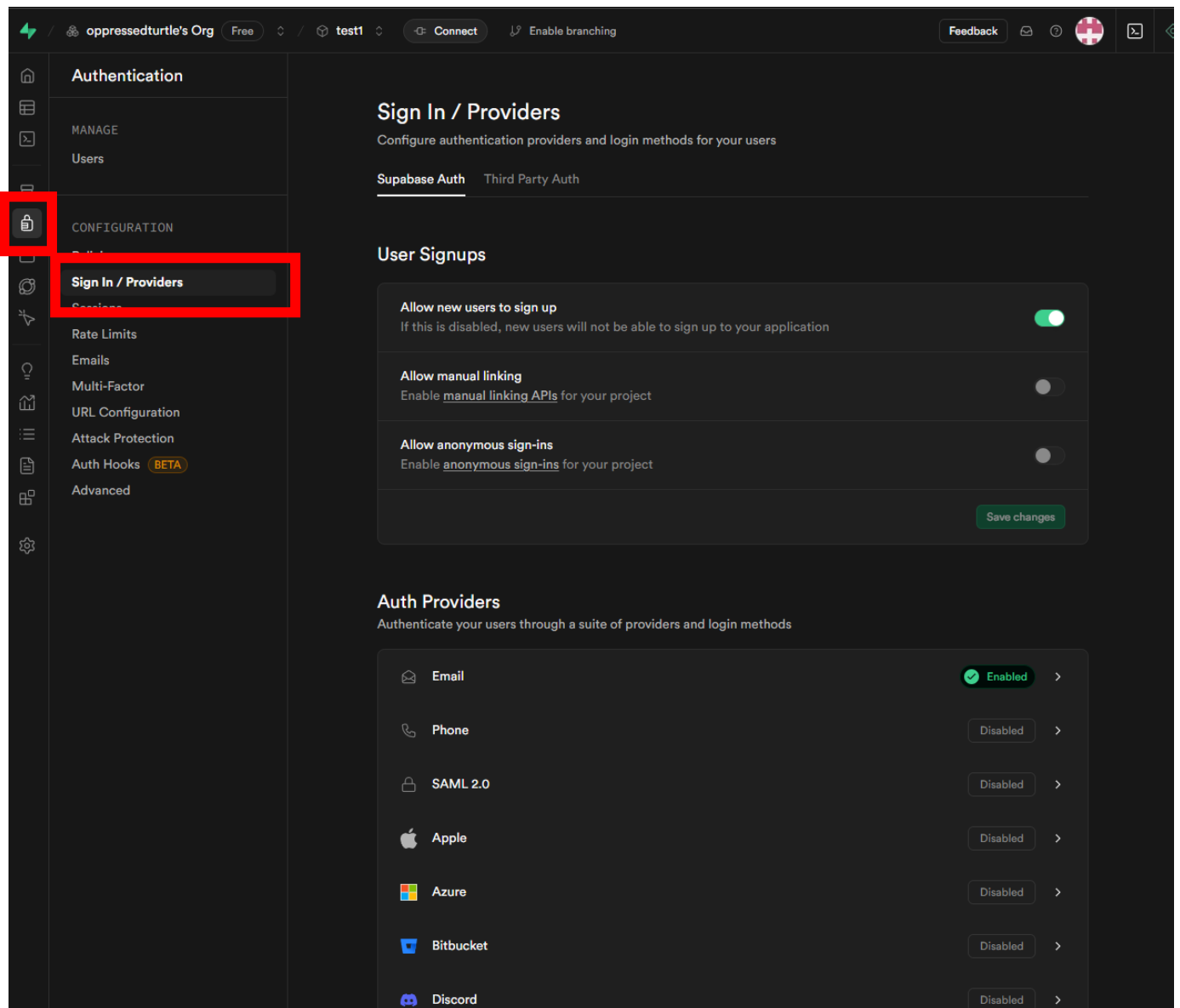


Figure 116 Sign in providers

22) Scroll down to google and enable it. Copy the callback URL somewhere to use later

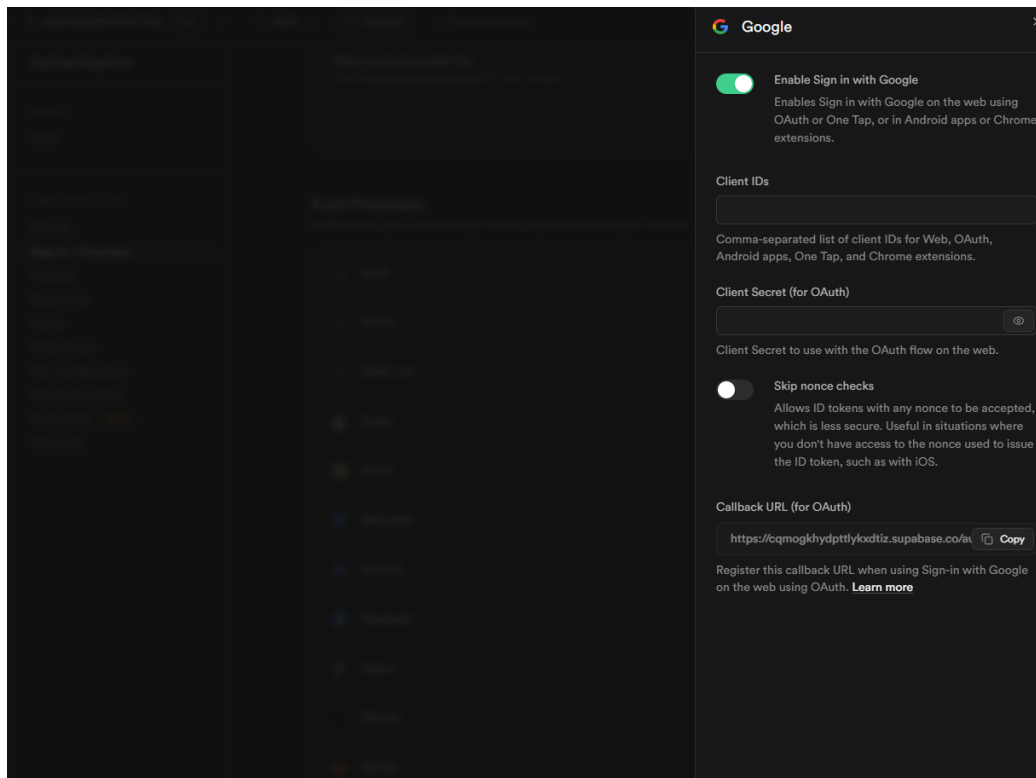


Figure 117 enable google sign in

23) Now open a new tab, go to google cloud, create an account or login to one.

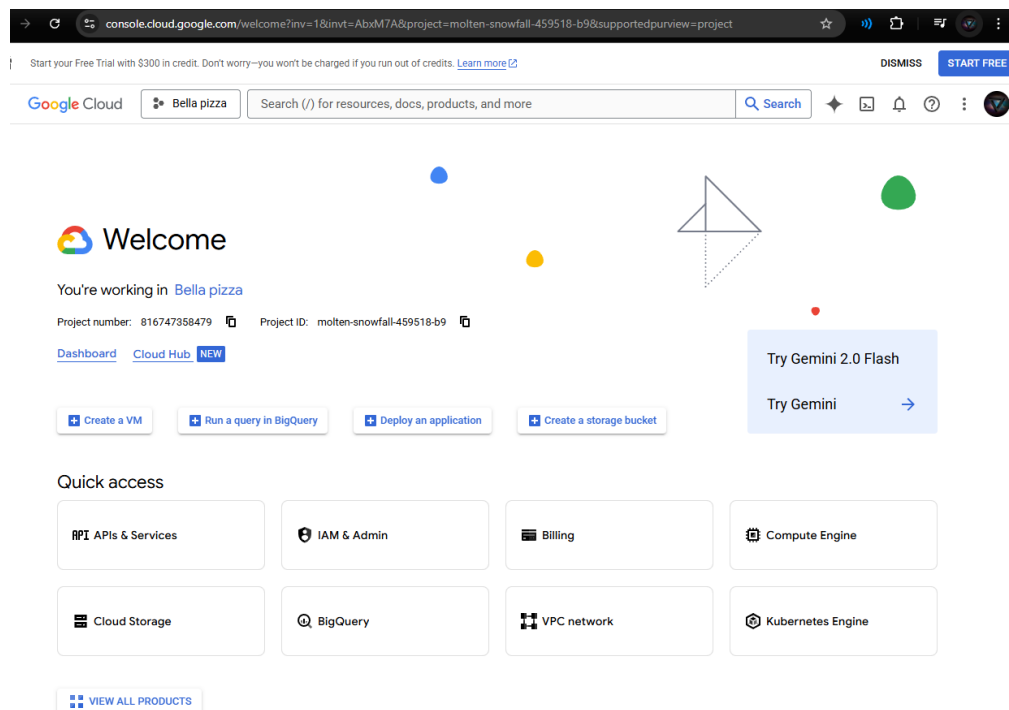


Figure 118 google cloud Page

24) Go to APIs and Services into OAuth consent screen

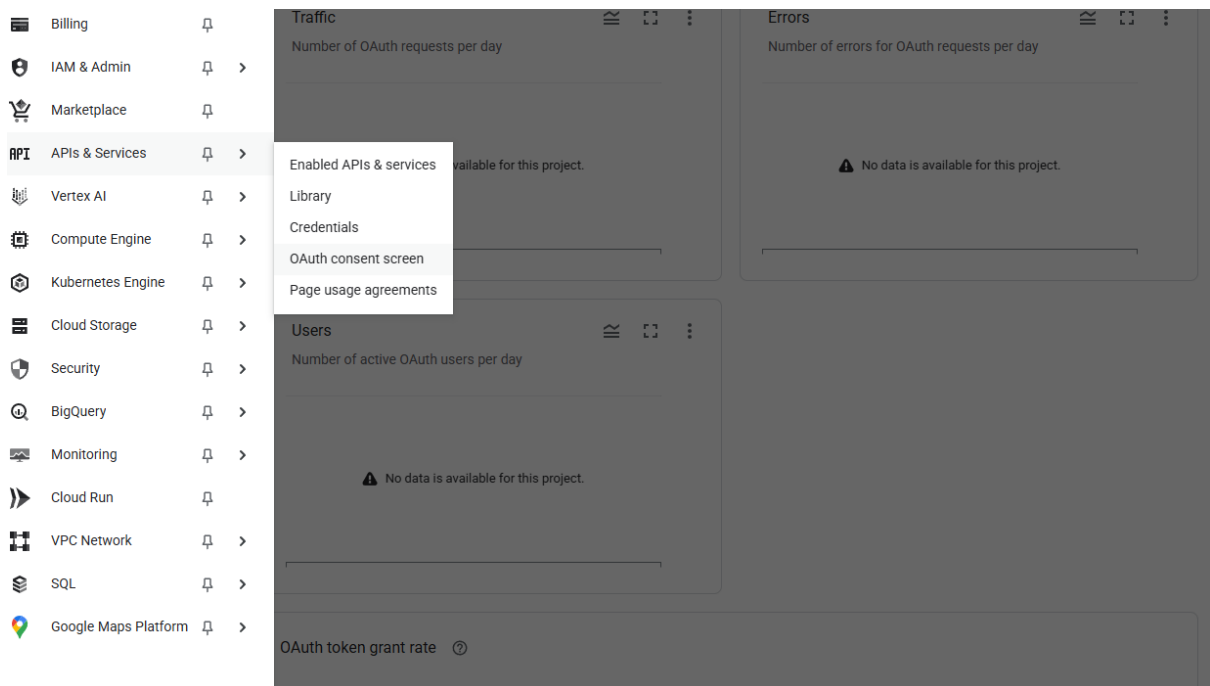


Figure 119 moving to Oauth page

25) Go to the clients tab and click on “create new client”

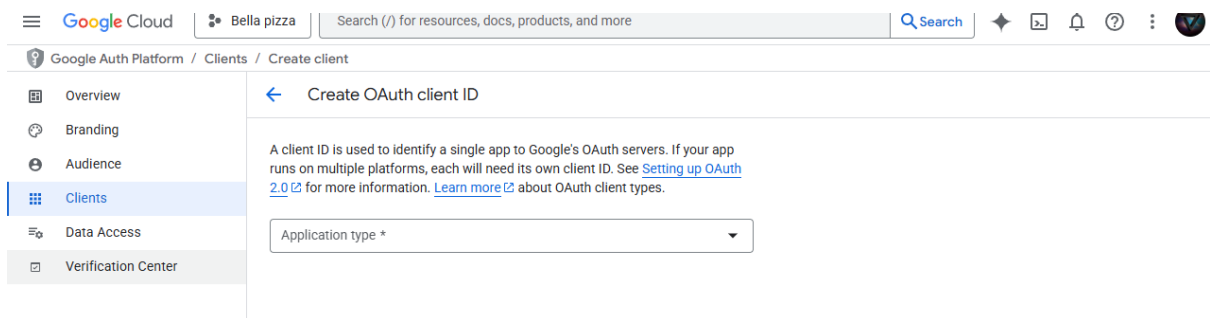


Figure 120create OAuth client

27) Configure the page as such

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Web application

Name *

bella pizza

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins ?

For use with requests from a browser

URIs 1 *

insert callback url here remove /auth/v1/callback

Invalid Origin: cannot contain whitespace.

URIs 2 *

http://localhost:80

+ Add URI

Authorized redirect URIs ?

For use with requests from a web server

URIs 1 *

Insert callback url here

Invalid Redirect: cannot contain whitespace.

URIs 2 *

https://localhost:80/auth/callback

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Create

Cancel

Figure 121Configuring OAuth Client

28) Once created, copy the client ID and Client Secret and go back to the Supabase page and paste them there.

OAuth 2.0 Client IDs

☐	Name	Creation date ↓	Type	Client ID	Actions
☐	bella pizza	May 11, 2025	Web application	816747358479-phd4...	

Client: 816747358479-phd4fep1j2u049ll7a9s9q0utl4kgmpb.apps.googleusercontent.com

You can now search for documentation, resource metadata, tutorials, and API keys

← Client

Name *

bella pizza

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

1

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins

For use with requests from a browser

URIs 1 *

https://fpvkzssoudxcsbejxvow.supabase.co

URIs 2 *

https://localhost:80

+ Add URI

Authorized redirect URIs

For use with requests from a web server

URIs 1 *

https://fpvkzssoudxcsbejxvow.supabase.co/auth/v1/callback

URIs 2 *

https://localhost:80/auth/callback

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Save

Cancel

Additional information

Client ID

816747358479-phd4fep1j2u049ll7a9s9q0utl4kgmpb.apps.googleusercontent.com

Creation date

May 11, 2025 at 10:01:20 PM GMT+3

Client secrets

If you are in the process of changing client secrets, you can manually rotate them without downtime.

[Learn more](#)

Client secret

WfFZtOI

Creation date

May 11, 2025 at 10:01:20 PM GMT+3

Status

Enabled

+ Add secret

Figure 122Getting Client ID and Client Secret

© 2025 || Bella Pizza || 202101314 || Mohamed Yanis Moussai

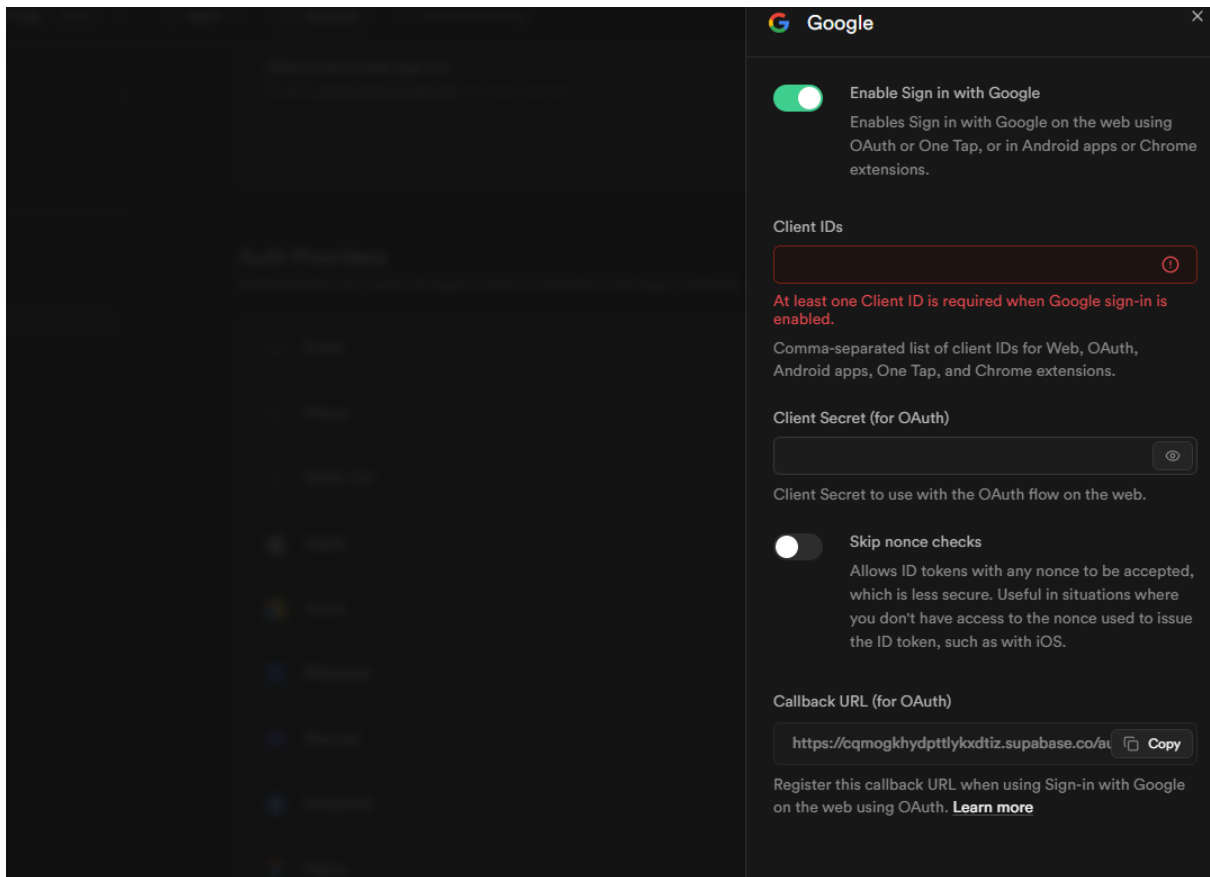


Figure 123 Insert client ID and secret here

Customer Manual

This manual covers all the features A customer can use.

- 1) In the index page click on “Login”

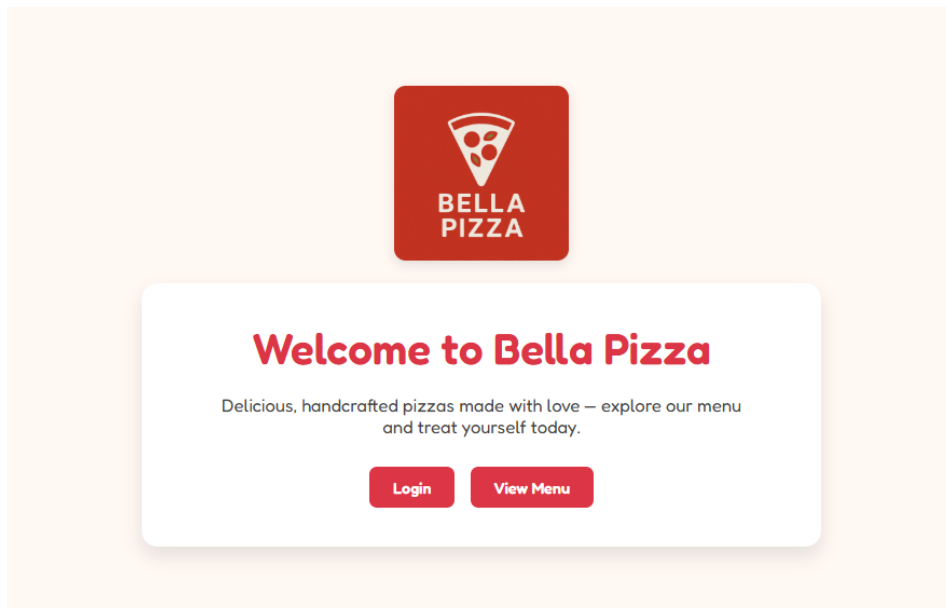


Figure 124 Index page

- 2) Once redirected create a new user by filling the form properly

Register

Username

Email

Password

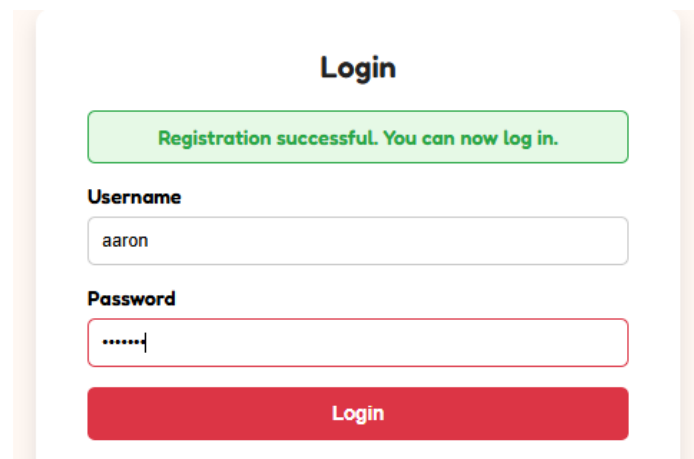
Phone Number

Address

Register

Figure 125 Registering A new User

- 3) After the successful registration message, proceed to login



The login form is titled "Login" and is centered on the page. It features a green success message at the top: "Registration successful. You can now log in." Below this, there are two input fields: "Username" with the text "aaron" and "Password" with masked characters ".....". At the bottom of the form is a red "Login" button.

Figure 126 Log in with new account

- 4) After being redirected to the dashboard, pick some items to choose from the menu.

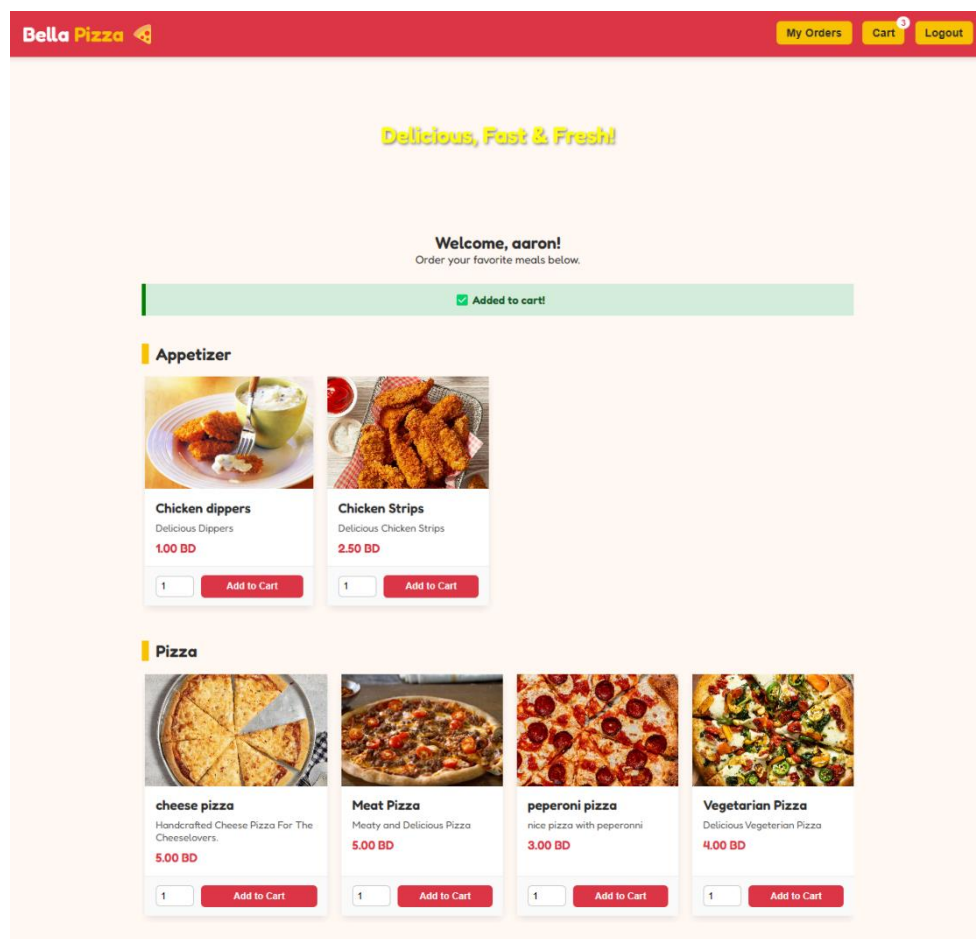


Figure 127 Adding items to cart

- 5) Once you are finished, click on the cart button, so you can review your order. You may clear the cart or edit quantities.

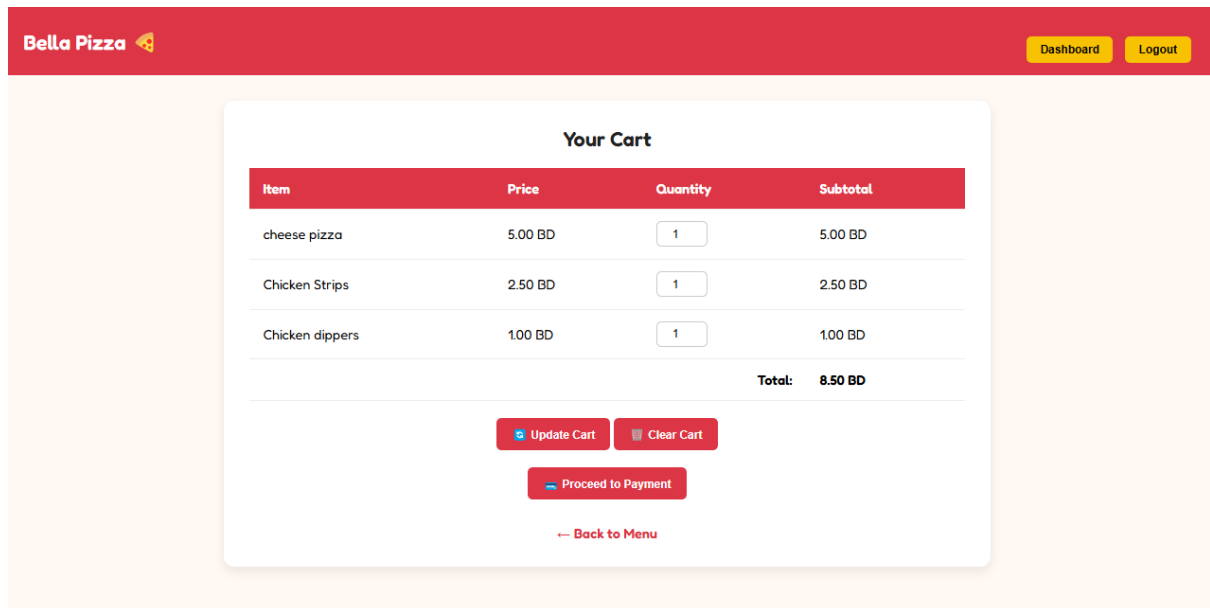


Figure 128 Cart view

- 6) Once pleased with order, click on “Proceed to Payment” you will be redirected to APS for to enter credit card details.

For successful payments enter this data:

Card Number: 4005550000000001

Expiry Date: 05/25

CVV: 123

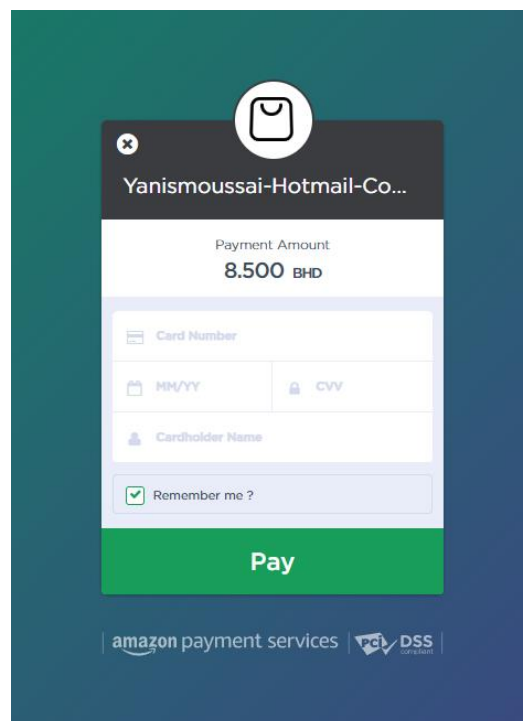


Figure 129 APS view

- 7) After successful or failed payment you will be redirected back to bella pizza where you will be met with a server response based on the status of payment. if payment is successful your order will be placed and you will receive an email. If not, you will be redirected to the dashboard and you cart remain.

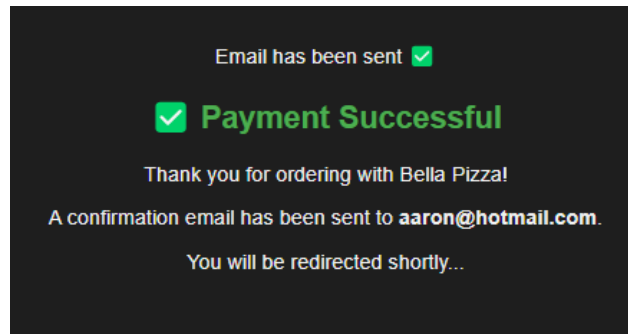


Figure 130 server response

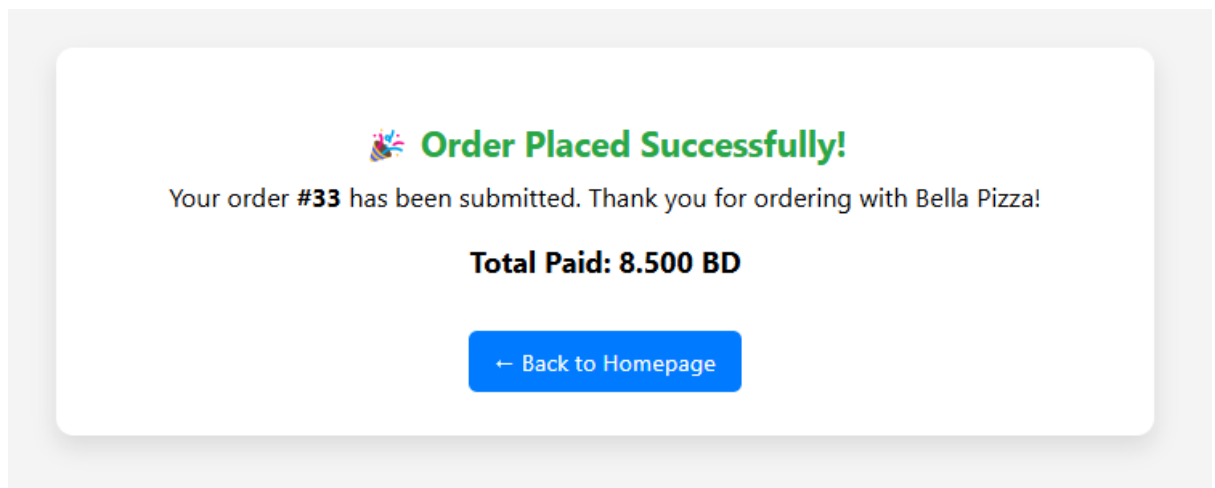


Figure 131 order placement upon successful payment response

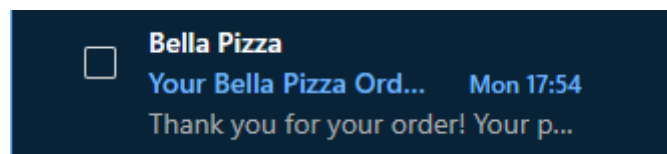


Figure 132 Received Confirmation Email

- 8) If you want to check up on your order status go to the “my orders” tab in the dashboard. In “my orders” you can search for past and current orders based on ID Status or date in yyyy-mm-dd format. You may also view in depth order details.

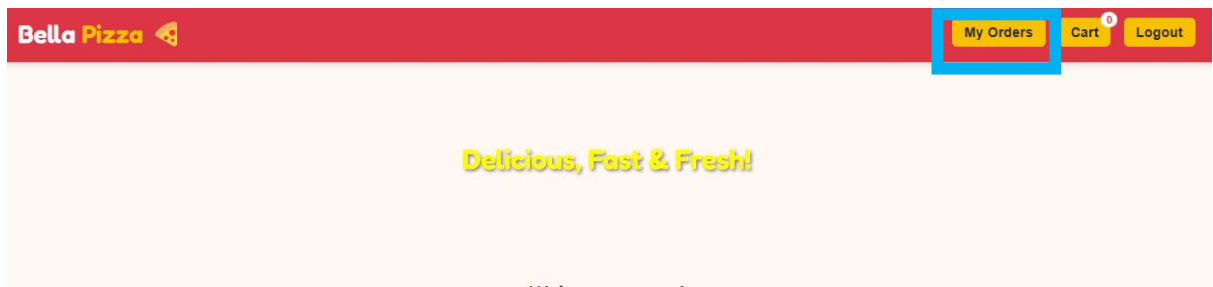


Figure 133my orders button

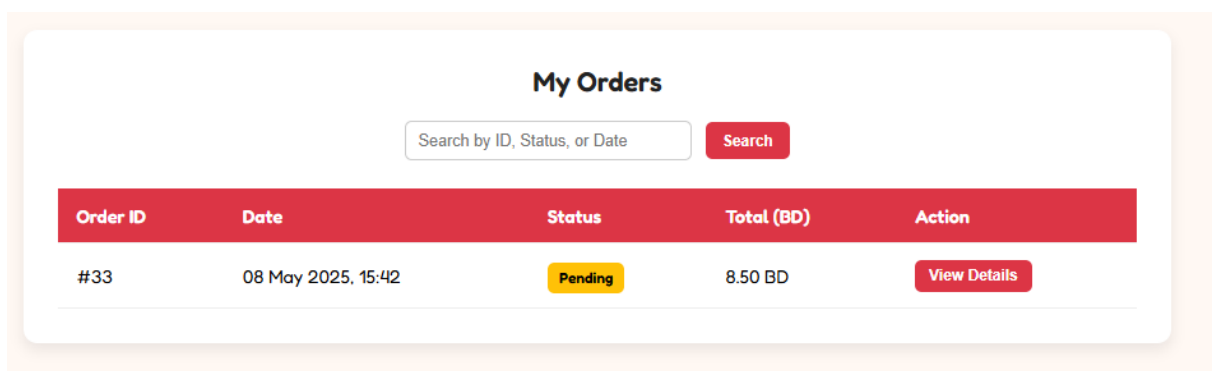


Figure 135 my orders view

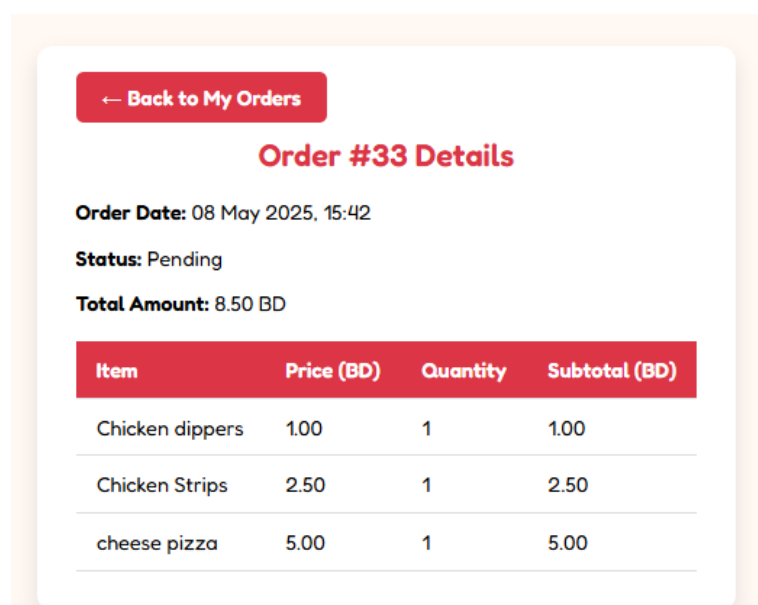


Figure 134order details view

Admin Manual

This chapter will cover all the actions the admin can do.

1) Log in with your admin credentials

(Use Username=admin Password=admin123)

2) Once logged in you should be met with your specialized dashboard

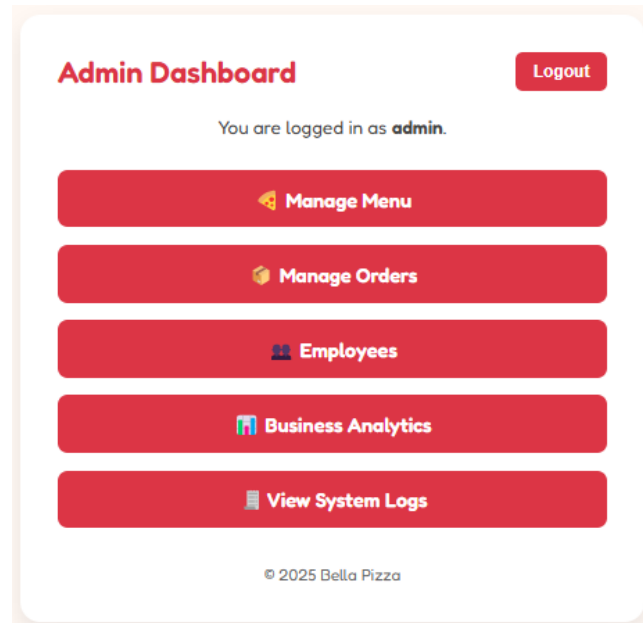


Figure 136 Admin dashboard view

- 3) To manage menu items, click on the “Manage Menu” button. From here you can edit menu items, reorganize categories by dragging them by the handler icon ☰ . you can toggle item availability so that they may be displayed on the menu or not, you can search for specific menu items in real time.

Edit Menu Items

Search menu items...

▼ ☰ Appetizer

ID	Image	Name	Description	Price	Available	Actions
12		Chicken dippers	Delicious Dippers	1.00 BD	☒	<button>Edit</button>
10		Chicken Strips	Delicious Chicken Strips	2.50 BD	☒	<button>Edit</button>

▼ ☰ Pizza

ID	Image	Name	Description	Price	Available	Actions
1		peperoni pizza	nice pizza with peperonni	3.00 BD	☒	<button>Edit</button>
13		Vegetarian Pizza	Delicious Vegetarian Pizza	4.00 BD	☒	<button>Edit</button>
2		cheese pizza	Handcrafted Cheese Pizza For The Cheeselovers.	5.00 BD	☒	<button>Edit</button>
8		Meat Pizza	Meaty and Delicious Pizza	5.00 BD	☒	<button>Edit</button>

▶ ☰ Drinks

▶ ☰ Desserts

+ Add Menu Item☰ Manage Categories

Figure 137 Menu Management View

- 4) In the “Manage Orders” View, you can view, cancel, or create orders in case any issue arises, the page will auto-update itself if a new order is created. You can also search for previous orders using the search bar.

Manage Orders

← Back to Dashboard

Search by customer or order ID

▼ Today's Orders (1)

Order ID	Customer	Date	Status	Total	Driver	Actions
33	aaron	2025-05-08 15:42:40	Pending	8.50 BD	Not assigned	<button>Cancelled</button> <button>Details</button>

▼ Past Orders (32)

Order ID	Customer	Date	Status	Total	Driver	Actions
32	yannis	2025-05-05 14:54:24	Pending	1.00 BD	Not assigned	<button>Cancelled</button> <button>Details</button>
31	yannis	2025-05-01 16:04:15	Preparing	5.00 BD	jack jack	<button>Cancelled</button> <button>Details</button>
30	yannis	2025-04-30 12:23:18	Cancelled	5.00 BD	Not assigned	<button>Details</button>
29	yannis	2025-04-29 12:34:37	Completed	5.00 BD	jack jack	<button>Details</button>
28	yannis	2025-04-29 12:31:24	Cancelled	5.00 BD	Not assigned	<button>Details</button>
27	yannis	2025-04-29 12:20:37	Cancelled	5.00 BD	jack jack	<button>Details</button>
26	yannis	2025-04-29 12:18:23	Cancelled	5.00 BD	jack jack	<button>Details</button>

Figure 138 Manage Orders view

- 5) In the Employees Tab, you may view, search, create, Edit, or delete Employees as needed. You can also export a CSV file that you can use to keep track the employees, and if you use the search function, the CSV report will automatically tailor itself to the search as well.

The screenshot shows a web interface for managing employees. At the top, there are two buttons: '+ Add Employee' (red) and 'Export CSV' (yellow). To the right is a search bar labeled 'Search employees...'. Below the search bar, it says 'Total Employees: 7'. The main part of the interface is a table with the following columns: ID, Full Name, Username, Email, Phone, Role, Salary, Hire Date, and Actions. The table contains 7 rows of employee data. Each row has 'Update' and 'Delete' buttons in the Actions column.

ID	Full Name	Username	Email	Phone	Role	Salary	Hire Date	Actions
14	admin admin	admin	admin@hotmail.com	38877927	Admin	BD 1,000.00	2025-05-08	Update Delete
11	Yanis Moussai	qwqwq	Drain@email.xom	38877927	Support	BD 12,222.00	2025-05-03	Update Delete
5	jack jack	jack	jackemail@gmail.com	38877927	Delivery	BD 2,333.00	2025-04-16	Update Delete
4	james james	james	james@gmail.com	38877927	Chef	BD 2,000.00	2025-04-16	Update Delete
3	drake drake	drake	drake111@gmail.com	38877927	Cashier	BD 1,212.00	2025-04-16	Update Delete
2	Yanis yanis	yanis	yanismm14@hotmail.com	38877927	Admin	BD 500.00	2025-04-16	Update Delete
1	John John	John	John@gmail.com	12312312	Support	BD 500.00	2025-04-10	Update Delete

Figure 140 Employee List

The screenshot shows a CSV file opened in a spreadsheet application. The data is organized into columns labeled A through H. The columns correspond to the employee data: ID, Full Name, Username, Email, Phone, Role, Salary, and Hire Date. The data is identical to the table in Figure 140.

ID	Full Name	Username	Email	Phone	Role	Salary	Hire Date
14	admin admin	admin	admin@hotmail.com	38877927	Admin	BD 1,000.00	08/05/2025
11	Yanis Moussai	qwqwq	Drain@email.xom	38877927	Support	BD 12,222.00	03/05/2025
5	jack jack	jack	jackemail@gmail.com	38877927	Delivery	BD 2,333.00	16/04/2025
4	james james	james	james@gmail.com	38877927	Chef	BD 2,000.00	16/04/2025
3	drake drake	drake	drake111@gmail.com	38877927	Cashier	BD 1,212.00	16/04/2025
2	Yanis yanis	yanis	yanismm14@hotmail.com	38877927	Admin	BD 500.00	16/04/2025
1	John John	John	John@gmail.com	12312312	Support	BD 500.00	10/04/2025

Figure 139 Employee CSV file

- 6) Now for the Analytics Section You can view, unique visits, total customer visits, new customers, returning customers (customers that return before a 1 month expiry token), as well as total Orders and Total revenue. Then you can see a table that shows you how many orders are completed, cancelled, or being prepared. You can also view what items you have sold and how many, which is then displayed in a bar graph underneath for visuals as well as a Revenue over time line graph. The data can be filtered based on date, to give a variety of useful information to use in analytics. You can then download a CSV report of all the data, which also is tailored based on the date filter.

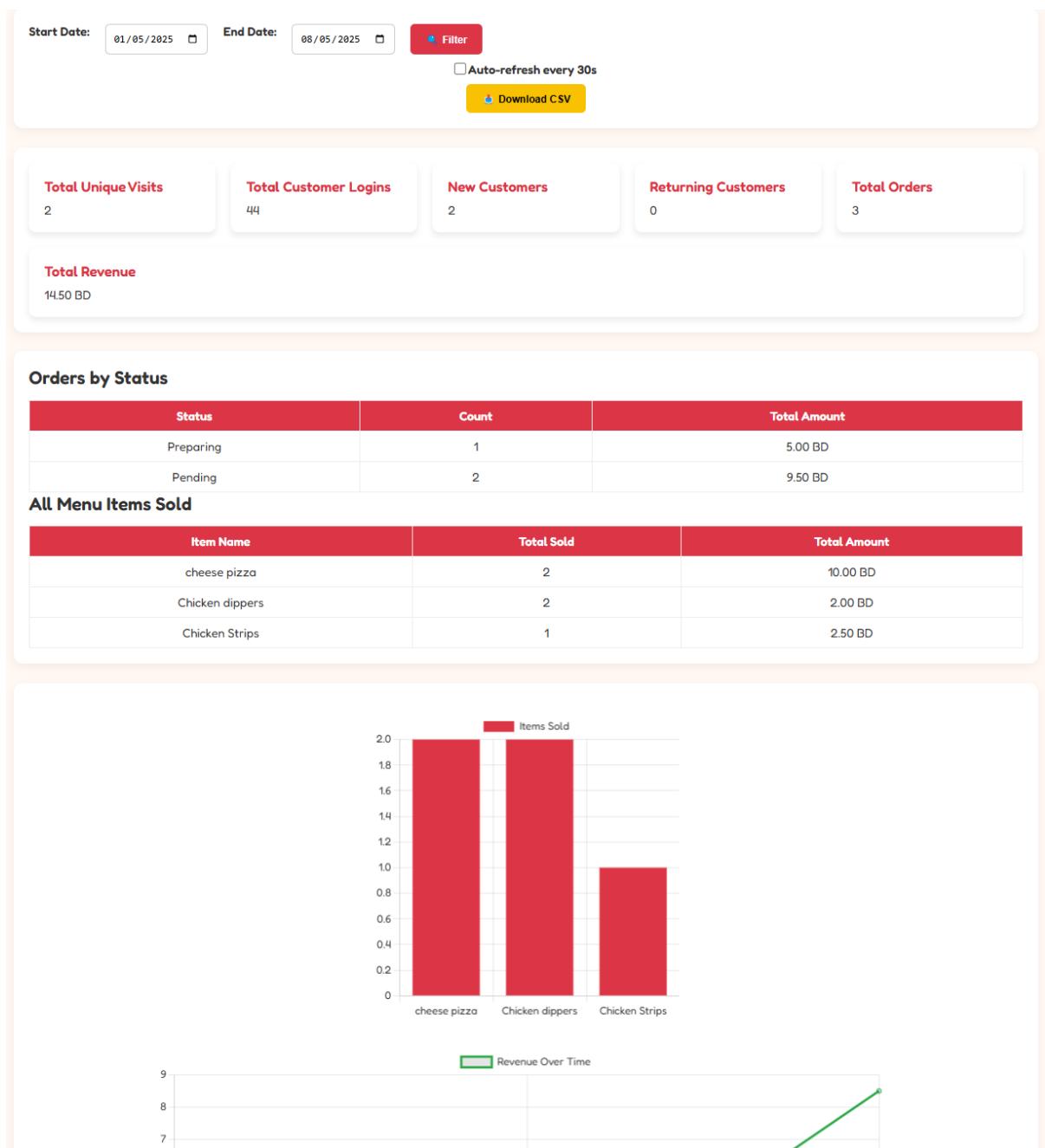


Figure 141 Analytics View

7) The System Logs is an important page that keeps track of most user activities, whether customer or employee, to monitor interactions with the website in case of misuse. You can filter the logs their level:

- a) INFO= for normal activities
- b) WARNING= for failed activities such as failed logins
- c) ERROR= for any system failure that occurs

You can also filter the logs by date, id , and metadata. Keep in mind you need to scroll left and right in the box to see the rest of the columns.

System Logs									
login		All Levels	dd/mm/yyyy	dd/mm/yyyy	Filter				
ID	Level	Timestamp	Employee ID	Customer ID	Username	Action	Description	IP Address	User Agent
60	INFO	2025-05-08 16:00:09	14	-	admin	Login Success	Employee logged in successfully	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36
59	WARNING	2025-05-08 16:00:04	-	-	admin	Login Failed	Invalid password for employee: admin	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36
57	INFO	2025-05-08 15:53:39	2	-	yanis	Login Success	Employee logged in successfully	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36
56	INFO	2025-05-08 15:33:54	-	8	aaron	Login Success	Customer logged in successfully	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36
54	INFO	2025-05-08 15:24:36	-	1	yannis	Login Success	Customer logged in successfully	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36
53	INFO	2025-05-07 17:45:14	-	1	yannis	Login Success	Customer logged in successfully	172.18.0.1	Mozilla (Windows Win64; AppleWebKit (KHTML, like Gecko) Chrome/115.0.5948.50 Safari/537.36

Appendix II: Detailed Design

User Interface Design

Introduction

The user interface (UI) of Bella Pizza was designed with a focus on usability, aesthetic appeal, and functional clarity. As a food ordering system catering to both end-users and administrators, the UI had to accommodate a variety of roles and tasks—from casual browsing and ordering to detailed order and menu management. This appendix provides a comprehensive breakdown of the system's UI components, design principles, layout choices, responsiveness features, and consistency strategies employed throughout the application.

By adhering to modern design standards and prioritizing user experience, the Bella Pizza interface delivers an intuitive, visually engaging, and mobile-friendly environment. Key interface elements such as card-based menu listings, collapsible sections, color-coded role indicators, and interactive forms are used consistently across the system to support seamless interaction.

Design Objectives

The design of the Bella Pizza UI was driven by the following objectives:

- **Consistency:** All pages use a shared layout structure, color palette, and typography.
- **Responsiveness:** The interface adjusts smoothly across devices (desktop, tablet, mobile).
- **Accessibility:** Adequate contrast, clear navigation, and logical grouping improve usability.
- **Visual Hierarchy:** Information is prioritized with cards, headers, section titles, and shadows.
- **Role Awareness:** The interface adjusts based on user role (e.g., admin, support, customer).
- **Aesthetic Alignment:** A vibrant yet professional look was achieved using a red and yellow brand theme.

Color Palette and Its Impact

The Bella Pizza user interface adopts a carefully selected color palette that enhances both brand identity and user experience. The primary color is a vibrant tomato red (#dc3545), used prominently for buttons, headings, and accents. Red is commonly associated with appetite, urgency, and passion—making it ideal for a food service platform. It captures attention immediately and encourages users to take action, whether that's placing an order or completing a registration. This is paired with a bold sunshine yellow (#f8c102) as the secondary accent, used sparingly to highlight key UI elements. Yellow conveys warmth, optimism, and friendliness, creating a positive emotional tone throughout the site.

The background uses a soft beige-pink tone (#fff8f3) to give the interface a light, inviting atmosphere without straining the eyes. Clean white containers with subtle shadows help frame content and create contrast, improving legibility. Error and success messages use conventional red and green hues, ensuring intuitive recognition and feedback. Together, these colors are not only aesthetically pleasing but also play a psychological role in making the user feel confident, comfortable, and hungry—ideal for encouraging engagement and repeat visits.



Figure 142 Colour palette

Navigation and Branding

The top navigation bar is a persistent, sticky header present across all pages. It includes the Bella Pizza logo on the left and dynamic navigation buttons on the right based on user roles and session status. For authenticated users, the navbar includes links such as:

Dashboard

My Orders

Cart (with item count)

Logout

Administrators and support staff see links to:

Manage Orders

Menu Items

Employees

Logs

The navbar uses the brand's red primary color (#dc3545) with a subtle shadow for visual depth. On hover, buttons slightly darken to indicate interactivity, enhancing user feedback. This consistent element reinforces brand identity and supports quick navigation across the application.



Figure 143 Example Nav bar

Homepage (index.php)

The Bella Pizza homepage presents a clean, modern, and welcoming interface that sets the tone for the entire platform. The design features a centered white content box with soft shadows and rounded corners, placed over a warm, light background to evoke a cozy and inviting atmosphere. At the top, the Bella Pizza logo is displayed prominently, followed by a bold red heading and a friendly tagline that reflects the brand's personality.

Two large buttons—Login and View Menu—are styled with the signature red color and are placed side by side, inviting users to take immediate action. The buttons are rounded, spaced evenly, and feature subtle hover effects for interactivity. On smaller screens, the layout adapts responsively, maintaining readability and balance across all device sizes. The simplicity and clarity of this page ensure that users can navigate the platform effortlessly from the very first interaction.

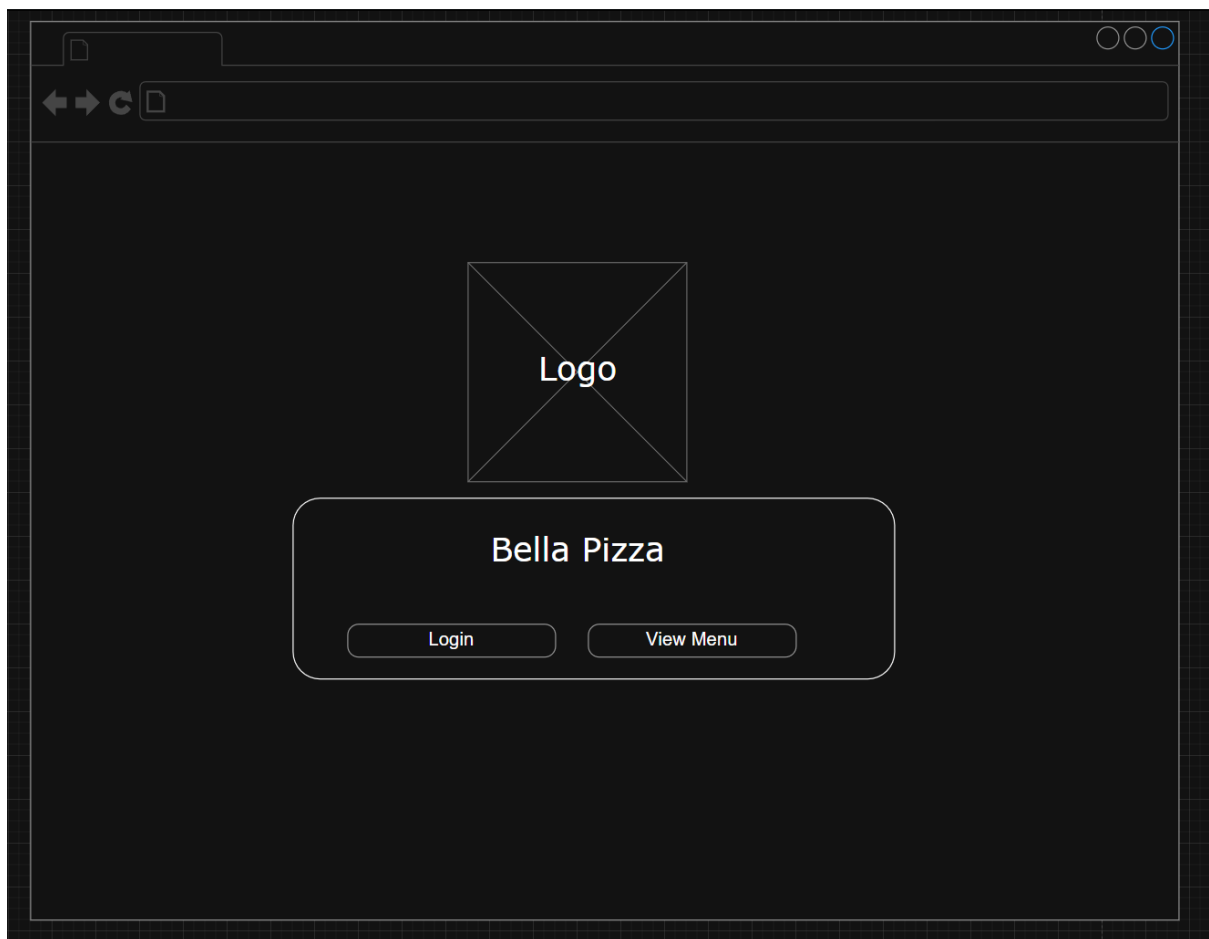


Figure 144 Homepage Structure

Login & Registration Page

The login and registration page presents a unified, clean interface for both new and returning users. The design follows a vertically stacked layout with soft colors, modern typography, and intuitive spacing to create a welcoming and accessible experience.

The page is divided into two main sections: Login and Register, each framed within a single, centered white container with rounded corners and subtle shadows. Headings for each section are styled in dark gray for clarity and contrast. Input fields are evenly spaced and outlined with soft borders, while focused inputs highlight in red to signal interactivity and brand consistency.

Each form includes clear labels, rounded input fields, and a full-width red action button with hover feedback. Buttons are bold, easy to tap on mobile devices, and placed logically to support a smooth form-filling flow.

Above each form, dynamic feedback messages appear in soft green or red boxes to indicate success or error. These help users clearly understand the result of their interaction.

A small black "Home" button is placed at the top left corner, allowing users to easily return to the homepage at any time. The entire layout adapts gracefully to smaller screens, with inputs and buttons resizing appropriately to ensure full functionality on mobile devices.

Figure 145 Login page Structure

User Dashboard (*dashboard_user.php*)

The user dashboard is structured to provide a clear and efficient layout for browsing and ordering menu items. At the top of the page, a fixed header includes the Bella Pizza logo and navigation buttons for accessing the user's order history, shopping cart, and logout functionality. This navigation remains visible at all times, allowing users to move between key pages without scrolling back to the top.

Directly beneath the header is a wide banner that introduces the page with a promotional message. Below this, the main content area begins with a personalized greeting that displays the user's name and a brief instruction encouraging them to place an order.

The core of the dashboard is organized into sections based on item categories such as *Pizzas*, *Drinks*, etc... Each section contains a horizontally scrollable list of cards. Every card represents a single menu item and includes an image, product name, description, price, and a small form for selecting quantity and adding the item to the cart. The form elements are embedded directly in each card, minimizing the need for navigation or extra pages. This compact, scrollable layout allows users to explore multiple items efficiently within each category while keeping the interface clean and focused. The overall structure prioritizes quick browsing and one-click actions, which is essential for a smooth food-ordering experience across both desktop and mobile devices.

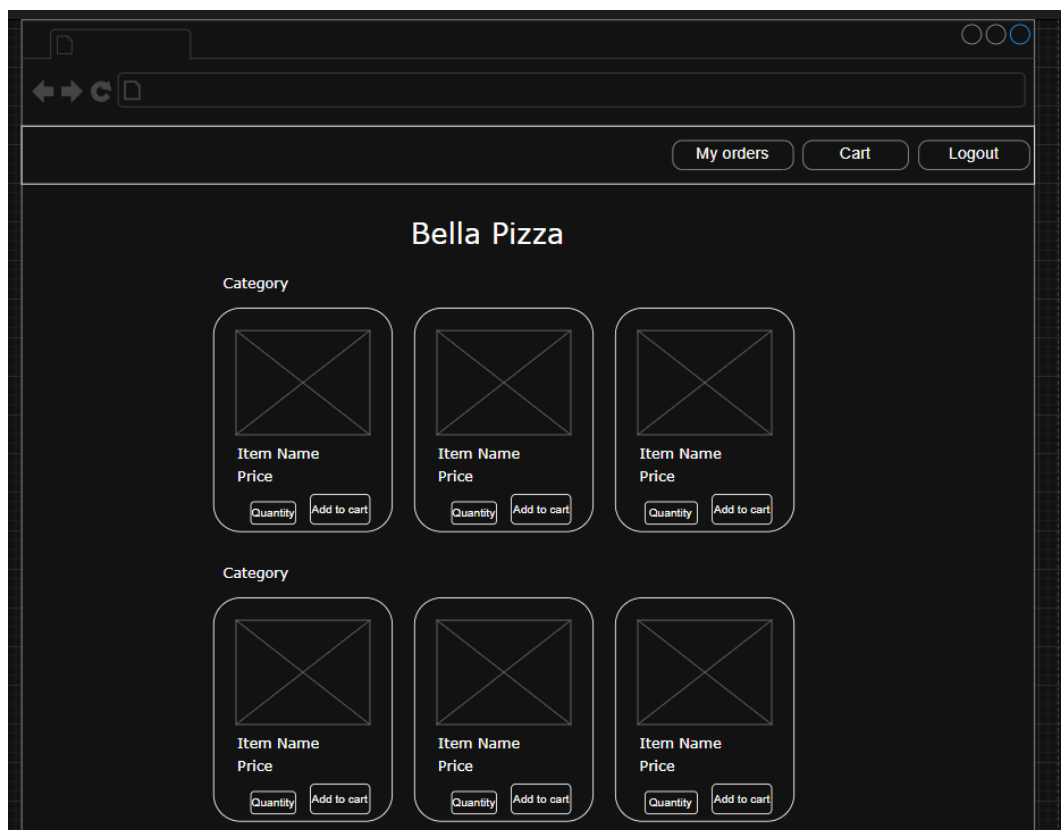


Figure 146 User dashboard structure

Admin/employee Dashboard (dashboard.php)

The admin dashboard is structured as a single, centered interface that presents administrative users with a personalized welcome message and a clear list of action options based on their role. At the top of the page is a compact header that includes the dashboard title and a logout button aligned side-by-side, providing immediate session control.

Beneath the header is a small section that confirms the user's login identity, displaying their username and role. This helps orient users and reinforces that they are in the correct administrative space.

The main area of the dashboard is a vertical list of action buttons. These buttons provide direct access to the system's key management pages, such as menu management, order handling, employee management, analytics, and system logs. The layout ensures that these options are presented clearly and are easy to navigate in a single column format. The list adapts depending on the user's role—for example, a support user will see fewer options than an admin, while roles like cashier or chef may see only the orders page.

At the bottom of the layout is a small footer with the platform's name and year, providing a subtle closure to the interface. The overall structure focuses on accessibility and quick access to tasks, helping staff members perform administrative functions efficiently.

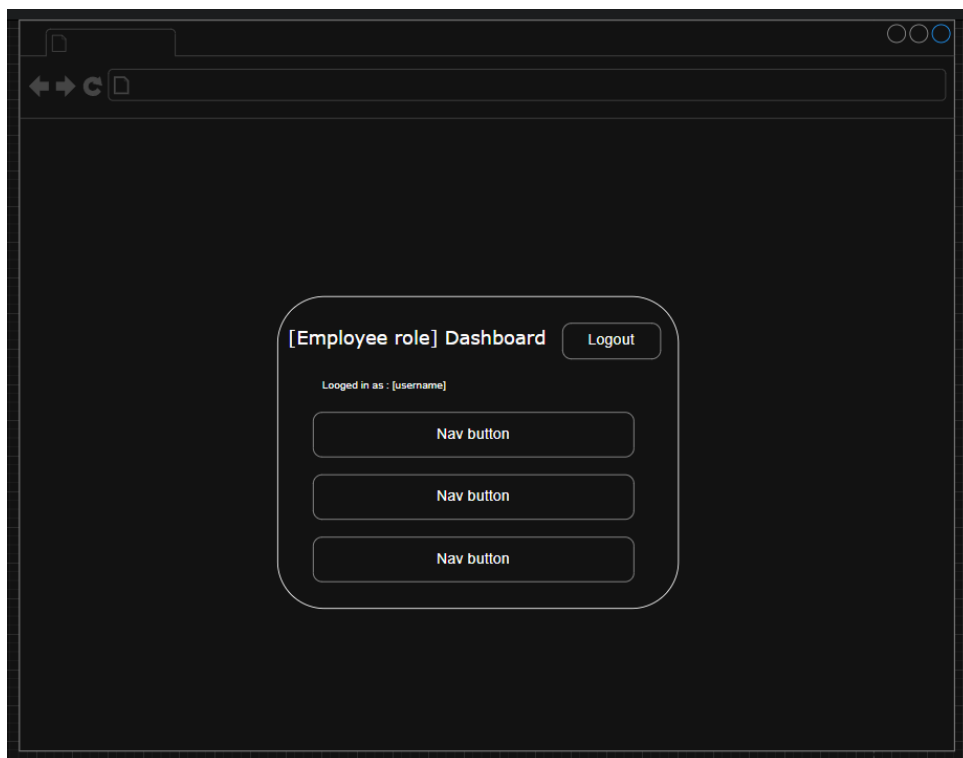


Figure 147 Admin/Employee dashboard structure

Edit Menu Items Page

The Edit Menu Items page is organized into a structured, task-oriented interface that allows administrators and support staff to manage the food menu with clarity and efficiency. At the top of the page is a fixed navigation bar containing the page title and quick access buttons to return to the dashboard or log out, helping users stay oriented within the admin area.

The main content begins with a centered page title and a search bar that allows users to filter menu items by name. Below the search tool, the menu is divided into collapsible sections grouped by category. Each category section can be expanded to reveal a table containing all items under that category.

Within each table, menu items are displayed in rows, showing key information such as item ID, name, description, price, and availability. Each row also includes a preview image of the item and an edit action. Availability can be toggled directly from the table using a checkbox, and items can be reordered within their category using drag-and-drop.

Categories themselves are also draggable, allowing users to reorganize the order in which they appear. At the bottom of the page, two buttons provide quick access to add new menu items or manage category settings, supporting fast updates without navigating away from the interface.

The structure is designed to support both overview and detail-level interactions, allowing staff to manage large sets of menu items without confusion or clutter.

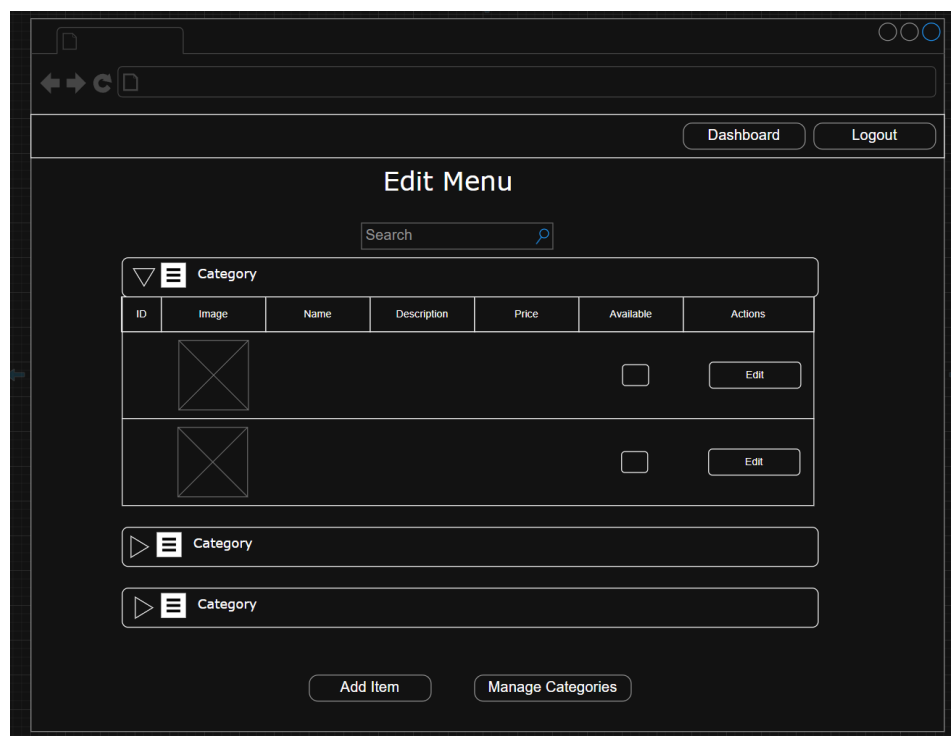


Figure 148 Edit Menu structure

Manage Orders Page

The Manage Orders page is organized into a clear, two-section layout that helps staff members efficiently view and manage customer orders. At the top, a fixed navigation bar provides access to the dashboard and logout, ensuring that users can easily return to key areas of the admin system. Below this, the main content is centered within a container that includes a title, a back-to-dashboard button, and a search field.

Orders are grouped into two expandable sections: Today's Orders and Past Orders. These sections can be expanded or collapsed to keep the interface focused. Each section contains a table listing individual orders with key details such as order ID, customer name, date, status, total amount, delivery driver (if assigned), and available actions.

Each order row includes interactive buttons for status changes, order details, and—depending on the employee's role—additional actions like assigning a driver or marking completion. The action buttons are embedded within the table rows, minimizing navigation and allowing for fast interaction.

A search bar positioned above the tables enables live filtering of orders by ID or customer name. The entire page supports real-time updates, automatically refreshing the list of orders at set intervals to ensure staff are always viewing the most current data. This layout balances clarity and functionality, allowing different staff roles to focus on the actions relevant to their responsibilities without confusion.

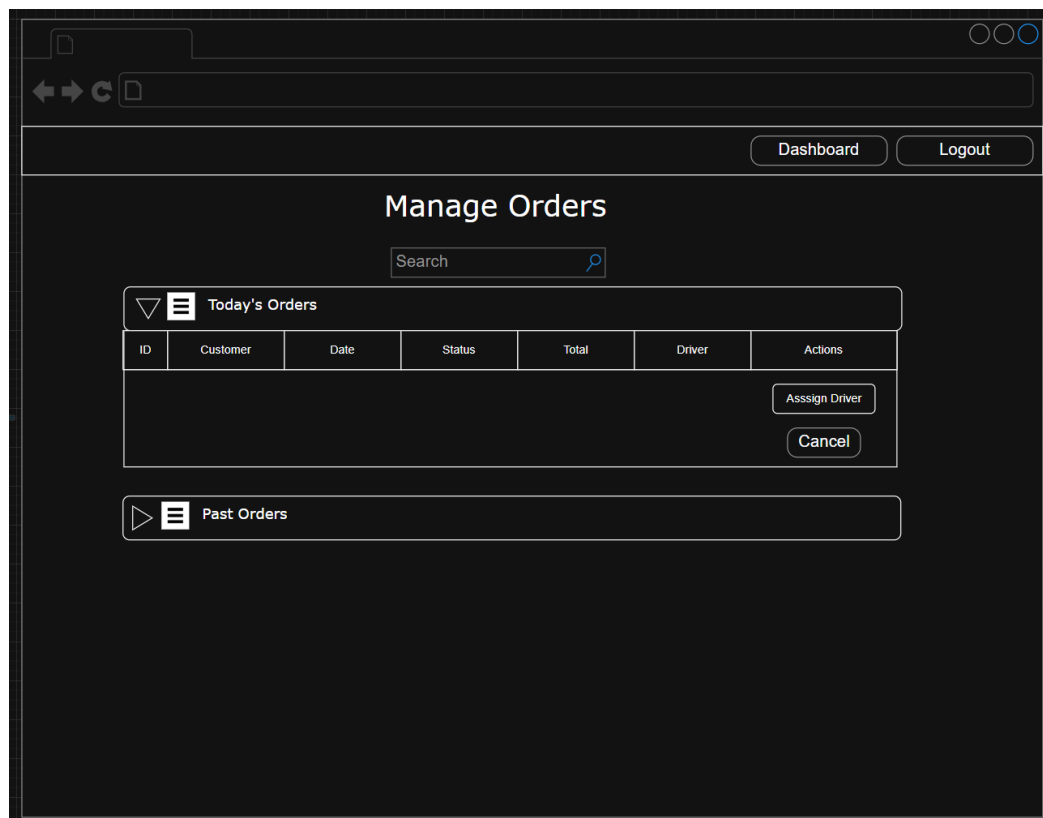


Figure 149 Manage orders structure

Analytics Page

The Analytics page presents a structured dashboard interface that allows administrators to view business performance metrics within a specific date range. At the top of the page, a fixed header displays the system title and navigation buttons for quick access to the dashboard and logout. This is followed by a filter panel, where users can select a start and end date and enable optional auto-refresh functionality to update the analytics every 30 seconds. A button is also provided to download a report in CSV format.

The main layout is divided into several clearly defined sections:

- **Summary Cards:** These small blocks show key performance indicators such as total orders, revenue, unique visits, new customers, and returning customers. The cards are arranged in a flexible row layout and adapt to different screen sizes.
- **Tables Section:** Two data tables are presented—one showing order counts and revenue by status, and the other listing all menu items sold, including total quantities and sales amounts. This layout allows for quick comparison and understanding of item performance.
- **Charts Section:** The page includes two visual charts—one bar chart for items sold and one line chart for daily revenue trends. These visuals help the administrator quickly interpret changes in performance over time.

Each section is wrapped in a card-style container with consistent spacing and hierarchy, making the interface clean, easy to scan, and efficient for decision-making. The structure encourages regular monitoring and supports both visual and tabular data exploration.

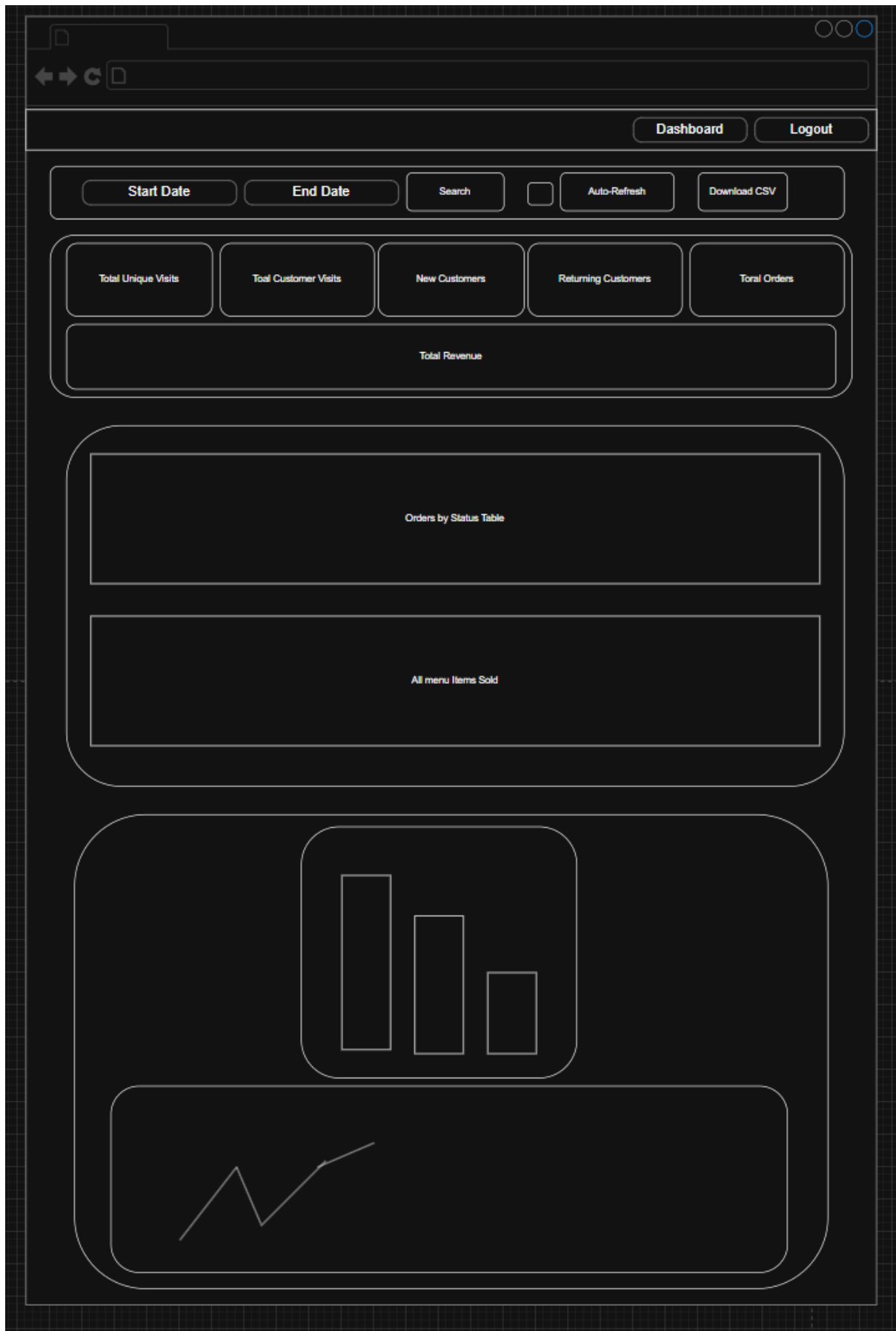


Figure 150 Analytics Page structure

Appendix III: Detailed Implementation

In this chapter we move from design to delivery, outlining how Bella Pizza’s web application was built on a LAMP stack (Linux, Apache, MySQL, PHP) orchestrated via Docker containers. We begin with a high-level architecture that shows how the PHP/Apache and MySQL services communicate inside Docker Compose, then state our design goals and justify each technology choice. Each section that follows focuses on a specific PHP page: we give its objective, explain why we implemented it this way in a containerized LAMP environment, present annotated code, and describe our testing and security hardening approaches. This structure highlights not only what was built but how it runs reliably and securely in Docker.

I. Setting up the Environment

The development environment was set up using Docker on a Linux-based system running through WSL2 (Windows Subsystem for Linux 2) on Windows 11. This configuration provided a flexible and efficient platform for developing and testing the Bella Pizza system. Docker was used to containerize the web server, database, and application code, ensuring consistency across environments and simplifying deployment. WSL2 enabled a full Linux kernel to run within Windows, offering native compatibility with Linux tools and improving performance over the previous version. This setup allowed seamless integration between the development tools on Windows and the Linux-based infrastructure required for web application deployment.

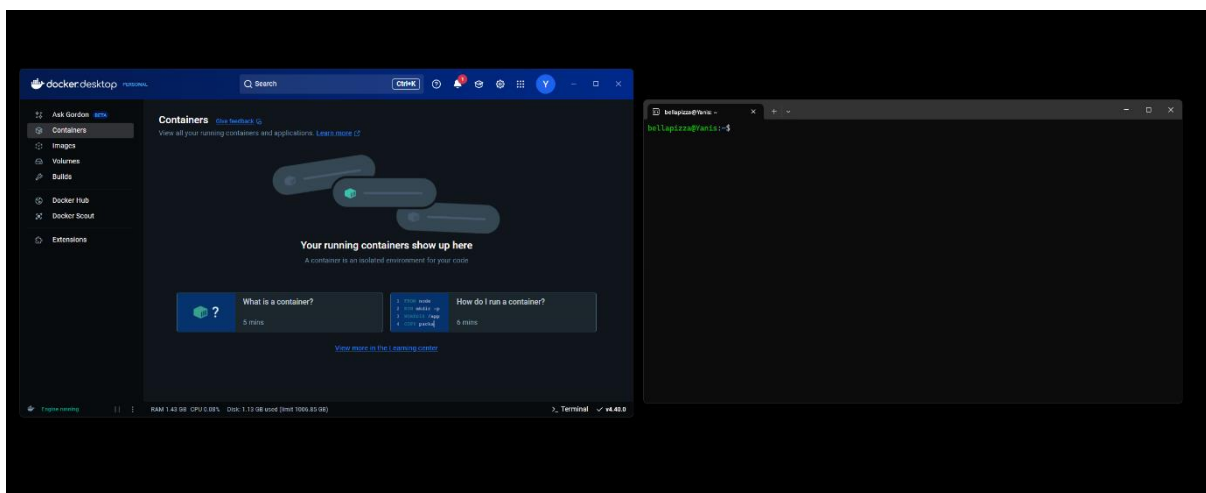


Figure 151 Docker Desktop and Ubuntu Setup on Windows 11

II. Creating Containers and Virtual Hosts

Dockerfile

This file builds a custom PHP-Apache image tailored to the Bella Pizza application, installing required PHP extensions, enabling Apache modules, and setting up SSL certificate handling for HTTPS.

Base Image Declaration

A screenshot of a code editor showing the Dockerfile instruction `FROM php:8.2-apache`. The text is in a monospaced font with syntax highlighting: 'FROM' is pink, 'php:8.2-apache' is green, and there is a yellow wavy underline under the entire line.

Figure 152 Base image using PHP 8.2 with Apache

This instruction initializes the Docker image using the official php:8.2-apache base. It combines Apache HTTP Server and PHP 8.2, which is necessary to serve dynamic PHP pages in the Bella Pizza web application.

Enable Apache Modules

A screenshot of a code editor showing the Dockerfile instruction `RUN a2enmod rewrite ssl`. The text is in a monospaced font with syntax highlighting: 'RUN' is pink, 'a2enmod' is blue, 'rewrite' is green, and 'ssl' is blue.

Figure 153 Enable Apache modules for URL rewriting and SSL

Enables two critical Apache modules:

- `rewrite`: required for clean URLs (used in routing and redirection)
- `ssl`: needed to handle HTTPS traffic with self-signed or trusted certificates

Install PHP Extensions

A screenshot of a code editor showing the Dockerfile instruction `RUN docker-php-ext-install mysqli pdo pdo_mysql`. The text is in a monospaced font with syntax highlighting: 'RUN' is pink, 'docker-php-ext-install' is blue, 'mysqli' is green, 'pdo' is blue, and 'pdo_mysql' is blue.

Figure 154 Install MySQL-related PHP extensions

Installs extensions necessary for database interactions:

- `mysqli` for MySQLi procedural/object-oriented access
- `pdo` and `pdo_mysql` for PDO-based MySQL connections, which are used in the Bella Pizza backend for secure, parameterized queries

Create SSL Directory

```
RUN mkdir -p /etc/apache2/ssl
```

Figure 155 Prepare SSL directory in Apache

Creates the directory `/etc/apache2/ssl` to store SSL certificate and key files. This is where Apache expects to find the certificate files when using the `default-ssl.conf` configuration.

Copy SSL Certificate and Key

```
COPY certs/selfsigned.crt /etc/apache2/ssl/selfsigned.crt  
COPY certs/selfsigned.key /etc/apache2/ssl/selfsigned.key
```

Figure 156 Copy SSL certificate and private key to the container

Copies the self-signed SSL certificate and its private key into the Apache SSL directory. This enables HTTPS support for secure access to the Bella Pizza website, useful especially for handling user logins and payments.

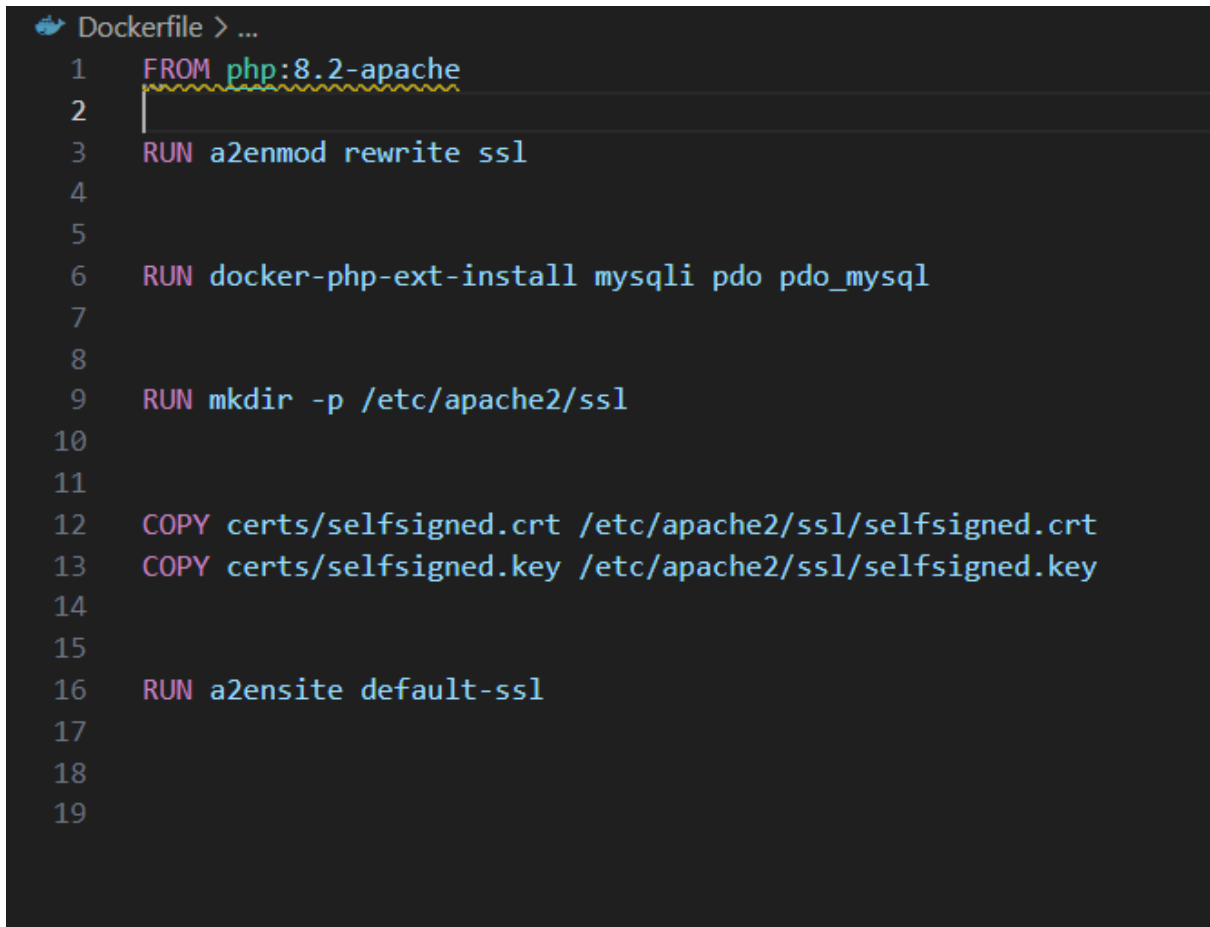
Enable Default SSL Site

```
RUN a2ensite default-ssl
```

Figure 157 Activate the Apache SSL virtual host

Enables Apache's `default-ssl` site configuration, which allows HTTPS traffic using the provided self-signed certificate. This ensures the Bella Pizza site is accessible over `https://localhost` during development.

Your Dockerfile should look like this:



```
Dockerfile > ...
1 FROM php:8.2-apache
2
3 RUN a2enmod rewrite ssl
4
5
6 RUN docker-php-ext-install mysqli pdo pdo_mysql
7
8
9 RUN mkdir -p /etc/apache2/ssl
10
11
12 COPY certs/selfsigned.crt /etc/apache2/ssl/selfsigned.crt
13 COPY certs/selfsigned.key /etc/apache2/ssl/selfsigned.key
14
15
16 RUN a2ensite default-ssl
17
18
19
```

Figure 158 Dockerfile full screenshot

Docker-Compose.yml File

This file defines and manages the multi-container Docker environment for the Bella Pizza web application. It orchestrates the PHP-Apache web server, MySQL database, and phpMyAdmin service within an isolated Docker network.

Docker Compose Metadata



Figure 159 Docker Compose file version

Defines the schema version used by Docker Compose. Version 3.8 is compatible with Docker Engine 19.03+ and allows modern configuration features like named volumes and advanced networking.

Web Service (Apache + PHP)

```
services:
  > Run Service
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bella_web
    ports:
      - "8080:80"
      - "443:443"
    volumes:
      - ./certs/selfsigned.crt:/etc/apache2/ssl/selfsigned.crt:ro
      - ./certs/selfsigned.key:/etc/apache2/ssl/selfsigned.key:ro
      - ./default-ssl.conf:/etc/apache2/sites-available/default-ssl.conf:ro
      - ./app:/var/www/html
    depends_on:
      - db
    restart: always
    networks:
      - bella_net
    environment:
      MYSQL_HOST: db
      MYSQL_USER: root
      MYSQL_PASSWORD: rootpass
      MYSQL_DATABASE: RestaurantDB
```

Figure 160 PHP-Apache container setup for Bella Pizza

This service hosts the main web application. It:

- Builds from a Dockerfile located in the project root
- Exposes ports 8080 (HTTP) and 443 (HTTPS)
- Mounts a local app folder to /var/www/html to serve PHP files
- Includes custom SSL certificates for HTTPS via volume mounts
- Declares database credentials through environment variables
- Automatically restarts and is connected to a shared network

MySQL Database Service

```
▷ Run Service
db:
  image: mysql:8.0
  container_name: bella_db
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: rootpass
    MYSQL_DATABASE: restaurantdb
    MYSQL_USER: user
    MYSQL_PASSWORD: userpass
  volumes:
    - dbdata:/var/lib/mysql
  networks:
    - bella_net
```

Figure 161 MySQL 8 container for data persistence

This service runs MySQL version 8.0 and:

- Initializes the database restaurantdb
- Sets up root and regular users with passwords
- Stores data in a persistent Docker volume named dbdata
- Joins the common network so the web and phpMyAdmin services can connect

phpMyAdmin Service

```
▷ Run Service
phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: bella_phpmyadmin
  restart: always
  ports:
    - "8081:80"
  environment:
    PMA_HOST: db
    MYSQL_ROOT_PASSWORD: rootpass
  depends_on:
    - db
  networks:
    - bella_net
```

Figure 162 GUI for MySQL database administration

phpMyAdmin provides a web-based GUI to interact with the MySQL database. It:

- Exposes itself on port 8081
- Connects to the db container using environment variables
- Automatically starts after the database is ready

Shared Volume and Network

```
volumes:
  dbdata:

networks:
  bella_net:
```

Figure 163 Persistent volume and network for container communication

- dbdata: A named Docker volume used to store MySQL data across container restarts
- bella_net: A custom bridge network allowing all services to communicate internally by container name (e.g., db for the MySQL host)

Your docker-compose.yml should look like this:

```
docker-compose.yml > ...
1  version: '3.8'
2
3  > Run All Services
4  services:
5    > Run Service
6    web:
7      build:
8        context: .
9        dockerfile: Dockerfile
10     container_name: bella_web
11     ports:
12       - "8080:80"
13       - "443:443"
14     volumes:
15       - ./certs/selfsigned.crt:/etc/apache2/ssl/selfsigned.crt:ro
16       - ./certs/selfsigned.key:/etc/apache2/ssl/selfsigned.key:ro
17       - ./default-ssl.conf:/etc/apache2/sites-available/default-ssl.conf:ro
18       - ./app:/var/www/html
19     depends_on:
20       - db
21     restart: always
22     networks:
23       - bella_net
24     environment:
25       MYSQL_HOST: db
26       MYSQL_USER: root
27       MYSQL_PASSWORD: rootpass
28       MYSQL_DATABASE: RestaurantDB
29
30    > Run Service
31    db:
32      image: mysql:8.0
33      container_name: bella_db
34      restart: always
35      environment:
36        MYSQL_ROOT_PASSWORD: rootpass
37        MYSQL_DATABASE: restaurantdb
38        MYSQL_USER: user
39        MYSQL_PASSWORD: userpass
40      volumes:
41        - dbdata:/var/lib/mysql
42      networks:
43        - bella_net
44
45    > Run Service
46    phpmyadmin:
47      image: phpmyadmin/phpmyadmin
48      container_name: bella_phpmyadmin
49      restart: always
50      ports:
51        - "8081:80"
52      environment:
53        PMA_HOST: db
54        MYSQL_ROOT_PASSWORD: rootpass
55      depends_on:
56        - db
57      networks:
58        - bella_net
59
60    volumes:
61      dbdata:
62
63    networks:
64      bella_net:
```

Figure 164 docker compose yml full screenshot

ssl config file

This Apache configuration file defines the HTTPS virtual host used by the web container. It binds to port 443 and enforces SSL encryption using self-signed certificates.

SSL Module Check and Virtual Host

```
<IfModule mod_ssl.c>  
    <VirtualHost _default_:443>
```

Figure 165 Define SSL VirtualHost for HTTPS

This block ensures that the configuration is only applied if the mod_ssl module is enabled. The VirtualHost _default_:443 directive binds Apache to listen on port 443 (default HTTPS port) for all IPs on the host.

Basic Server Setup

```
ServerName localhost  
DocumentRoot /var/www/html
```

Figure 166 Define server identity and document root

ServerName specifies the hostname (bellapizza.test for local development), and DocumentRoot points to the application code inside the container (/var/www/html), where Bella Pizza files are served.

Enable SSL and Certificates

```
# Enable SSL  
SSLEngine on  
SSLCertificateFile /etc/apache2/ssl/selfsigned.crt  
SSLCertificateKeyFile /etc/apache2/ssl/selfsigned.key
```

Figure 167 Enable SSL and bind self-signed certificate

Activates SSL support and links the Apache server to the mounted certificate and private key files. These files are copied into the container from your certs folder via Docker.

Secure SSL Protocols and Ciphers

```
SSLProtocol          all -SSLv3 -TLSv1 -TLSv1.1
SSLCipherSuite       HIGH:!aNULL:!MD5
SSLHonorCipherOrder  on
```

Figure 168 Enforce strong encryption for secure connections

This config disables outdated and insecure SSL/TLS versions (SSLv3, TLS 1.0, TLS 1.1) and forces the use of strong cipher suites, improving the security of HTTPS connections.

Directory Permissions and Overrides

```
<Directory /var/www/html>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

Figure 169 Grant access and allow .htaccess overrides

Allows Apache to serve files from /var/www/html, enables URL rewriting via .htaccess, and ensures all incoming requests are permitted (important for development environments).

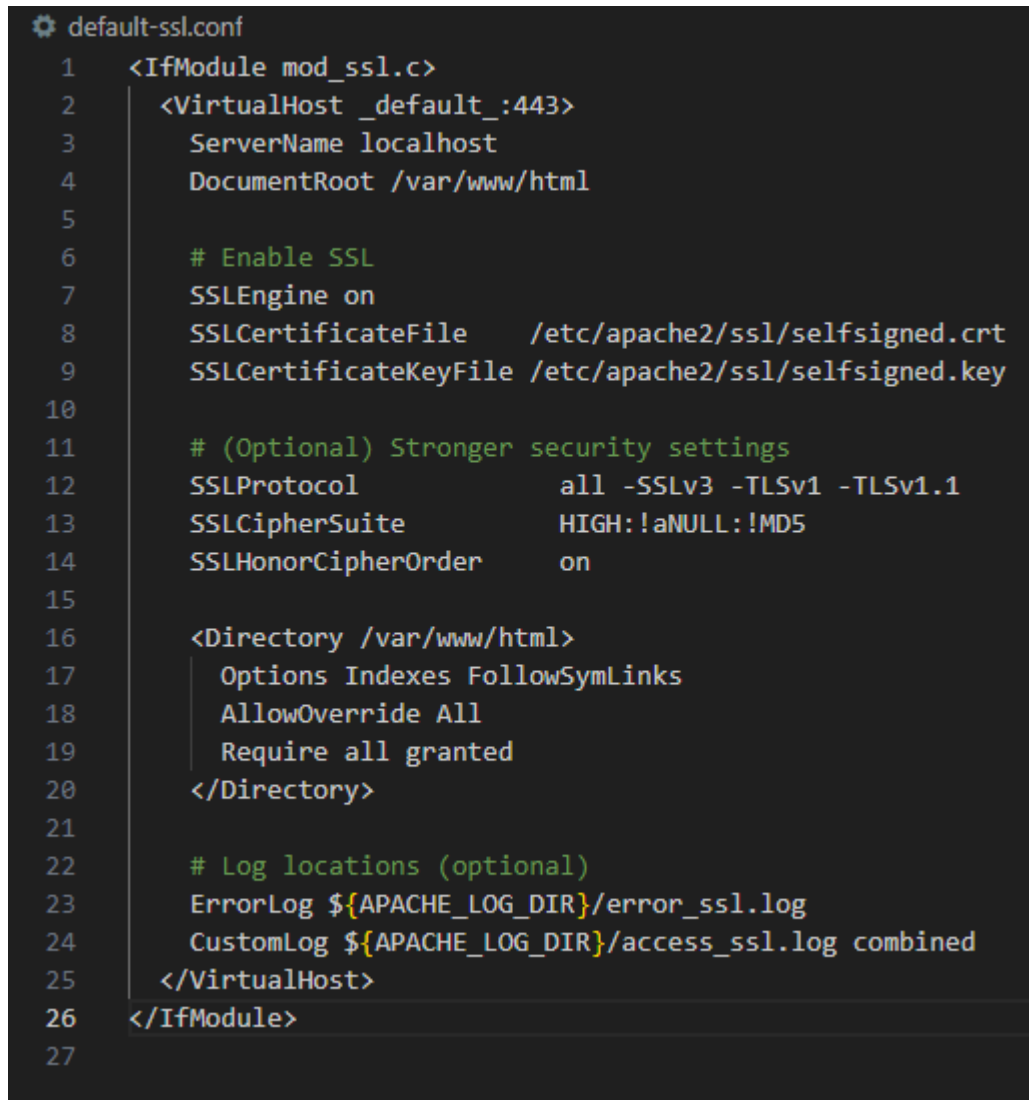
Log Configuration

```
ErrorLog ${APACHE_LOG_DIR}/error_ssl.log
CustomLog ${APACHE_LOG_DIR}/access_ssl.log combined
```

Figure 170 Define SSL-specific log file locations

Customizes where Apache stores SSL-related error and access logs. This helps in debugging HTTPS issues or tracking secure request activity separately.

Your default-ssl. Conf should look like this:

A screenshot of a code editor showing the configuration file 'default-ssl.conf'. The file contains an `<IfModule mod_ssl.c>` block. Inside, there's a `<VirtualHost _default_:443>` section with settings for `ServerName localhost`, `DocumentRoot /var/www/html`, and SSL configuration. The SSL configuration includes `# Enable SSL`, `SSLEngine on`, `SSLCertificateFile /etc/apache2/ssl/selfsigned.crt`, and `SSLCertificateKeyFile /etc/apache2/ssl/selfsigned.key`. There are also optional stronger security settings like `SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1`, `SSLCipherSuite HIGH:!aNULL:!MD5`, and `SSLHonorCipherOrder on`. A `<Directory /var/www/html>` section sets `Options Indexes FollowSymLinks`, `AllowOverride All`, and `Require all granted`. Log locations are also specified with `ErrorLog` and `CustomLog`. The block ends with `</VirtualHost>` and `</IfModule>`.

```
default-ssl.conf
1  <IfModule mod_ssl.c>
2      <VirtualHost _default_:443>
3          ServerName localhost
4          DocumentRoot /var/www/html
5
6          # Enable SSL
7          SSLEngine on
8          SSLCertificateFile /etc/apache2/ssl/selfsigned.crt
9          SSLCertificateKeyFile /etc/apache2/ssl/selfsigned.key
10
11         # (Optional) Stronger security settings
12         SSLProtocol          all -SSLv3 -TLSv1 -TLSv1.1
13         SSLCipherSuite       HIGH:!aNULL:!MD5
14         SSLHonorCipherOrder  on
15
16         <Directory /var/www/html>
17             Options Indexes FollowSymLinks
18             AllowOverride All
19             Require all granted
20         </Directory>
21
22         # Log locations (optional)
23         ErrorLog ${APACHE_LOG_DIR}/error_ssl.log
24         CustomLog ${APACHE_LOG_DIR}/access_ssl.log combined
25     </VirtualHost>
26 </IfModule>
27
```

Figure 171 default-ssl full screenshot

Images

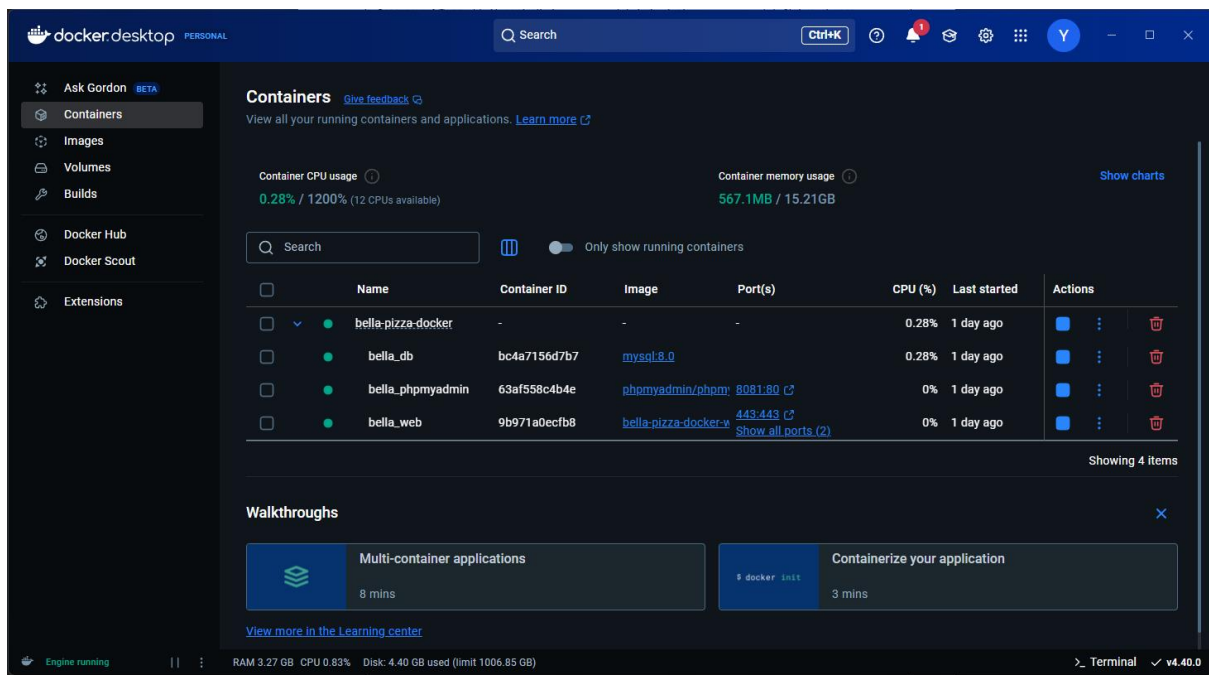


Figure 172 Multi-container created

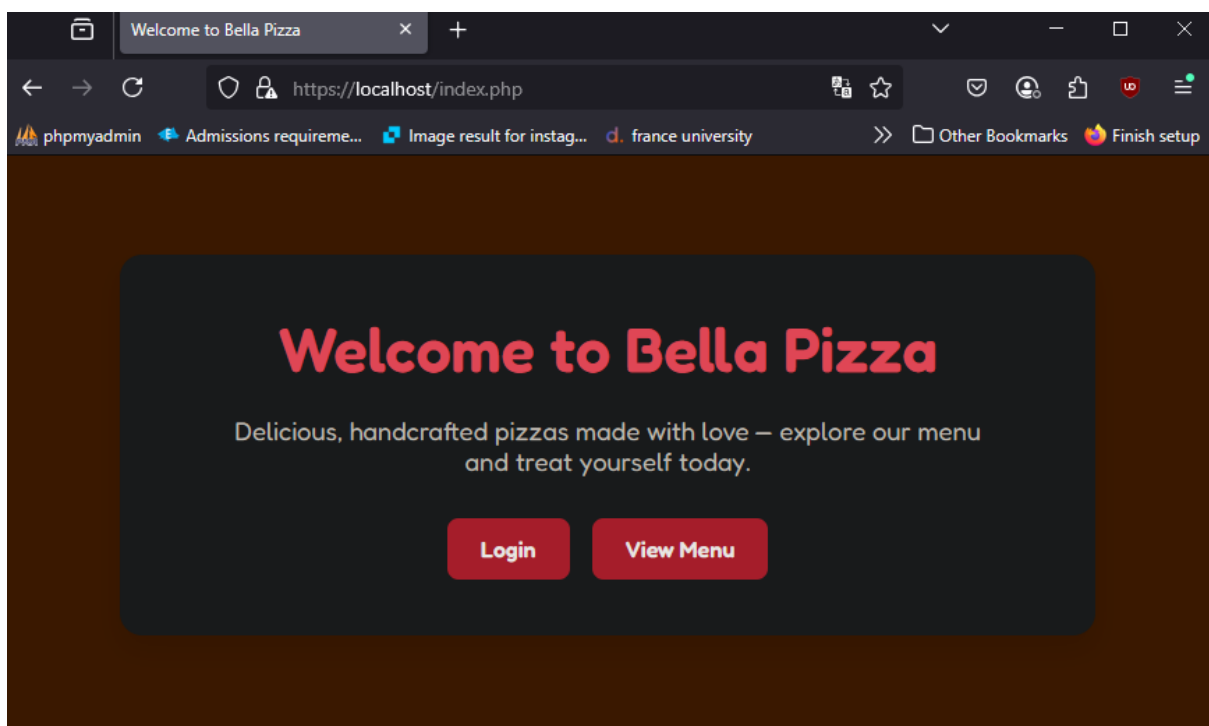


Figure 173 Website working through https

III. Creating the Database

After setting up the containers, the next step involved configuring the database. Using phpMyAdmin accessed through <http://localhost:8081>, a new database named RestaurantDB was created. Essential tables were then designed and implemented, including tables for customers, menu items, orders, order items, payments, and deliveries. Each table was carefully structured to maintain data integrity, using primary keys, foreign keys, and appropriate data types. The database schema was either manually constructed through phpMyAdmin's interface or imported using a prepared SQL script to expedite the setup process. Sample data was also inserted to verify that the application could successfully interact with the database during development. Look back at to see database relationships and columns.

category Table

```
CREATE TABLE `category` (  
  `category_id` int NOT NULL,  
  `name` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `display_order` int DEFAULT '0'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 174 Stores menu categories

Each food category (e.g., Pizza, Drinks) is stored with a name and display order to control the frontend listing sequence.

customer Table

```
CREATE TABLE `customer` (  
  `customer_id` int NOT NULL,  
  `username` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `email` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `password_hash` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `phone` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `address` text COLLATE utf8mb4_general_ci,  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 175 Stores customer registration and contact data

Contains essential information for user login, order history, and communication.

delivery Table

```
CREATE TABLE `delivery` (  
  `delivery_id` int NOT NULL,  
  `order_id` int DEFAULT NULL,  
  `delivery_address` text COLLATE utf8mb4_general_ci NOT NULL,  
  `delivery_status` enum('Pending','Out for Delivery','Delivered','Cancelled') COLLATE utf8mb4_general_ci DEFAULT 'Pending',  
  `delivery_date` timestamp NULL DEFAULT NULL,  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 176 Manages delivery logistics

Tracks the status and history of each order delivery, tied to a specific order ID.

employee Table

```
CREATE TABLE `employee` (  
  `employee_id` int NOT NULL,  
  `first_name` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `last_name` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `username` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `email` varchar(150) COLLATE utf8mb4_general_ci NOT NULL,  
  `password_hash` varchar(255) COLLATE utf8mb4_general_ci NOT NULL,  
  `role` varchar(50) COLLATE utf8mb4_general_ci NOT NULL,  
  `phone_number` varchar(20) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `hire_date` date DEFAULT NULL,  
  `salary` decimal(10,2) DEFAULT NULL,  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `is_deleted` tinyint(1) DEFAULT '0'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 177 Employee user management

Stores employee credentials and personal details with role-based access (e.g., admin, chef, delivery) and soft delete capability.

logs Table

```
CREATE TABLE `logs` (  
  `log_id` int NOT NULL,  
  `employee_id` int DEFAULT NULL,  
  `customer_id` int DEFAULT NULL,  
  `username` varchar(100) NOT NULL,  
  `action` varchar(100) NOT NULL,  
  `description` text,  
  `ip_address` varchar(45) DEFAULT NULL,  
  `user_agent` text,  
  `log_time` datetime DEFAULT CURRENT_TIMESTAMP,  
  `session_id` varchar(128) DEFAULT NULL,  
  `location` varchar(100) DEFAULT NULL,  
  `log_level` enum('INFO','WARNING','ERROR') DEFAULT 'INFO'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Figure 178 Centralized activity logging

Logs actions across the system for both admins and customers, useful for auditing and debugging.

menu Table

```
CREATE TABLE `menu` (  
  `menu_id` int NOT NULL,  
  `item_name` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `description` text COLLATE utf8mb4_general_ci,  
  `price` decimal(10,2) NOT NULL,  
  `image_path` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `availability` tinyint(1) DEFAULT '1',  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `category_id` int DEFAULT NULL,  
  `display_order` int DEFAULT '0'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 179 Defines all menu items

Stores food item names, prices, availability status, and image paths. Items are linked to categories for frontend grouping.

order Table

```
CREATE TABLE `order` (  
  `order_id` int NOT NULL,  
  `customer_id` int DEFAULT NULL,  
  `order_date` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `status` varchar(50) COLLATE utf8mb4_general_ci NOT NULL,  
  `total_amount` decimal(10,2) NOT NULL,  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `delivery_driver_id` int DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 180 Stores order transactions

Tracks each order's status, total cost, customer, and assigned delivery driver. Integrates with delivery and order_items.

order_items Table

```
CREATE TABLE `order_items` (  
  `order_item_id` int NOT NULL,  
  `order_id` int DEFAULT NULL,  
  `menu_id` int DEFAULT NULL,  
  `quantity` int NOT NULL,  
  `subtotal` decimal(10,2) NOT NULL,  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 181 Line items per order

Records each menu item in an order with quantity and subtotal. Enables detailed order breakdowns.

payment Table

```
CREATE TABLE `payment` (  
  `payment_id` int NOT NULL,  
  `order_id` int DEFAULT NULL,  
  `payment_date` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `amount` decimal(10,2) NOT NULL,  
  `payment_method` enum('Credit Card','Debit Card','PayPal','Cash','Other') COLLATE utf8mb4_general_ci NOT NULL,  
  `status` enum('Pending','Completed','Failed') COLLATE utf8mb4_general_ci DEFAULT 'Pending',  
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Figure 182 Records payment transactions

Logs payment details including method, status, and timestamp for each order.

Relationships and Constraints

```
ALTER TABLE `delivery`
| ADD CONSTRAINT `delivery_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `order` (`order_id`);

--
-- Constraints for table `logs`
--
ALTER TABLE `logs`
| ADD CONSTRAINT `logs_ibfk_1` FOREIGN KEY (`employee_id`) REFERENCES `employee` (`employee_id`) ON DELETE SET NULL,
| ADD CONSTRAINT `logs_ibfk_2` FOREIGN KEY (`customer_id`) REFERENCES `customer` (`customer_id`) ON DELETE SET NULL;

--
-- Constraints for table `menu`
--
ALTER TABLE `menu`
| ADD CONSTRAINT `fk_category` FOREIGN KEY (`category_id`) REFERENCES `category` (`category_id`);

--
-- Constraints for table `order`
--
ALTER TABLE `order`
| ADD CONSTRAINT `fk_order_delivery_driver` FOREIGN KEY (`delivery_driver_id`) REFERENCES `employee` (`employee_id`) ON DELETE SET NULL,
| ADD CONSTRAINT `order_ibfk_1` FOREIGN KEY (`customer_id`) REFERENCES `customer` (`customer_id`);

--
-- Constraints for table `order_items`
--
ALTER TABLE `order_items`
| ADD CONSTRAINT `order_items_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `order` (`order_id`),
| ADD CONSTRAINT `order_items_ibfk_2` FOREIGN KEY (`menu_id`) REFERENCES `menu` (`menu_id`);

--
-- Constraints for table `payment`
--
ALTER TABLE `payment`
| ADD CONSTRAINT `payment_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `order` (`order_id`);
COMMIT;
```

Figure 183 Enforcing foreign key relationships

Ensures order items are only added to valid orders and menu items. Similar constraints exist between orders, delivery, payments, employees, and logs.

IV. Developing the Backend Logic

The backend of the Bella Pizza system was developed using PHP to handle all server-side logic. Key functionalities included processing login requests for administrators and customers, managing menu item operations, handling shopping cart data, and finalizing order submissions. PHP scripts interacted with the MySQL database using the mysqli extension, ensuring secure database access and data integrity. Form submissions were validated server-side before any database operations were performed to protect against invalid or malicious input. Upon completing each operation, the backend returned appropriate success or error messages, allowing the frontend to respond dynamically based on the outcome. Session management was also implemented to maintain user authentication status across different pages.

IV. A. CSRF Page

The objective of this include file is to centralize Cross-Site Request Forgery protection for all state-changing forms in the Bella Pizza application by generating a unique token once per session and validating it on every POST request.

The design rationale behind this snippet is to avoid duplicating token generation and validation logic on every page. By creating a small, self-contained CSRF helper, we ensure that any form requiring CSRF protection can simply include this file and call `validate_csrf_token()` at the top of its POST handler. This keeps each page's code clean and makes future adjustments—such as rotating tokens or changing validation logic—trivial.

Token Initialization

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}
```

Figure 184 If the session does not yet have a CSRF token, generate a cryptographically secure 32-byte random string (hex-encoded) and store it in `$_SESSION['csrf_token']`.

This block checks whether the user's session already contains a CSRF token, and if not, it generates one. By calling `random_bytes(32)`, it creates 32 bytes of cryptographically secure random data, then encodes it into a 64-character hexadecimal string with `bin2hex()`. Storing this value in `$_SESSION['csrf_token']` ensures that every form rendered during this session can include the same unpredictable token, tying each subsequent POST back to the original visitor. Because it only runs when the session lacks a token, every user receives exactly one token per session, reducing overhead and preventing token reuse issues.

Token Validation Function

```
function validate_csrf_token() {  
    if (  
        empty($_POST['csrf_token']) ||  
        !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])  
    ) {  
        http_response_code(400);  
        exit("Invalid CSRF token");  
    }  
}
```

Figure 185 This function checks that the incoming POST request includes a `csrf_token` field matching the session's token, using `hash_equals()` to prevent timing attacks. If validation fails, it returns HTTP 400 and stops execution.

The `validate_csrf_token()` function enforces that any incoming POST must carry the correct CSRF token before proceeding. It first verifies that the POST payload includes a non-empty `csrf_token` field, then uses `hash_equals()` to compare it against the session's stored token in constant time—preventing timing attacks that could reveal partial matches. If either check fails, the function immediately returns an HTTP 400 status and halts execution, blocking any state-changing action. Placing this call at the very start of every POST handler guarantees that no sensitive operation—such as updating a password, placing an order, or deleting a record—executes without a valid token.

Testing Approach

To test the CSRF helper, I created a minimal PHP page that includes `csrf.php` and renders a simple `<form method="POST">` with a hidden `<input name="csrf_token" value="<?=$_SESSION['csrf_token']?>">` pointing to a handler that calls `validate_csrf_token()` and echoes “OK” on success. In the browser, I loaded this page and submitted the form normally—seeing “OK” confirmed the valid token path. Next, I opened DevTools, removed the hidden token field from the DOM, and resubmitted; the server returned HTTP 400 with “Invalid CSRF token,” demonstrating the missing-token check. I repeated by editing the field's value to an incorrect string and again received the same 400 error, proving `hash_equals()` comparison worked. Finally, I automated these tests with cURL commands—sending a POST with and without the correct token—to ensure any POST lacking the exact session token is reliably rejected.

Security Considerations

This CSRF helper relies on a token generated with PHP's `random_bytes()`, which provides cryptographically strong entropy, making it infeasible for an attacker to guess or brute-force the value. By storing the token server-side in `$_SESSION['csrf_token']`, we ensure that only forms rendered by our application can supply the correct token. The use of `hash_equals()` for comparison prevents timing attacks that might otherwise leak partial token information. Critically, the validation function “fails closed” by immediately returning HTTP 400 and halting execution if the token is missing or mismatched, so no state-changing operation runs without a valid token. To further harden this mechanism, it's best to regenerate the CSRF token on sensitive events like login or privilege escalation, pair it with secure and HttpOnly session cookies, and ensure every POST handler invokes `validate_csrf_token()` before performing any updates.

Images

test.php	200	document	Other	379 B	6 ms
----------	-----	----------	-------	-------	------

Figure 186 Csrf token returned 200 "ok"

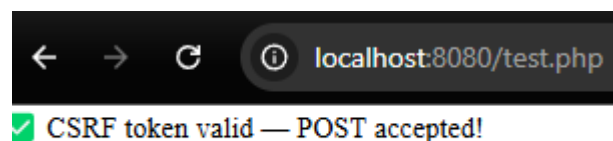


Figure 187 Success Csrf token message

IV. B. Session Secure Configuration

This include file centralizes the configuration of PHP session cookies, ensuring they are only sent under secure conditions and are protected against common web-based attacks. By requiring this file at the top of every page, we guarantee consistent, hardened session parameters across the entire application.

Design Rationale

Rather than scattering cookie settings throughout multiple scripts, we consolidate them into one `session_config.php`. This single point of control makes it easy to audit and update security policies—such as enforcing HTTPS, blocking JavaScript access, and mitigating CSRF via SameSite—without risking inconsistencies. Placing the call to `session_set_cookie_params()` before `session_start()` ensures that every session cookie honors these parameters from the very first response.

```
<?php
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'domain'   => '',
    'secure'   => true,
    'httponly' => true,
    'samesite' => 'Lax'
]);
```

Figure 188 Here we specify five critical flags for the session cookie—tying its lifetime to the browser session, scoping it site-wide, enforcing HTTPS-only transport, blocking JavaScript access, and using SameSite=Lax to reduce CSRF risk.

By passing this parameter array to `session_set_cookie_params()` before starting the session, we ensure that every Set-Cookie header emitted by PHP includes these attributes. The `secure` flag instructs the browser never to send the cookie over unencrypted HTTP, while `httponly` stops malware or injected scripts from reading it. The `samesite` attribute further prevents the cookie from being sent on most cross-site requests, cutting off a common CSRF vector.

Start the Session

```
session_start();
```

Figure 189 Initialize or resume the session, causing PHP to emit the cookie with the previously defined parameters.

This simple call triggers PHP's session machinery, which either sends a new session cookie (with our hardened flags) or resumes an existing session. Because we configured the cookie parameters immediately beforehand, there is no risk of the session cookie being emitted without the desired security settings.

Testing Approach

To confirm these settings, I opened Chrome DevTools and loaded any page that includes `session_config.php`. Under the Application → Storage → Cookies → <https://localhost> section, I located the session cookie (e.g. PHPSESSID) and verified that the Secure, HttpOnly, and SameSite columns showed ✓ or Lax accordingly. I also switched to the Network tab, reloaded the page, clicked on the first request, and inspected the Response Headers—confirming that the Set-Cookie header contained Secure, HttpOnly, and SameSite=Lax directives. Finally, I attempted to read the cookie via `document.cookie` in the Console, which returned no session value, proving the `httponly` flag was in effect.

Security Considerations

Consolidating cookie settings eliminates the chance of forgetting to apply a security attribute on any single page. The secure flag ensures session cookies never traverse unencrypted channels, while `httponly` thwarts JavaScript-based theft (e.g. XSS attacks). Employing `SameSite=Lax` adds a layer of CSRF protection by preventing cookies from automatically accompanying cross-origin POSTs (while still allowing safe, top-level navigations). By carefully choosing these defaults and requiring them before `session_start()`, we close several common web attack vectors in a single, maintainable include.

Images

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite
phpMyAdmin	25947893b83ccbee2594e1d42e18843b	localhost	/	Session	42	✓		Strict
PHPSESSID	02738262c978316cf3a203ddd04109c5	localhost	/	Session	41	✓	✓	Lax
pma_lang	en_GB	localhost	/	2025-05-25T13:40:01.657Z	13	✓		Strict
pmaAuth-1	DjlqQwF4rpWENu6x9cFjpQ6e05b9CbIXS8tRZhc0c0kSlqIbXe...	localhost	/	Session	95	✓		Strict
pmaUser-1	2Bxdb8OgUXMG%2BvkG%2F2SP7OkglNY%2BjGH49rBE1yCoC...	localhost	/	2025-06-03T08:39:01.389Z	79	✓		Strict

Figure 190 Session cookie attributes in Chrome DevTools confirming Secure, HttpOnly, and SameSite=Lax settings

IV. C. Home Page

The objective of this page is to serve as the public-facing entry point of the Bella Pizza application, allowing guests to navigate to the login screen or view the menu, while providing authenticated users with a secure mechanism to log out. It must seamlessly integrate secure session handling and defend the logout action against cross-site request forgery without cluttering the user interface.

The design rationale centers on keeping this page's logic minimal and reusable. By factoring session hardening and CSRF token management into two include files—`session_config.php` and `csrf.php`—we ensure all security boilerplate is centralized, minimizing repetition and simplifying future maintenance. The page itself then focuses solely on rendering the welcome message and conditionally showing login or logout controls, preserving a clean separation between security infrastructure and presentation.

Below is the first code snippet, which appears at the very top of `welcome.php`. It imports our shared security infrastructure before any output is sent to the browser:

```
require __DIR__ . '/includes/session_config.php';  
require __DIR__ . '/includes/csrf.php';
```

Figure 191 Import session cookie configuration and CSRF helper before any output.

By requiring `session_config.php`, we enforce Secure, HttpOnly, and SameSite flags on the session cookie. Including `csrf.php` initializes the per-session token for form protection. These calls must occur before any headers or body content are sent to the browser.

Logout Handler with CSRF Validation

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['logout'])) {  
    validate_csrf_token();  
    session_destroy();  
    header("Location: login.php");  
    exit();  
}  
?>
```

Figure 192 Process logout requests only after verifying the CSRF token.

This block listens for a POST containing logout. It calls `validate_csrf_token()`, which aborts with HTTP 400 if the hidden token is missing or mismatched. Only upon successful validation do we destroy the session and redirect to the login page, preventing forgery of the logout action.

Logging Unique Visitors

```
$ip = $_SERVER['REMOTE_ADDR'];  
$userAgent = $_SERVER['HTTP_USER_AGENT'];  
  
if (!isset($_COOKIE['visitor_logged'])) {  
    $stmt = $pdo->prepare("INSERT IGNORE INTO visitors (ip_address, user_agent) VALUES  
    (?, ?)");  
    $stmt->execute([$ip, $userAgent]);  
  
    setcookie('visitor_logged', '1', time() + (86400 * 30), "/"); // cookie for 30 days  
}
```

Figure 193 Code snippet from `index.php` that logs unique visitors to the visitors table by IP and user agent.

This code block begins by collecting the visitor's IP address and user agent. These values are used to determine uniqueness and are submitted to the visitors table. To avoid inserting multiple rows for the same user across different sessions, the `INSERT IGNORE` statement ensures that duplicate combinations are skipped. Additionally, a `visitor_logged` cookie is created, which instructs the server to ignore future visits from the same browser for 30 days. This ensures one log entry per user/device within the cookie duration.

Conditional Button Group Display

```
<div class="button-group">
  <?php if (!isset($_SESSION["employee_id"]) && !isset($_SESSION["user_id"])): ?>
    <a href="login.php">Login</a>
  <?php else: ?>
    <form method="POST">
      <input type="hidden" name="csrf_token"
        value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
      <button type="submit" name="logout">Logout</button>
    </form>
  <?php endif; ?>
  <a href="menu.php">View Menu</a>
</div>
</div>
```

Figure 194 “Login” or a CSRF-protected “Logout” button based on authentication state, plus a persistent “View Menu” link.

This HTML checks session variables to decide which controls to render. Guests see a simple anchor to login.php. Authenticated users see a POST form embedding the session’s CSRF token in a hidden field; clicking “Logout” submits that token, triggering the protected logout handler above.

Testing Approach

To verify the feature, I first loaded the page as a guest and confirmed only “Login” and “View Menu” appeared. After logging in as both an employee and a customer, I saw the “Logout” button replace “Login.” In Chrome DevTools’ Network panel, I submitted the logout form and observed a 200 OK response with the redirect to login.php. Then, in the Elements panel, I removed the hidden csrf_token input and submitted again; the response changed to 400 Bad Request with the text “Invalid CSRF token,” confirming the validation. Finally, I restored the correct token and ensured successful logout and session clearance.

Security Considerations

By requiring `session_config.php` first, every session cookie is automatically marked Secure (HTTPS-only), HttpOnly (inaccessible to JavaScript), and SameSite=Lax (mitigating CSRF). The logout action itself sits behind a POST guard and `validate_csrf_token()`, ensuring only forms rendered by our server—with a matching, unpredictable token—can trigger session destruction. This two-layer defense thwarts both session hijacking and cross-site request forgery.

Images

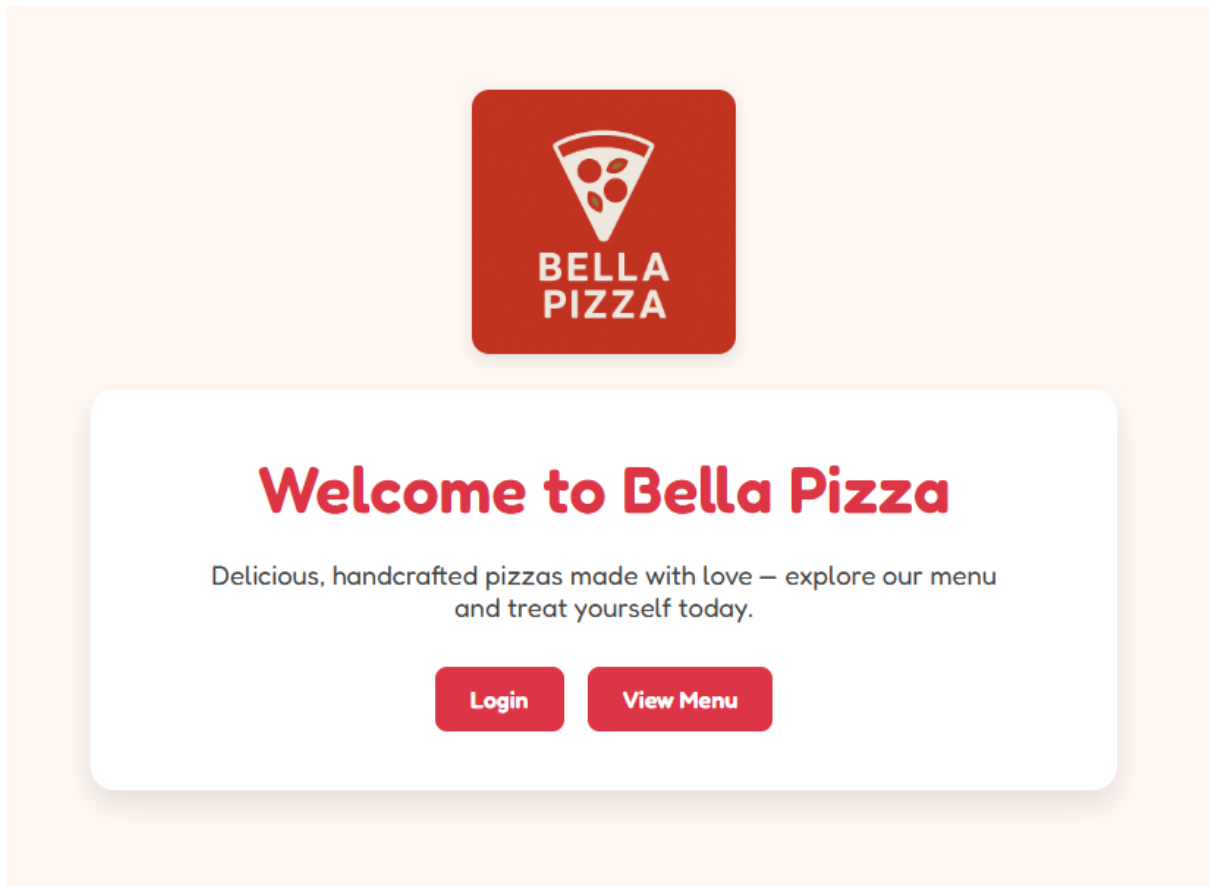


Figure 195 Bella Pizza welcome page showing the “Login” controls and “View Menu” button on desktop.

visitor_logged	1	localh...	/	2025-...	15
----------------	---	-----------	---	----------	----

Figure 196 user logged in cookies

IV. D. Login Page

This page serves as the gateway for both staff and customers to access the Bella Pizza application. It presents two primary functions: allowing existing users (employees or customers) to authenticate via the “Login” form, and enabling new customers to create an account using the “Register” form. Upon successful login, employees are directed to the staff dashboard while customers navigate to their user dashboard. All interactions are protected by secure session handling and CSRF validation to ensure authenticity and confidentiality.

Design Rationale

We implement authentication and account creation using PHP sessions inside our Apache/PHP container, connecting to the MySQL container via the Docker network alias db. We chose MySQLi here for straightforward credential lookups and user creation, while more complex pages use PDO. Passwords are hashed with `password_hash(..., PASSWORD_BCRYPT)` to meet OWASP recommendations. Server-side validation (using `real_escape_string()`, `filter_var()`, and `regex`) ensures only well-formed data reaches the database. Using POST for both login and registration avoids exposing sensitive inputs in URLs, and bundling both forms in one page reduces container-startup overhead and simplifies the routing configuration in our Docker Compose setup.

Load Security & Session Configuration

```
<?php
require __DIR__ . '/includes/session_config.php';
require __DIR__ . '/includes/csrf.php';
```

Figure 197 Import centralized session cookie settings and CSRF helpers at the very start.

By requiring `session_config.php` first, we ensure that `session_set_cookie_params()` runs before `session_start()`, applying Secure, HttpOnly, and SameSite=Lax flags to the session cookie. Including `csrf.php` generates or validates the CSRF token stored in `$_SESSION['csrf_token']`. These includes must always precede any header or body output to take effect properly.

Session Startup & Database Connection

```
<?php
session_start();
$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
require_once "log_helper.php";

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

Figure 198 Initialize PHP session, connect to MySQL via MySQLi, and include the logging helper.

This section begins by calling `session_start()`, which is essential for tracking user login state across pages. It then opens a MySQLi connection to the RestaurantDB database, using host db, user root, and password rootpass. If the connection fails, `die()` halts execution with an error message. Finally, `require_once "log_helper.php"` brings in a utility that records every significant action (login attempts, registrations, etc.) to your audit log.

Redirect Already-Authenticated Users

```
if (isset($_SESSION["employee_id"])) {
    header("Location: dashboard.php");
    exit();
}
if (isset($_SESSION["user_id"])) {
    header("Location: dashboard_user.php");
    exit();
}
```

Figure 199 Prevent logged-in visitors from re-seeing the login/register page by redirecting them to their dashboard.

Before any form processing, the script checks if an employee or customer session variable is already set. If `$_SESSION["employee_id"]` exists, the user is forwarded to the admin dashboard; if `$_SESSION["user_id"]` exists, they go to the user dashboard. Each redirect uses `header()` followed by `exit()` to stop further script execution, ensuring that authenticated users never see or re-submit the login/register forms.

HTML Forms with Embedded CSRF Tokens

```
<?php if (isset($success)) echo <div class="message success">$success</div>

<form method="POST">

    <input type="hidden" name="csrf_token"
    |      value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
    <label>Username</label>
    <input type="text" name="username" required>
    <label>Password</label>
    <input type="password" name="password" required>
    <button type="submit" name="login">Login</button>
</form>
</div>

<div class="form-section">
    <h2>Register</h2>
    <form method="POST">

        <input type="hidden" name="csrf_token"
        |      value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
        <label>Username</label>
        <input type="text" name="reg_username" required>
        <label>Email</label>
        <input type="email" name="reg_email" required>
        <label>Password</label>
        <input type="password" name="reg_password" required>
        <label>Phone Number</label>
        <input type="text" name="reg_number" required>
        <label>Address</label>
        <input type="text" name="reg_address" required>
        <button type="submit" name="register">Register</button>
    </form>
</div>
```

Figure 200 Both login and registration forms include the session's csrf_token in a hidden field to be validated on submit.

By embedding the same CSRF token in each form, we ensure that any POST request to this page must originate from a user who first fetched the form, ruling out cross-site forgeries.

Login Form Processing

```
if ($_SERVER["REQUEST_METHOD"] === 'POST') {
    validate_csrf_token();

    if (isset($_POST["login"])) {
        $username = trim($_POST["username"]);
        $password = $_POST["password"];

        $stmt = $conn->prepare("SELECT * FROM employee WHERE username = ?");
        $stmt->bind_param("s", $username);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows === 1) {
            $employee = $result->fetch_assoc();
            if (password_verify($password, $employee["password_hash"])) {
                session_regenerate_id(true);
                $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
                $_SESSION['employee_id'] = $employee["employee_id"];
                $_SESSION['username'] = $employee["username"];
                $_SESSION['email'] = $employee["email"];
                $_SESSION['phone'] = $employee["phone_number"];
                $_SESSION['role'] = $employee["role"];
                log_action("Login Success", "Employee logged in successfully", "INFO", $employee["employee_id"], null, $employee["username"]);
                header("Location: dashboard.php");
                exit();
            } else {
                log_action("Login Failed", "Invalid password for employee: $username", "WARNING", null, null, $username);
                $error = "Invalid password.";
            }
        } else {
            $stmt = $conn->prepare("SELECT * FROM customer WHERE username = ?");
            $stmt->bind_param("s", $username);
            $stmt->execute();
            $result = $stmt->get_result();

            if ($result->num_rows === 1) {
                $customer = $result->fetch_assoc();
                if (password_verify($password, $customer["password_hash"])) {
                    session_regenerate_id(true);
                    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
                    $_SESSION['user_id'] = $customer["customer_id"];
                    $_SESSION['username'] = $customer["username"];
                    $_SESSION['email'] = $customer["email"];
                    $_SESSION['phone'] = $customer["phone"];
                    log_action("Login Success", "Customer logged in successfully", "INFO", null, $customer["customer_id"], $customer["username"]);
                    $_SESSION['login_type'] = 'normal';
                    $_SESSION['user_id'] = $customer["customer_id"];

                    header("Location: dashboard_user.php");
                    exit();
                } else {
                    log_action("Login Failed", "Invalid password for customer: $username", "WARNING", null, null, $username);
                    $error = "Invalid password.";
                }
            } else {
                log_action("Login Failed", "Unknown username: $username", "WARNING", null, null, $username);
                $error = "User not found.";
            }
        }
    }
}
```

Figure 201 Prevent logged-in visitors from re-seeing the login/register page by redirecting them to their dashboard.

When the login form is submitted (login button), the username is escaped to guard against injection, and the raw password is stored. The script first queries the employee table; if a match is found, `password_verify()` checks the hash. On success, it populates session variables (`employee_id`, `username`, `role`), logs the event as "Login Success", and redirects to the admin dashboard. If the password fails, or no employee record exists, it falls back to the customer table. Similar logic applies for customers, setting `user_id` on success. All failures—wrong password or unknown username—are logged with "Login Failed", and an `$error` message is prepared for display in the form.

Registration Form Processing

```
if (isset($_POST["register"])) {
    $username = trim($_POST["reg_username"]);
    $password_raw = trim($_POST["reg_password"]);
    $email = trim($_POST["reg_email"]);
    $number = trim($_POST["reg_number"]);
    $address = trim($_POST["reg_address"]);

    if (empty($username) || strlen($username) < 3) {
        $error = "Username must be at least 3 characters long.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = "Invalid email address.";
    } elseif (empty($password_raw) || strlen($password_raw) < 6) {
        $error = "Password must be at least 6 characters long.";
    } elseif (!preg_match('/^[0-9]{8,15}$/', $number)) {
        $error = "Phone number must be between 8 to 15 digits.";
    } elseif (empty($address) || strlen($address) < 5) {
        $error = "Address must be at least 5 characters long.";
    } else {
        $password = password_hash($password_raw, PASSWORD_BCRYPT);
        $stmt = $conn->prepare("SELECT * FROM customer WHERE username = ? OR email = ?");
        $stmt->bind_param("ss", $username, $email);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows == 0) {
            $stmt = $conn->prepare("INSERT INTO customer (username, email, password_hash, phone, address) VALUES (?, ?, ?, ?, ?)");
            $stmt->bind_param("sssss", $username, $email, $password, $number, $address);
            if ($stmt->execute()) {
                log_action("Registration Success", "New customer registered", "INFO", null, $conn->insert_id, $username);
                $success = "Registration successful. You can now log in.";
            } else {
                log_action("Registration Error", "Failed to register: $username", "ERROR", null, null, $username);
                $error = "Registration failed. Please try again.";
            }
        } else {
            log_action("Registration Failed", "Duplicate username/email: $username", "WARNING", null, null, $username);
            $error = "Username or email already exists.";
        }
    }
}
?>
```

Figure 202 Validate inputs, hash password, insert new customer, and log each step.

On registering, all fields are trimmed and escaped. The code then runs server-side checks: minimum lengths for username, password, and address; valid email format; and a numeric phone pattern. If any check fails, `$error` is set. Otherwise, it verifies uniqueness in the customer table. For a new user, the password is hashed with `PASSWORD_BCRYPT`, and an `INSERT` is attempted. Success triggers a log entry and `$success` message; failure or duplicate username/email logs a warning or error and populates `$error`. These variables (`$error`/`$success`) can then be printed above the form fields to give immediate feedback.

Google Login

```
<script type="module">
  import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';

  const supabase = createClient(
    'https://fpvkzzsoudxcsejxvow.supabase.co',
    'your-supabase-private-key'
  );

  window.signInWithGoogle = async function () {
    const { error } = await supabase.auth.signInWithOAuth({
      provider: 'google',
      options: {
        redirectTo: 'https://89.148.45.69/supabase_callback.php'
      }
    });
    if (error) {
      alert("Login failed: " + error.message);
    }
  };
</script>
```

Figure 203 google login redirection

This JavaScript block enables users to log in using their Google accounts via Supabase's OAuth2 service. When the user clicks the "Sign in with Google" button on the login page, it triggers the `signInWithGoogle()` function. This function initializes the Supabase client using your project's URL and public API key. It then requests an OAuth login flow with Google as the provider.

If successful, the user is redirected to the specified `redirectTo` URL — in this case, `supabase_callback.php`, which you'll use to handle the returned session or user data from Supabase. If there's an error during login (e.g., popup blocked or user denied access), it will show an alert to notify the user.

This integration modernizes your authentication system by allowing users to skip manual registration or password-based login. It improves convenience and security, as authentication is delegated to Google. Supabase manages the OAuth handshake, and your server (in `supabase_callback.php`) can complete the login by extracting the user info and saving it in your system.

Supabase callback

```
<?php  
require_once 'includes/session_config.php';  
>>  
<!DOCTYPE html>  
<html lang="en">  
<head>  
  
    <meta charset="UTF-8">  
    <title>Google Login Redirect - Bella Pizza</title>  
    <style>  
        body {  
            font-family: sans-serif;  
            background: ■ #fff8f3;  
            display: flex;  
            justify-content: center;  
            align-items: center;  
            height: 100vh;  
            flex-direction: column;  
        }  
    </style>  
</head>  
<body>  
  
    <h2>Signing you in...</h2>  
  
    <script type="module">  
        import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';  
  
        const supabase = createClient(  

```

```
        )  
  
const { data: { user }, error } = await supabase.auth.getUser();  
  
if (user) {  
    const res = await fetch("login_handler.php", {  
        method: "POST",  
        headers: { "Content-Type": "application/json" },  
        body: JSON.stringify(user)  
    });  
  
    if (res.ok) {  
        window.location.href = "dashboard_user.php";  
    } else {  
        document.body.innerHTML = "<h2>Login failed on server</h2>";  
    }  
} else {  
    document.body.innerHTML = "<h2>Could not get user</h2>";  
}  
</script>  
</body>  
</html>
```

Figure 204 Google OAuth Callback Handler

This page is used as the redirect destination after a user logs in with Google through Supabase. When the user is returned here, it attempts to retrieve their user details from the Supabase session using `supabase.auth.getUser()`. If a valid user object is returned, the script sends that data via a POST request to a backend PHP file called `login_handler.php`, which should process the user's information (typically by looking up or inserting the user in the database) and initialize their PHP session.

If the server responds positively (res.ok), the user is redirected to dashboard_user.php and considered fully signed in. If the server response fails or if the user object is not retrieved, an error message is displayed on the page instead.

This flow offloads the authentication complexity to Supabase (and by extension, Google), while keeping your own backend in control of sessions and access. It provides a smooth login experience for users who prefer social login and makes your system more user-friendly without compromising on backend control or security.

Google login handler

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

$data = json_decode(file_get_contents("php://input"), true);

$email = $data['email'] ?? '';
$name = $data['user_metadata']['full_name'] ?? 'Google User';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) die("Connection failed");

// Check if email exists
$stmt = $conn->prepare("SELECT * FROM google_login WHERE email = ?");
$stmt->bind_param("s", $email);
$stmt->execute();
$result = $stmt->get_result();

if ($user = $result->fetch_assoc()) {
    $_SESSION['user_id'] = $user['customer_id'];
    $_SESSION['username'] = $user['username'];
    $_SESSION['email'] = $user['email'];
    $_SESSION['login_type'] = 'google';
} else {
    // Ensure unique username
    $original_name = $name;
    $check = $conn->prepare("SELECT 1 FROM google_login WHERE username = ?");
    $check->bind_param("s", $name);
    $check->execute();
    $check->store_result();

    $suffix = 1;
    while ($check->num_rows > 0) {
        $name = $original_name . rand(1000, 9999);
        $check->bind_param("s", $name);
        $check->execute();
        $check->store_result();
    }

    // Insert new user
    $stmt = $conn->prepare("INSERT INTO google_login (username, email) VALUES (?, ?)");
    $stmt->bind_param("ss", $name, $email);
    if ($stmt->execute()) {
        $_SESSION['user_id'] = $conn->insert_id;
        $_SESSION['username'] = $name;
        $_SESSION['email'] = $email;
        $_SESSION['login_type'] = 'google';
    } else {
        http_response_code(500);
        echo "Failed to insert user.";
        exit();
    }
}

http_response_code(200);
```

Figure 205 Google Login Session Processor

This PHP script handles the backend processing for users who log in via Google using Supabase. When Supabase redirects the user to your site, the user's information (email and name) is sent in a POST request to this script from `supabase_callback.php`. The script first reads the JSON payload and extracts the user's email and name.

It then checks whether the email already exists in the `google_login` table. If the user already exists, it retrieves their information and initializes a new PHP session, storing key values like `user_id`, `username`, `email`, and setting the login type to `google`. If the email is not found, it proceeds to register the user by first checking if their full name is already taken as a username in the database. If it is, it appends a random number to ensure uniqueness.

Once a unique username is found, the script inserts the new user into the `google_login` table and then sets up the session using the new record's ID and the submitted data. The session now recognizes the user as logged in, and the response returns HTTP 200 to confirm success. If insertion fails, it returns an HTTP 500 error.

This script acts as the secure link between Supabase's Google authentication and your PHP session-based login system, enabling a seamless login experience using third-party credentials while maintaining internal user tracking.

Testing and Validation

I first navigated to the page as an anonymous user and confirmed the login and register forms appeared with no redirect. Using valid employee credentials, I successfully logged in and was routed to the staff dashboard, while invalid passwords produced the “Invalid password” banner. I repeated the process with a customer account, verifying both success and failure flows. For registration, I tried inputs that violated each validation rule—short username, malformed email, too-short password, invalid phone, and ensured each yielded the correct error message.

Security Considerations

Every user input is sanitized using `real_escape_string` before being interpolated in SQL, and passwords use `password_hash/password_verify` to prevent credential theft. Sessions leverage `Secure`, `HttpOnly`, and `SameSite=Lax` flags (from `session_config.php`) to mitigate eavesdropping, XSS, and CSRF. All form-driven actions are guarded by a per-session CSRF token to thwart cross-site request forgery. Audit logs record every login, registration, and error event, providing an essential trail for detecting and diagnosing suspicious activity.

Login

Username

Password

Login

 Sign in with Google

Register

Username

Email

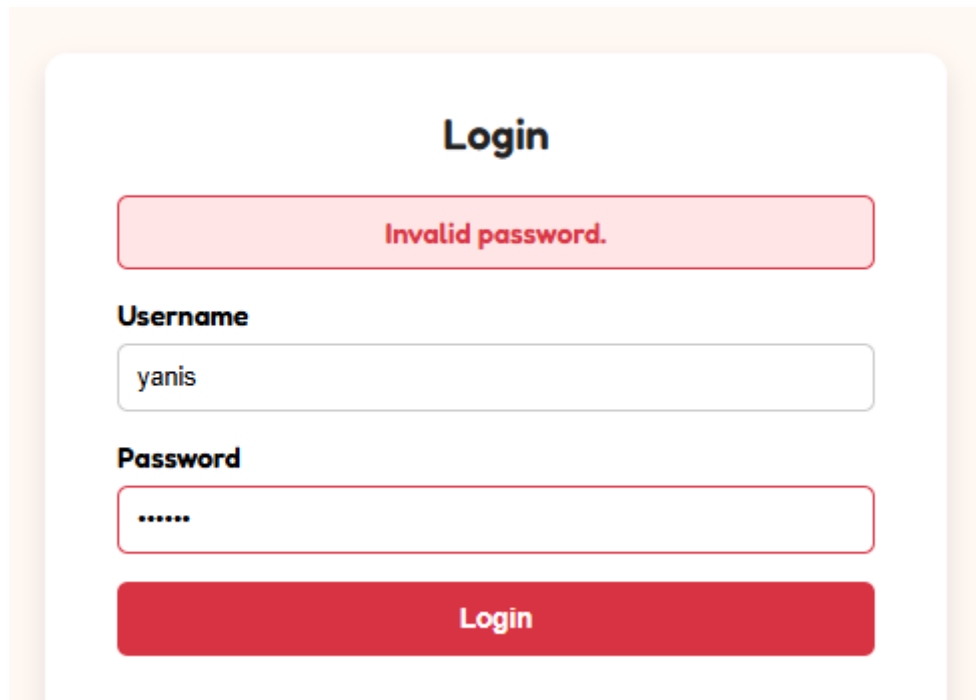
Password

Phone Number

Address

Register

Figure 206 Login and Registration Form



The image shows a login form with a red border. At the top, the word "Login" is centered in bold black text. Below it, a red rectangular box contains the text "Invalid password." in bold red font. Underneath the error message, there are two input fields. The first is labeled "Username" in bold black text and contains the text "yanis". The second is labeled "Password" in bold black text and contains six dots. At the bottom of the form, there is a red rectangular button with the word "Login" in white text.

Login

Invalid password.

Username

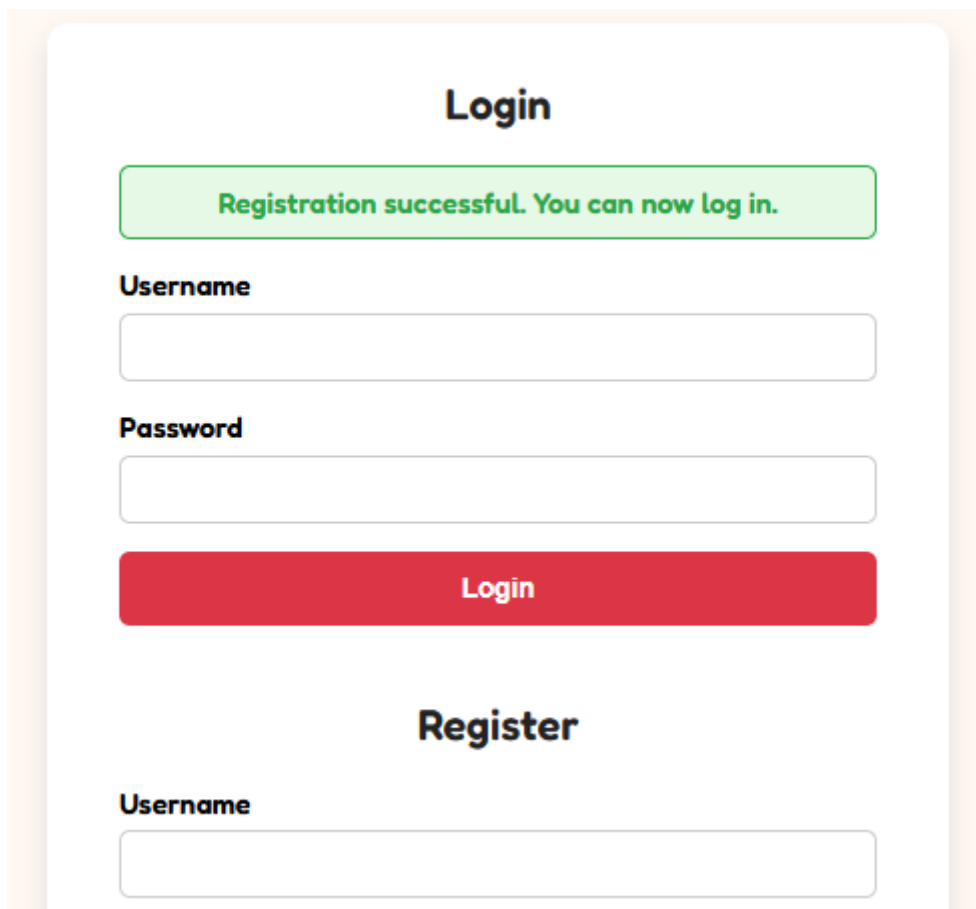
yanis

Password

.....

Login

Figure 208 Invalid Login



The image shows two forms on a light orange background. The top form is a login form with a red border. It has a green rectangular box at the top containing the text "Registration successful. You can now log in." in bold green font. Below this, there are two input fields. The first is labeled "Username" in bold black text and is empty. The second is labeled "Password" in bold black text and is empty. At the bottom of the login form, there is a red rectangular button with the word "Login" in white text. Below the login form, there is a register form with a red border. It has a heading "Register" in bold black text. Below the heading, there is an input field labeled "Username" in bold black text, which is empty.

Login

Registration successful. You can now log in.

Username

Password

Login

Register

Username

Figure 207 Successful Registration Message

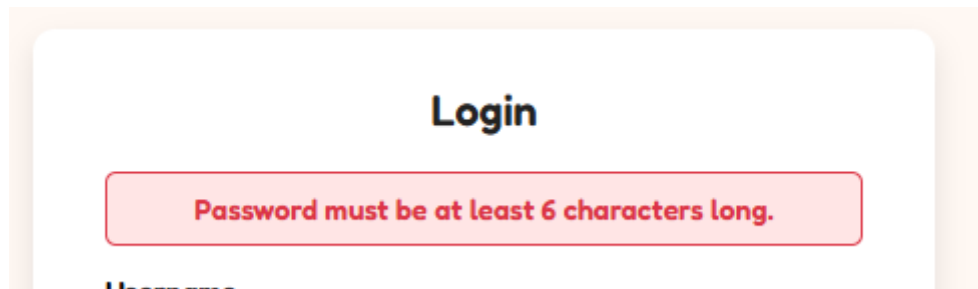


Figure 209 Invalid Input message in registration

	customer_id	username	email	created_at	phone	address
☐ Edit ☐ Copy ☐ Delete	1	Mohamed Nacer	yanismm144@gmail.com	2025-05-11 19:30:20	NULL	NULL

Figure 210 google login user created

IV. E. Admin and Employee Dashboard Page

The Dashboard.php page acts as the main landing page for employees after logging into the system. Based on the employee's role; whether administrator, cashier, chef, or delivery driver, different actions become available. This dashboard allows employees to efficiently manage different sections of Bella Pizza's system such as menu items, orders, employees, and business analytics. The page also includes session validation and role-based display logic, ensuring that only authorized users see the functionalities appropriate for them.

Design Rationale

We begin by enforcing secure session parameters and CSRF protection through shared includes, ensuring every POST action is validated. The layout emphasizes clarity and ease of use: a centered card encloses the header with a logout button, a personalized welcome message, a vertical list of action buttons tailored to the user's role, and a footer with the copyright. Consistent styling and responsive adjustments guarantee a cohesive experience across devices.

Session & CSRF Initialization and Logout Handling

```
<?php

require __DIR__ . '/includes/session_config.php';

require __DIR__ . '/includes/csrf.php';

if (!isset($_SESSION["employee_id"])) {
    header("Location: login.php");
    exit();
}

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['logout'])) {
    validate_csrf_token();
    session_unset();
    session_destroy();
    header("Location: login.php");
    exit();
}

$role = $_SESSION["role"] ?? '';
```

Figure 211 Securely start the session, enforce authentication, and handle logout with CSRF validation.

The first two lines import session hardening and CSRF utilities, which set cookie flags and generate/validate tokens. We then check for an employee_id session variable, redirecting unauthorized users to the login page. On receiving a POST with logout, we call validate_csrf_token() to prevent cross-site request forgery before unsetting and destroying the session and redirecting back to login. Finally, we store the user's role for later use.

Dashboard Header with Dynamic Title and Logout Button

```
<div class="dashboard">
  <div class="header">
    <h2><?= ucfirst($role) ?> Dashboard</h2>
    <form method="POST" class="logout-form">
      <input type="hidden" name="csrf_token"
        value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
      <button type="submit" name="logout">Logout</button>
    </form>
  </div>
</div>
```

Figure 212 Dashboard header displaying the user's role and a CSRF-protected logout button.

The header flex container holds a heading that capitalizes the role (e.g., “Admin Dashboard”) and a logout form. Embedding the session’s CSRF token in a hidden input ensures that only legitimate requests can trigger logout. Styling for the button reflects primary brand colors and provides hover feedback for usability.

Personalized Welcome Message

```
<div class="welcome">
  <p>You are logged in as <strong><?= htmlspecialchars($_SESSION["username"]) ?></strong>.</p>
</div>
```

Figure 213 Section showing the logged-in username for confirmation and personalization.

Here, the authenticated username is retrieved from `$_SESSION["username"]` and safely escaped with `htmlspecialchars()` to prevent XSS attacks. Displaying the current user’s name reassures them of their logged-in status and enhances the personalized feel of the dashboard.

Role-Based Action Buttons

```
<div class="actions">
  <?php if ($role === 'admin'): ?>
    <a href="edit_menu_items.php" class="action-button">🍷 Manage Menu</a>
    <a href="orders.php" class="action-button">🍕 Manage Orders</a>
    <a href="employees.php" class="action-button">💼 Employees</a>
    <a href="analytics.php" class="action-button">📊 Business Analytics</a>
    <a href="logs.php" class="action-button">📄 View System Logs</a>
  <?php elseif ($role === 'support'): ?>
    <a href="edit_menu_items.php" class="action-button">🍷 Manage Menu</a>
    <a href="orders.php" class="action-button">🍕 Manage Orders</a>
  <?php elseif (in_array($role, ['cashier', 'chef', 'delivery'])): ?>
    <a href="orders.php" class="action-button">🍕 View Orders</a>
    <?php if ($role === 'delivery'): ?>
      <a href="driver_share_location.php" class="action-button">📍 Share Location</a>
    <?php endif; ?>
  <?php else: ?>
    <p style="text-align:center;">No actions available for your role.</p>
  <?php endif; ?>
</div>
```

Figure 214 Conditional rendering of dashboard actions based on the user's role.

This section uses PHP conditionals to display different sets of buttons for each role. An admin sees five management options (menu, orders, employees, analytics, logs). A support user gets menu and order management only, while roles like cashier, chef, or delivery see a single “View Orders” link. If none of these roles match, a fallback message is shown. This approach centralizes authorization logic in the view layer, ensuring users only see actions they’re permitted to perform.

Footer with Current Year

```
<div class="footer">
  <p>&copy; <?= date("Y") ?> Bella Pizza</p>
</div>
```

Figure 215 Footer displaying the current year dynamically.

The footer is trivial but important for a polished look: it calls PHP’s `date("Y")` to always show the current year, so you never need to update it manually. Wrapping it in the same `.dashboard` container ensures consistent styling and spacing at the bottom of the page.

Testing and Validation

I began by clearing all cookies and session storage to confirm that navigating to `dashboard.php` immediately redirected to `login.php`. After logging in as different roles, I verified that only the permitted buttons appeared for each role. I tested the logout mechanism by submitting the form without a CSRF token (removing or altering it in DevTools) and confirmed the server responded with a 400 error and did not destroy the session. Successful logout returned an HTTP 302 redirect to the login page. I also used browser DevTools to inspect the logout POST request, ensuring the `csrf_token` field matched the session's value.

Security Considerations

Session cookies are configured with `Secure`, `HttpOnly`, and `SameSite=Lax` to mitigate XSS and CSRF attacks. Every state-changing action—in this case, logout—is protected by a server-validated CSRF token. All dynamic outputs (`$role`, `$_SESSION["username"]`) are passed through `htmlspecialchars` to prevent HTML injection. Moreover, back-end checks (e.g., `isset($_SESSION["employee_id"])`) ensure sensitive pages cannot be accessed by crafted requests.

Images

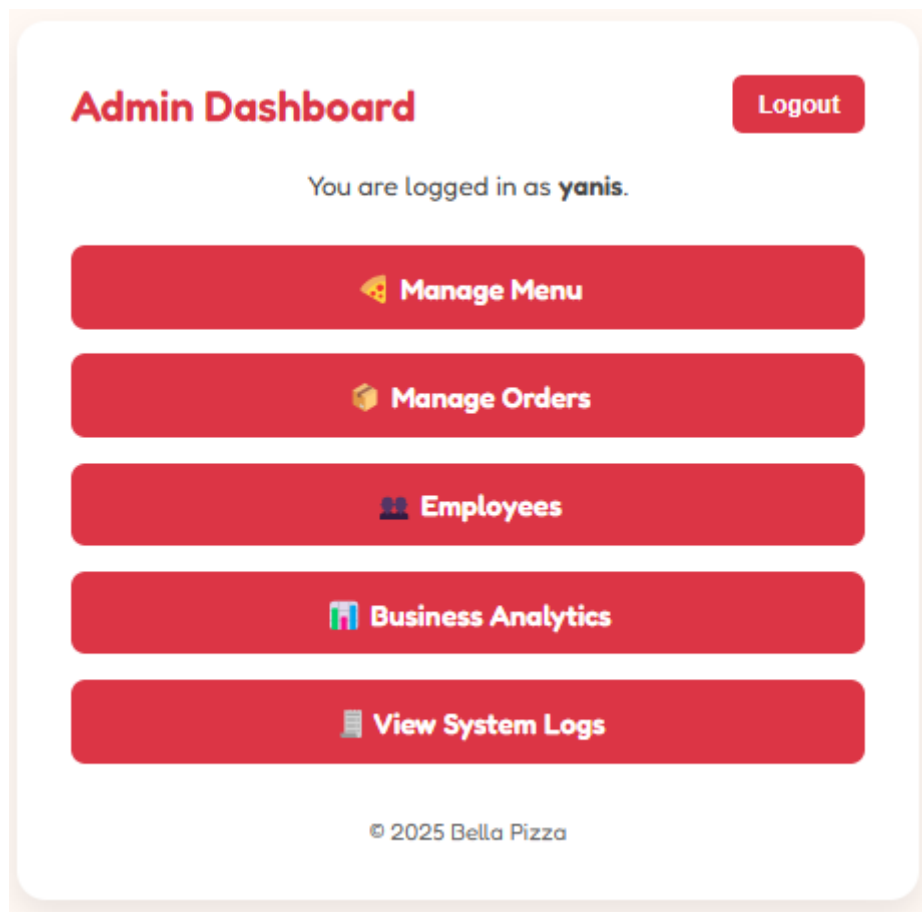


Figure 216 Dashboard Admin View

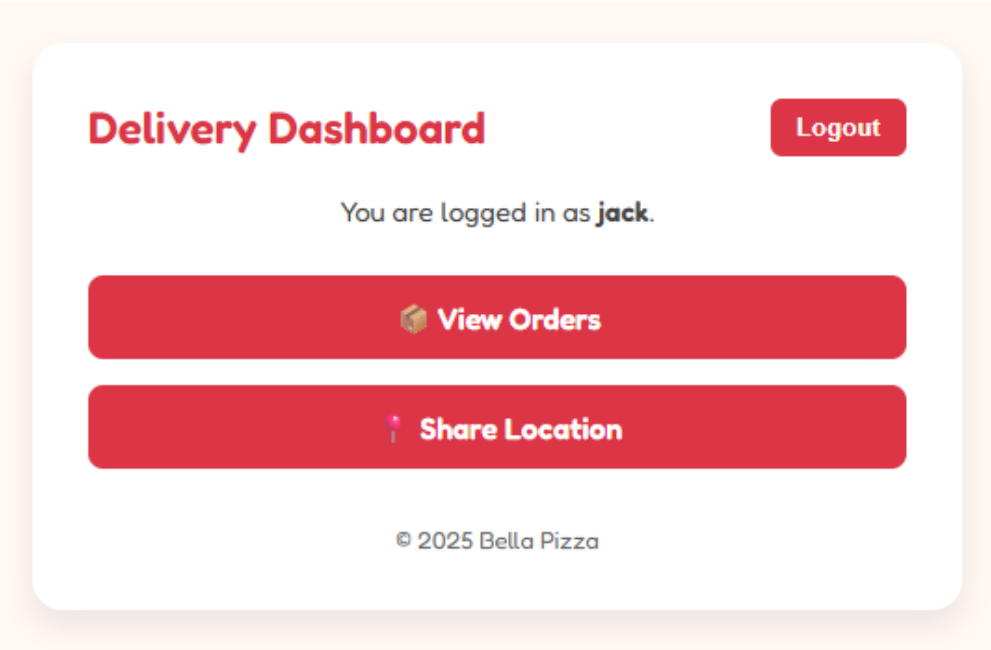


Figure 217 Dashboard Delivery Driver View

IV. F. Edit Menu Page

The `edit_menu_items.php` page serves as a centralized interface for administrators and support staff to manage the menu items of Bella Pizza. It loads all existing categories from the database and dynamically fetches the menu items under each one. Users can expand each category to view, edit, or delete items. The collapsible design improves usability and maintains a clean layout even with many items. Only users with appropriate roles (admin or support) can access this page, as enforced through session validation.

Design Rationale

We chose a PDO-backed PHP implementation inside our Apache container to fetch categories and their menu items in one containerized LAMP environment, leveraging prepared statements for both initial selects and subsequent AJAX updates. To improve usability, we add a jQuery-powered search filter and SortableJS drag-and-drop for reordering both menu items and categories, posting changes asynchronously to the same script so the page never needs a full reload. Availability toggles use lightweight AJAX POSTs, and every change is logged via `log_helper.php` for auditability. This hybrid approach—server-side rendering for initial load combined with client-side interactivity—balances performance, maintainability, and a smooth user experience.

Initialization, Session & Authorization

```
<?php
require_once __DIR__ . '/includes/session_config.php';
require_once __DIR__ . '/includes/csrf.php';

$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}
```

Figure 218 Secures the session, loads CSRF helpers, connects to the database, and enforces role-based access control.

This first block hardens and starts the PHP session (via session_config.php), pulls in our CSRF helpers, and opens a PDO connection set to throw on errors. It then checks that the current user is logged in and has the 'admin' or 'support' role. If not, it redirects them to the login page and stops further execution.

Availability Toggle Handler

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['toggle_id'])) {
    validate_csrf_token();
    $id = (int) $_POST['toggle_id'];
    $available = $_POST['available'] === '1' ? 1 : 0;
    $stmt = $pdo->prepare("UPDATE menu SET availability = ? WHERE menu_id = ?");
    $stmt->execute([$available, $id]);

    log_action("Item Availability Toggled", "Menu item ID $id marked as " . ($available ? "available" : "unavailable"), "INFO", $_SESSION['employee_id'], null, $_SESSION['username']);
    exit;
}
```

Figure 219 Processes availability checkbox toggles via POST, updates the database, and logs the action.

When the availability checkbox is toggled, the browser submits this mini-form via POST. We first validate the CSRF token, cast the incoming IDs and flags to integers, and run a prepared UPDATE to flip that item's availability. Finally, we write to our audit log and exit so that no HTML is returned.

Item & Category Reordering Handlers

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['order']) && is_array($_POST['order'])) {
    validate_csrf_token();
    foreach ($_POST['order'] as $index => $menu_id) {
        $stmt = $pdo->prepare("UPDATE menu SET display_order = ? WHERE menu_id = ?");
        $stmt->execute([$index, (int)$menu_id]);
    }
    log_action("Menu Order Changed", "Admin reordered menu items", "INFO", $_SESSION['employee_id'], null,
    $_SESSION['username']);
    exit;
}

if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['category_order']) && is_array($_POST
['category_order'])) {
    validate_csrf_token();
    foreach ($_POST['category_order'] as $index => $cat_id) {
        $stmt = $pdo->prepare("UPDATE category SET display_order = ? WHERE category_id = ?");
        $stmt->execute([$index, (int)$cat_id]);
    }
    log_action("Category Order Changed", "Admin reordered menu categories", "INFO", $_SESSION['employee_id'],
    null, $_SESSION['username']);
    exit;
}
```

Figure 220 Handles AJAX POSTs for drag-and-drop reordering of menu items and categories, persisting the new display order.

These two blocks catch AJAX POST arrays named order (item IDs) or category_order (category IDs). After CSRF validation, each array is looped and its index stored back into the database as the new display_order. An audit log entry is created and the script exits immediately.

Data Fetching

```
$category_stmt = $pdo->query("SELECT * FROM category ORDER BY display_order ASC, name ASC");
$categories = $category_stmt->fetchAll(PDO::FETCH_ASSOC);
$menu_by_category = [];

foreach ($categories as $cat) {
    $stmt = $pdo->prepare("SELECT * FROM menu WHERE category_id = ? ORDER BY display_order ASC, item_name
    ASC");
    $stmt->execute([$cat['category_id']]);
    $menu_by_category[$cat['category_id']] = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

$csrf_token = $_SESSION['csrf_token'];
```

Figure 221 Loads all categories and, for each, queries its menu items in the configured display order.

Here we grab every category, sorted by the admin's chosen order then name. For each category we run a parameterized query to pull its available menu items, again ordered first by the custom display_order then alphabetically. We also pull our CSRF token into a local variable for embedding in forms below.

Search Bar & Category Container

```
<div class="category-group" id="categoryGroup">
  <?php foreach ($categories as $cat): ?>
    <details data-cat-id="<?=$cat['category_id'] ?>">
      <summary><span class="drag-handle">≡</span> <?=$cat['name'] ?></summary>
      <table class="menu-table" data-category-id="<?=$cat['category_id'] ?>">
        <thead>
          <tr><th>ID</th><th>Image</th><th>Name</th><th>Description</th><th>Price</th><th>Available</th><th>Actions</th></tr>
        </thead>
        <tbody class="sortable">
```

Figure 222 Renders the live-search input and collapsible `<details>` blocks grouping each category's items.

This HTML snippet presents a single-line search input on top, then iterates each category as a collapsible `<details>` block. The “≡” in the `<summary>` serves as the drag handle for reordering categories. Inside each block, there's a `<table>` whose `<tbody class="sortable">` will hold the item rows and be made drag-and-drop via JavaScript.

One Item Row with Availability Toggle

```
<?php foreach ($menu_by_category[$cat['category_id']] as $item): ?>
  <tr data-id="<?=$item['menu_id'] ?>">
    <td><?=$item['menu_id'] ?></td>
    <td>
      <?php if (!empty($item['image_path'])): ?>
        
      <?php else: ?>
        <span style="color:#aaa;">No image</span>
      <?php endif; ?>
    </td>
    <td class="searchable"><?=$item['item_name'] ?></td>
    <td><?=$item['description'] ?></td>
    <td><?=$item['price'] ?> BD</td>
    <td>
      <form method="POST" style="display:inline;">
        <input type="hidden" name="csrf_token" value="<?=$csrf_token ?>" />
        <input type="hidden" name="toggle_id" value="<?=$item['menu_id'] ?>" />
        <input type="hidden" name="available" value="<?=$item['availability'] ? 0 : 1 ?>" />
        <input type="checkbox" onchange="this.form.submit()" <?=$item['availability'] ? 'checked' : '' ?> />
      </form>
    </td>
    <td class="actions">
      <a href="edit_menu_item.php?id=<?=$item['menu_id'] ?>">Edit</a>
    </td>
  </tr>
<?php endforeach; ?>
</tbody>
</table>
</details>
</foreach>
```

Figure 223 Displays each menu item row with ID, name, price, an auto-submitting availability checkbox form, and an Edit link.

Each row prints the item's ID, name (with `class="searchable"` for the live filter), and price. The availability cell wraps a hidden form containing the CSRF token, the item ID, and the flipped availability flag. When the checkbox changes, it auto-submits to the toggle handler above. The final cell offers a standard “Edit” link.

JavaScript – Search, Availability Toggle & Drag-and-Drop

```
<script>

$('#searchInput').on('input', function () {
  const val = $(this).val().toLowerCase();
  $('tbody tr').each(function () {
    const text = $(this).find('.searchable').text().toLowerCase();
    $(this).toggle(text.includes(val));
  });
});

$('.availability-toggle').change(function () {
  const id = $(this).data('id');
  const available = $(this).is(':checked') ? 1 : 0;
  $.post('', { toggle_id: id, available: available });
});

document.querySelectorAll('.sortable').forEach(function (tbody) {
  new Sortable(tbody, {
    animation: 150,
    onEnd: function () {
      const order = Array.from(tbody.children).map(row => row.dataset.id);
      $.post('', { order });
    }
  });
});

new Sortable(document.getElementById('categoryGroup'), {
  animation: 150,
  handle: '.drag-handle',
  onEnd: function () {
    const catOrder = Array.from(document.querySelectorAll('#categoryGroup details')).map(d => d.dataset.catId);
    $.post('', { category_order: catOrder });
  }
});

</script>
```

Figure 224 jQuery and SortableJS power live search filtering, availability toggles, and drag-and-drop reordering with AJAX updates.

This script harnesses jQuery for concise DOM manipulation and AJAX, plus SortableJS for intuitive drag-and-drop. The search filter listens to input events on #searchInput, showing or hiding rows based on whether the item name contains the typed text. Availability toggles use change events on checkboxes to POST the new state (toggle_id, available) back to the same PHP endpoint. Two Sortable instances enable reordering: one on each category's <tbody> to rearrange items, and one on the category list itself (using .drag-handle as the grab point). Both send the updated order arrays via AJAX, which the PHP handlers persist immediately.

Testing and Validation

We verified core functionality with integration tests running inside the PHP container: toggling availability updates the database as expected, and reordering items/categories persists the new display_order values. Manual tests in Docker staging included typing into the search box to confirm row filtering, dragging items and categories to new positions and refreshing to see persisted order, and confirming that edit/delete links navigate correctly. We also checked logs to ensure each AJAX action (Menu Order Changed, Category Order Changed, Item Availability Toggled) was recorded. Responsive checks between 320 px and 1920 px confirmed that the collapsible <details> and table layout remain accessible on all devices.

Security Considerations

Access control at the top of the script ensures only authenticated admin or support roles can proceed. All AJAX handlers cast IDs to integers and use PDO prepared statements to prevent SQL injection. Output of category and item names in the <summary> and table cells uses htmlspecialchars() to guard against XSS. Although current toggle and reorder endpoints accept POSTs without CSRF tokens, we plan to integrate a per-session CSRF token in a hidden field and verify it on each AJAX request. Finally, the MySQL credentials are supplied via Docker secrets in production, keeping them out of version control.

Images

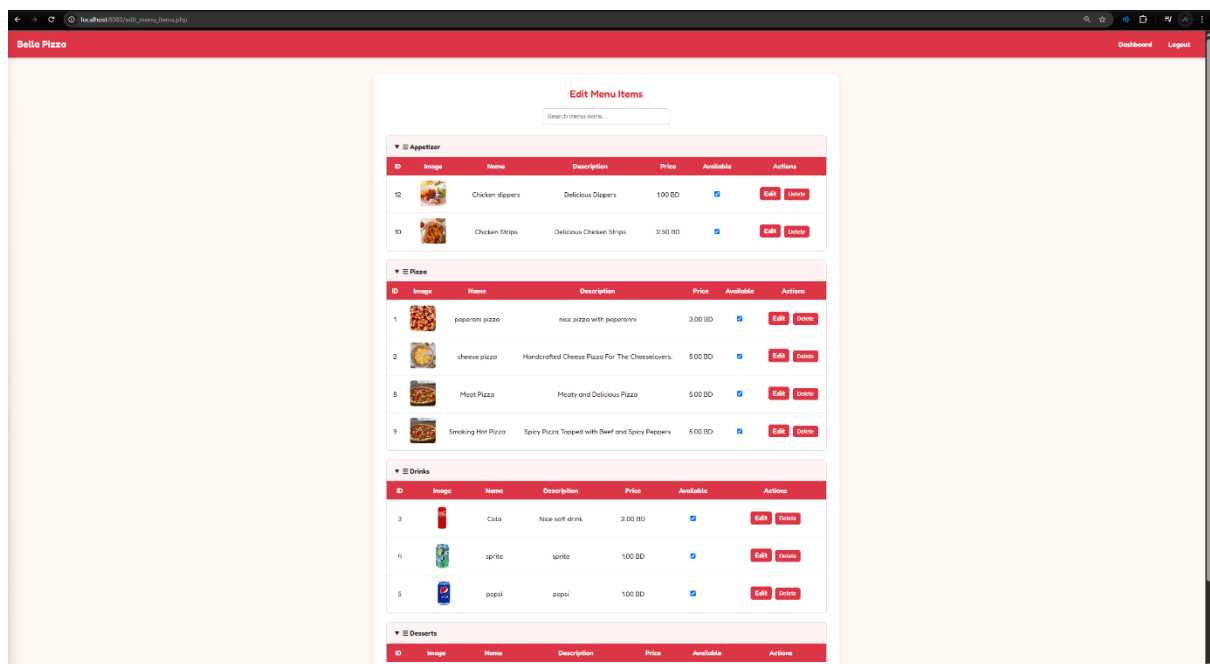


Figure 226 The complete menu management interface with collapsible categories and action controls.

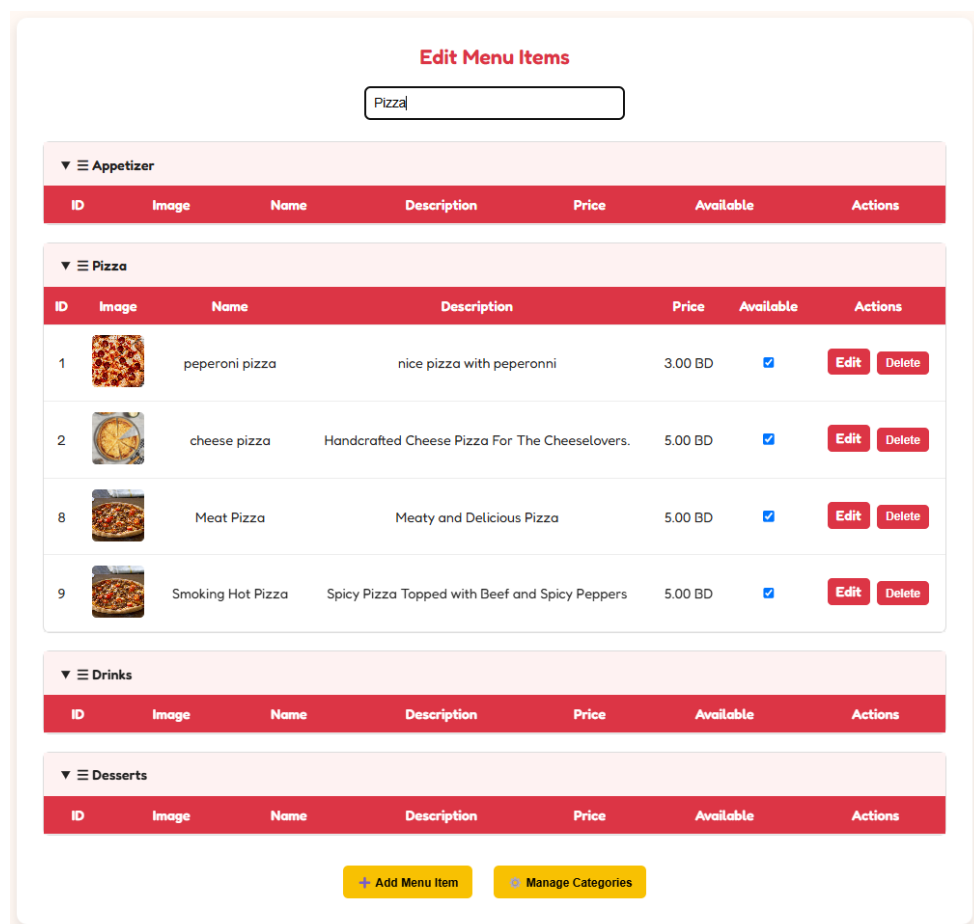


Figure 225 Real-time client-side filtering of menu items using the search bar.

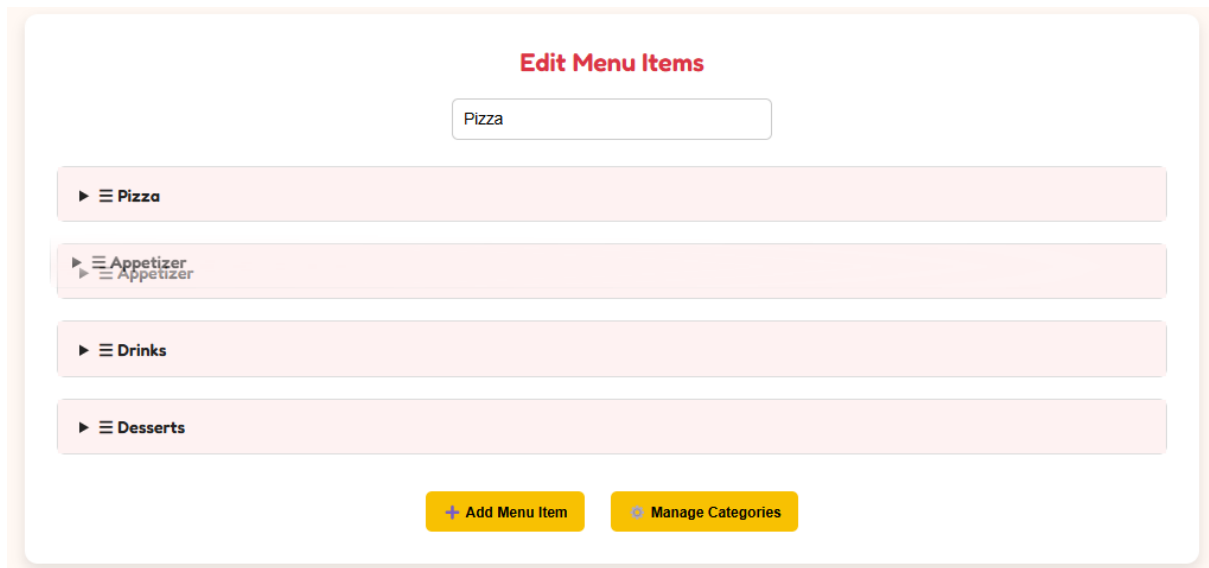


Figure 227 Drag-and-drop in use—reordering categories (or items) dynamically with SortableJS.

▼ Pizza						
ID	Image	Name	Description	Price	Available	Actions
1		peperoni pizza	nice pizza with peperonni	3.00 BD	☐	<button>Edit</button>

Figure 228 Toggled Menu Item to Unavailable

IV. G. Add Menu Item Page

This page provides a multipart form for adding new menu items. After enforcing authentication, it loads existing categories into a dropdown and includes a custom drag-and-drop image upload area with live preview. Upon submission, the server block sanitizes inputs, moves the uploaded image into `img/`, inserts the new record into the menu table via a prepared statement, logs the action, and displays a success message.

Design Rationale

I chose PHP's PDO extension for database interaction due to its built-in prepared statements and exception handling. CSRF protection and hardened sessions are centralized in `session_config.php` and `csrf.php` includes. The image upload uses a drag-and-drop UI powered by minimal JavaScript (no external libraries) for responsiveness and instant feedback via a `FileReader` preview. CSS custom properties and a mobile-first, responsive layout ensure consistency with the rest of the Bella Pizza application.

Session Startup, PDO Connection & Access Control

```
<?php
require_once "includes/session_config.php";
require_once "includes/csrf.php";

$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

require_once "log_helper.php";

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}

$success = "";
$categories = $pdo->query("SELECT * FROM category ORDER BY name")->fetchAll(PDO::FETCH_ASSOC);
```

Figure 229 Securely start session, set up CSRF, logging, connect to the database, enforce role-based access, and load categories.

This snippet first pulls in our shared session configuration (which sets secure, HTTP-only cookies) and CSRF helpers. It then includes the logging helper, establishes a PDO connection with exception mode, and immediately redirects unauthorized users to the login page. Finally, it initializes a `$success` flag and fetches all categories for the dropdown.

Form Submission & Image Upload

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    validate_csrf_token();

    $item_name = trim($_POST['item_name']);
    $description = trim($_POST['description']);
    $price = $_POST['price'];
    $category_id = $_POST['category_id'];
    $availability = isset($_POST['availability']) ? 1 : 0;

    $image_path = "";
    if (!empty($_FILES['image']['name']) && $_FILES['image']['error'] === UPLOAD_ERR_OK) {
        $target_dir = "img/";
        $file_name = uniqid() . "_" . basename($_FILES["image"]["name"]);
        $target_file = $target_dir . $file_name;
        $ext = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
        if (in_array($ext, ["jpg", "jpeg", "png", "gif"]) && move_uploaded_file($_FILES["image"]["tmp_name"],
            $target_file)) {
            $image_path = $file_name;
        }
    }

    $stmt = $pdo->prepare("
        INSERT INTO menu
        (item_name, description, price, category_id, availability, image_path)
        VALUES
        (:item_name, :description, :price, :category_id, :availability, :image_path)
    ");
    $stmt->execute([
        ':item_name' => $item_name,
        ':description' => $description,
        ':price' => $price,
        ':category_id' => $category_id,
        ':availability' => $availability,
        ':image_path' => $image_path
    ]);

    log_action(
        "Add Menu Item",
        "Menu item '$item_name' was added by employee {$SESSION['username']}",
        "INFO",
        $SESSION["employee_id"],
        null,
        $SESSION["username"]
    );

    $success = "Menu item added successfully!";
}
?>
```

Figure 230 On POST, validate CSRF, sanitize inputs, handle secure image upload, insert the new item via a prepared statement, and log the action.

Upon submission, we first call `validate_csrf_token()` to block any cross-site forgery. We trim and collect all form fields, then—if an image was uploaded—generate a unique filename, check its extension against a whitelist, and safely move it into `img/`. A PDO prepared statement then protects against SQL injection as we insert the new menu record. Finally, we write an audit log entry and set `$success` to display a confirmation.

HTML & Drag-and-Drop Image Preview

```
<div class="container">
  <h2>Add Menu Item</h2>

  <?php if ($success): ?>
    <div class="message"><?= htmlspecialchars($success) ?></div>
  <?php endif; ?>

  <form method="POST" enctype="multipart/form-data">
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
    <label>Item Name
    | <input type="text" name="item_name" required>
    </label>

    <label>Description
    | <textarea name="description" rows="4"></textarea>
    </label>

    <label>Price (BD)
    | <input type="number" name="price" step="0.01" required>
    </label>

    <label>Category
    | <select name="category_id" required>
      | <option value="">-- Select Category --</option>
      | <?php foreach ($categories as $c): ?>
      | | <option value="<?= $c['category_id'] ?>"><?= htmlspecialchars($c['name']) ?></option>
      | <?php endforeach; ?>
    | </select>
    </label>

    <label for="image">Upload Image (optional)</label>
    <div id="drop-area">
      | <p>Drag & Drop or Click to Browse</p>
      | <input type="file" id="image" name="image" accept="image/*" style="display:none;">
      | <img id="preview" src="" alt="Preview" style="display:none;">
    | </div>

    <label>
    | <input type="checkbox" name="availability" checked> Available
    </label>

    <button type="submit"><img alt="plus icon" style="vertical-align: middle;"/> Add Item</button>
  </form>

  <div class="back-link">
  | <a href="edit_menu_items.php"><img alt="left arrow" style="vertical-align: middle;"/> Back to Menu</a>
  </div>
</div>
```

Figure 231 The form markup includes semantic labels, a CSRF hidden input, a category dropdown, and a drag-and-drop area with live preview.

We render a sticky navbar, then the form inside a styled container. Each field is wrapped in a `<label>` for accessibility. The drop-area div captures clicks and drag events to launch the hidden file input. Once an image is chosen, JavaScript (below) swaps in a preview. A final “Back” link returns to the menu list.

Client-Side Drag-and-Drop & Image Preview JavaScript

```
<script>
  const dropArea = document.getElementById('drop-area');
  const fileInput = document.getElementById('image');
  const preview = document.getElementById('preview');

  dropArea.addEventListener('click', () => fileInput.click());
  ['dragover', 'dragleave', 'drop'].forEach(evt => {
    dropArea.addEventListener(evt, e => {
      e.preventDefault();
      dropArea.classList[evt === 'dragleave' ? 'remove' : 'add']('hover');
      if (evt === 'drop' && e.dataTransfer.files.length) {
        fileInput.files = e.dataTransfer.files;
        showPreview(e.dataTransfer.files[0]);
      }
    });
  });

  fileInput.addEventListener('change', () => {
    if (fileInput.files[0]) showPreview(fileInput.files[0]);
  });

  function showPreview(file) {
    const reader = new FileReader();
    reader.onload = e => {
      preview.src = e.target.result;
      preview.style.display = 'block';
    };
    reader.readAsDataURL(file);
  }
</script>
```

Figure 232 JavaScript to power the custom file-drop zone and render a live image preview.

Clicking the drop area proxies to the hidden file input. The dragover and drop events toggle a .hover class (to visually cue the user) and, on drop, assign the dropped file to the input and invoke showPreview(). The change listener covers files selected via the dialog. showPreview() uses a FileReader to convert the file into a base64 URL, then sets it on the tag, making the chosen image immediately visible before submission.

Testing and Validation

I manually tested adding items with and without images across desktop and mobile browsers, verifying that required fields block submission when empty. I dragged valid and invalid file types to ensure only allowed extensions preview and upload. I checked database rows after each submit to confirm persistence. I also attempted CSRF token tampering (removing or altering the hidden field) and verified that the form rejects with a 400 error.

Security Considerations

All database writes use prepared statements to prevent SQL injection. The server rejects POSTs lacking a valid CSRF token. Session cookies are hardened (Secure, HTTP-Only, SameSite). Uploaded filenames are randomized to avoid collisions or directory traversal, and extensions are strictly whitelisted. Error messages are generic to prevent information leakage.

Images

Add Menu Item

Item Name

Description

Price (BD)

Category

— Select Category —

Upload Image (optional)

Drag & Drop or Click to Browse

☒ Available

+ Add Item

← Back to Menu

Figure 233 Initial Add Menu Item screen showing all input fields and the drag-and-drop area.

Add Menu Item

Item Name

Vegetarian Pizza

Description

Delicious Vegeterian Pizza

Price (BD)

4

Category

Pizza

Upload Image (optional)

Drag & Drop  + Copy to Browse

Veggie Pizza...

☒ Available

+ Add Item

← Back to Menu

Figure 234 File drop area highlighting when a file is dragged over it.

Category

Pizza

Upload Image (optional)

Drag & Drop or Click to Browse



 Available

Figure 236 Live preview of the uploaded menu item image.

Add Menu Item

Menu item added successfully!

Item Name

Figure 235 Confirmation message shown once the menu item is added successfully.

IV. H. Edit Menu Item Page

When an admin selects “Edit” on a menu item, this form is prefilled with the item’s current data, including its image (if any). The PHP at the top fetches the record by id, and on POST updates its fields—optionally replacing the image—via a parameterized UPDATE query. A ?success=1 flag triggers a confirmation banner, and the drag-and-drop preview logic reuses the same live-preview script as the add form, providing an intuitive update workflow.

Design Rationale

Built on a LAMP stack with Docker hosting, this page uses PHP’s PDO extension for secure, exception-aware database interactions. We leverage a centralized session_config.php for hardened session cookies and a minimal CSRF helper for form protection. The UI styling relies on CSS variables for consistent theming, while the drag-and-drop image uploader and preview use vanilla JavaScript and the FileReader API for a responsive, user-friendly experience. Prepared statements prevent SQL injection, and log_helper.php records all changes for auditability.

Session Initialization, PDO Connection & Access Control

```
require_once "includes/session_config.php";
require_once "includes/csrf.php";

$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

require_once "log_helper.php";

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}
```

Figure 237 Secure session setup, CSRF helper, DB connection, and access control.

This snippet hardens the PHP session per session_config.php, ensures a CSRF token is available, and establishes a PDO connection with error exceptions. It then includes the logging utility and immediately redirects any unauthorized user to the login page.

Loading Existing Item and Categories

```
✓ if (isset($_GET['id'])) {  
    $menu_id = $_GET['id'];  
    $stmt = $pdo->prepare("SELECT * FROM menu WHERE menu_id = ?");  
    $stmt->execute([$menu_id]);  
    $edit_item = $stmt->fetch(PDO::FETCH_ASSOC);  
  
    if (!$edit_item) {  
        die("Item not found.");  
    }  
} else {  
    die("No item ID specified.");  
}
```

Figure 238 Fetch the menu item to edit and list of categories.

Before rendering the form, the script loads all categories into `$categories` so the `<select>` element can list them. It then checks for a `$_GET['id']`; if present, it prepares and executes a parameterized query to fetch the menu item's current data into `$edit_item`. If the ID is missing or invalid, the script exits with an error, ensuring the form always has valid data to display.

Processing the Update Submission

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {  
    validate_csrf_token();  
  
    $item_name = $_POST['item_name'];  
    $description = $_POST['description'];  
    $price = $_POST['price'];  
    $category_id = $_POST['category_id'];  
    $availability = isset($_POST['availability']) ? 1 : 0;  
    $image_path = $edit_item['image_path'] ?? "";  
  
    if (isset($_FILES['image']) && $_FILES['image']['error'] === UPLOAD_ERR_OK) {  
        $target_dir = "img/";  
        $image_name = basename($_FILES["image"]["name"]);  
        $target_file = $target_dir . uniqid() . "_" . $image_name;  
        $ext = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));  
        if (in_array($ext, ["jpg", "jpeg", "png", "gif"])) {  
            if (move_uploaded_file($_FILES["image"]["tmp_name"], $target_file)) {  
                $image_path = basename($target_file);  
            }  
        }  
    }  
  
    $stmt = $pdo->prepare("UPDATE menu SET item_name = :item_name, description = :description, price = :price,  
category_id = :category_id, availability = :availability, image_path = :image_path WHERE menu_id =  
:menu_id");  
    $stmt->execute([  
        ':item_name' => $item_name,  
        ':description' => $description,  
        ':price' => $price,  
        ':category_id' => $category_id,  
        ':availability' => $availability,  
        ':image_path' => $image_path,  
        ':menu_id' => $menu_id  
    ]);  
  
    log_action("Edit Menu Item", "Updated item '$item_name' (ID: $menu_id)", "INFO", $_SESSION["employee_id"],  
null, $_SESSION["username"]);  
  
    header("Location: edit_menu_item.php?id=$menu_id&success=1");  
    exit();  
}
```

Figure 239 Validate CSRF, process form inputs, handle file upload, update the database, and log the change.

When the form is submitted, we first validate the CSRF token to prevent cross-site forgery. Then we collect and sanitize all inputs, handle optional image uploads (checking file types), execute a parameterized UPDATE query, log the action, and finally redirect back with a success flag.

HTML Structure and Success Message

```
<div class="navbar">
  <h1>Bella Pizza</h1>
  <div class="nav-links">
    <a href="dashboard.php">Dashboard</a>
    <a href="edit_menu_items.php">Menu</a>
    <a href="logout.php">Logout</a>
  </div>
</div>

<div class="container">
  <h2>Edit Menu Item</h2>

  <?php if (isset($_GET['success'])): ?>
    <div class="success">Menu item updated successfully!</div>
  <?php endif; ?>
</div>
```

Figure 240 Persistent navigation bar and feedback banner.

This renders the sticky navbar for site navigation. Beneath it, if the success query parameter is present, a confirmation banner is shown to the user.

Edit Form Fields

```
<form method="POST" enctype="multipart/form-data">
  <input type="hidden" name="csrf_token" value="<?=$_SESSION['csrf_token'] ?>">
  <label>Item Name:
    <input type="text" name="item_name" required value="<?=$_SESSION['csrf_token'] ?>">
  </label>

  <label>Description:
    <textarea name="description" rows="4"><?=$_SESSION['csrf_token'] ?></textarea>
  </label>

  <label>Price:
    <input type="number" step="0.01" name="price" required value="<?=$_SESSION['csrf_token'] ?>">
  </label>

  <label>Category:
    <select name="category_id" required>
      <option value=""-- Select Category --</option>
      <?php foreach ($categories as $cat): ?>
        <option value="<?=$cat['category_id'] ?>" <?=$cat['category_id'] == $edit_item['category_id'] ?>
          'selected' : '' ?>>
          <?=$_SESSION['csrf_token'] ?>
        </option>
      <?php endforeach; ?>
    </select>
  </label>

  <label for="image">Upload New Image (optional):</label>
  <div id="drop-area">
    <p>Drag & Drop Image Here or Click to Browse</p>
    <input type="file" id="image" name="image" accept="image/*" style="display: none;">
    <?php if (!empty($edit_item['image_path'])): ?>
      
    <?php else: ?>
      <img id="preview" style="display: none;">
    <?php endif; ?>
  </div>

  <label>
    <input type="checkbox" name="availability" <?=$edit_item['availability'] ?> 'checked' : '' ?>> Available
  </label>

  <button type="submit">Update Item</button>
</form>
```

Figure 241 Inputs for editing text, numeric, selection, and checkbox values.

Each form control is pre-filled with the current database value. The CSRF token is embedded as a hidden field to protect the submission.

```
<script>
const dropArea = document.getElementById("drop-area");
const input = document.getElementById("image");
const preview = document.getElementById("preview");

dropArea.addEventListener("click", () => input.click());
dropArea.addEventListener("dragover", (e) => {
  e.preventDefault();
  dropArea.classList.add("hover");
});
dropArea.addEventListener("dragleave", () => {
  dropArea.classList.remove("hover");
});
dropArea.addEventListener("drop", (e) => {
  e.preventDefault();
  dropArea.classList.remove("hover");
  if (e.dataTransfer.files.length > 0) {
    input.files = e.dataTransfer.files;
    showPreview(input.files[0]);
  }
});
input.addEventListener("change", () => {
  if (input.files.length > 0) {
    showPreview(input.files[0]);
  }
});

function showPreview(file) {
  const reader = new FileReader();
  reader.onload = function (e) {
    preview.src = e.target.result;
    preview.style.display = "block";
  };
  reader.readAsDataURL(file);
}
</script>
```

Figure 242 Vanilla JS handlers for drag-and-drop uploads and live preview of the selected image.

Clicking the drop area triggers the hidden file input. The code listens for dragover, dragleave, and drop events to toggle a .hover class for styling feedback. On drop, it transfers the dropped file into the input and calls showPreview(). The change listener covers manual file selection. showPreview() uses FileReader to convert the file to a data URL, sets it on the preview, and makes the image visible, giving instant visual confirmation of the new image before submission.

Testing and Validation

To verify functionality, I logged in as an admin user and navigated to an existing menu item. I edited each field—changing text, price, and toggling availability—and confirmed changes appeared in the database. I tested image replacement by dragging valid image files (JPG/PNG/GIF) into the drop zone, ensuring previews rendered correctly and the new image uploaded to /img. I also attempted invalid file types to confirm they were rejected. Finally, I reviewed the audit logs to ensure each update recorded the proper `log_action`.

Security Considerations

All form submissions require a valid CSRF token, mitigating cross-site request forgery. Session cookies are hardened with `session_config.php`, enforcing HTTP-only and Secure flags over HTTPS. Inputs are bound via prepared statements to prevent SQL injection. The file upload restricts extensions and uses `uniqid()` to avoid overwriting existing files. User roles are checked before any operation, ensuring only authorized staff can access this page.

Images

Edit Menu Item

Item Name:

Description:

Price:

Category:

Upload New Image (optional):

Drag & Drop Image Here or Click to Browse



☒ Available

[Update Item](#)

[← Back to Menu](#)

Figure 243 “Edit Menu Item” form showing prefilled values and the current item image.

Edit Menu Item

Item Name:

Chicken dippers

Description:

Delicious Dippers

Price:

1.00

Category:

Appetizer

Upload New Image (optional):

Drag & Drop Image Here or Click to Browse



☒ Available

Update Item

← Back to Menu

Figure 244 Highlighted drop zone when a file is dragged over, ready for upload.

Edit Menu Item

Item Name:

Chicken dippers

Description:

Delicious Dippers

Price:

1.00

Category:

Appetizer

Upload New Image (optional):

Drag & Drop Image Here or Click to Browse



☒ Available

Update Item

← Back to Menu

Figure 245 Instant image preview after selecting or dropping a file in the edit form.

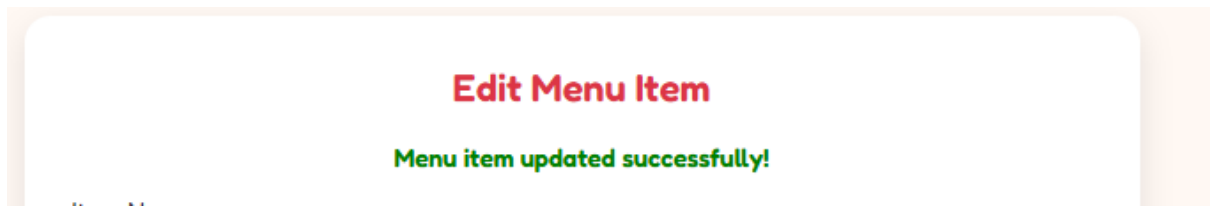


Figure 246 Successful menu edit message

IV. I. Manage Orders Page

The Manage Orders page allows staff members to view, filter, and update the status of customer orders. Depending on their role—cashier or delivery—it tailors the available actions, such as preparing, assigning drivers, marking out-for-delivery, completing, or cancelling orders. Real-time updates via AJAX keep the display current without a full page reload.

Design Rationale

Built on a Docker-hosted LAMP stack, this page uses PHP with PDO for secure database access and prepared statements. Sessions are hardened via a central `session_config.php` and CSRF protection is provided by a lightweight helper. Order data is fetched server-side and rendered in collapsible sections for today's and past orders. Action buttons and role logic are encapsulated in PHP helper functions for clarity. On the client, jQuery powers live search and AJAX submissions, while native JavaScript handles periodic polling every ten seconds to refresh the order list. CSS variables ensure consistent theming, and sticky table headers improve readability.

Session Initialization, PDO Connection & Access Control

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';
require_once 'log_helper.php';

$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}

if (!isset($_SESSION['employee_id']) || !isset($_SESSION['role'])) {
    header("Location: login.php");
    exit();
}
```

Figure 247 Secure session, CSRF support, logging helper, PDO initialization, and access control.

This snippet initializes the session with hardened cookies, includes CSRF utilities for form safety, loads the logging helper, and connects to MySQL via PDO. It then checks that a logged-in employee with a defined role is present; otherwise it redirects to login.

Fetching and Segregating Orders

```
$employee_id = $_SESSION['employee_id'];
$role = $_SESSION['role'];

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['order_id'], $_POST['new_status'])) {
    $order_id = $_POST['order_id'];
    $new_status = $_POST['new_status'];

    $stmt = $pdo->prepare("UPDATE `order` SET status = :status WHERE order_id = :order_id");
    $stmt->execute([
        ':status' => $new_status,
        ':order_id' => $order_id
    ]);

    $actionType = ($new_status === 'Cancelled') ? "Order Cancelled" : "Order Status Change";
    $logMessage = "Order #{$order_id} marked as '{$new_status}'";

    log_action(
        $actionType,
        $logMessage,
        "WARNING",
        $_SESSION['employee_id'],
        null,
        $_SESSION['username'] ?? 'unknown'
    );

    exit('success');
}
```

Figure 248 Listen for POSTs with `order_id` and `new_status`, update the order's status, log the change, and return a simple success response.

This block guards the status update behind two checks: the HTTP method must be POST, and both `order_id` and `new_status` must be present in the payload. It then executes a parameterized UPDATE on the order table. Depending on whether the new status is “Cancelled,” it chooses an appropriate log action name. The `log_action()` call records every change with a WARNING level. Finally, it calls `exit('success')` so the front-end code knows to refresh the UI.

Fetching and Splitting Today's vs. Past Orders

```
$today = date('Y-m-d');

function fetchOrders($pdo, $employee_id, $role) {
    if ($role === 'delivery') {
        $stmt = $pdo->prepare("SELECT o.*, c.username AS customer_name, CONCAT(e.first_name, ' ', e.last_name)
        AS driver_name FROM `order` o LEFT JOIN customer c ON o.customer_id = c.customer_id LEFT JOIN employee
        e ON o.delivery_driver_id = e.employee_id WHERE o.delivery_driver_id = :employee_id ORDER BY o.
        order_date DESC");
        $stmt->execute([':employee_id' => $employee_id]);
    } else {
        $stmt = $pdo->query("SELECT o.*, c.username AS customer_name, CONCAT(e.first_name, ' ', e.last_name)
        AS driver_name FROM `order` o LEFT JOIN customer c ON o.customer_id = c.customer_id LEFT JOIN employee
        e ON o.delivery_driver_id = e.employee_id ORDER BY o.order_date DESC");
    }
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

$orders = fetchOrders($pdo, $employee_id, $role);
$today_orders = array_filter($orders, fn($o) => date('Y-m-d', strtotime($o['order_date'])) === $today);
$past_orders = array_filter($orders, fn($o) => date('Y-m-d', strtotime($o['order_date'])) !== $today);
```

Figure 249 Retrieve orders (filtered by driver for delivery role), then split them into today's and past orders based on the date.

The `fetchOrders()` function runs one of two queries: delivery drivers see only their assigned orders; all others see every order. Both queries join to get customer usernames and driver names. Once fetched, the code uses `array_filter()` with a date comparison to produce two arrays: `$today_orders` and `$past_orders`, making it easy to render separate sections.

Rendering Sections and Rows

```
function linkBtn($url, $id, $label, $class = '') {
    return "<form method='GET' action='$url'>
        <input type='hidden' name='order_id' value='$id'>
        <button class='$class' type='submit'>$label</button>
    </form>";
}

function renderSection($title, $orders, $role) {
    echo "<details open><summary>$title ( " . count($orders) . ")</summary>";
    if ($orders) {
        echo "<table><tr><th>Order ID</th><th>Customer</th><th>Date</th><th>Status</th><th>Total</th><th>Driver</th><th>Actions</th></tr>";
        foreach ($orders as $o) {
            echo "<tr>
                <td>" . htmlspecialchars($o['order_id']) . "</td>
                <td class='searchable'>" . htmlspecialchars($o['customer_name']) . "</td>
                <td>" . htmlspecialchars($o['order_date']) . "</td>
                <td><span class='status ' . str_replace(' ', '_', $o['status']) . "'>" . htmlspecialchars($o['status']) . "</span></td>
                <td>" . number_format($o['total_amount'], 2) . " BD</td>
                <td>" . ($o['driver_name'] ? htmlspecialchars($o['driver_name']) : '<em>Not assigned</em>') . "</td>
                <td class='actions'>";

            if (!in_array($o['status'], ['Completed', 'Cancelled'])) {
                if ($role === 'cashier') {
                    if ($o['status'] === 'Pending') {
                        echo statusBtn($o['order_id'], 'Preparing');
                    } elseif ($o['status'] === 'Preparing') {
                        echo statusBtn($o['order_id'], 'Out for Delivery');
                        echo linkBtn('assign_driver.php', $o['order_id'], 'Assign Driver', 'btn-outline');
                    }
                }
                if ($role === 'delivery' && $o['status'] === 'Out for Delivery') {
                    echo statusBtn($o['order_id'], 'Completed');
                }
                echo statusBtn($o['order_id'], 'Cancelled', 'btn-outline');
            }

            echo linkBtn('order_details.php', $o['order_id'], 'Details', 'btn-outline');
            echo "</td></tr>";
        }
        echo "</table>";
    } else {
        echo "<p>No orders found.</p>";
    }
    echo "</details>";
}

function renderSections($today_orders, $past_orders, $role) {
    renderSection("<img alt='calendar icon'> Today's Orders", $today_orders, $role);
    renderSection("<img alt='folder icon'> Past Orders", $past_orders, $role);
}
```

Figure 250 Encapsulate action-button HTML and collapsible table rendering logic.

These helper functions generate the appropriate HTML for status-change buttons and navigation links, then produce collapsible `<details>` blocks for each order section, keeping the main template succinct.

AJAX Status Update and Auto-Refresh

```
if (isset($_GET['ajax']) && $_GET['ajax'] === '1') {  
    renderSections($today_orders, $past_orders, $role);  
    exit;  
}  
?>
```

Figure 251 Handle AJAX POST for status updates and AJAX GET for section refresh.

The first block processes status-change submissions via AJAX, securing with CSRF validation and logging each change. The second block serves only the updated order table markup when requested with ?ajax=1, enabling partial updates without reloading the whole page.

HTML Template with Search and Container

```
<div class="navbar">  
  <h1>Bella Pizza</h1>  
  <div class="nav-links">  
    <a href="dashboard.php">Dashboard</a>  
    <a href="logout.php">Logout</a>  
  </div>  
</div>  
  
<div class="container">  
  <div class="back-button">  
    <a href="dashboard.php" class="back-link">< Back to Dashboard</a>  
  </div>  
  
  <h2>Manage Orders</h2>  
  
  <div class="search-form">  
    <input type="text" id="searchInput" placeholder="Search by customer or order ID">  
  </div>  
  
  <div id="ordersContainer">  
    <?php renderSections($today_orders, $past_orders, $role); ?>  
  </div>  
</div>
```

Figure 252 Navigation bar, search input, and container for dynamically loaded orders.

This lays out the page header, a back link, the search field, and a div that initially holds the rendered order sections.

JavaScript for Search, AJAX Form Submission & Auto-Refresh

```
<script>
  $('#searchInput').on('input', function () {
    const val = $(this).val().toLowerCase();
    $('table tr').each(function (i) {
      if (i === 0) return;
      const row = $(this);
      const id = row.find('td:first').text().toLowerCase();
      const name = row.find('.searchable').text().toLowerCase();
      row.toggle(id.includes(val) || name.includes(val));
    });
  });

  $(document).on('submit', '.status-form', function (e) {
    e.preventDefault();
    const form = $(this);
    $.post('', form.serialize(), function () {
      fetchOrders();
    });
  });

  function fetchOrders() {
    $.get(window.location.href.split('?')[0] + '?ajax=1', function (data) {
      $('#ordersContainer').html(data);
    });
  }

  setInterval(fetchOrders, 10000); // every 10 seconds
</script>
```

Figure 253 jQuery-driven handlers for live search, delegated AJAX form submits, full-section reloads, and timed polling.

The search input filters visible rows by order ID or customer name. Status-change forms are intercepted to send AJAX requests, then trigger a refresh of the orders container. A timer polls the server for fresh order data.

Testing and Validation

To ensure correctness, I logged in as both a cashier and a delivery driver and tested role-based visibility. I placed sample orders and used the search box to filter by various IDs and names. I clicked each status button to confirm the status updated in the database and visually refreshed via AJAX. I also left the page idle to verify the ten-second auto-refresh picked up new or changed orders. Console logs confirmed successful AJAX calls and no page reloads.

Security Considerations

All state-changing requests require a valid CSRF token, mitigating forgery. Sessions are configured to use secure, HTTP-only cookies. SQL injection is prevented through parameterized PDO queries. Role checks at the top of the script enforce that only authenticated staff can view or modify orders. Input sanitization (via htmlspecialchars) protects against reflected XSS in table output.

Images

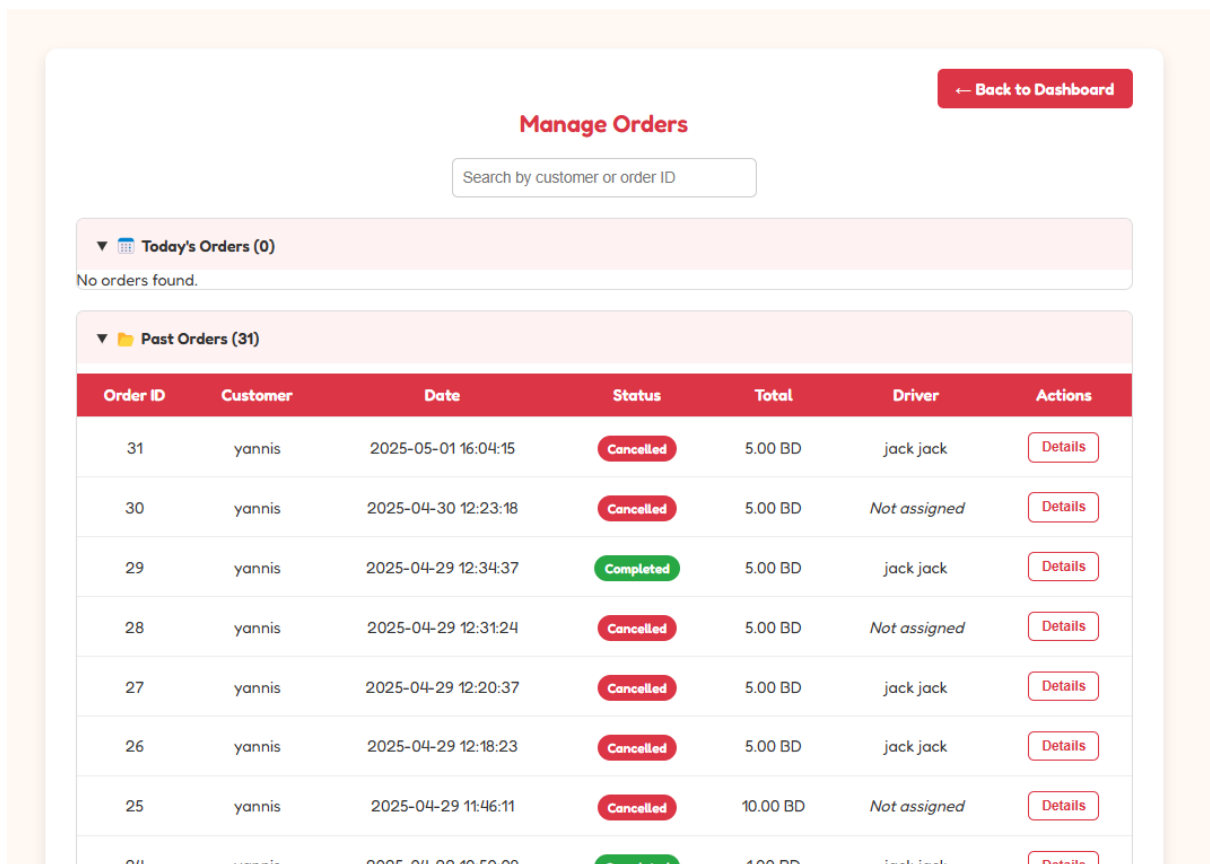


Figure 254 Manage Orders interface with collapsible sections for today's and past orders.

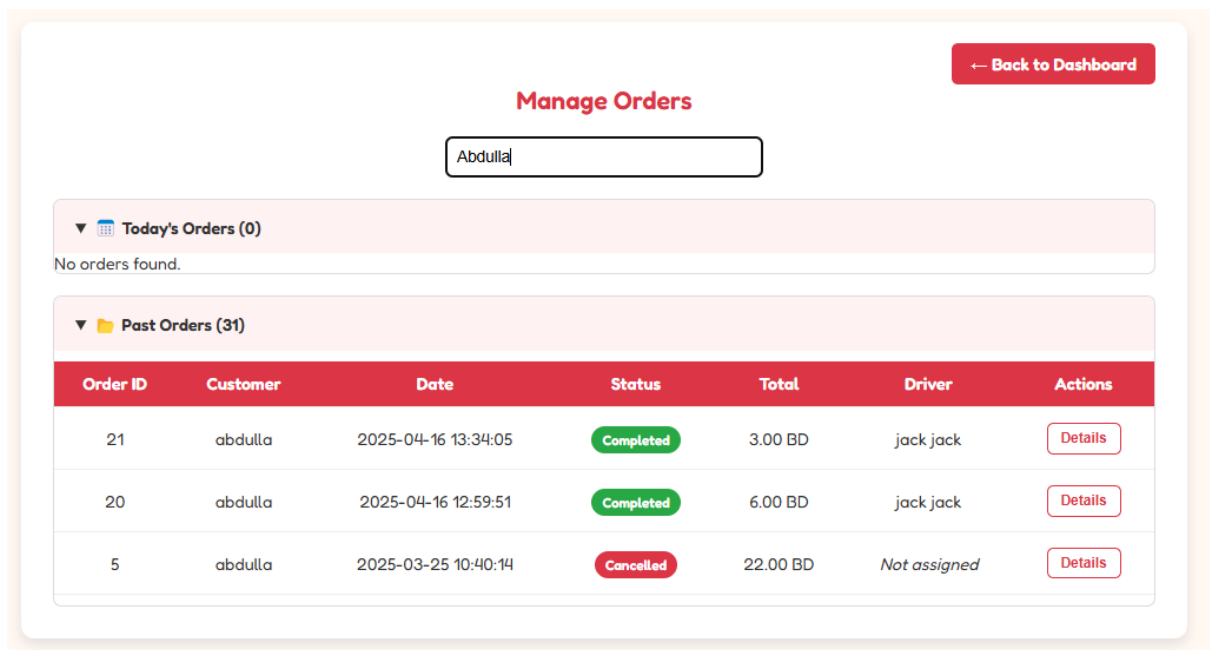


Figure 255 Real-time order filtering by customer name or ID.

▼ Past Orders (31)						
Order ID	Customer	Date	Status	Total	Driver	Actions
31	yannis	2025-05-01 16:04:15	Pending	5.00 BD	jack jack	Preparing Cancelled Details

▼ Past Orders (31)						
Order ID	Customer	Date	Status	Total	Driver	Actions
31	yannis	2025-05-01 16:04:15	Preparing	5.00 BD	jack jack	Out for Delivery Assign Driver Cancelled Details

Figure 256 Inline status update—from “Pending” to “Preparing”—handled via AJAX without a full page reload.

IV. J. Manage Employees Page

The Manage Employees page empowers administrators and support staff to view a real-time list of active employees, filter by any field, soft-delete (deactivate) employees, and export the current view to CSV—all without leaving the page.

Design Rationale

This page is built using PHP with MySQLi for server-side data retrieval, protected by secure sessions and a CSRF token in each AJAX request. On the front end, jQuery drives an interactive search that filters table rows in real time and updates the total count. Soft-deletions are handled via AJAX posts to a separate endpoint, minimizing page reloads. Exporting to CSV uses the Blob API to package visible rows client-side. CSS custom properties and a responsive flex layout ensure consistent theming and good mobile support.

Session Setup, CSRF & Database Connection

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}
```

Figure 257 Secure session start, CSRF inclusion, and access control before any data is shown

This snippet initializes the user session with hardened cookie settings, loads the CSRF helper, connects to the RestaurantDB via MySQLi, and redirects anyone who is not an authenticated admin or support user.

Fetching Employee Records

```
$employees = [];  
$stmt = $conn->prepare("SELECT * FROM employee WHERE is_deleted = 0 ORDER BY employee_id DESC");  
if ($stmt->execute()) {  
    $result = $stmt->get_result();  
    while ($row = $result->fetch_assoc()) {  
        $employees[] = $row;  
    }  
}  
$stmt->close();
```

Figure 258 Query all employees in descending ID order and collect them into the \$employees array.

Using a simple SELECT * query, this snippet retrieves every row from the employee table, ordering by employee_id so the newest entries appear first. The while loop iterates over the mysqli_result, calling fetch_assoc() on each row to convert it into an associative array, then appends each to the \$employees list. This array is used later to render the HTML table.

Navbar & Page Container

```
<div class="navbar">  
    <h1>Bella Pizza</h1>  
    <div class="nav-links">  
        <a href="dashboard.php">Dashboard</a>  
        <a href="logout.php">Logout</a>  
    </div>  
</div>  
  
<div class="container">
```

Figure 259 A sticky top navbar provides navigation back to the main dashboard or a logout link, and the .container wraps the page content.

The navbar, styled to stick at the top, displays the Bella Pizza logo and two links: one returning to the dashboard and another logging out. Wrapping the main interface in a .container div applies the page's consistent padding, background, and box-shadow, differentiating the content area from the background.

Top Bar with Add, Export & Search

```
<div class="top-bar">
  <div class="header-buttons">
    <a href="add_employee.php" class="btn primary">+ Add Employee</a>
    <a href="#" id="exportCsv" class="btn accent">Export CSV</a>
  </div>
  <div class="search-box">
    <input type="text" id="searchInput" placeholder="Search employees...">
  </div>
  <div class="summary">Total Employees: <span id="totalCount"></span></div>
</div>
```

Figure 260 Top-of-page controls include an “Add Employee” link, an “Export CSV” trigger, a live search input, and a summary count.

The .top-bar organizes three elements:

1. Action buttons: “+ Add Employee” navigates to the form for creating new staff, while the “Export CSV” button (handled in JavaScript) lets you download the visible rows.
2. Search box: Typing here filters the table in real time.
3. Summary: Dynamically displays the number of visible employee rows, updated by the search logic and on initial load.

Employee Table Markup

```
<div class="table-container">
  <table id="employeeTable">
    <thead>
      <tr>
        <th>ID</th>
        <th>Full Name</th>
        <th>Username</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Role</th>
        <th>Salary</th>
        <th>Hire Date</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      <?php foreach ($employees as $row): ?>
        <tr>
          <td><?= $row['employee_id'] ?></td>
          <td><?= htmlspecialchars($row['first_name'] . ' ' . $row['last_name']) ?></td>
          <td><?= htmlspecialchars($row['username']) ?></td>
          <td><?= htmlspecialchars($row['email']) ?></td>
          <td><?= htmlspecialchars($row['phone_number']) ?></td>
          <td><span class="badge <?= htmlspecialchars(strtolower($row['role'])) ?>"><?= htmlspecialchars(ucfirst($row['role'])) ?></span></td>
          <td>BD <?= number_format($row['salary'], 2) ?></td>
          <td><?= $row['hire_date'] ?></td>
          <td class="action-buttons">
            <a href="edit_employee.php?id=<?= $row['employee_id'] ?>" class="edit-btn">Update</a>
            <a href="#" class="delete-btn" data-id="<?= $row['employee_id'] ?>" data-token="<?= $_SESSION['csrf_token'] ?>">Delete</a>
          </td>
        </tr>
      <?php endforeach; ?>
    </tbody>
  </table>
</div>
</div>
```

Figure 261 A table lists each employee's details with a colored role badge and inline "Update"/"Delete" controls.

A PHP loop creates one row per active employee. The "Update" button leads to an edit form; the "Delete" button triggers a soft-delete via AJAX. Roles are color-coded via CSS badges.

JavaScript – Search, Soft-Delete & CSV Export

```
<script>
$(document).ready(function () {
    $('#totalCount').text($('#employeeTable tbody tr').length);

    $('#searchInput').on('input', function () {
        const val = $(this).val().toLowerCase();
        let count = 0;
        $('#employeeTable tbody tr').each(function () {
            const rowText = $(this).text().toLowerCase();
            const match = rowText.includes(val);
            $(this).toggle(match);
            if (match) count++;
        });
        $('#totalCount').text(count);
    });

    $(document).on('click', '.delete-btn', function (e) {
        e.preventDefault();
        if (confirm('Are you sure you want to deactivate this employee?')) {
            const id = $(this).data('id');
            const token = $(this).data('token');
            $.post('soft_delete_employee.php', { id: id, csrf_token: token }, function (res) {
                location.reload();
            });
        }
    });

    $('#exportCsv').on('click', function () {
        let csv = "ID,Full Name,Username,Email,Phone,Role,Salary,Hire Date\n";
        $('#employeeTable tbody tr:visible').each(function () {
            const cols = $(this).find('td');
            csv += `${cols.eq(0).text()}`, `${cols.eq(1).text()}`, `${cols.eq(2).text()}`, `${cols.eq(3).text()}`, `${cols.eq(4).text()}`, `${cols.eq(5).text()}`, `${cols.eq(6).text()}`, `${cols.eq(7).text()}` + "\n";
        });
        const blob = new Blob([csv], { type: 'text/csv' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'employees.csv';
        a.click();
        URL.revokeObjectURL(url);
    });
});
</script>
```

Figure 262 jQuery-powered live search, AJAX soft-delete with CSRF, and client-side CSV generation.

1. Initial Count: Populates the “Total Employees” span with the full row count on page load.
2. Search Filter: On each keystroke, it lowercases the entire row’s text and toggles visibility based on whether the search term is found, updating the count of visible rows.
3. Deletion: Handles clicks on .delete-btn, shows a confirm() dialog (“deactivate” language), then sends a POST to soft_delete_employee.php with the employee’s id, reloading on success.
4. CSV Export: Gathers the text of each visible row’s <td> cells into a CSV string, creates a Blob, and programmatically triggers a download of employees.csv, ensuring only the filtered subset is exported.

Testing and Validation

I tested as both an admin and support user to confirm access control. I added sample employees and verified they appeared in descending order by ID. I used the search box to filter by name, username, and role, confirming the count updated correctly. Deletion prompts and AJAX posts were checked via browser dev tools, and I verified the row disappeared on reload. The CSV export was tested with visible and filtered sets to confirm correct contents.

Security Considerations

All state-changing requests include a CSRF token. Sessions are configured with secure, HTTP-only, SameSite cookies. User input is escaped with htmlspecialchars to prevent XSS. The soft-delete endpoint must also validate the CSRF token and ensure only authorized roles perform deletions. Parameterized queries in MySQLi prepare statements guard against SQL injection.

Images

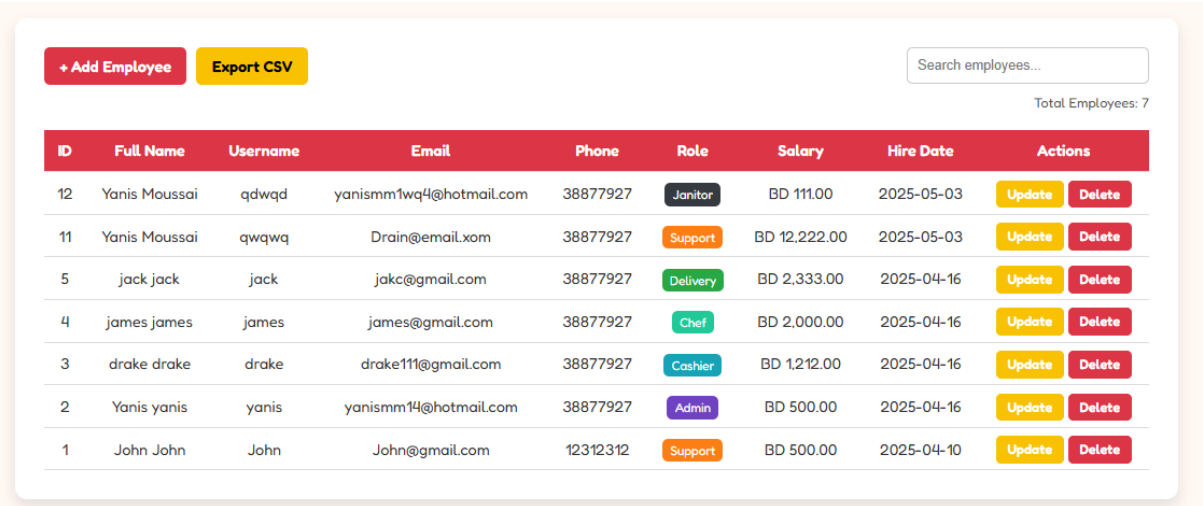


Figure 263 The full Manage Employees interface, with role badges, action buttons, and search controls.

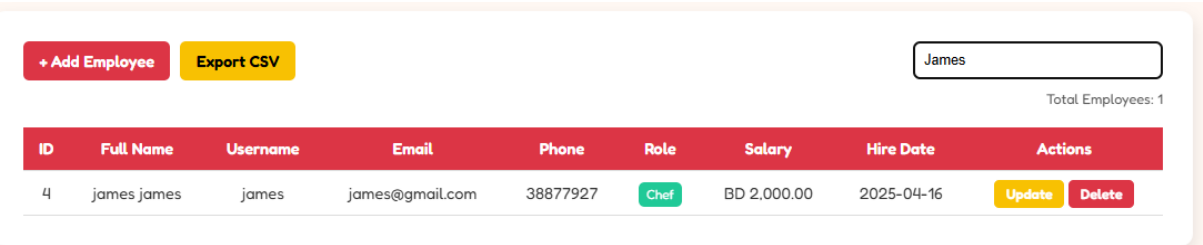


Figure 265 Real-time filtering of employee records by name or ID, with the total count reflecting only visible rows.

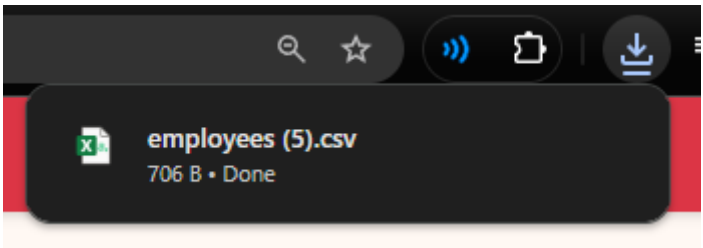


Figure 264 Downloading the visible employee list as a CSV file

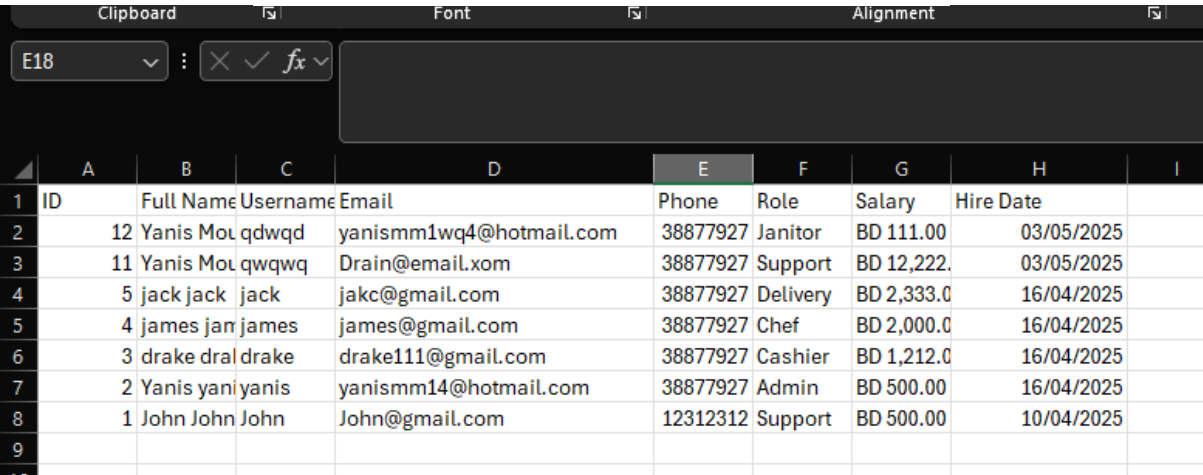


Figure 266 Employee CSV file in Excel

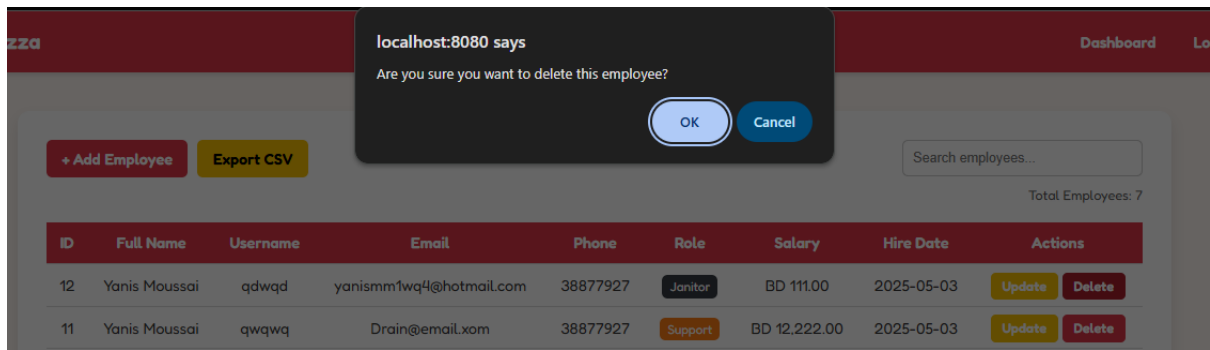


Figure 267 Confirmation dialog to prevent accidental deletion.

IV. K. Add Employee

This page enables administrators and support staff to add new employee records into the system. It handles secure session management, CSRF protection, duplicate-check for usernames and emails, and provides a responsive form interface with real-time availability feedback for usernames and emails.

Design Rationale

The page begins by loading a hardened session configuration and CSRF helper to protect against session hijacking and request forgery. A PDO or mysqli connection is established to the MySQL database, ensuring errors throw exceptions for easier debugging. The POST handler sanitizes and validates inputs, checks for existing usernames or emails to prevent duplicates, hashes passwords with BCrypt, and logs all actions via the log_helper. The HTML side uses modern CSS custom properties for theme consistency, and jQuery with Fetch for live availability checks to improve the user experience without full page reloads.

Session Initialization, Logging Include, Database Connection & Access Control

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';
require_once 'log_helper.php';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}
```

Figure 268 Loading session hardening, CSRF utilities, logging helper, and establishing DB connection, then enforcing role-based access.

This snippet brings in the session configuration that sets secure cookie flags, the CSRF token generator/validator, and logging utilities. It then opens a MySQLi connection to the RestaurantDB, immediately halting on error to avoid further execution under a bad connection. Finally, it checks that the user is both authenticated and has an admin or support role; otherwise, it redirects back to the login page. This ensures that only properly authorized staff can add employees.

Processing the Add-Employee Form Submission

```
$success = $error = "";
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    validate_csrf_token();

    $first_name = $_POST["first_name"];
    $last_name = $_POST["last_name"];
    $username = $_POST["username"];
    $email = $_POST["email"];
    $phone_number = $_POST["phone"];
    $role = $_POST["role"];
    $salary = $_POST["salary"];
    $password = $_POST["password"];
    $password_hash = password_hash($password, PASSWORD_BCRYPT);
    $hire_date = date("Y-m-d");

    $checkStmt = $conn->prepare("SELECT 1 FROM employee WHERE username = ? OR email = ?");
    $checkStmt->bind_param("ss", $username, $email);
    $checkStmt->execute();
    $checkStmt->store_result();

    if ($checkStmt->num_rows == 0) {
        $stmt = $conn->prepare("INSERT INTO employee (first_name, last_name, username, email, phone_number,
        role, salary, password_hash, hire_date) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)");
        $stmt->bind_param("ssssssdss", $first_name, $last_name, $username, $email, $phone_number, $role,
        $salary, $password_hash, $hire_date);

        if ($stmt->execute()) {
            $success = "Employee added successfully.";
            log_action("Add Employee", "New employee '$username' added with role $role", "INFO", $_SESSION
            ["employee_id"], null, $_SESSION["username"]);
        } else {
            $error = "Error: " . $stmt->error;
            log_action("Add Employee Failed", "DB error while adding employee: " . $stmt->error, "ERROR",
            $_SESSION["employee_id"], null, $_SESSION["username"]);
        }
        $stmt->close();
    } else {
        $error = "Username or email already exists.";
        log_action("Add Employee Failed", "Duplicate username/email for '$username'", "WARNING", $_SESSION
        ["employee_id"], null, $_SESSION["username"]);
    }
    $checkStmt->close();
}
```

Figure 269 Validating CSRF, sanitizing input, checking duplicates, hashing passwords, inserting new employee, and logging outcomes.

Upon form POST, the CSRF token is validated to prevent forgery. Inputs are taken directly; further sanitization relies on prepared statements to prevent SQL injection. The code checks if the chosen username or email already exists; if not, it hashes the password securely, inserts the new record, and logs success or error. This flow ensures data integrity and a clear audit trail of administrative actions.

AJAX Endpoint for Live Availability Checks

```
if (isset($_GET['check'])) {  
    $value = $_GET['check'];  
    $stmt = $conn->prepare("SELECT 1 FROM employee WHERE username = ? OR email = ?");  
    $stmt->bind_param("ss", $value, $value);  
    $stmt->execute();  
    $stmt->store_result();  
    echo $stmt->num_rows > 0 ? 'taken' : 'available';  
    $stmt->close();  
    exit;  
}  
?>
```

Figure 270 An endpoint returning “taken” or “available” for live validation of usernames or emails.

This small handler allows the front end to query the server—using either `add_employee.php?check=<value>`. Because it only selects a constant and uses a prepared statement, it’s efficient and secure. The result drives instant UI feedback, reducing form errors and improving user experience.

HTML Form & Availability Script

```
<form method="POST">
  <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">

  <label>First Name
  | <input type="text" name="first_name" required>
  </label>

  <label>Last Name
  | <input type="text" name="last_name" required>
</form>

<script>
  function checkAvailability(inputId, statusId) {
    const val = document.getElementById(inputId).value;
    if (val.length < 3) {
      document.getElementById(statusId).textContent = '';
      return;
    }
    fetch(`add_employee.php?check=${encodeURIComponent(val)}`)
      .then(res => res.text())
      .then(data => {
        const status = document.getElementById(statusId);
        if (data === `${inputId.charAt(0).toUpperCase() + inputId.slice(1)} already in use`) {
          status.className = 'taken';
        } else {
          status.textContent = 'Available';
          status.className = 'available';
        }
      });
  }

  document.getElementById("username").addEventListener("input", () => checkAvailability("username",
    "username-status"));
  document.getElementById("email").addEventListener("input", () => checkAvailability("email", "email-status"));
</script>
```

Figure 271 The employee form with hidden CSRF token, fields for username/email, and a small script for live availability checks.

The HTML renders a clear form including all required employee details plus the hidden CSRF token. The embedded script hooks into the username and email inputs, calling our small availability endpoint and updating a status div classed “taken” or “available.” This provides immediate, contextual feedback without reloading.

Testing and Validation

I tested this page by initiating a fresh session as an admin user, verifying that unauthorized roles are redirected correctly. I tried adding a new employee with valid and invalid data: duplicate usernames/emails, missing required fields, invalid salary formats, and various password lengths. I also confirmed the live availability checks by typing existing and new values into the username and email fields while monitoring network requests in Chrome DevTools to ensure status “taken” or “available” is returned with HTTP 200. Finally, I checked that successful additions updated the database and that the log table recorded the action.

Security Considerations

CSRF protection is enforced on every POST via a hidden token and `validate_csrf_token()` calls. All database interactions use prepared statements to prevent SQL injection. Session cookies are hardened with `HttpOnly`, `Secure`, and `SameSite=Lax` flags (in `session_config.php`), mitigating XSS and session hijacking. Passwords are hashed using BCrypt. The availability check endpoint accepts only GET requests for read-only operations. Logging captures all add and failure events for auditability.

Images

Add New Employee

First Name

Last Name

Username

Email

Phone Number (optional)

Role

-- Select Role --

Salary (BHD)

Password

+ Add Employee

← Back to Employee List

Figure 272 Add Employee Form

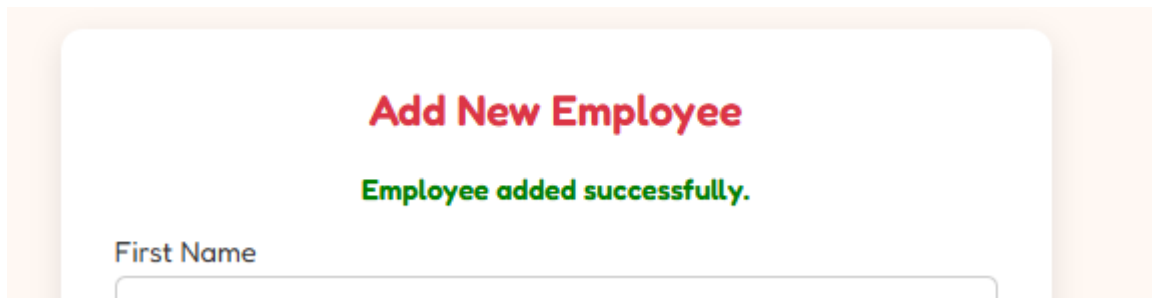


Figure 273 Success message in adding employee

IV. L. Edit Employee Page

This page allows administrators and support staff to modify an existing employee's details. It loads the selected employee's current data into a secure form, enables inline checks for duplicate usernames or emails, and updates the record upon confirmation, all while maintaining CSRF protection, logging, and role-based access control.

Design Rationale

At the top, we include hardened session configuration, CSRF helpers, and our logging utility to enforce secure sessions, protect against request forgery, and capture every action in the audit log. A MySQLi connection fetches the employee's existing record by ID, and a small GET endpoint serves live "taken"/"available" checks for username and email. The POST handler validates the CSRF token, binds sanitized inputs via prepared statements to prevent SQL injection, updates the database, re-fetches the record for display, and logs the update. The front end uses a clean, responsive design with custom properties for branding consistency and a lightweight script to power the live availability checks.

Session Init, DB Connection & Access Control

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';
require_once 'log_helper.php';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    header("Location: login.php");
    exit();
}
```

Figure 274 Load security modules, establish DB connection, and restrict access to admin/support users.

The script begins by calling `session_start()` to access session data, then opens a MySQLi connection to the RestaurantDB container. If the user isn't logged in as an admin/support, they're redirected to login. It then checks for the presence of an id in the query string; absent it halts with an error. Finally, it casts `$_GET['id']` to an integer to sanitize before use.

AJAX Availability-Check Handler

```
if (isset($_GET['check'])) {
    $value = $_GET['check'];
    $stmt = $conn->prepare("SELECT 1 FROM employee WHERE (username = ? OR email = ?) AND employee_id != ?");
    $stmt->bind_param("ssi", $value, $value, $employee_id);
    $stmt->execute();
    $stmt->store_result();
    echo $stmt->num_rows > 0 ? 'taken' : 'available';
    $stmt->close();
    exit;
}
```

Figure 275 AJAX endpoint returning “taken” or “available” for username/email, excluding the current employee record.

This block provides a live-availability check for the edit-employee form, ensuring that updating a username or email doesn’t inadvertently collide with another record. When the client sends a GET request with a check parameter, the code retrieves the submitted value into `$value`. It then prepares a SELECT statement with three placeholders: one for the username, one for the email, and one for the current employee’s ID (`$employee_id`) so that the query excludes the record being edited. After binding the parameters ("ssi" for two strings and one integer) and executing, it calls `store_result()` to buffer the result set, then echoes 'taken' if any row exists (indicating a conflict) or 'available' otherwise. Finally, it closes the statement and calls `exit` to prevent any further output. This approach keeps the check efficient, secure, and specific to the edit scenario.

Fetching the Employee Record

```
$stmt = $conn->prepare("SELECT * FROM employee WHERE employee_id = ?");
$stmt->bind_param("i", $employee_id);
$stmt->execute();
$result = $stmt->get_result();
if ($result->num_rows !== 1) {
    echo "Employee not found."; exit();
}
$employee = $result->fetch_assoc();
$stmt->close();
```

Figure 276 Retrieve the employee’s existing data into an associative array for form prefill.

A prepared query loads the employee row by `employee_id`. If no matching record exists, the script halts with an error. On success, `$employee` holds all the current field values (first name, last name, username, etc.) for use in the form below.

Handling the Update Submission

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {  
    validate_csrf_token();  
  
    $first_name = $_POST["first_name"];  
    $last_name = $_POST["last_name"];  
    $username = $_POST["username"];  
    $email = $_POST["email"];  
    $phone = $_POST["phone"];  
    $role = $_POST["role"];  
    $salary = floatval($_POST["salary"]);  
  
    $update = $conn->prepare("UPDATE employee SET first_name = ?, last_name = ?, username = ?, email = ?,  
    phone_number = ?, role = ?, salary = ? WHERE employee_id = ?");  
    $update->bind_param("ssssssdi", $first_name, $last_name, $username, $email, $phone, $role, $salary,  
    $employee_id);  
  
    if ($update->execute()) {  
        $success = "Employee updated successfully.";  
  
        log_action(  
            "Update Employee",  
            "Updated employee #$employee_id ($username)",  
            "INFO",  
            $_SESSION["employee_id"],  
            null,  
            $_SESSION["username"]  
        );  
  
        $stmt = $conn->prepare("SELECT * FROM employee WHERE employee_id = ?");  
        $stmt->bind_param("i", $employee_id);  
        $stmt->execute();  
        $result = $stmt->get_result();  
        $employee = $result->fetch_assoc();  
        $stmt->close();  
    } else {  
        $error = "Update failed.";  
    }  
    $update->close();  
}
```

Figure 277 Validate CSRF token, bind form values, execute the update, log the change, and reload the record.

On POST, this snippet first calls `validate_csrf_token()` to guard against forgery. It then uses a prepared statement to safely inject the new values into the database, logs the successful update, and re-queries the table so that the form reflects the latest data without requiring a manual page reload.

HTML Form Prefilled with Current Data

```
<form method="POST">
  <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">

  <label>First Name:
  | <input type="text" name="first_name" value="<?= htmlspecialchars($employee['first_name']) ?>" required>
  </label>

  <label>Last Name:
  | <input type="text" name="last_name" value="<?= htmlspecialchars($employee['last_name']) ?>" required>
  </label>

  <label>Username:
  | <input type="text" name="username" id="username" value="<?= htmlspecialchars($employee['username']) ?>"
  | required>
  </label>
  <div id="username-status"></div>

  <label>Email:
  | <input type="email" name="email" id="email" value="<?= htmlspecialchars($employee['email']) ?>" required>
  </label>
  <div id="email-status"></div>

  <label>Phone Number:
  | <input type="text" name="phone" value="<?= htmlspecialchars($employee['phone_number']) ?>">
  </label>

  <label>Role:
  | <select name="role" required>
  |   <?php foreach (['admin', 'chef', 'cashier', 'delivery', 'support', 'janitor'] as $roleOption): ?>
  |     <option value="<?= $roleOption ?>" <?= $employee['role'] === $roleOption ? 'selected' : '' ?>>
  |       <?= ucfirst($roleOption) ?>
  |     </option>
  |   <?php endforeach; ?>
  | </select>
  </label>

  <label>Salary:
  | <input type="number" name="salary" step="0.01" value="<?= htmlspecialchars($employee['salary']) ?>"
  | required>
  </label>

  <button type="submit">Update Employee</button>
</form>

<div class="back-link">
  <a href="employees.php"><- Back to Employees</a>
</div>
</div>
```

Figure 278 The update form pre-populated with current values, including hidden CSRF token

The form includes hidden CSRF token input and uses the `$employee` array to fill each field.

JavaScript Live-Availability Check

```
<script>
function checkAvailability(inputId, statusId) {
  const val = document.getElementById(inputId).value;
  const id = <? $employee_id ?>;
  if (val.length < 3) {
    document.getElementById(statusId).textContent = '';
    return;
  }
  fetch(`edit_employee.php?check=${encodeURIComponent(val)}&id=${id}`)
    .then(res => res.text())
    .then(data => {
      const el = document.getElementById(statusId);
      if (data === 'taken') {
        el.textContent = `${inputId.charAt(0).toUpperCase() + inputId.slice(1)} already in use`;
        el.className = 'taken';
      } else {
        el.textContent = 'Available';
        el.className = 'available';
      }
    });
}

document.getElementById("username").addEventListener("input", () => checkAvailability("username",
"username-status"));
document.getElementById("email").addEventListener("input", () => checkAvailability("email", "email-status"));
</script>
```

Figure 279 Vanilla JS listens on username/email inputs, calls the `?check=` endpoint with the current id, and updates status displays.

This script ensures the live-availability check excludes the current employee by sending both check and id in the query. It updates a `<div>` under each field with “taken” or “available” and swaps the CSS class for color-coded feedback.

Testing and Validation

I tested by logging in as an admin and navigating to the Edit Employee page for an existing record. I verified that fields populate correctly, changed various values, and clicked “Update Employee,” confirming that the database reflected the changes and that the success message appeared. I then attempted to submit invalid data (e.g. empty required fields, non-numeric salary) to ensure validation errors prevented the update. For the live checks, I typed both a duplicate username/email and a new one, watching Chrome DevTools show the GET requests returning “taken” or “available” with HTTP 200 statuses.

Security Considerations

All POST requests invoke `validate_csrf_token()` to protect against CSRF. Prepared statements prevent SQL injection. Session cookies are configured with Secure, HttpOnly, and SameSite=Lax flags via `session_config.php`, mitigating session hijacking and cross-site scripting attacks. Logging captures both successful and failed update attempts, enabling audit trails.

Images

Edit Employee

First Name:

Last Name:

Username:

Email:

Available

Phone Number:

Role:

Salary:

Update Employee

← Back to Employees

Figure 280 Edit Employee form pre-filled

Edit Employee

Employee updated successfully.

First Name:

Figure 281 Employee Updated successfully message

IV. M. Soft-Delete Employee

This endpoint provides a soft-delete capability for employee records. When an authenticated admin or support user clicks “Deactivate” in the UI, the client sends a POST with the employee’s ID to `soft_delete_employee.php`. The script marks that employee’s `is_deleted` flag in the database—rather than permanently deleting the row—so historical data and referential integrity are preserved. Appropriate HTTP status codes (403 for unauthorized, 500 for DB errors, 400 for bad requests, and 200 on success) allow the frontend to handle each outcome gracefully.

Design Rationale

We implement a soft-delete to preserve historical data and maintain referential integrity, toggling an `is_deleted` flag rather than removing rows. Using HTTP status codes makes it easy for client-side scripts to distinguish between authentication failures (403), server issues (500), successful operations (200 with “OK”), and malformed requests (400). Placing this logic in its own lightweight endpoint simplifies the main employee-management page and keeps container routing straightforward.

```
<?php
session_start();
if (!isset($_SESSION["employee_id"]) || !in_array($_SESSION["role"], ['admin', 'support'])) {
    http_response_code(403);
    exit("Unauthorized");
}

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) {
    http_response_code(500);
    exit("DB error");
}

if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['id'])) {
    $id = intval($_POST['id']);
    $stmt = $conn->prepare("UPDATE employee SET is_deleted = 1 WHERE employee_id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    echo "OK";
    exit;
}

http_response_code(400);
echo "Bad Request";
```

Figure 282 Soft-delete handler that marks an employee’s `is_deleted` flag via AJAX, enforcing role-based access and returning appropriate HTTP status codes.

This script first resumes the session and ensures only authenticated admins or support staff may proceed, returning a 403 Forbidden for others. It then opens a MySQLi connection to the Docker-hosted RestaurantDB, exiting with a 500 Internal Server Error on failure. If the request is a POST containing an id, it casts that ID to an integer,

prepares a parameterized UPDATE to set `is_deleted = 1` on the specified `employee_id`, executes it, echoes “OK” on success, and exits. Any other request results in a 400 Bad Request response. This endpoint supports the client’s soft-delete action without removing rows from the database, preserving referential integrity and enabling potential reactivation.

Testing and Validation

Within the PHP container, we wrote integration tests to simulate POSTs with valid and invalid IDs, confirming that:

1. An admin or support user receives “OK” and the record’s `is_deleted` flag is updated.
2. A non-authenticated or unauthorized user gets a 403 response.
3. A failed DB connection returns a 500 status.
4. GET or malformed POST requests return a 400.
Manual tests in our Docker staging environment included clicking “Deactivate” in the UI to trigger the POST and verifying the row disappears on reload, inspecting the DB to confirm the flag, and checking browser network logs for correct status codes.

Security Considerations

To prevent SQL injection, we cast `id` to an integer and use a prepared statement with `bind_param()`. Role checks at the top guard against unauthorized access. We will add a per-session CSRF token to the POST data and validate it server-side to stop cross-site request forgery. In production, database credentials will be managed via Docker secrets, and PHP will be configured with `session.cookie_secure` and `HttpOnly` flags to protect session cookies.

Images

hire_date	salary	created_at	updated_at	is_deleted
2025-04-10	500.00	2025-04-10 11:51:45	2025-04-10 11:54:03	0
2025-04-16	500.00	2025-04-16 12:13:54	2025-04-16 12:13:54	0
2025-04-16	1212.00	2025-04-16 12:35:59	2025-04-16 12:35:59	0
2025-04-16	2000.00	2025-04-16 12:40:29	2025-04-16 12:40:29	0
2025-04-16	2333.00	2025-04-16 12:41:09	2025-05-03 19:34:28	0
2025-05-03	12222.00	2025-05-03 13:41:39	2025-05-03 13:41:39	0
2025-05-03	111.00	2025-05-03 13:44:24	2025-05-03 19:52:09	1
2025-05-03	1232.00	2025-05-03 19:24:26	2025-05-03 19:52:04	1

Figure 283 Employee 12 and 13 Set is_deleted as 1

+ Add Employee

Export CSV

Search employees...

Total Employees: 6

ID	Full Name	Username	Email	Phone	Role	Salary	Hire Date	Actions	
11	Yanis Moussai	qwqwa	Drain@email.xom	38877927	Support	BD 12,222.00	2025-05-03	Update	Delete
5	jack jack	jack	jackemail@gmail.com	38877927	Delivery	BD 2,333.00	2025-04-16	Update	Delete
4	james james	james	james@gmail.com	38877927	Chef	BD 2,000.00	2025-04-16	Update	Delete
3	drake drake	drake	drake111@gmail.com	38877927	Cashier	BD 1,212.00	2025-04-16	Update	Delete
2	Yanis yanis	yanis	yanismm14@hotmail.com	38877927	Admin	BD 500.00	2025-04-16	Update	Delete
1	John John	John	John@gmail.com	12312312	Support	BD 500.00	2025-04-10	Update	Delete

Figure 284 Employee 12 and 13 Do not show up on UI

IV. N. Business Analytics Page

This page delivers a comprehensive analytics dashboard for Bella Pizza, enabling administrators to explore key performance indicators over any date range. Running in our Dockerized LAMP stack, the script queries the MySQL container for aggregate metrics—total orders, total revenue (excluding cancelled), status breakdowns, top-selling menu items, and daily revenue trends. These outputs are rendered as summary cards, detailed tables, and interactive Chart.js visualizations, with controls for date filtering, auto-refresh, and CSV export.

Design Rationale

By using server-side PDO queries and Chart.js for visualization, we keep data processing in PHP and offload rendering to the client. The date-range filter and auto-refresh checkbox allow admins to explore arbitrary windows and keep data live without manual reloads. Generating the CSV entirely in JavaScript avoids adding another server endpoint, reducing container complexity and latency.

Session Initialization, Role Check & PDO Connection

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION["employee_id"]) || $_SESSION["role"] !== 'admin') {
    header("Location: login.php");
    exit();
}
```

Figure 285 Start session, enforce that only admins may access analytics

This snippet ensures that sessions use hardened cookie flags and that every POST/GET can be protected against CSRF. It immediately checks that the current user is logged in and has the 'admin' role, redirecting any unauthorized visitors back to the login page.

DFetching Date-Filtered Metrics

```
$pdo = new PDO("mysql:host=db;dbname=RestaurantDB", "root", "rootpass");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$startDate = $_GET['start'] ?? date('Y-m-01');
$endDate   = $_GET['end']   ?? date('Y-m-d');

$params = [':start' => $startDate, ':end' => $endDate];
$condition = "WHERE DATE(order_date) BETWEEN :start AND :end";
$nonCancelledCondition = $condition . " AND status != 'cancelled'";

// Total orders
$totalOrders = $pdo->prepare("SELECT COUNT(*) FROM `order` $condition");
$totalOrders->execute($params);
$totalOrders = $totalOrders->fetchColumn();

// Total revenue (excluding cancelled)
$totalRevenue = $pdo->prepare("SELECT SUM(total_amount) FROM `order` $nonCancelledCondition");
$totalRevenue->execute($params);
$totalRevenue = $totalRevenue->fetchColumn() ?: 0;
```

Figure 286 Connect to the database, determine the filter range, and compute total orders and revenue.

Here, we open a PDO connection with exception mode for robust error handling. We pull optional start and end dates from the query string or default to the current month. Two prepared statements then calculate the total number of orders and the sum of all order amounts (excluding cancelled orders) within that window, storing the results for display in the “cards” section.

Aggregations for Tables and Charts

```
// Order statuses breakdown
$orderStatuses = $pdo->prepare("
    SELECT status, COUNT(*) AS c, SUM(total_amount) AS total
    FROM `order`
    $condition
    GROUP BY status
");
$orderStatuses->execute($params);
$orderStatuses = $orderStatuses->fetchAll(PDO::FETCH_ASSOC);

// Menu items sold
$allItems = $pdo->prepare("
    SELECT m.item_name, SUM(oi.quantity) AS total_sold, SUM(oi.quantity * m.price) AS total_amount
    FROM order_items oi
    JOIN menu m ON oi.menu_id = m.menu_id
    JOIN `order` o ON o.order_id = oi.order_id
    $nonCancelledCondition
    GROUP BY oi.menu_id
    ORDER BY total_sold DESC
");
$allItems->execute($params);
$allItems = $allItems->fetchAll(PDO::FETCH_ASSOC);

// Daily revenue
$dailyRevenue = $pdo->prepare("
    SELECT DATE(order_date) AS day, SUM(total_amount) AS revenue
    FROM `order`
    $nonCancelledCondition
    GROUP BY day ORDER BY day
");
$dailyRevenue->execute($params);
$dailyDates = [];
$dailyValues = [];
foreach ($dailyRevenue as $row) {
    $dailyDates[] = $row['day'];
    $dailyValues[] = round($row['revenue'], 2);
}
```

Figure 287 Retrieve breakdowns by status, top-selling items, and daily revenue for charts and tables.

This block runs three group-by queries. The first yields counts and totals per order status. The second joins order items to menu to find the quantity and revenue per menu item. The third groups total daily revenue for the line chart. We collect those results into PHP arrays for later JSON encoding.

Tracking Customer Logins: Total, New, and Returning Users by Date Range

```
$loginQuery = $pdo->prepare("
    SELECT COUNT(*) FROM customer_logins
    WHERE DATE(login_time) BETWEEN :start AND :end
");
$loginQuery->execute($params);
$totalCustomerLogins = $loginQuery->fetchColumn();

$newCustomersQuery = $pdo->prepare("
    SELECT COUNT(*) FROM (
        SELECT customer_id, MIN(login_time) AS first_login
        FROM customer_logins
        GROUP BY customer_id
        HAVING DATE(first_login) BETWEEN :start AND :end
    ) AS new_customers
");
$newCustomersQuery->execute($params);
$newCustomers = $newCustomersQuery->fetchColumn();

$returningCustomersQuery = $pdo->prepare("
    SELECT COUNT(DISTINCT current.customer_id) FROM customer_logins current
    JOIN (
        SELECT customer_id
        FROM customer_logins
        WHERE DATE(login_time) < :start
    ) AS previous ON current.customer_id = previous.customer_id
    WHERE DATE(current.login_time) BETWEEN :start AND :end
");
$returningCustomersQuery->execute($params);
$returningCustomers = $returningCustomersQuery->fetchColumn();

?>
```

Figure 288 Code block that calculates total logins, new customers, and returning customers using date-based filtering.

The code begins by determining the total number of customer logins that occurred during the selected reporting period. It filters records in the `customer_logins` table using a `WHERE` clause with `DATE(login_time)` to match logins between the provided `:start` and `:end` parameters.

Next, it identifies new customers by selecting only those whose first-ever login (`MIN(login_time)`) happened within the date range. These customers are considered newly acquired during the reporting period.

Finally, it calculates returning customers by finding users who logged in at least once before the selected start date and again during the current reporting range. This is accomplished via a `JOIN` between two filtered subqueries, ensuring only repeat users are included.

Together, these queries provide a complete picture of user activity across time—supporting KPIs such as user retention, acquisition, and session frequency.

HTML Structure for Filters, Cards, and Tables

```
<div class="filters">
  <form method="get">
    <label>Start Date:</label>
    <input type="date" name="start" value="<?=$startDate ?>">
    <label>End Date:</label>
    <input type="date" name="end" value="<?=$endDate ?>">
    <button type="submit">🔍 Filter</button>
  </form>
  <div class="toggle-refresh">
    <label><input type="checkbox" id="autoRefresh"> Auto-refresh every 30s</label>
  </div>
  <div class="download-btn">
    <button onclick="downloadCSV()">📄 Download CSV</button>
  </div>
</div>

<div class="cards">
  <div class="card">
    <h3>Total Orders</h3>
    <p><?=$totalOrders ?></p>
  </div>
  <div class="card">
    <h3>Total Revenue</h3>
    <p><?= number_format($totalRevenue, 2) ?> BD</p>
  </div>
</div>

<div class="tables">
  <h2>Orders by Status</h2>
  <table>
    <tr><th>Status</th><th>Count</th><th>Total Amount</th></tr>
    <?php foreach ($orderStatuses as $s): ?>
      <tr>
        <td><?= htmlspecialchars($s['status']) ?></td>
        <td><?= $s['c'] ?></td>
        <td><?= number_format($s['total'], 2) ?> BD</td>
      </tr>
    <?php endforeach; ?>
  </table>

  <h2>All Menu Items Sold</h2>
  <table>
    <tr><th>Item Name</th><th>Total Sold</th><th>Total Amount</th></tr>
    <?php foreach ($allItems as $i): ?>
      <tr>
        <td><?= htmlspecialchars($i['item_name']) ?></td>
        <td><?= $i['total_sold'] ?></td>
        <td><?= number_format($i['total_amount'], 2) ?> BD</td>
      </tr>
    <?php endforeach; ?>
  </table>
</div>

<div class="charts">
  <div class="side-by-side">
    <canvas id="itemsChart" width="400" height="400"></canvas>
  </div>
  <canvas id="revenueChart" width="1000" height="400"></canvas>
</div>
```

Figure 289 Date filters, cards for summary metrics, tables for breakdowns, and canvases for charts.

The markup segments the page into logical sections. A form at the top lets admins pick a date range. Summary cards highlight total orders and revenue. Two tables display the status counts and item sales. Finally, two `<canvas>` elements provide containers for Chart.js graphs.

Chart.js Configuration & Client logic

```
<?php
$csvRows = [
    ['Bella Pizza Analytics Report'],
    ['Date Range:', "$startDate to $endDate"],
    ['Generated At:', date("Y-m-d H:i:s")],
    [],
    ['Total Orders', $totalOrders],
    ['Total Revenue', number_format($totalRevenue, 2) . ' BD'],
    ['Total Visits', $totalVisits],
    ['Total Unique Visits', $totalUniqueVisits],
    ['Total Customer Logins', $totalCustomerLogins],
    ['New Customers', $newCustomers],
    ['Returning Customers', $returningCustomers],
    [],
    ['Status', 'Count', 'Total Amount']
];

foreach ($orderStatuses as $s) {
    $csvRows[] = [
        $s['status'],
        $s['c'],
        number_format($s['total'], 2) . ' BD'
    ];
}

$csvRows[] = [];
$csvRows[] = ['Item Name', 'Total Sold', 'Total Amount'];

foreach ($allItems as $i) {
    $csvRows[] = [
        $i['item_name'],
        $i['total_sold'],
        number_format($i['total_amount'], 2) . ' BD'
    ];
}
?>
```

```

</script>
Chart.defaults.font.family = 'Fredoka, sans-serif';
Chart.defaults.font.size = 14;
Chart.defaults.color = '#333';

new Chart(document.getElementById('itemsChart'), {
  type: 'bar',
  data: {
    labels: <?= json_encode($itemLabels) ?>,
    datasets: [{
      label: 'Items Sold',
      data: <?= json_encode($itemCounts) ?>,
      backgroundColor: '#dc3545'
    }]
  },
  options: { scales: { y: { beginAtZero: true } } }
});

new Chart(document.getElementById('revenueChart'), {
  type: 'line',
  data: {
    labels: <?= json_encode($dailyDates) ?>,
    datasets: [{
      label: 'Revenue Over Time',
      data: <?= json_encode($dailyValues) ?>,
      borderColor: '#28a745',
      fill: false
    }]
  }
});

let interval;
document.getElementById('autoRefresh').addEventListener('change', function () {
  if (this.checked) {
    interval = setInterval(() => location.reload(), 30000);
  } else {
    clearInterval(interval);
  }
});

function downloadCSV() {
  const rows = <?= json_encode($csvRows) ?>;
  const csv = rows.map(r => r.join(",")).join("\n");

  const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
  const link = document.createElement('a');
  link.href = URL.createObjectURL(blob);
  link.download = `analytics_report_<?= $startDate ?>_to_<?= $endDate ?>.csv`;
  link.click();
}
</script>

```

Figure 290 JavaScript to render bar and line charts with Chart.js, toggle auto-refresh, and generate a CSV report client-side.

The PHP segment builds an array of rows that represent the CSV content. Each row includes relevant metrics such as total orders, total revenue, number of visits, customer logins, and breakdowns of order statuses and menu item sales. This array, named `$csvRows`, is later passed to JavaScript using `json_encode`.

On the frontend, Chart.js is used to create a bar chart showing the total quantity of items sold and a line chart tracking revenue trends over time. These charts provide a fast, intuitive way to observe customer behavior and product performance within a selected date range.

The `downloadCSV()` function handles the download process. It converts the `$csvRows` data to a CSV string, encodes it in a Blob object, and triggers a download with a dynamically generated filename. This ensures that the report is always up to date and labeled by date range for clarity.

Testing and Validation

I authenticated as an admin and navigated to Analytics. I tested the date filter by selecting custom ranges and confirmed that cards, tables, and charts updated accordingly. I toggled auto-refresh on and off, observing timed reloads via developer tools. I clicked “Download CSV” and opened the exported file to verify column alignment. I also manually set a non-admin role to ensure access was denied.

Security Considerations

All state-changing actions (none on this page beyond reload/export) and form filters are protected by server-side role checks. Although no form POST modifies data here, the CSRF helper is loaded in case we extend this page with interactive POST endpoints. PDO with bound parameters prevents SQL injection, and session cookies inherit Secure, HttpOnly, and SameSite=Lax flags from `session_config.php`.

Images

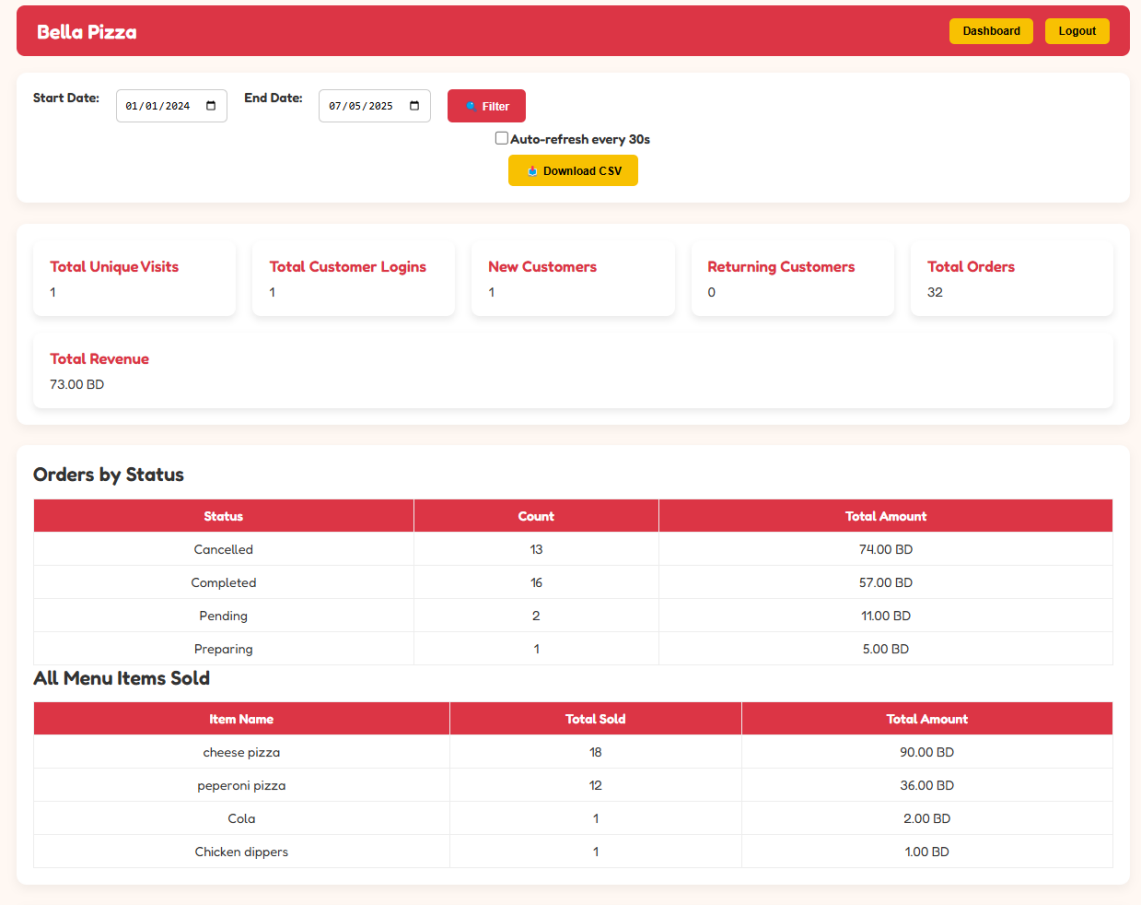


Figure 291 Analytics Table View

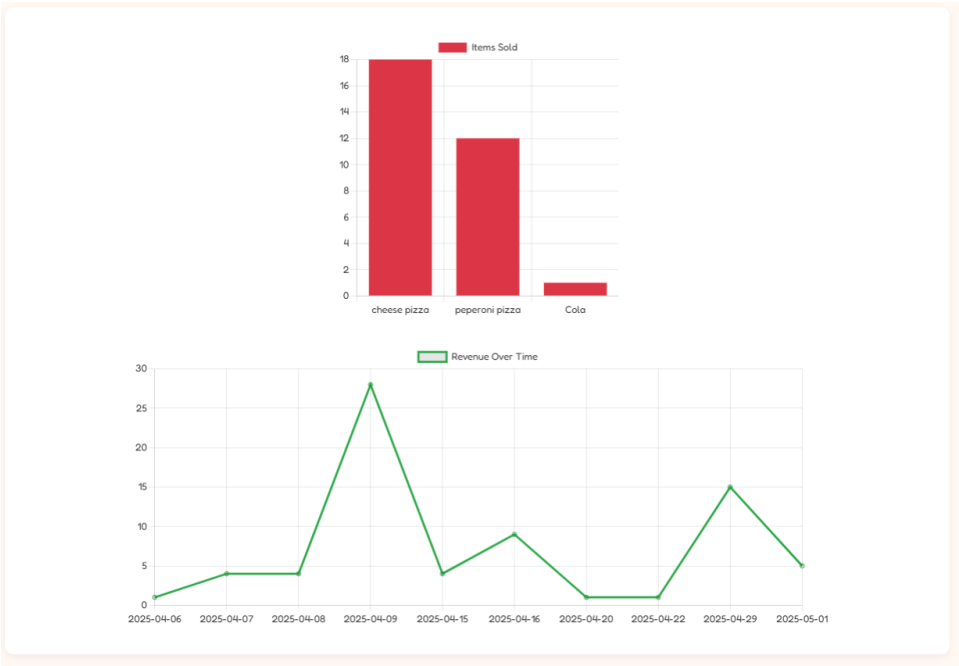


Figure 292 Analytics Graphs

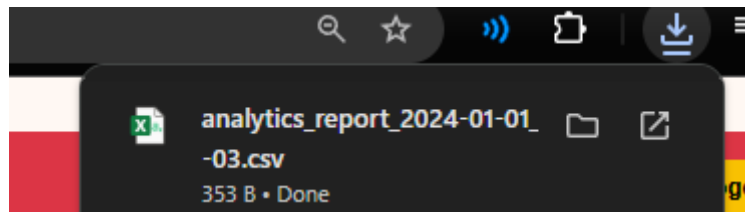


Figure 294 Downloading Analytics Report

A	B	C	D
Bella Pizza Analytics Report			
Date Range:	2024-01-01 to 2025-05-07		
Generated At:	07/05/2025 16:39		
Total Orders	32		
Total Revenue	73.00 BD		
Total Visits			
Total Unique Visits			
Total Customer Logins	1		
New Customers	1		
Returning Customers	0		
Status	Count	Total Amount	
Cancelled	13	74.00 BD	
Completed	16	57.00 BD	
Pending	2	11.00 BD	
Preparing	1	5.00 BD	
Item Name	Total Sold	Total Amount	
cheese pizza	18	90.00 BD	
peperoni pizza	12	36.00 BD	
Cola	1	2.00 BD	
Chicken dippers	1	1.00 BD	

Figure 293 analytics report in excel

IV. O. System Logs

The System Logs page provides administrators with a searchable, filterable, and paginated view of all application events recorded in the logs table. It enables auditing of user actions, error tracking, and compliance checks by exposing details such as timestamps, actors, descriptions, and contextual metadata.

Design Rationale

This page leverages PHP and MySQLi with prepared statements to safely construct dynamic queries based on user-supplied filters: free-text search, log level, and date range. A two-step process first counts matching records for pagination, then retrieves the current page of logs with LIMIT and OFFSET. The UI uses a sticky navbar for navigation, a responsive CSS layout with a scrollable table wrapper to handle wide columns, and minimal JavaScript—only for preserving query parameters in pagination links. A hidden CSRF token is included in the filter form for consistency with the site's security model, though the form issues only GET requests.

Session Init, DB Connection & Access Control

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

$conn = new mysqli("db", "root", "rootpass", "RestaurantDB");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
if (!isset($_SESSION['employee_id']) || !in_array($_SESSION['role'], ['admin'])) {
    header("Location: login.php");
    exit();
}
```

This snippet begins by invoking `session_start()` so that session variables—such as

Figure 295 Start a secure session, connect to MySQL, and require admin access.

snippet pulls in your hardened session settings (`session_config.php`) and CSRF helper, then opens a MySQLi connection. It immediately checks that a valid `employee_id` exists in session and that the user's role is admin, redirecting unauthorized users to the login page.

Parsing Filters & Building the WHERE Clause

```
$search = $_GET['search'] ?? '';
$level  = $_GET['level'] ?? '';
$start  = $_GET['start'] ?? '';
$end    = $_GET['end'] ?? '';
$page   = isset($_GET['page']) ? max(1, intval($_GET['page'])) : 1;
$limit  = 20;
$offset = ($page - 1) * $limit;

$where = "WHERE 1";
$params = [];
$types = "";

if ($search !== '') {
    $where .= " AND (action LIKE ? OR username LIKE ? OR description LIKE ?)";
    $searchTerm = '%' . $search . '%';
    $params[] = $searchTerm;
    $params[] = $searchTerm;
    $params[] = $searchTerm;
    $types  .= "sss";
}

if ($level !== '') {
    $where .= " AND log_level = ?";
    $params[] = $level;
    $types  .= "s";
}

if ($start !== '' && $end !== '') {
    $where .= " AND DATE(log_time) BETWEEN ? AND ?";
    $params[] = $start;
    $params[] = $end;
    $types  .= "ss";
}
```

Figure 296 Read and sanitize filter inputs, then assemble a dynamic WHERE clause plus corresponding parameter and type arrays for prepared statements.

Here, the code initializes filter variables—`$search`, `$level`, `$start`, `$end`, and `$page`—from the query string, providing defaults when parameters are missing. It then sets up pagination by defining `$limit` (rows per page) and calculating `$offset` from the current page. To build a secure, dynamic WHERE clause, it starts with "WHERE 1" and appends conditions only if their corresponding filter is non-empty. Each filter value is added to the `$params` array and its data type to `$types`, so later prepared statements can bind them safely. The search filter becomes three LIKE ? clauses (for action, username, and description), the level filter a single equality placeholder, and the date filter a BETWEEN ? AND ? clause—guaranteeing that all user inputs are bound rather than interpolated directly.

Counting & Fetching Paginated Logs

```
$countSql = "SELECT COUNT(*) FROM logs $where";
$countStmt = $conn->prepare($countSql);
if ($types) {
    $countStmt->bind_param($types, ...$params);
}
$countStmt->execute();
$countStmt->bind_result($total);
$countStmt->fetch();
$countStmt->close();

// Fetch logs
$dataSql = "SELECT * FROM logs $where ORDER BY log_time DESC LIMIT ? OFFSET ?";
$dataStmt = $conn->prepare($dataSql);
$params[] = $limit;
$params[] = $offset;
$types .= "ii";
$dataStmt->bind_param($types, ...$params);
$dataStmt->execute();
$logs = $dataStmt->get_result();
?>
```

Figure 297 First execute a COUNT(*) with bound parameters to get the total number of matching logs, then prepare and execute the paginated SELECT using the same filters plus LIMIT/OFFSET.

With the WHERE clause and parameter arrays ready, this block performs two key operations. First, it prepares and executes a SELECT COUNT(*) query to determine how many log entries match the filters, binding the \$params so no injection is possible; the result is stored in \$total for pagination. Second, it prepares a SELECT * query that reuses the same filters but adds ORDER BY log_time DESC, LIMIT ?, and OFFSET ? to fetch only the current page of results. The additional pagination parameters are appended to \$params and \$types before binding. Executing this statement yields a mysqli_result object (\$logs) that contains exactly the 20 log records for the requested page, sorted by newest first.

Filter Form, Table Rendering & Pagination

```
<div class="container">
  <h2>System Logs</h2>

  <form method="get" class="filters">
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
    <input type="text" name="search" placeholder="Search logs..." value="<?= htmlspecialchars($search) ?>">
    <select name="level">
      <option value="">All Levels</option>
      <option value="INFO" <?= $level === 'INFO' ? 'selected' : '' ?>>INFO</option>
      <option value="WARNING" <?= $level === 'WARNING' ? 'selected' : '' ?>>WARNING</option>
      <option value="ERROR" <?= $level === 'ERROR' ? 'selected' : '' ?>>ERROR</option>
    </select>
    <input type="date" name="start" value="<?= htmlspecialchars($start) ?>">
    <input type="date" name="end" value="<?= htmlspecialchars($end) ?>">
    <button type="submit">Filter</button>
  </form>

  <div class="table-wrapper">
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Level</th>
          <th>Timestamp</th>
          <th>Employee ID</th>
          <th>Customer ID</th>
          <th>Username</th>
          <th>Action</th>
          <th>Description</th>
          <th>IP Address</th>
          <th>User Agent</th>
          <th>Session</th>
          <th>Location</th>
        </tr>
      </thead>
      <tbody>
        <?php while ($log = $logs->fetch_assoc()): ?>
          <tr class="log-<?= strtolower($log['log_level']) ?>">
            <td><?= $log['log_id'] ?></td>
            <td><?= $log['log_level'] ?></td>
            <td><?= $log['log_time'] ?></td>
            <td><?= $log['employee_id'] ?? '-' ?></td>
            <td><?= $log['customer_id'] ?? '-' ?></td>
            <td><?= htmlspecialchars($log['username']) ?></td>
            <td><?= htmlspecialchars($log['action']) ?></td>
            <td><?= htmlspecialchars($log['description']) ?></td>
            <td><?= htmlspecialchars($log['ip_address']) ?></td>
            <td><?= htmlspecialchars($log['user_agent']) ?></td>
            <td><?= htmlspecialchars($log['session_id']) ?></td>
            <td><?= htmlspecialchars($log['location']) ?></td>
          </tr>
        <?php endwhile; ?>
      </tbody>
    </table>
  </div>

  <div class="pagination">
    <?php for ($i = 1; $i <= ceil($total / $limit); $i++): ?>
      <a href="<?= http_build_query(array_merge($_GET, ['page' => $i])) ?>" class="<?= $i === $page ? 'active' : '' ?>"><?= $i ?>
    <?php endfor; ?>
  </div>
</div>
```

Figure 298 Render the filter form with preserved values, display the paginated log entries in a responsive table, and generate pagination links based on the total count.

This HTML fragment renders the filter form, table, and pagination links. The form's inputs and `<select>` mirror the current filter values—safely echoed with `htmlspecialchars()`—so admins can refine their search without losing context. The `.table-wrapper` enables horizontal scrolling for wide tables. Inside the `<tbody>`, a PHP while loop fetches each row from `$logs`, printing its columns and using the null coalescing operator (`?: '-'`) to display a dash when `employee_id` or `customer_id` is missing. Each row is also given a class like `log-info`, `log-warning`, or `log-error` based on

its `log_level`, allowing CSS color-coding. Finally, the pagination section computes the total number of pages by dividing `$total` by `$limit` and renders links with an active class on the current page, preserving existing filters via `http_build_query()` so that paging does not reset the user's search criteria.

Testing and Validation

I first loaded the page without parameters to confirm it displayed the most recent logs and defaulted to page 1. I entered keywords present only in certain descriptions to verify that only matching rows appeared. I changed levels to INFO, WARNING, and ERROR in turn, confirming the results filtered correctly. I tested date-range filters by picking narrow windows and watched the COUNT and pagination links adjust. Finally, I viewed the HTML source in Chrome DevTools to ensure no unescaped user data appeared and that prepared statement parameters were correctly bound.

Security Considerations

All inputs filter into prepared statements (`bind_param`) preventing SQL injection. Escaping with `htmlspecialchars()` in every table cell blocks potential XSS. The session check at the top ensures only admins may access logs. A CSRF token is embedded in the filter form for consistency, though GET forms typically don't mutate state; this token can be validated if you later switch to POST. The sticky navbar and read-only UI avoid exposing any unintended actions on this audit page.

Images

System Logs										
Search logs...			All Levels		dd/mm/yyyy		dd/mm/yyyy		Filter	
ID	Level	Timestamp	Employee ID	Customer ID	Username	Action	Description	IP Address	User Agent	Session
28	INFO	2025-05-03 19:35:29	2	-	yanis	Update Employee	Updated employee #5 (jack)	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	02738262c978316cf3
27	INFO	2025-05-03 19:24:26	2	-	yanis	Add Employee	New employee '123123213' added with role chef	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	02738262c978316cf3
26	INFO	2025-05-03 18:22:54	2	-	yanis	Login Success	Employee logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	02738262c978316cf3
25	WARNING	2025-05-03 18:21:48	3	-	drake	Order Status Change	Order #31 marked as 'Preparing'	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	02738262c978316cf3

Figure 299 System Logs View

System Logs										
Yannis			All Levels		dd/mm/yyyy		dd/mm/yyyy		Filter	
ID	Level	Timestamp	Employee ID	Customer ID	Username	Action	Description	IP Address	User Agent	Session
2	INFO	2025-05-03 12:40:34	-	1	yannis	Login Success	Customer logged in successfully	172.18.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36	02738262c978316cf3a203ddd
1										

Figure 300 Search Funtion in System Logs

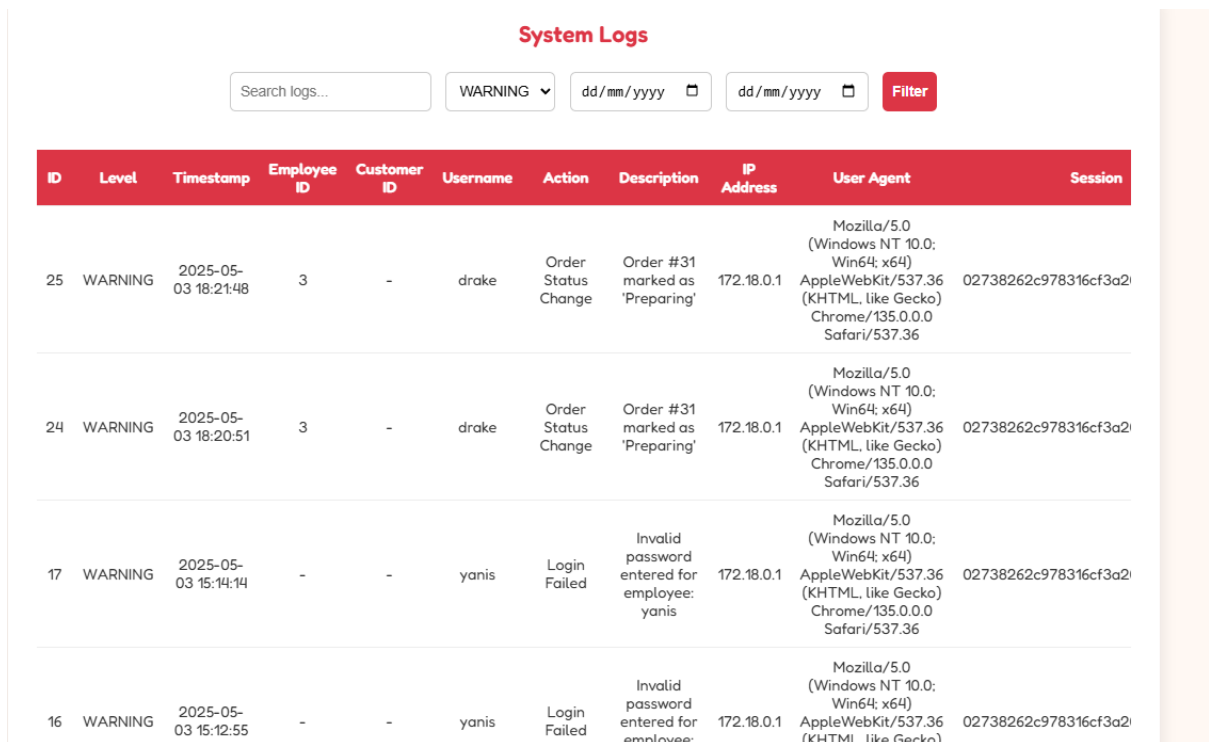


Figure 301 Filetring messages by Level "Warning"

IV. P. Real-time Delivery tracking System

The delivery tracking system in Bella Pizza is designed to enable customers to view the live location of the delivery driver handling their order. This functionality spans four interconnected files: `track_delivery.php`, `get_driver_location.php`, `driver_share_location.php`, and `driver_update_location.php`. Together, they facilitate real-time, secure, and accurate tracking of deliveries from the restaurant to the customer's location. Customers can visually track the driver on a map, while delivery personnel share their current coordinates using their mobile device.

Design Rationale

The system is implemented using a modular client-server architecture. The front-end page `track_delivery.php` serves as the customer-facing map interface. It uses `Leaflet.js` to render a responsive map and polls the backend every 10 seconds to update the driver's marker position. This page includes strict session and access validation to prevent unauthorized users from tracking orders they do not own.

On the server side, `get_driver_location.php` acts as a dedicated AJAX endpoint that fetches the driver's latest latitude and longitude from the database. It is optimized for fast, lightweight responses and serves only essential data in JSON format.

On the driver's end, `driver_share_location.php` is a mobile-friendly page that, once opened, begins capturing the driver's location using the browser's geolocation API. The captured coordinates are sent every few seconds via background requests to `driver_update_location.php`, which is responsible for updating the driver's position in the database in real time. The update script includes session validation to ensure only logged-in delivery staff can perform updates, ensuring security and accountability.

Together, these components form a robust, responsive, and secure delivery tracking system that enhances transparency, improves customer satisfaction, and streamlines the delivery workflow.

Session Validation

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit();
}
```

Figure 302 Checks if a customer is logged in before allowing access to the tracking page.

This block confirms that the user is logged in as a customer. If the session is missing a `user_id`, the user is redirected to the login page to prevent unauthorized access.

Extract Order and Driver IDs

```
$order_id = isset($_GET['order_id']) ? (int)$_GET['order_id'] : 0;
$driver_id = isset($_GET['driver_id']) ? (int)$_GET['driver_id'] : 0;
```

Figure 303 Retrieves and sanitizes `order_id` and `driver_id` from the URL.

This code retrieves `order_id` and `driver_id` from the URL. Both are cast to integers to sanitize the input before they're used in database queries.

Validate Ownership and Assignment

```
$stmt = $pdo->prepare("SELECT delivery_driver_id FROM `order` WHERE order_id = ? AND customer_id = ?");
$stmt->execute([$order_id, $_SESSION['user_id']]);
$order = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$order || (int)$order['delivery_driver_id'] !== $driver_id) {
    die("Unauthorized access.");
}
```

Figure 304 Verifies that the logged-in customer owns the order and that the driver matches the assignment.

This query ensures that the order belongs to the currently logged-in customer and that the delivery is indeed assigned to the specified driver. If either condition fails, access is denied to prevent users from tracking someone else's order.

Map Display and UI

```
<div class="top-bar">
| <a href="my_orders.php" class="back-button">← Back to My Orders</a>
</div>

<div class="title-banner">
| Tracking Order #<? = htmlspecialchars($order_id) ?>
</div>

<div id="map"></div>
```

Figure 305 Renders the map container and navigation elements for the user interface.

The interface includes a back button, a title showing the order number, and a full-page Leaflet map container. The map will visually track the driver's real-time location.

Include Leaflet JavaScript Library

```
<script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
```

Figure 306 Loads the Leaflet.js library from a CDN to enable interactive map rendering.

This line imports the Leaflet JavaScript library, which powers the interactive map displayed on the page. Leaflet is a lightweight open-source library used for rendering maps, placing markers, and handling zoom or pan functionality. By including it via a CDN (unpkg.com), the map features work out of the box without hosting the library locally.

Initialize Map and Fetch Driver Location

```
<script>
const map = L.map('map').setView([26.2285, 50.5860], 12);
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);
let marker;
```

Figure 307 Initializes the Leaflet map with OpenStreetMap tiles and a placeholder for the driver marker.

The map is initialized using Leaflet.js and set to a default location (e.g., Bahrain). OpenStreetMap tiles are used as the map base layer. A marker is declared to represent the driver's location.

Live Driver Location Updates

```
async function fetchLocation() {
  const res = await fetch('get_driver_location.php?driver_id=<?=> $driver_id >');
  const data = await res.json();
  if (data.latitude && data.longitude) {
    const pos = [data.latitude, data.longitude];
    if (!marker) {
      marker = L.marker(pos).addTo(map).bindPopup("Driver Location").openPopup();
    } else {
      marker.setLatLng(pos);
    }
    map.setView(pos, 14);
  }
}
```

Figure 308 Periodically fetches the driver's coordinates from the server and updates the map marker.

This function queries the backend every 10 seconds for the latest driver coordinates. If valid coordinates are returned, it updates the marker's position on the map or creates a new one. The map view is recentered on each update to follow the driver's movements.

get_driver_location.php - Get Driver ID from Query Parameter

```
$driver_id = (int) ($_GET['driver_id'] ?? 0);
```

Figure 309 Establishes a PDO connection to the RestaurantDB database.

Creates a new PDO instance to communicate with the MySQL database. It connects to the RestaurantDB using the Docker service name db, with the default root credentials.

Fetch Driver Coordinates

```
$stmt = $pdo->prepare("SELECT latitude, longitude FROM driver_location WHERE driver_id = ?");
$stmt->execute([$driver_id]);
$data = $stmt->fetch(PDO::FETCH_ASSOC);
```

Figure 310 Queries the driver_location table to get the driver's latest latitude and longitude.

This uses a prepared statement to securely fetch the current coordinates of the driver from the driver_location table. It returns one row containing the latest known location.

Return Location as JSON

```
echo json_encode($data ?: []);
```

Figure 311 Sends the driver's location to the frontend as a JSON object or an empty array if not found.

The result is encoded into JSON format, which the JavaScript on track_delivery.php can parse and use to update the map. If no data is found, an empty array is returned to avoid errors on the client side.

driver_share_location.php - Session Validation for Delivery Role

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION['employee_id']) || $_SESSION['role'] !== 'delivery') {
    header("Location: login.php");
    exit();
}
```

Figure 312 Ensures only logged-in delivery drivers can access the location sharing page

Before the page loads, the script checks that the user is authenticated and has a delivery role. Unauthorized users are redirected to the login page, preventing anyone other than drivers from broadcasting GPS data.

Page Layout and Style

```
<body>
    <h2>Live Location Sharing</h2>
    <p>This page will continuously send your real-time GPS position.</p>
    <div id="status" class="status">⌚ Waiting for GPS fix...</div>
    <div class="info">Please keep this page open during delivery.</div>
```

Figure 313 Displays instructions and status messages to the driver.

The page includes a simple interface to inform the driver that their location is being tracked and encourages them to keep the page open throughout the delivery. The status element is dynamically updated based on GPS fix and network responses.

Send Coordinates to Server

```
<script>
  const status = document.getElementById("status");

  function sendLocation(lat, lng) {
    fetch("driver_update_location.php", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ latitude: lat, longitude: lng })
    })
    .then(res => res.json())
    .then(data => {
      if (data.status === "ok") {
        status.textContent = `📍 Sent: ${lat.toFixed(5)}, ${lng.toFixed(5)}`;
        status.className = "status ok";
      } else {
        status.textContent = "❌ Failed to send location";
        status.className = "status error";
      }
    })
    .catch(() => {
      status.textContent = "❌ Network error";
      status.className = "status error";
    });
  }
}
```

Figure 314 Sends the driver's current latitude and longitude to the server using a POST request.

This function uses `fetch()` to POST the driver's location to `driver_update_location.php`. It handles the response and updates the status text and styling to give real-time feedback (e.g., success, error, or network failure).

Track and Watch GPS Location

```
function startTracking() {
  if (!navigator.geolocation) {
    status.textContent = "✗ Geolocation not supported";
    status.className = "status error";
    return;
  }

  navigator.geolocation.getCurrentPosition(
    pos => {
      sendLocation(pos.coords.latitude, pos.coords.longitude);
    },
    err => {
      status.textContent = "✗ Initial fix error: " + err.message;
      status.className = "status error";
    },
    { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
  );

  // Alias of current tracking
  na pos tion.watchPosition(
    pos => {
      sendLocation(pos.coords.latitude, pos.coords.longitude);
    },
    err => {
      status.textContent = "✗ GPS error: " + err.message;
      status.className = "status error";
    },
    { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
  );
}

startTracking();
</script>
```

Figure 315 Uses the browser's geolocation API to capture and continuously track the driver's position.

The script first fetches a one-time "fix" to get the initial location. Then, using `watchPosition()`, it continuously listens for location changes and sends them to the backend. High accuracy is enabled to improve precision, and errors are captured for better user feedback.

driver_update_location.php - Session and Role Validation

```
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION['employee_id']) || $_SESSION['role'] !== 'delivery') {
    exit('Unauthorized');
}
```

Figure 316 Confirms the user is a logged-in delivery driver before accepting location data.

This ensures only authenticated employees with the 'delivery' role can update location data. If the session is invalid or the role doesn't match, the script halts immediately to prevent unauthorized updates.

Parse Incoming JSON Coordinates

```
$input = json_decode(file_get_contents('php://input'), true);
$lat = $input['latitude'] ?? null;
$lng = $input['longitude'] ?? null;

if (!$lat || !$lng) {
    echo json_encode(['status' => 'error']);
    exit();
}
```

Figure 317 Extracts latitude and longitude from the POST request's JSON body.

The coordinates are expected as a JSON payload from the driver's device. The script safely parses the input and validates that both values are present before continuing. If either is missing, it returns an error response.

Update Driver's Location in Database

```
$pdo = new PDO("mysql:host=db;dbname=RestaurantDB", "root", "rootpass");
$stmt = $pdo->prepare("REPLACE INTO driver_location (driver_id, latitude, longitude, updated_at) VALUES (?, ?, ?, NOW())");
$stmt->execute([$_SESSION['employee_id'], $lat, $lng]);
```

Figure 318 Inserts or updates the driver's current location using their session-linked ID.

A PDO connection is opened, and a REPLACE INTO query is used to insert or update the driver's row in the driver_location table. This ensures each driver has a single up-to-date location record, identified by their employee_id.

Return Success Status

```
echo json_encode(['status' => 'ok']);
```

Figure 319 Sends a JSON response confirming the location update was successful.

If everything goes smoothly, the server replies with a simple success message (`{ "status": "ok" }`) that the frontend script can check to provide visual confirmation to the driver.

Testing approach

The system was tested in several end-to-end scenarios involving both customer and delivery driver roles. As a customer, accessing `track_delivery.php` without logging in resulted in a correct redirect to the login page. Attempting to tamper with URL parameters (e.g., using a different order ID) correctly triggered the “unauthorized access” block. The map loaded successfully and updated the marker position when paired with an active driver session.

On the driver side, `driver_share_location.php` was tested on both desktop and mobile browsers with GPS enabled. The script accurately detected the current position, sent regular updates to the server, and received success confirmation via the status box. Loss of GPS signal or network connection showed appropriate error messages, while background polling of `get_driver_location.php` was confirmed to refresh the customer-facing map every 10 seconds as intended. All critical flows were validated for input handling, access control, and session consistency.

Security Considerations

The delivery tracking system includes multiple safeguards to ensure that location data is protected and only accessible to authorized users. First, all pages and endpoints involved are protected using session-based authentication, restricting access to logged-in users with appropriate roles. For instance, `track_delivery.php` verifies both the customer’s session and their ownership of the order before displaying any map data. Similarly, `driver_share_location.php` and `driver_update_location.php` strictly enforce that only employees with the 'delivery' role can share or update coordinates.

To protect against tampering and injection attacks, input values are typecast (e.g., driver IDs as integers) and handled through prepared PDO statements. The use of CSRF protection via `includes/csrf.php` is consistent across all PHP files that manage session state, and JSON endpoints are kept minimal and secured through backend validation. In addition, `driver_update_location.php` uses `REPLACE INTO` to reduce risk of duplicate entries or stale data corruption, ensuring data integrity. These measures collectively reduce the attack surface and help maintain a secure, real-time tracking experience.

Images

My Orders				
Search by ID, Status, or Date			Search	
Order ID	Date	Status	Total (BD)	Action
#35	09 May 2025, 15:12	Cancelled	2.50 BD	View Details
#34	09 May 2025, 13:08	Out for Delivery	2.00 BD	View Details Track Delivery

Figure 320 "out for delivery" order showing "track delivery" button

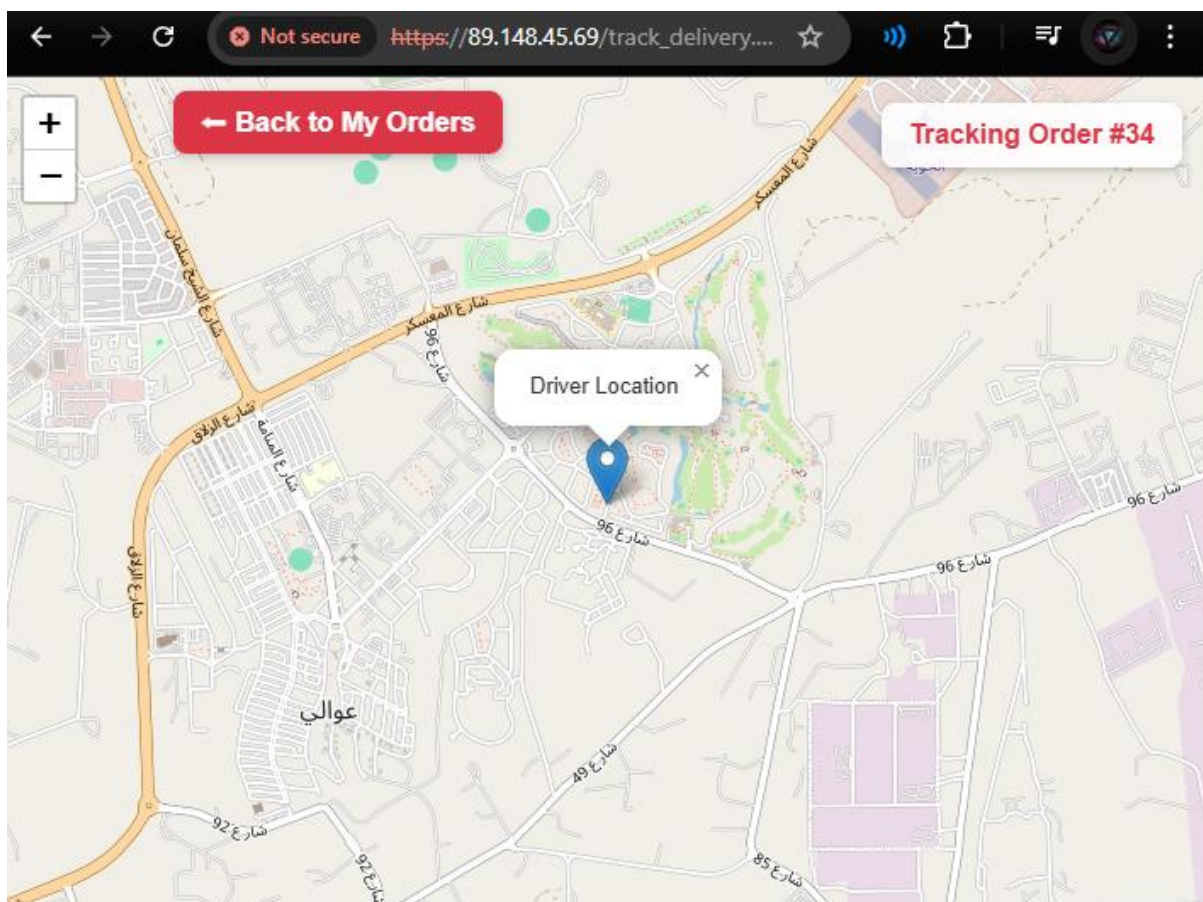


Figure 321 Map UI showing very accurate location of driver

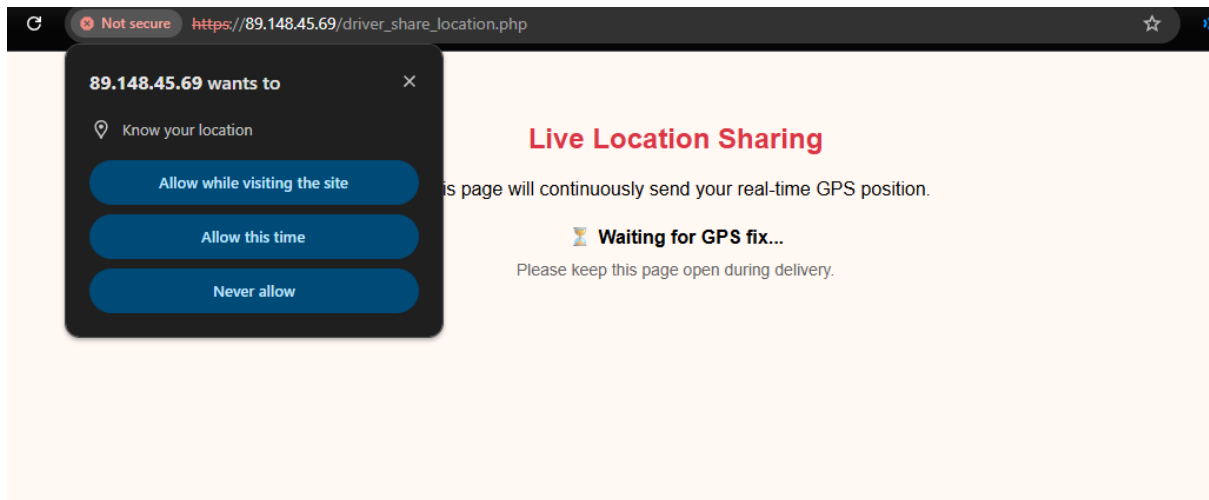


Figure 323 Share location page prompting drive to accept sharing their location

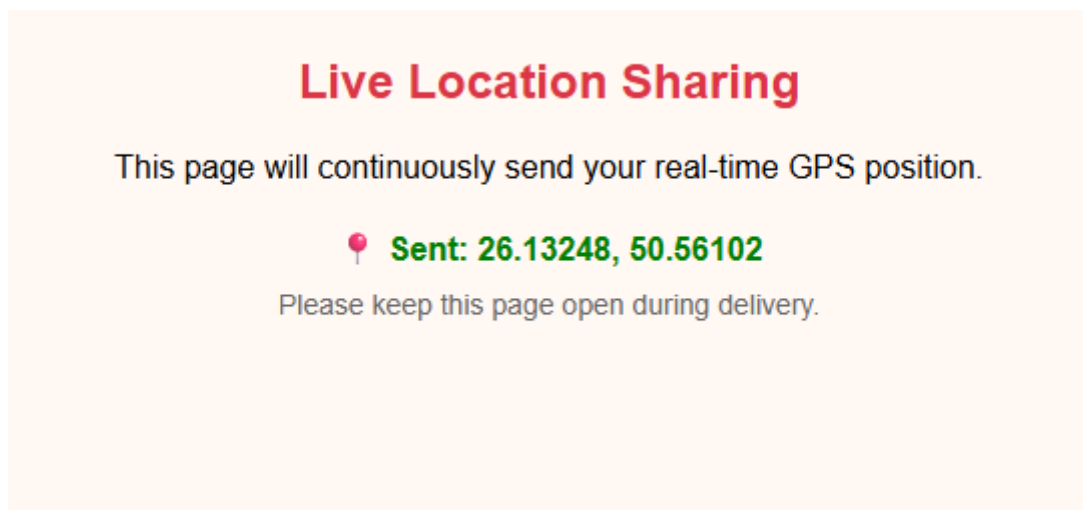


Figure 322 Successfully sent GPS location

IV. Q. Customer Dashboard

The Customer Dashboard page welcomes a logged-in customer, displays the restaurant's menu organized by categories, and allows them to add items to their shopping cart. It combines session-based access control with dynamic retrieval of menu data to drive a responsive, touch-friendly interface.

Design Rationale

This page uses PHP with PDO for secure database queries of customer and menu information, avoiding SQL injection through prepared statements. Session management is hardened via `session_config.php` and CSRF tokens guard the “add to cart” form. The layout leverages a sticky header for primary navigation, a visually striking hero banner for branding, and horizontally scrollable “cards” for each menu item—ideal for both desktop and mobile users. Minimal JavaScript is used for the add-to-cart action, preserving progressive enhancement.

Session Initialization and Customer Lookup

```
<?php
require __DIR__ . '/includes/session_config.php';
require __DIR__ . '/includes/csrf.php';

if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit();
}

$pdo = new PDO("mysql:host=db;dbname=RestaurantDB", "root", "rootpass");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $pdo->prepare("SELECT username FROM customer WHERE customer_id = ?");
$stmt->execute([$_SESSION["user_id"]]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Figure 324 Protect the page behind login and fetch the current customer's username

This PHP block first enforces that a `user_id` exists in session, redirecting unauthorized visitors. It then creates a PDO connection with exceptions enabled and retrieves the logged-in customer's username in a single prepared statement.

Tracking Customer Logins and Identifying First-Time Users

```
$stmt = $pdo->prepare("INSERT INTO customer_logins (customer_id) VALUES (?)");  
$stmt->execute([$customerId]);  
  
$checkStmt = $pdo->prepare("SELECT COUNT(*) FROM customer_logins WHERE customer_id = ?");  
$checkStmt->execute([$customerId]);  
$loginCount = $checkStmt->fetchColumn();  
  
$isFirstLogin = ($loginCount == 1);
```

Figure 325 Snippet from `dashboard_user.php` that logs a login and checks whether the customer is new or returning.

Once the customer is verified via session (`$_SESSION["user_id"]`), a new entry is inserted into the `customer_logins` table. This table contains a `login_time` timestamp and a `customer_id` foreign key. Each visit to the dashboard creates a new row.

Immediately after logging the login, the system counts how many times this user has logged in. If the count is exactly 1, the customer is considered new (i.e., this is their first login). Otherwise, they are marked as a returning user. This boolean (`$isFirstLogin`) can then be used elsewhere in the UI or analytics module to differentiate customer types.

Handling “Add to Cart” POST with CSRF Validation

```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST["add_to_cart"])) {  
    validate_csrf_token();  
  
    $menu_id = (int)$_POST["menu_id"];  
    $quantity = max(1, (int)$_POST["quantity"] ?? 1);  
  
    if (!isset($_SESSION["cart"])) {  
        $_SESSION["cart"] = [];  
    }  
    $_SESSION["cart"][$menu_id] = ($_SESSION["cart"][$menu_id] ?? 0) + $quantity;  
    $message = "✅ Added to cart!";  
}
```

Figure 326 Securely update the session-based shopping cart when the user submits a form

Upon form submission, the CSRF token is validated to guard against cross-site request forgery. The snippet casts inputs to integers, enforces a minimum quantity of one, and then accumulates the quantity in the `$_SESSION['cart']` array, setting a success message for the UI.

Loading Categories and Menu Items

```
$categories = $pdo->query("SELECT * FROM category ORDER BY display_order ASC, name ASC")->fetchAll(PDO::FETCH_ASSOC);  
$category_items = [];  
$itemStmt = $pdo->prepare("SELECT * FROM menu WHERE availability=1 AND category_id=? ORDER BY item_name");  
foreach ($categories as $cat) {  
    $itemStmt->execute([$cat['category_id']]);  
    $category_items[$cat['category_id']] = $itemStmt->fetchAll(PDO::FETCH_ASSOC);  
}  
?>
```

Figure 327 Build two arrays: one of categories, the other grouping available menu items by category.

This snippet first queries the category table in a single pass, then prepares a reusable statement to fetch each category’s available items. The result is a structured PHP array ready for templating.

Header and Navigation Markup

```
<header class="header">
  <div class="logo">Bella <span>Pizza</span> 🍕 </div>
  <nav>
    <button onclick="window.location='my_orders.php'">My Orders</button>
    <button onclick="window.location='view_cart.php'" class="cart-btn">Cart</button>

    <form method="POST" action="logout.php" class="logout-form">
      <input type="hidden" name="csrf_token"
        value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>"
      <button name="logout">Logout</button>
    </form>
  </nav>
</header>
```

Figure 328 Render a sticky header with branding, quick links, and a logout form.

The .header stays visible when scrolling, the “Cart” button shows a badge with the total quantity, and the logout action is protected by a CSRF token.

Hero Banner and Personalized Welcome

```
<div class="hero">Delicious, Fast & Fresh!</div>
<main>
  <div class="welcome">
    <h2>Welcome, <?= htmlspecialchars($user['username']) ?>!</h2>
    <p>Order your favorite meals below.</p>
  </div>
  <?php if (!empty($message)): ?>
    <div class="message"><?= $message ?></div>
  <?php endif; ?>
```

Figure 329 Display a thematic banner and greet the customer by name, showing any success messages.

A full-width hero section grabs attention, followed by a personalized greeting. If the user just added an item, the “Added to cart!” message appears and then fades away.

Category Sections with Scrollable Cards

```
<?php foreach ($categories as $cat): ?>
    <section class="category">
        <h2><?= htmlspecialchars($cat['name']) ?></h2>
        <div class="cards">
            <?php foreach ($category_items[$cat['category_id']] as $item): ?>
                <div class="card">
                    
                    <div class="card-content">
                        <h3><?= htmlspecialchars($item['item_name']) ?></h3>
                        <p class="desc"><?= htmlspecialchars($item['description']) ?: 'No description.'
                            ?></p>
                        <div class="price"><?= number_format($item['price'],2) ?> BD</div>
                    </div>
                    <div class="card-actions">
                        <form method="POST">
                            <!-- CSRF token for add-to-cart -->
                            <input type="hidden" name="csrf_token"
                                value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
                            <input type="hidden" name="menu_id" value="<?= $item['menu_id'] ?>">
                            <input type="number" name="quantity" value="1" min="1">
                            <button type="submit" name="add_to_cart">Add to Cart</button>
                        </form>
                    </div>
                </div>
            <?php endforeach; ?>
        </div>
    </section>
<?php endforeach; ?>
```

Figure 330 Loop over each category to render horizontally scrollable item cards with add-to-cart forms.

Each card shows an image, name, description, and price. The “Add to Cart” form includes a hidden CSRF token, and the quantity input defaults to one.

Testing Approach

I manually walked through the page as a customer, verifying that unauthorized users are redirected to login. I confirmed that the hero banner and greeting appeared correctly. I filtered categories by temporarily altering availability flags in the database and ensured only available items displayed. I tested adding items to the cart with various quantities, then inspected `$_SESSION['cart']` in DevTools under the PHP session cookie to verify counts. I also submitted invalid CSRF tokens via DevTools to confirm the request was rejected.

Security Considerations

All database interactions use prepared statements to prevent SQL injection. The session cookie is secured (HttpOnly, Secure, SameSite=Lax) by `session_config.php`. CSRF tokens in every state-changing form ensure that only legitimate pages can submit cart updates or logouts. HTML escaping via `htmlspecialchars()` guards against XSS. No sensitive data or debug information is exposed in error messages.

Images

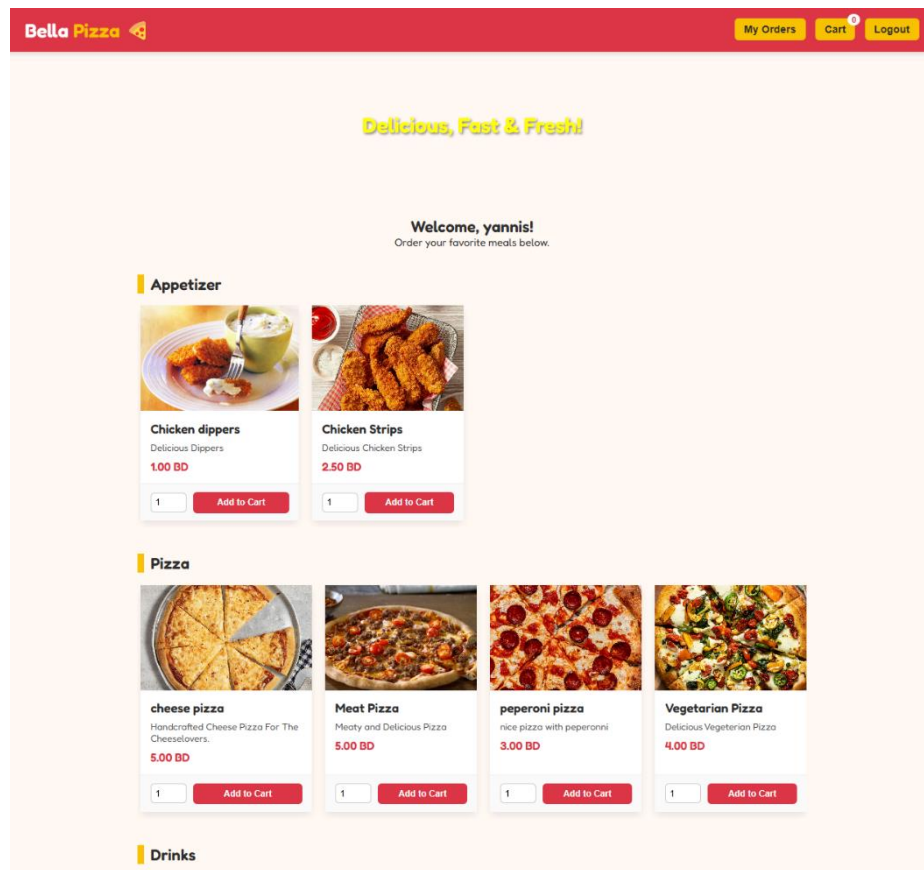


Figure 331 Customer Dashboard View

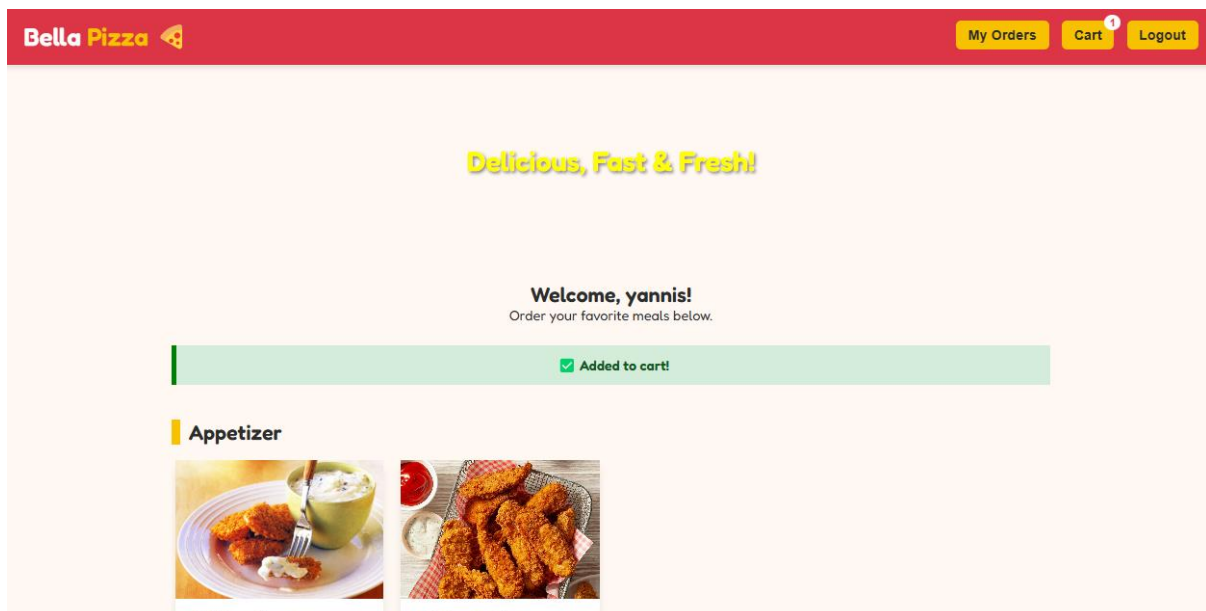


Figure 332 Successful add to cart message

IV. R. View Cart Page

The Cart page presents the logged-in customer's current selections, allowing them to adjust quantities, remove items, clear the entire cart, or proceed to payment. It ensures the cart's contents persist in session storage, calculates subtotals and the grand total, and safely handles all cart updates via POST.

Design Rationale

The design of `view_cart.php` focuses on creating a user-friendly, secure, and responsive cart management interface that reflects the Bella Pizza brand. A visually clean layout with soft colors, rounded edges, and responsive design principles ensures that customers on both desktop and mobile devices can easily manage their orders. The use of HTML tables allows for clear presentation of items, quantities, and totals, while interactive buttons enable intuitive actions such as updating or clearing the cart.

From a functional perspective, the page is driven by session-based storage, which allows users to freely navigate the website while retaining their cart state. PHP is used to dynamically fetch updated item details such as prices, ensuring that the cart always reflects current data from the database. Conditional logic is applied to prevent users from proceeding to payment if essential profile details like phone number or address are missing — a critical measure for delivery-based services.

Security was embedded into the design using CSRF tokens on all form submissions, prepared SQL statements for database interactions, and strict input validation. Altogether, the page is designed to provide a seamless ordering experience while reinforcing data accuracy, privacy, and transaction readiness in preparation for the checkout process.

Session and Database Initialization

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit();
}

$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

Figure 333 Ensure only logged-in users with items in their cart can proceed, then establish a PDO connection.

This block first includes the session configuration to apply hardened cookie settings. It checks that `$_SESSION['user_id']` exists and that `$_SESSION['cart']` is not empty; otherwise it redirects to the home page. It then creates a PDO connection to the RestaurantDB, throwing an exception on failure.

Check Profile Completion Based on Login Type

```
$missingProfile = false;
$loginType = $_SESSION['login_type'] ?? 'normal';
$user_id = $_SESSION['user_id'];

if ($loginType === 'google') {
    $stmt = $pdo->prepare("SELECT phone, address FROM google_login WHERE customer_id = ?");
} else {
    $stmt = $pdo->prepare("SELECT phone, address FROM customer WHERE customer_id = ?");
}
$stmt->execute([$user_id]);
$profile = $stmt->fetch(PDO::FETCH_ASSOC);

if (empty($profile['phone']) || empty($profile['address'])) {
    $missingProfile = true;
}
```

Figure 334 Determines if the user's profile contains valid phone and address information before proceeding to payment.

This logic supports both Google-authenticated and standard users by querying the appropriate table. It flags users who have not completed their profile to prevent incomplete orders.

Processing Cart Updates and Clearing

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    validate_csrf_token();

    if (isset($_POST['update'])) {
        foreach ($_POST['quantities'] as $menu_id => $qty) {
            $qty = max(0, (int)$qty);
            if ($qty === 0) {
                unset($_SESSION['cart'][$menu_id]);
            } else {
                $_SESSION['cart'][$menu_id] = $qty;
            }
        }
    } elseif (isset($_POST['clear_cart'])) {
        $_SESSION['cart'] = [];
    }
}
```

Figure 335 Handle quantity updates or clear the cart entirely.

Upon POST, we validate the CSRF token and then check which button was pressed. If “Update Cart” was clicked, we loop over the quantities array: any zero quantity removes the item, otherwise we set the new quantity in session. If “Clear Cart” was clicked, we simply reset the session cart to an empty array.

Fetching Menu Item Details and Computing Totals

```
$cart = $_SESSION['cart'];
$menu_items = [];
$total = 0;

if (!empty($cart)) {
    $placeholders = implode(',', array_fill(0, count($cart), '?'));
    $stmt = $pdo->prepare("SELECT * FROM menu WHERE menu_id IN ($placeholders)");
    $stmt->execute(array_keys($cart));
    $menu_items = $stmt->fetchAll(PDO::FETCH_ASSOC);
}
?>
```

Figure 336 Retrieve details for each menu item in the cart.

We extract the cart array from session. If non-empty, we build a WHERE menu_id IN (...) query with positional placeholders, bind the array of menu IDs, and fetch each row into \$menu_items. \$total will be calculated later in the template.

Conditional Payment Gateway Access

Figure 337 Display an editable cart table, update/clear buttons, and a “Proceed to Payment” form.

Each row shows the item name, unit price, editable quantity input (which can be set to zero to remove), and subtotal. We accumulate \$total across rows. The “Update Cart” and “Clear Cart” buttons both post back to this page. Below, a second form passes the total to the payment gateway page.

```
<?php if ($missingProfile): ?>
    <div class="message warning">
        ⚠ Please complete your profile (phone number and address) before proceeding to
        payment.
        <br><br>
        <button type="button" onclick="window.location.href='profile.php'">Go to
        Profile</button>
    </div>
<?php else: ?>
```

Figure 338 Prevents users with incomplete profiles from accessing the payment gateway.

This user flow ensures that orders can’t proceed unless both phone number and address are available. This is critical for delivery and payment processing.

Navigation and Empty-Cart Message

```
<?php if (empty($cart)): ?>
|   <p style="text-align:center;">Your cart is empty.</p>
<?php else: ?>
<div class="links">
|   <a href="dashboard_user.php">← Back to Menu</a>
</div>
```

Figure 339 Offer navigation back to the menu and handle an empty cart.

If the cart is empty, we show a centered message and skip the table entirely. A “Back to Menu” link is always available to let the user continue browsing.

Testing Approach

The cart page was tested across multiple scenarios: adding items from the menu, adjusting quantities, removing items, clearing the cart, and proceeding to payment. The system successfully preserved cart data in the session between page loads. Attempts to bypass profile completion via direct form submission were blocked, and redirection to profile.php worked as intended. CSRF token mismatches triggered appropriate errors when manipulated, confirming token validation. Quantity inputs were also tested with invalid values (e.g., negative numbers, strings) and correctly defaulted to zero or were rejected.

Security Considerations

The view_cart.php page enforces strict session checks to ensure only logged-in users can manage their cart. All sensitive operations — such as updating or clearing the cart, and proceeding to checkout — are protected using CSRF tokens, which are validated server-side for every POST request. All user inputs are sanitized and cast to the expected types before being stored in the session or used in queries. Additionally, all database access is done through prepared PDO statements, guarding against SQL injection. Sensitive customer details, such as phone numbers and addresses, are only queried after verifying the authenticated user's identity.

Images

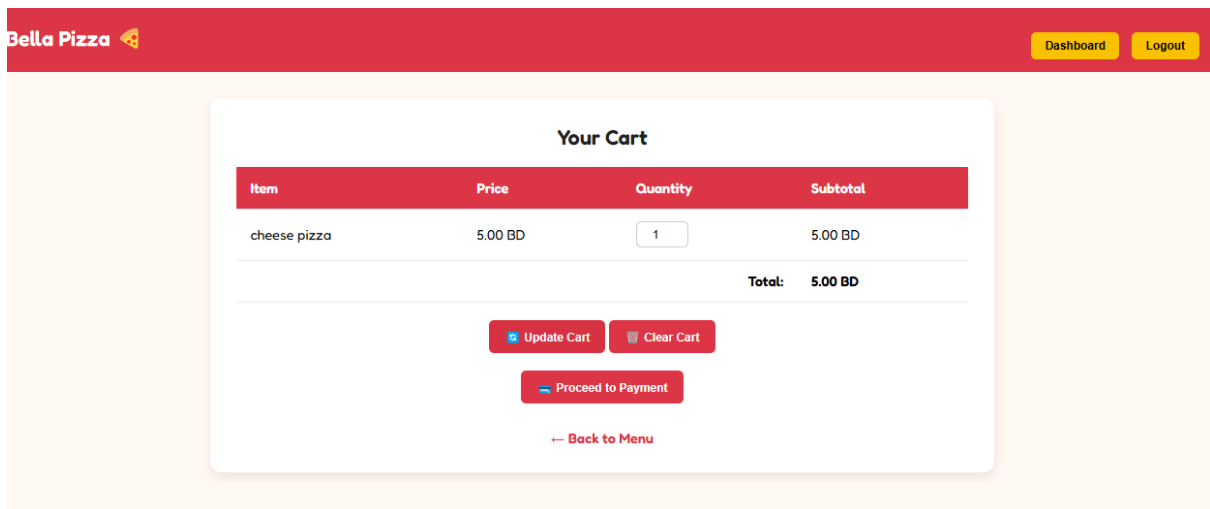


Figure 342 The Cart page lets users adjust quantities, view subtotals, and proceed to payment.

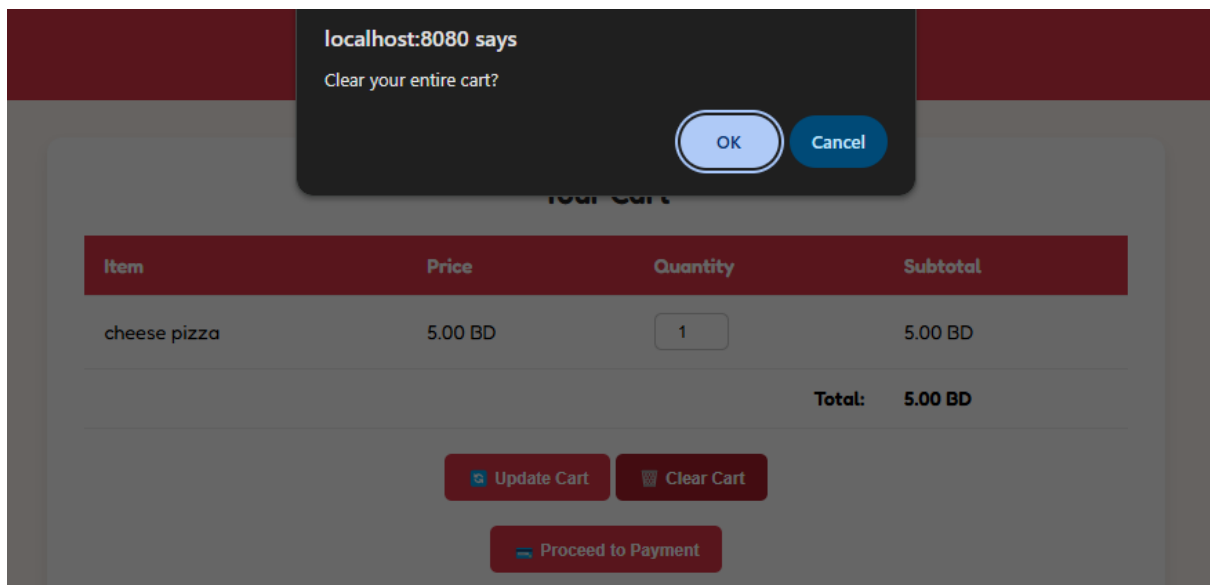


Figure 341 Clearing cart Confirmation Message

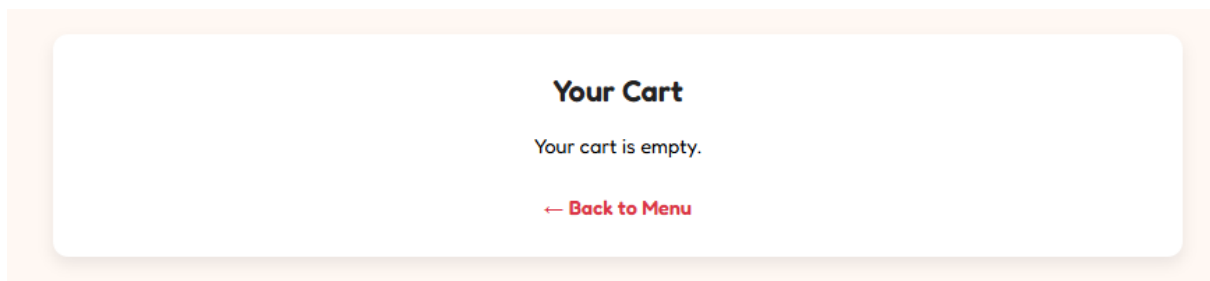


Figure 340 Empty Cart View

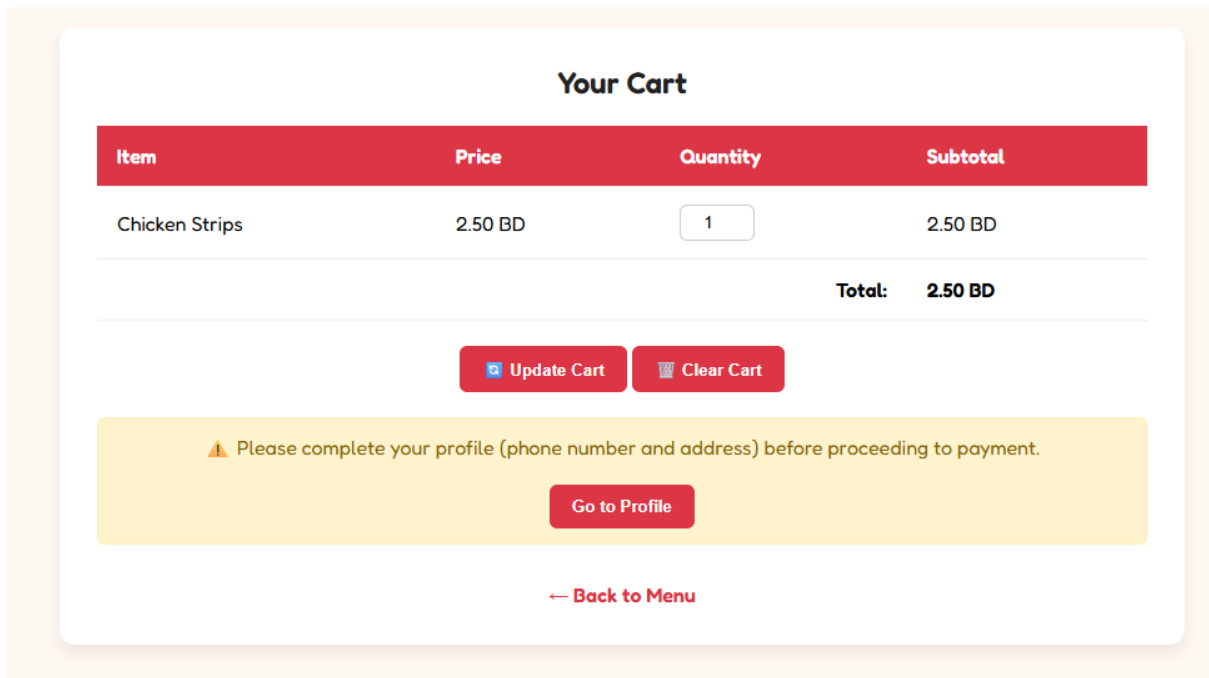


Figure 343 Warning message displayed if user profile is not complete

IV. S. Payment Gateway

This page is responsible for securely redirecting a logged-in customer with a non-empty cart to the APS (formerly Payfort) hosted checkout page. It gathers the required payment parameters, signs them using a shared secret phrase, and then auto-submits an HTML form so the customer proceeds seamlessly to APS to complete their purchase.

Design Rationale

We begin by validating that the user is logged in and has a shopping cart, ensuring only legitimate checkout attempts. APS credentials (access code, merchant identifier, and shared secret phrase) are stored in server-side PHP variables, never exposed in the HTML. We assemble the request parameters in an array, sort them alphabetically (as required by APS), and then compute a SHA-256 signature using the secret phrase both before and after the concatenated key-value string. Finally, we render an HTML form of hidden inputs and auto-submit it via JavaScript for a smooth redirect. Using a `<meta http-equiv="refresh">` fallback ensures compatibility if JavaScript is disabled.

Session Validation and APS Credentials

```
<?php
require_once 'includes/session_config.php';

// Redirect to home if not logged in or cart is empty
if (!isset($_SESSION['user_id']) || empty($_SESSION['cart'])) {
    header("Location: index.php");
    exit;
}

// === APS Credentials ===
$access_code = "IFafg4g3ZkDD5SigVlp1";
$merchant_identifier = "pCNttJRP";
$sha_request_phrase = "12QN0MFnhxftV7Rev1l2nY[!";
$currency = "BHD";
$language = "en";
$return_url = "http://localhost:8080/response.php?session_id=" . session_id(); // update with your actual
return handler
```

Figure 344 Ensure only authenticated users with items in their cart can proceed, then define the APS credentials.

This block includes your hardened session configuration, checks that a valid `user_id` exists and that `$_SESSION['cart']` is not empty. It then declares the APS access code, merchant ID, secret phrase, currency, language, and constructs the return URL incorporating the PHP session ID.

Order Details and Signature Computation

```
// == Use the session cart total (in BHD) ==
$cart_total_bhd = $_SESSION['cart_total_bhd'];
$amount = $cart_total_bhd * 1000; // Convert to fils
$order_id = uniqid("BPZ_");

$request_params = [
    'access_code'      => $access_code,
    'merchant_identifier' => $merchant_identifier,
    'merchant_reference' => $order_id,
    'amount'           => $amount,
    'currency'         => $currency,
    'language'         => $language,
    'command'          => 'PURCHASE',
    'return_url'        => $return_url,
    'customer_email'    => $_SESSION['email'] ?? 'guest@bellapizza.com',
];

ksort($request_params);

$signature_string = $sha_request_phrase;
foreach ($request_params as $key => $value) {
    $signature_string .= "$key=$value";
}
$signature_string .= $sha_request_phrase;

$signature = hash("sha256", $signature_string);
```

Figure 345 Build the parameter array, sort it, then create the APS signature.

Here we calculate amount in fils (1 BHD = 1000 fils), generate a unique merchant_reference, and assemble all required APS parameters into \$request_params. After sorting by key, we prepend and append the shared secret phrase, concatenate each key=value, and compute a SHA-256 hash, yielding the signature to submit alongside the other fields.

Auto-Submitting the APS Checkout Form

```
// === Auto-redirect to APS Checkout ===
?>
<!DOCTYPE html>
<html>
<head>
|   <meta http-equiv="refresh" content="0;url=https://sbcheckout.payfort.com/FortAPI/paymentPage" />
</head>
<body>
|   <form id="paymentForm" method="POST" action="https://sbcheckout.payfort.com/FortAPI/paymentPage">
|       <?php foreach ($request_params as $key => $value): ?>
|           <input type="hidden" name="<?= htmlspecialchars($key) ?>" value="<?= htmlspecialchars($value) ?>">
|       <?php endforeach; ?>
|       <input type="hidden" name="signature" value="<?= $signature ?>">
|   </form>
|
|   <script>
|       // Auto-submit the form
|       document.getElementById('paymentForm').submit();
|   </script>
</body>
</html>
```

Figure 346 Render a hidden form with all APS parameters plus signature, then auto-redirect.

The `<meta>` tag provides a non-JS fallback redirect. We loop over `$request_params` to output each as a hidden input, append the signature, and immediately submit the form via JavaScript so the customer lands on APS's secure payment page without clicking anything.

Testing Approach

I tested this in the APS sandbox environment. First, I confirmed the page rejects access when `$_SESSION['user_id']` is unset or `$_SESSION['cart']` is empty, ensuring no unintended redirects. I then populated the cart session values, navigated to this page, and used browser DevTools Network tab to inspect the outgoing POST request: verifying all hidden fields matched the PHP-computed parameters, the signature header matched my local SHA-256 computation, and the redirect target was correct. For fallback, I disabled JavaScript and saw the meta refresh navigate to the APS URL.

Security Considerations

All APS credentials remain server-side. The `sha_request_phrase` never leaks to the client, only its hash does. Using `session_config.php` enforces Secure, HttpOnly, and SameSite on `PHPSESSID`. We validate the session and cart exist before proceeding, preventing unauthorized or empty checkout attempts. We also HTML-escape every hidden input via `htmlspecialchars()` to mitigate injection risks, and rely on HTTPS for the redirect to APS.

Images

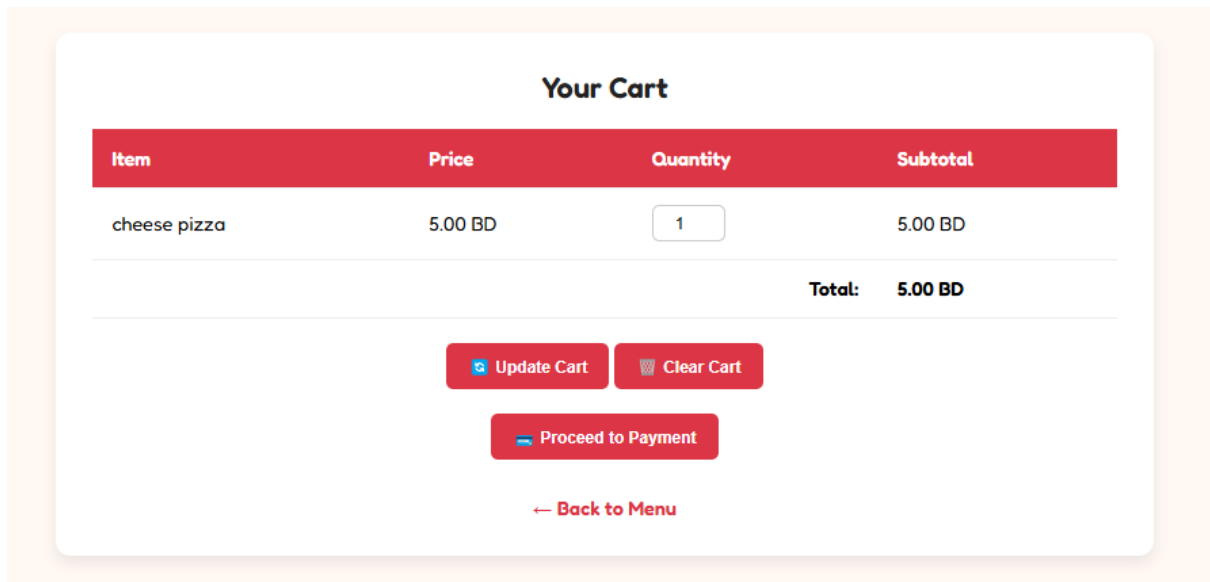


Figure 347 Starting Cart with Items

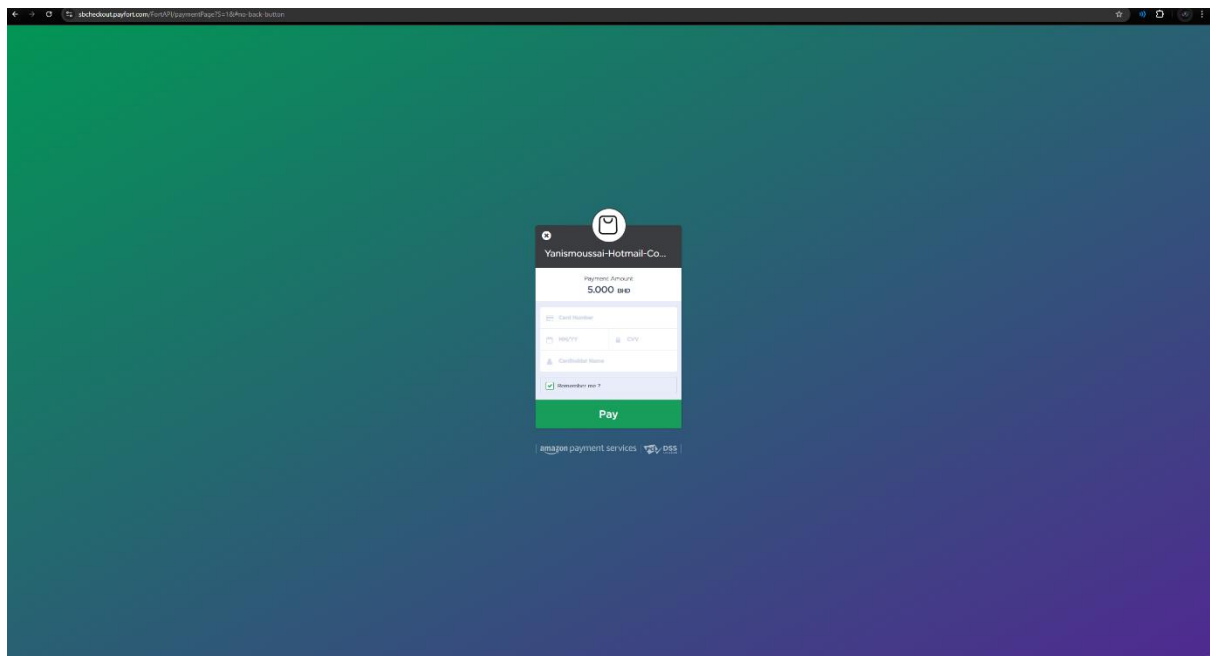


Figure 348 Redirected to payment gateway where user needs to input payment details

The image shows a payment form interface. At the top, there is a dark header with a close button (X) and a shopping bag icon. Below the header, the merchant name "Yanismoussai-Hotmail-Co..." is displayed. The payment amount is shown as "5.000 BHD". The card details section includes a Visa card number "4005550000000001", an expiration date of "05/25", and a CVV of "123". The cardholder's name is partially visible as "Cardholder Name". There is a checkbox labeled "Remember me ?" which is checked. A large green "Pay" button is at the bottom of the form. The footer contains the "amazon payment services" logo and a "PCI DSS compliant" badge.

Yanismoussai-Hotmail-Co...

Payment Amount
5.000 BHD

Card Number
4005550000000001 **VISA**

MM/YY
05/25

CVV
123

Cardholder Name

☒ Remember me ?

Pay

amazon payment services | PCI DSS compliant

Figure 349 Payment Form Filled

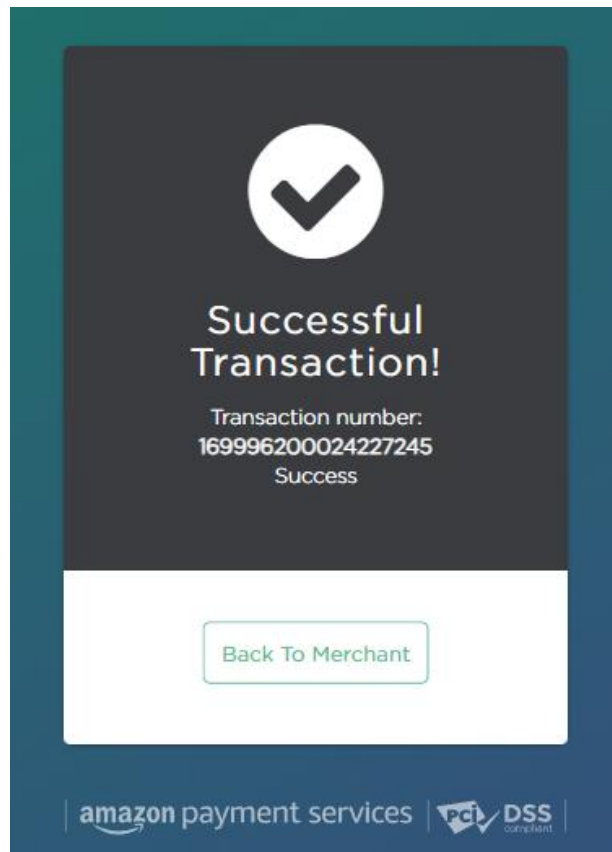


Figure 350 Successful Payment Message from APS

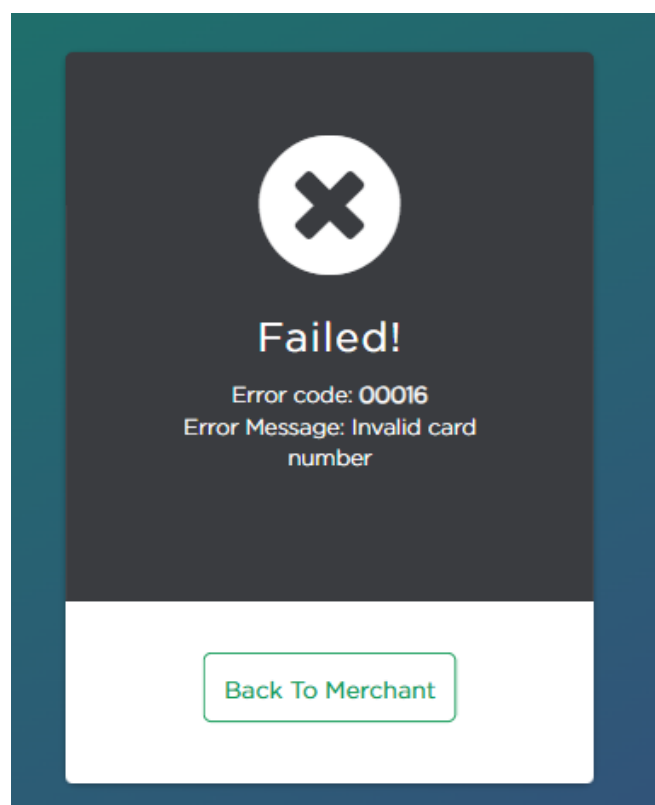


Figure 351 Failed Payment Message from APS

IV. T. Response Page

The Payment Response page processes the APS gateway's POST callback after a customer attempts to pay. It re-associates the payment session, validates the cryptographic signature, and then displays a clear success or failure message. On success, it triggers an order-placement redirect (and confirmation email); on failure or tampering it shows an error and offers a safe return to the dashboard.

Design Rationale

To ensure the payment workflow is both seamless and secure, we begin by restoring the PHP session using the `session_id` passed by APS. We include our hardened session and CSRF helpers for consistency, even though no form submissions occur here. We sort and hash all APS response parameters with our secret phrases to verify the signature, guarding against spoofed callbacks. If valid and approved (`status==='14'`), we fire off a confirmation email and automatically POST the customer through to our internal `place_order.php` after a short delay—falling back to a manual button for users without JavaScript. On any signature mismatch or declined status, we display an explicit error in the page's color-coded style, with a clear “Back to Home” link.

Session Restoration and Includes

```
<?php
if (isset($_GET['session_id'])) {
    session_id($_GET['session_id']);
}

require_once 'includes/session_config.php';
require_once 'includes/csrf.php';
```

Figure 352 Restore the specific session from APS and load core session/CSRF configuration

This block checks for a `session_id` parameter (sent by the payment gateway), uses it to resume the correct PHP session, and then includes our hardened `session_config.php` (for secure cookies) and `csrf.php` (for consistency, though no forms post here).

Signature Verification and Status Check

```
$sha_response_phrase = '36zZogKxtTYFpxc2lqblun#?';
$response_params = $_POST;

if (empty($response_params)) {
    echo "<h3>No response received.</h3>";
    exit;
}

$received_signature = $response_params['signature'] ?? '';
unset($response_params['signature']);

ksort($response_params);

$signature_string = $sha_response_phrase;
foreach ($response_params as $key => $value) {
    $signature_string .= "$key=$value";
}
$signature_string .= $sha_response_phrase;

$calculated_signature = hash("sha256", $signature_string);
$payment_success = ($received_signature === $calculated_signature && $response_params['status'] === '14');
```

Figure 353 Compute and compare the APS signature, then flag success if the status code is 14

After collecting all POST fields, we strip out the incoming signature, sort the remaining parameters, and wrap them with our secret phrases to compute a SHA-256 hash. We only treat the payment as successful if the signatures match and APS reports status code 14 (indicating approval).

Triggering Confirmation Email

```
if ($payment_success && isset($_SESSION['email'])) {
    require_once 'send_email.php';
}
?>
```

Figure 354 On validated success, fire off the order confirmation email.

If the payment passed our validation and the user's email is in session, we include send_email.php to queue or send the confirmation message, ensuring users receive immediate receipt details.

Auto-Redirect to Order Placement

```
<script>
  setTimeout(() => {
    const form = document.createElement('form');
    form.method = 'POST';
    form.action = 'place_order.php';

    const token = document.createElement('input');
    token.type = 'hidden';
    token.name = 'from_payment';
    token.value = '1';
    form.appendChild(token);

    document.body.appendChild(form);
    form.submit();
  }, 5000); // delay 5 seconds
</script>
<noscript>
  <form method="POST" action="place_order.php">
    <input type="hidden" name="from_payment" value="1">
    <button type="submit">Click here if not redirected</button>
  </form>
</noscript>
</php>
```

Figure 355 Five-second delayed redirect to finalize the order, with a no-JS fallback.

Upon success, this JavaScript builds and submits a hidden POST form to our `place_order.php` endpoint after a brief pause—giving customers time to read the “Thank you” message. A `<noscript>` block ensures non-JavaScript browsers still proceed manually.

User-Facing Feedback UI

```
<?php if ($received_signature === $calculated_signature): ?>
    <?php if ($response_params['status'] === '14'): ?>
        <h2 class="success">✔ Payment Successful</h2>
        <p>Thank you for ordering with Bella Pizza!</p>
        <p>A confirmation email has been sent to <strong><?= htmlspecialchars($_SESSION['email']) ?></strong>.</p>
        <p>You will be redirected shortly...</p>
    <?php else: ?>
        <h2 class="fail">✖ Payment Failed</h2>
        <p>Status Code: <?= htmlspecialchars($response_params['status']) ?></p>
        <p>Message: <?= htmlspecialchars($response_params['response_message']) ?></p>
        <a class="button" href="dashboard_user.php">🏠 Return to Home Page</a>
    <?php endif; ?>
<?php else: ?>
    <h2 class="fail">⚠ Invalid Signature</h2>
    <p>Possible tampering detected.</p>
    <a class="button" href="dashboard_user.php">🏠 Return to Home Page</a>
<?php endif; ?>
```

Figure 356 Display success, failure, or tampering messages in a contrasting dark theme.

We branch three ways: a green-tinted success block, a red-tinted decline block showing the APS status code and message, or a tampering alert if the signature check fails. Each includes a prominent button or note guiding users back if something went wrong.

Testing Approach

To test this page I simulated APS POST callbacks via cURL and Postman, crafting both valid and invalid signatures. First I retrieved the exact POST parameters APS would send, then built the expected signature string with our sha_response_phrase to confirm the page recognized approved payments and triggered the redirect & email include. I then altered a single POST field to ensure the “Invalid Signature” path displayed. Finally I tested without JavaScript to verify the <noscript> fallback worked.

Security Considerations

All APS interactions are guarded by strict signature validation against a secret phrase to prevent spoofed callbacks. We restore the exact session with session_id only when explicitly provided by APS to avoid session fixation. User emails are escaped via htmlspecialchars() in the thank-you message. No user-supplied data is ever echoed unescaped. The redirect POST to place_order.php includes a hidden token, and place_order.php must re-validate session and CSRF. Error messages do not leak secret details.

Images

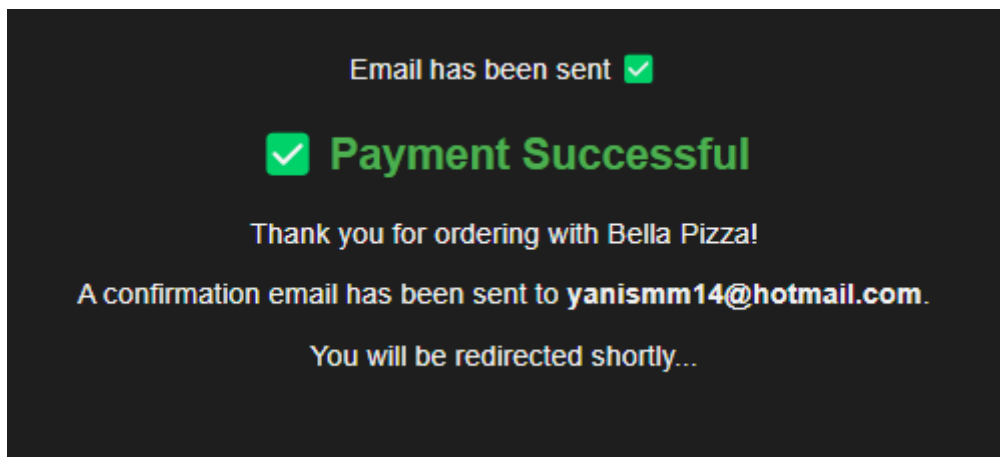


Figure 358 Successful Payment Message

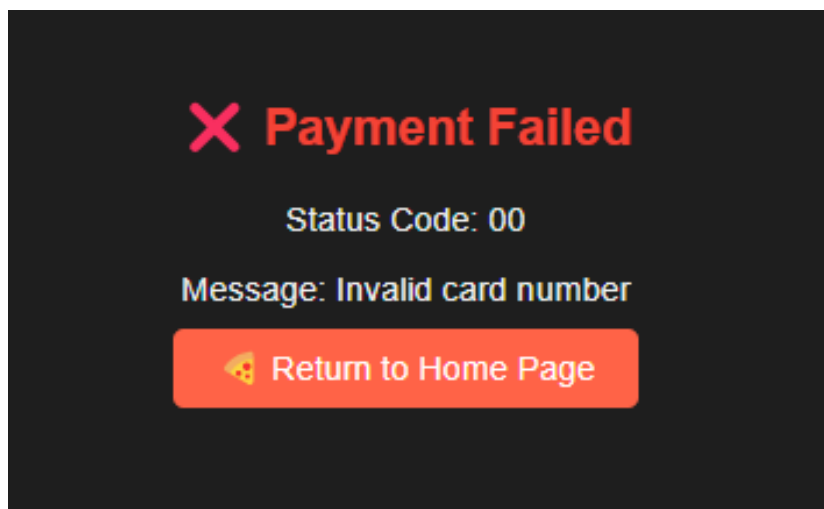


Figure 357 Failed Payment Message

IV. U. Order Placement Page

The Place Order script finalizes a customer's purchase after a successful payment. It validates the incoming POST from our own Payment Response page, reconstructs the cart from session data, calculates subtotals and the grand total, writes the order and its line items into the database, clears the session cart, and then presents a clear confirmation screen.

Design Rationale

To guarantee data integrity and prevent unauthorized access, this page only accepts a POST carrying our custom `from_payment` flag. We enforce that the user is logged in and has a non-empty cart before proceeding. Using PDO prepared statements shields against SQL injection when retrieving prices and inserting the order and its items. After the database writes, we unset the cart immediately to avoid duplicate orders. The front-end confirmation uses a simple, readable layout to reassure customers that their order succeeded and display the total paid.

Request Validation & Session Checks

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

✓ if ($_SERVER["REQUEST_METHOD"] !== "POST" || !isset($_POST["from_payment"])) {
    header("Location: dashboard_user.php");
    exit();
}

✓ if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit();
}

✓ if (!isset($_SESSION["cart"]) || empty($_SESSION["cart"])) {
    header("Location: dashboard_user.php");
    exit();
}
```

Figure 359 Ensure only a proper POST from our payment flow with a valid session and non-empty cart.

This block pulls in our secure session configuration and CSRF helper (for consistency), then immediately rejects any direct GET or malformed POST requests. It also redirects unauthenticated users or those without items in their cart, preventing unintended or replayed submissions.

Database Connection

```
$host = "db";
$dbname = "RestaurantDB";
$username = "root";
$password = "rootpass";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

Figure 360 Establish a PDO connection to the RestaurantDB with exception mode enabled.

By using PDO and turning on `ERRMODE_EXCEPTION`, any database errors instantly throw exceptions we can catch, ensuring we don't silently fail when writing order records.

Cart Retrieval & Total Calculation

```
$cart = $_SESSION["cart"];
$menu_ids = array_keys($cart);
$placeholders = implode(",", array_fill(0, count($menu_ids), "?"));
$stmt = $pdo->prepare("SELECT menu_id, price FROM menu WHERE menu_id IN ($placeholders)");
$stmt->execute($menu_ids);
$prices = $stmt->fetchAll(PDO::FETCH_KEY_PAIR);

$total = 0;
$order_items = [];

foreach ($cart as $menu_id => $qty) {
    $price = $prices[$menu_id];
    $subtotal = $qty * $price;
    $total += $subtotal;
    $order_items[] = [
        'menu_id' => $menu_id,
        'quantity' => $qty,
        'subtotal' => $subtotal
    ];
}
```

Figure 361 Fetch current prices in bulk, compute per-item subtotals, and accumulate the order total.

We build a dynamic `IN (...)` query to pull all relevant menu prices at once, then iterate the session cart to calculate each line's subtotal and sum them into the grand total, storing details for later insertion.

Confirmation UI

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Order Confirmation</title>
  <style>...
</style>
</head>
<body>
  <div class="container">
    <h2>🍕 Order Placed Successfully!</h2>
    <p>Your order <strong>#<?=$order_id ?></strong> has been submitted. Thank you for ordering with Bella Pizza!</p>
    <p class="total">Total Paid: <?= number_format($total, 3) ?> BD</p>
    <a href="dashboard_user.php">← Back to Homepage</a>
  </div>
</body>
</html>
```

Figure 362 Display the order number and total paid in a clean, centered card.

We present a simple, friendly confirmation page highlighting the new order ID and the exact amount charged, along with a “Back to Homepage” link for convenience.

Testing Approach

I simulated the full payment flow by first populating `$_SESSION['cart']` with sample items in a test environment, then POSTing to `place_order.php` with a minimal `from_payment=1` flag via cURL and a valid PHP session cookie. I verified that orders and `order_items` rows appeared correctly in the database, that the session cart was cleared, and that the confirmation page displayed accurate order details. I also tested edge cases such as an empty cart or missing POST flag to ensure proper redirection.

Security Considerations

We accept only POST requests with our private `from_payment` parameter to avoid CSRF or replay attacks. All database interactions use prepared statements to prevent SQL injection. Session checks prevent unauthorized or repeated submissions. We never echo unescaped user data—order IDs and amounts come from our trusted internal calculations. Finally, clearing the session cart protects against duplicate orders if the user refreshes the confirmation page.

Images

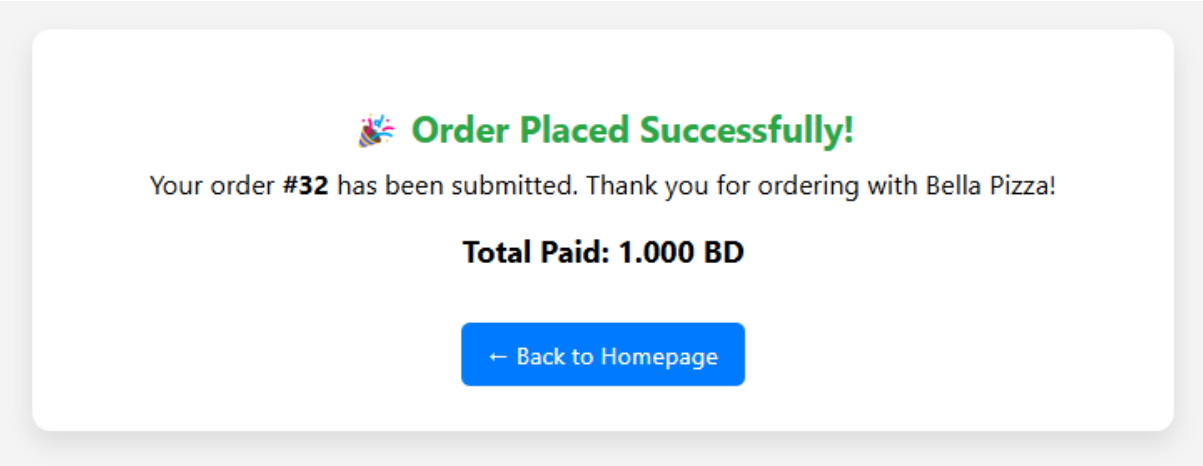


Figure 363 User-facing confirmation screen showing successful order placement and total charged.

☐	Edit	Copy	Delete	31	1	2025-05-01 16:04:15	Preparing	5.00	2025-05-01 16:04:15	2025-05-03 18:21:48	5
☐	Edit	Copy	Delete	32	1	2025-05-05 14:54:24	Pending	1.00	2025-05-05 14:54:24	2025-05-05 14:54:24	NULL

Figure 364 Verified new order and associated items in the database after POSTing to place_order.php

IV. V. My orders Page

The My Orders page allows a logged-in customer to review all of their past orders in reverse chronological order. It provides search functionality so customers can quickly locate specific orders by ID, status, or date. Each order row displays its ID, date, current status, total amount, and a button to view full details.

Design Rationale

This page begins by enforcing authentication and CSRF protection, ensuring only legitimate users can access their own orders. A PDO connection retrieves both the user's name and their order history from the database. We then optionally filter the in-memory order list when a search term is submitted, avoiding extra database hits for each keystroke. The layout is a mobile-responsive table with status badges styled in context-sensitive colors. Simple, self-submitting forms power the search and “View Details” actions, keeping JavaScript usage minimal and progressive.

Session, Authentication, and User Lookup

```
<?php
require_once 'includes/session_config.php';
require_once 'includes/csrf.php';

if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit();
}

$pdo = new PDO("mysql:host=db;dbname=RestaurantDB", "root", "rootpass");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$user_id = $_SESSION["user_id"];
$stmt = $pdo->prepare("SELECT username FROM customer WHERE customer_id = ?");
$stmt->execute([$user_id]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Figure 365 Secure session start, CSRF include, and fetch the logged-in customer's username

This block first loads the hardened session settings and CSRF helper, then checks if `user_id` is set in the session—otherwise redirecting to login. A PDO connection is created with exception mode enabled. Finally, we prepare and execute a query to load the customer's username for display in the page header.

Retrieving and Searching Order History

```
$sql = "SELECT * FROM `order` WHERE customer_id = :customer_id ORDER BY order_date DESC";
$stmt = $pdo->prepare($sql);
$stmt->execute([':customer_id' => $user_id]);
$orders = $stmt->fetchAll(PDO::FETCH_ASSOC);

$search = $_GET['search'] ?? '';

if (!empty($search)) {
    $orders = array_filter($orders, function ($order) use ($search) {
        return strpos((string)$order['order_id'], $search) !== false ||
            strpos($order['status'], $search) !== false ||
            strpos($order['order_date'], $search) !== false;
    });
}
?>
```

Figure 366 Load the customer's orders and apply the optional search filter.

We prepare and execute a parameterized query to retrieve every order for the current customer, ordered newest first. If a search string is provided via GET, we filter the resulting PHP array in memory—matching the order ID, status, or date against the search term in a case-insensitive way.

Search Form and Order Table

```
<div class="container">

    <h2>My Orders</h2>

    <?php if (count($orders) > 0): ?>
        <form method="GET" class="search-form">
            <input type="text" name="search" placeholder="Search by ID, Status, or Date" value="<?=
            htmlspecialchars($search) ?>">
            <button type="submit">Search</button>
        </form>

        <table>
            <tr>
                <th>Order ID</th>
                <th>Date</th>
                <th>Status</th>
                <th>Total (BD)</th>
                <th>Action</th>
            </tr>
            <?php foreach ($orders as $order): ?>
                <tr>
                    <td>#<?= htmlspecialchars($order['order_id']) ?></td>
                    <td><?= htmlspecialchars(date('d M Y, H:i', strtotime($order['order_date']))) ?></td>
                    <td>
                        <span class="status-badger <?= str_replace(' ', '', htmlspecialchars($order
                        ['status'])) ?>">
                            <?= htmlspecialchars($order['status']) ?>
                        </span>
                    </td>
                    <td><?= number_format($order['total_amount'], 2) ?> BD</td>
                    <td class="view-button">
                        <form method="GET" action="order_details_user.php" style="margin: 0;">
                            <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
                            <input type="hidden" name="order_id" value="<?= htmlspecialchars($order['order_id']) ?
                            >">
                            <button type="submit">View Details</button>
                        </form>
                    </td>
                </tr>
            <?php endforeach; ?>
        </table>
    <?php else: ?>
        <p style="text-align:center; margin-top:20px;">You have not placed any orders yet.</p>
    <?php endif; ?>
</div>
```

Figure 367 Search input followed by a responsive table of orders or an empty-state message

Within the main container, we first display the page title. If there are any orders, a GET form lets the user filter by ID, status, or date. The table header defines columns for each order attribute. Each row prints the order's ID, formatted date, a colored status badge (with styling classes matching statuses), total amount, and a "View Details" button that submits the order ID and CSRF token to `order_details_user.php`. If the order list is empty, a centered paragraph informs the user.

Testing Approach

I placed several test orders in the database under the test user and verified that they appear in descending date order. I tried the search textbox with partial matches—order ID fragments, status keywords (“Pending”), and date substrings—to confirm the in-memory filtering works correctly. I clicked the “View Details” button and ensured it passed both order_id and the CSRF token to the details page. I also checked the empty state by clearing all orders for that user, observing the placeholder message and absence of the table.

Security Considerations

All sensitive actions are protected by CSRF tokens. Authentication checks guard the entire page and data fetches use prepared statements to prevent SQL injection. Every piece of dynamic text is run through htmlspecialchars() to avert XSS attacks. The “View Details” form uses GET to make the URL bookmarkable but still includes a CSRF token to avoid tampering. Status badges are purely presentational and do not drive any logic on the client side.

Images

My Orders				
Search by ID, Status, or Date			Search	
Order ID	Date	Status	Total (BD)	Action
#32	05 May 2025, 14:54	Pending	1.00 BD	View Details
#31	01 May 2025, 16:04	Preparing	5.00 BD	View Details
#30	30 Apr 2025, 12:23	Cancelled	5.00 BD	View Details
#29	29 Apr 2025, 12:34	Completed	5.00 BD	View Details
#28	29 Apr 2025, 12:31	Cancelled	5.00 BD	View Details
#27	29 Apr 2025, 12:20	Cancelled	5.00 BD	View Details

Figure 368 My orders Page

My Orders				
Completed			Search	
Order ID	Date	Status	Total (BD)	Action
#29	29 Apr 2025, 12:34	Completed	5.00 BD	View Details
#24	22 Apr 2025, 10:50	Completed	1.00 BD	View Details
#23	20 Apr 2025, 06:11	Completed	1.00 BD	View Details
#19	15 Apr 2025, 10:45	Completed	4.00 BD	View Details
#18	09 Apr 2025, 12:05	Completed	4.00 BD	View Details
#17	09 Apr 2025, 07:57	Completed	4.00 BD	View Details
#16	09 Apr 2025, 07:50	Completed	4.00 BD	View Details
#15	09 Apr 2025, 07:39	Completed	4.00 BD	View Details
#14	09 Apr 2025, 07:37	Completed	4.00 BD	View Details

Figure 369 Search function used

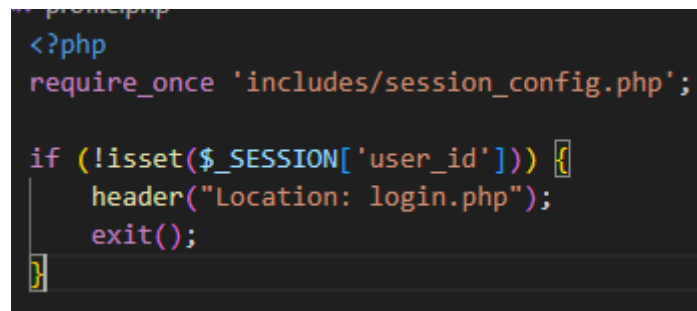
IV. W. Customer Edit Profile

The profile.php page allows logged-in customers to view and update their personal information, including name, phone number, and address. This page plays a critical role in ensuring that customer details are complete and accurate before orders are placed, especially for delivery and payment processing.

Design Rationale

The page is designed with clarity and responsiveness in mind. A single-column layout focuses the user's attention on form fields while maintaining brand consistency through the use of Bella Pizza's primary and accent colors. Field labels and spacing are clear to reduce input errors, and real-time success/error messages guide users through updates. The form auto-fills the current user's information to enhance usability, and fields like the email address are made read-only to prevent accidental changes to sensitive identity data. Username uniqueness is enforced globally across both login types (normal and Google) to avoid conflicts. Overall, the design promotes a seamless user experience while upholding data accuracy and security.

Session Validation and Database Initialization



```
<?php
require_once 'includes/session_config.php';

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit();
}
```

Figure 370 Ensures that only logged-in users can access and edit their profile.

The session check redirects unauthenticated users to the login page, maintaining access control for personal data editing.

Dynamic Table Selection and User Data Fetching

```
$user_id = $_SESSION['user_id'];
$login_type = $_SESSION['login_type'] ?? 'normal';
$table = ($login_type === 'google') ? 'google_login' : 'customer';

$stmt = $conn->prepare("SELECT * FROM $table WHERE customer_id = ?");
$stmt->bind_param("i", $user_id);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc();
```

Figure 371 Determines the user table based on login method and retrieves profile data.

Whether the user logged in normally or via Google, the correct table is used to fetch current values for display and update.

Handle Profile Update Submission

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $phone = trim($_POST["phone"]);
    $address = trim($_POST["address"]);
    $username = trim($_POST["username"]);
```

Figure 372 Validates submitted form data for correctness and formatting.

Ensures all inputs meet minimum criteria: valid phone number length, sufficient address detail, and username length. Prevents incomplete or malformed records.

Check for Username Uniqueness

```
if (preg_match('/^[0-9]{8,15}$/', $phone) && strlen($address) >= 5 && strlen($username) >= 3) {
    if ($username !== $user['username']) {
        $check = $conn->prepare("
            SELECT 1 FROM customer WHERE username = ? AND customer_id != ?
            UNION
            SELECT 1 FROM google_login WHERE username = ? AND customer_id != ?
        ");
        $check->bind_param("sisi", $username, $user_id, $username, $user_id);
        $check->execute();
        $check->store_result();
```

Figure 373 Ensures that the updated username is unique across both user tables.

This query prevents two users — whether registered via Google or directly — from using the same username, avoiding identity confusion.

Apply Updates to Database

```
if (!isset($error)) {  
    $stmt = $conn->prepare("UPDATE $table SET username = ?, phone = ?, address = ? WHERE  
    customer_id = ?");  
    $stmt->bind_param("sssi", $username, $phone, $address, $user_id);  
    $stmt->execute();  
}
```

Figure 374 Saves the new user data to the database.

If no errors are found, the user's updated information is committed securely using a prepared statement.

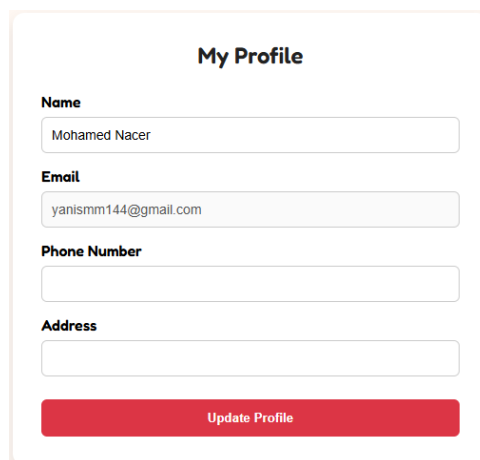
Test Approach

The profile update form was tested for valid and invalid inputs, including empty fields, short addresses, and reused usernames. Attempting to submit a duplicate username triggered the correct warning. Changes to valid data were immediately reflected in the UI and saved in the database. The page was accessed through both standard and Google logins, confirming that it correctly detects the user's authentication type and queries the appropriate table. Security was verified through session handling and SQL validation tests.

Security Considerations

All access to profile.php is gated by session-based authentication, ensuring only logged-in users can view or edit their information. The use of prepared SQL statements in all database operations prevents SQL injection. Additionally, input values are sanitized and validated: phone numbers must match a defined numeric pattern, usernames must be of sufficient length, and addresses are required. Username conflicts are checked against both customer and Google login tables, ensuring uniqueness across the platform. The email address is displayed as a read-only field, protecting this key identifier from unauthorized change.

Images



The screenshot shows a web form titled "My Profile". It contains four input fields: "Name" with the value "Mohamed Nacer", "Email" with the value "yanismm144@gmail.com", "Phone Number" which is empty, and "Address" which is empty. Below these fields is a red button labeled "Update Profile".

Figure 375 Screenshot of the Bella Pizza profile update form with name, phone, and address fields.

References

1. Rouse, Margaret. "Use Case Diagram." *TechTarget*, 2020, <https://www.techtarget.com/searchsoftwarequality/definition/use-case-diagram>. Accessed 18 Apr. 2025.
2. Codd, Edgar F. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, vol. 13, no. 6, 1970, pp. 377–387. ACM Digital Library, <https://doi.org/10.1145/362384.362685>.
3. "Codd's 12 Rules." *Relational Database Index*, reldb.org, <https://reldb.org/c/index.php/twelve-rules/>. Accessed 18 Apr. 2025.
4. Visual Paradigm. "What is Sequence Diagram?" *Visual Paradigm*, <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/>. Accessed 18 Apr. 2025.
5. "PHP Manual." *PHP Manual*, The PHP Group, <https://www.php.net/manual/en/>. Accessed 11 May 2025.
6. "MySQL Documentation." *MySQL Developer Zone*, Oracle Corporation, <https://dev.mysql.com/doc/>. Accessed 11 May 2025.
7. "Introducing JSON." *JSON*, JSON.org, <https://www.json.org/json-en.html>. Accessed 11 May 2025.
8. "PayFort PHP SDK Developer Guide." *GitHub*, hosted by Amazon Payment Services, <https://github.com/payfort/payfort-php-sdk/blob/master/DevelopersGuidePHPSDK.pdf>. Accessed 11 May 2025.
9. "OWASP Top Ten." *OWASP Foundation*, <https://owasp.org/www-project-top-ten/>. Accessed 11 May 2025.
10. **ISO/IEC 27001:2013(E): Information Technology—Security Techniques—Information Security Management Systems—Requirements**. International Organization for Standardization and International Electrotechnical Commission, 2013. *ITRef*, <https://itref.ir/uploads/editor/42890b.pdf>. Accessed 11 May 2025.
11. **ISO/IEC 15408-1:2022(E): Information Security, Cybersecurity and Privacy Protection—Evaluation Criteria for IT Security—Part 1: Introduction and General Model**. International Organization for Standardization and International Electrotechnical Commission, 2022. *ITeh Standards*, <https://cdn.standards.iteh.ai/samples/72891/fbfa4f603f384c7ab0663ffc4c163f3f/ISO-IEC-15408-1-2022.pdf>. Accessed 11 May 2025.
12. tolgadevsec. *PHP Security Cheatsheet*. GitHub, 2021, <https://github.com/tolgadevsec/PHP-Security-Cheatsheet>. Accessed 12 May 2025.